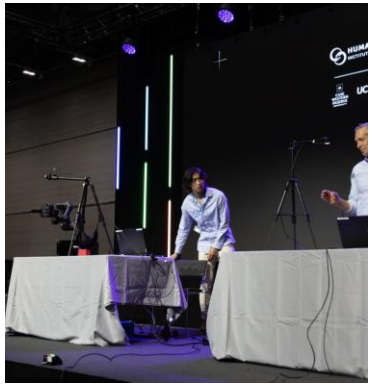
Stephen Mock Work Portfolio

Link to Non-PDF Version with Videos

About Me

- Graduated with Bachelor's and Masters in Mechanical Engineering at Georgia Tech
  - Concentration in Robotics
- Worked at UCLA Biomechatronics Lab, Miso Robotics, NASA JPL, iRobot, SharkNinja
- Interested in space, research, consumer products, IoT
  - Enjoy systems engineering, robotics, mechatronics, coding, mechanical design
- Outside of work
  - Sports, Music, IoT Projects
- Contact Info
  - sjmock99@gmail.com
  - Personal Website



Skills

Hardware:

- CAD Design
  - OnShape, SolidWorks, Creo
    - EPDM
- Prototyping
  - 3D Printing, Laser Cutting
  - Mill, Lathe, etc
  - Soldering
- Controllers / Controls
  - Arduino, ESP32, Teensy
  - Raspberry Pi, Intel NUC
  - PID, Robot Kinematics, Robot Control
- Sensors
  - Tactile (visuo-optical), Force Torque, Thermocouples, IMU, Encoder, Infrared, Thermal, Ultrasonic, Hall, Humidity/Temperature, Laser Displacement
- Components
  - Haptics, Stepper Motors, Brushed Motors, Motor Drivers, 8020, Servos, Relays, Power Supplies, Thermal Controllers, various electronics

Software:

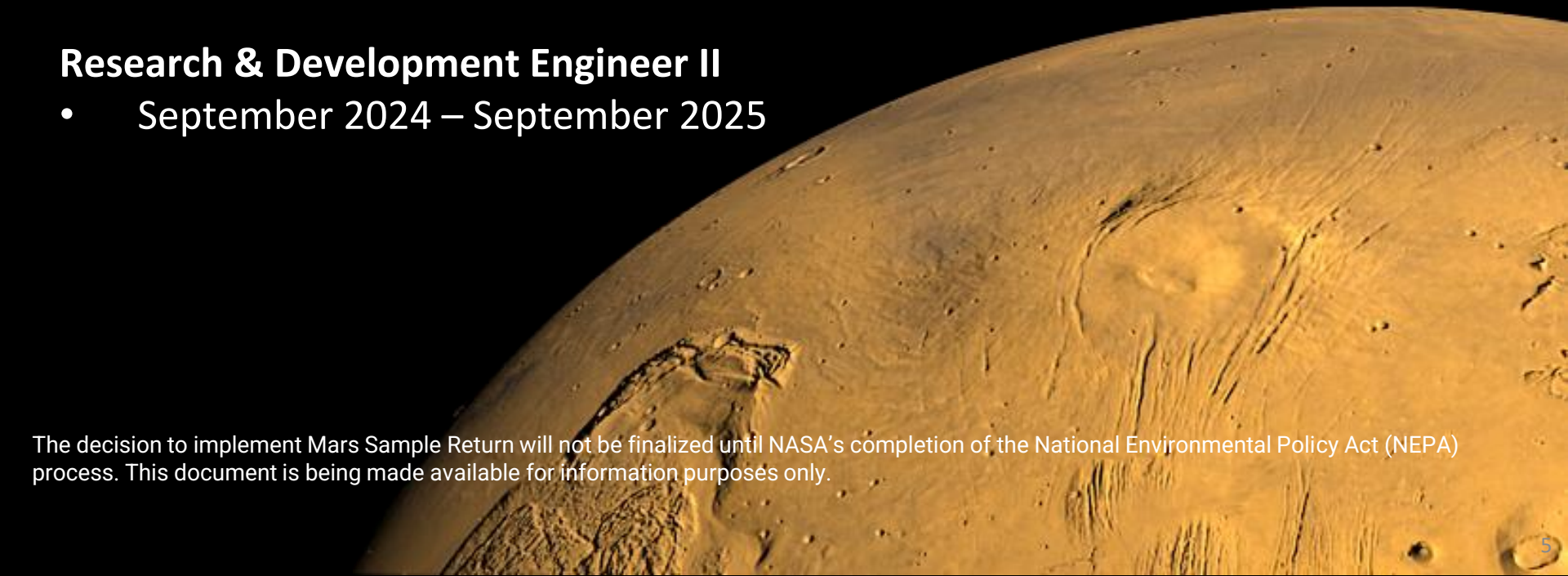
- Programming Languages / Frameworks / OS
  - Python, C/C++ [mainly for Microcontrollers], MATLAB, LabView
    - Basic HTML, CSS, JS, Java
  - ROS1, ROS2, MicroROS
  - Linux (Ubuntu)
- Networking / Protocols
  - Serial Protocols: I2C, SPI, UART
  - MODBUS TCP, TCP/IP, SSH, CAN, VISA, SCPI
  - MQTT
- Applications
  - MATLAB, Git, SciKit Learn (Machine Learning), OpenCV, Linux, Jupyter Notebook, Notion, LabView, PlatformIO, VSCode

Portfolio Table of Contents

UCLA Summary of Work

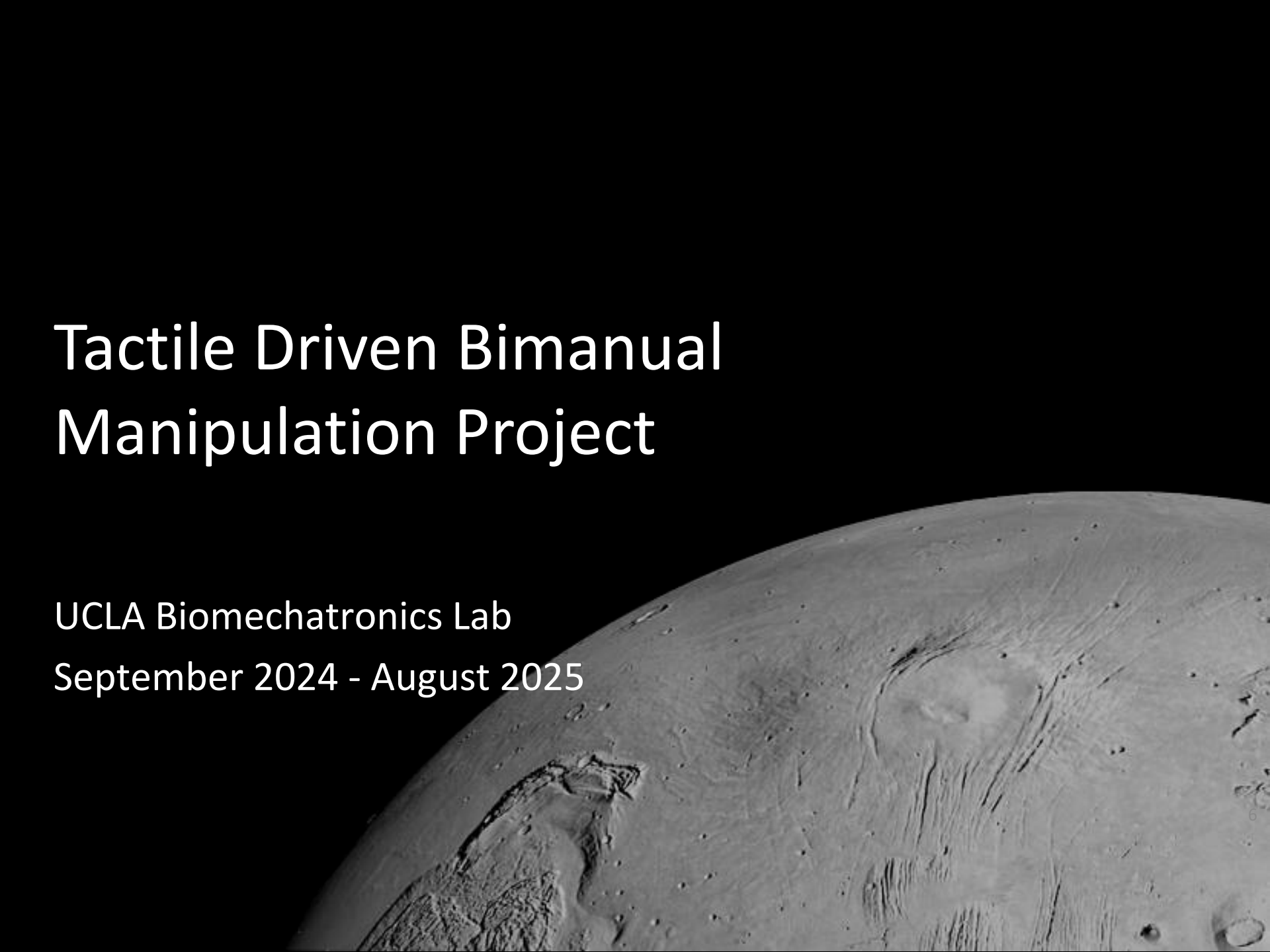
Research & Development Engineer II

- September 2024 – September 2025



The decision to implement Mars Sample Return will not be finalized until NASA's completion of the National Environmental Policy Act (NEPA) process. This document is being made available for information purposes only.

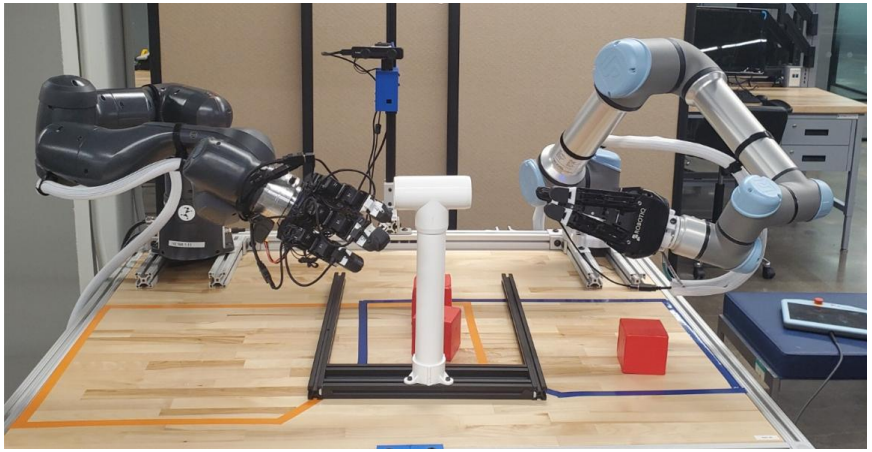
Tactile Driven Bimanual Manipulation Project



UCLA Biomechatronics Lab

September 2024 - August 2025

Project Overview



Current Team Members

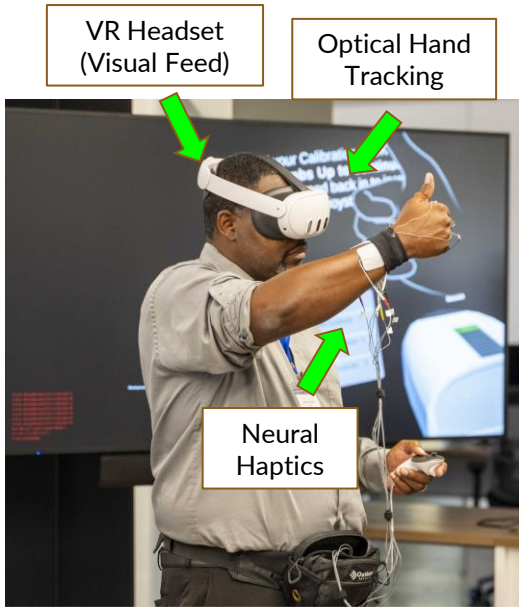
- **Stephen:** System
- **Mumbi:** Tactile Sensors
- **Ben:** Shared Autonomy
- **Cole:** Controller + Architecture

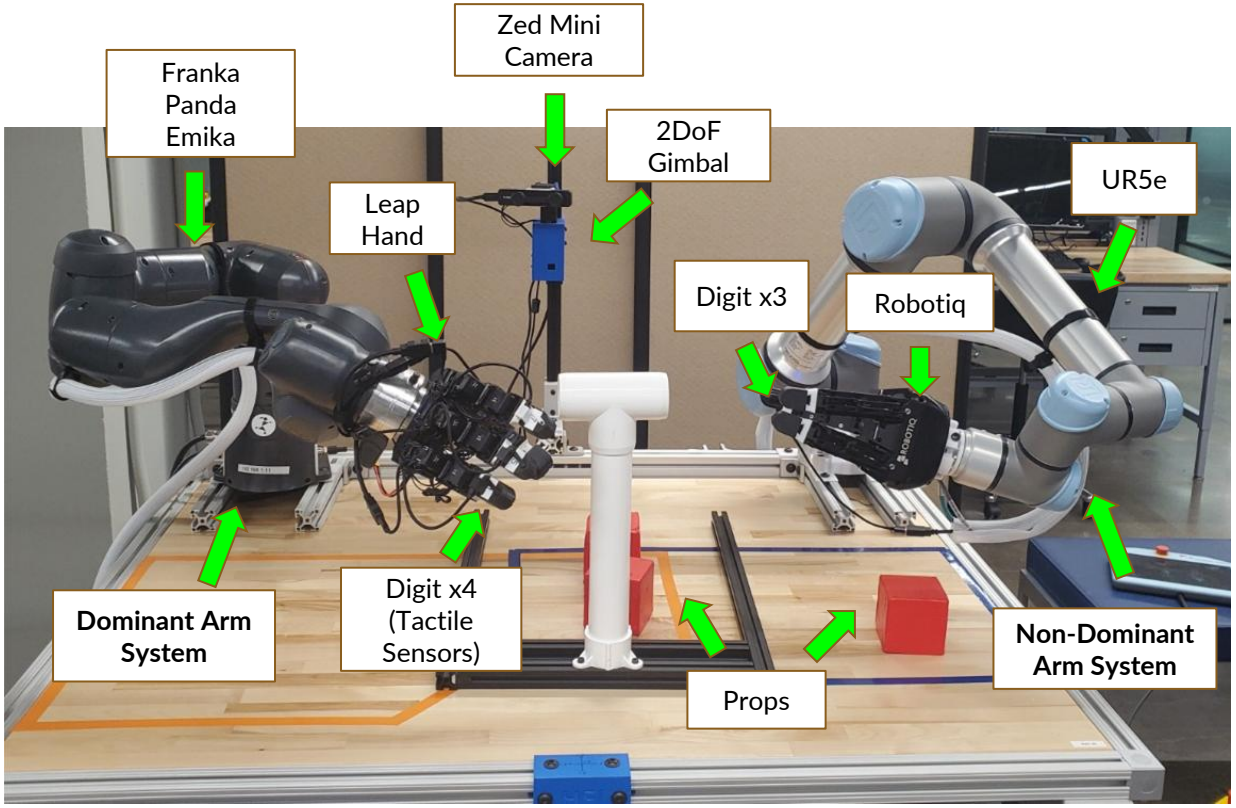
Collaboration with:

- CWRU: Haptics, Hand Tracking
- CS: Human Factors
- UW: Tactile Sensors

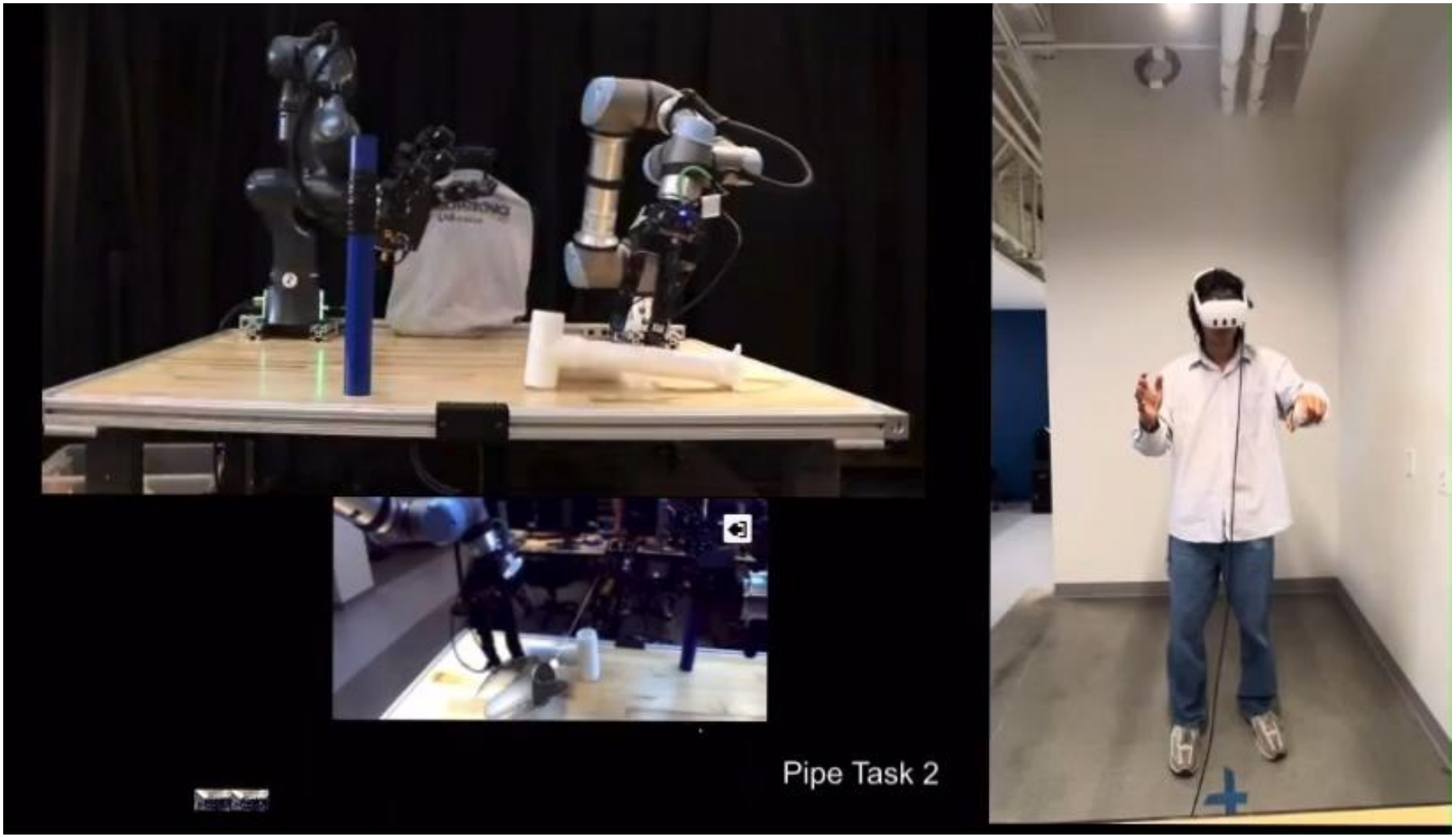
Objective: Create a teleoperated bimanual haptic robotic system in order to explore the applications and control for remote shared embodiment

Project Overview (cont.)





Bimanual Control with Operator



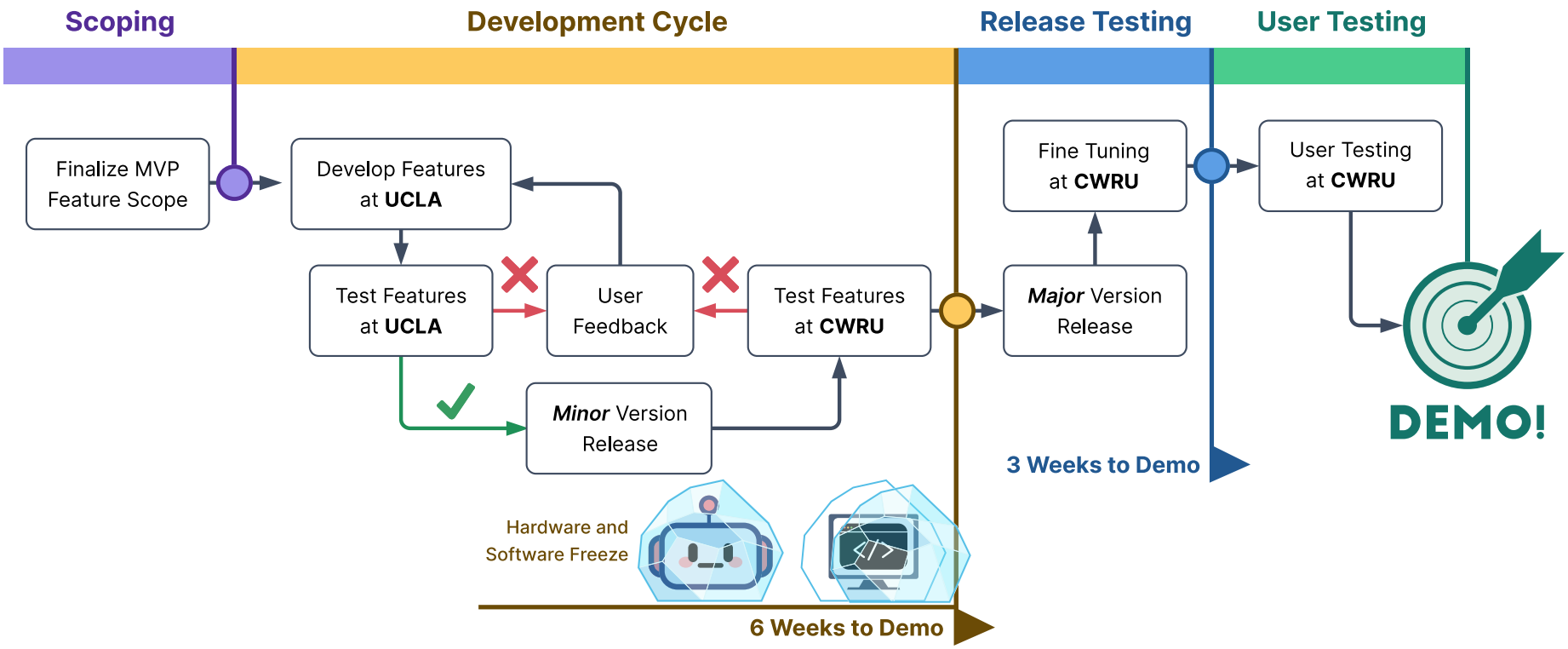
Project Summary

System Level Improvements since start in September

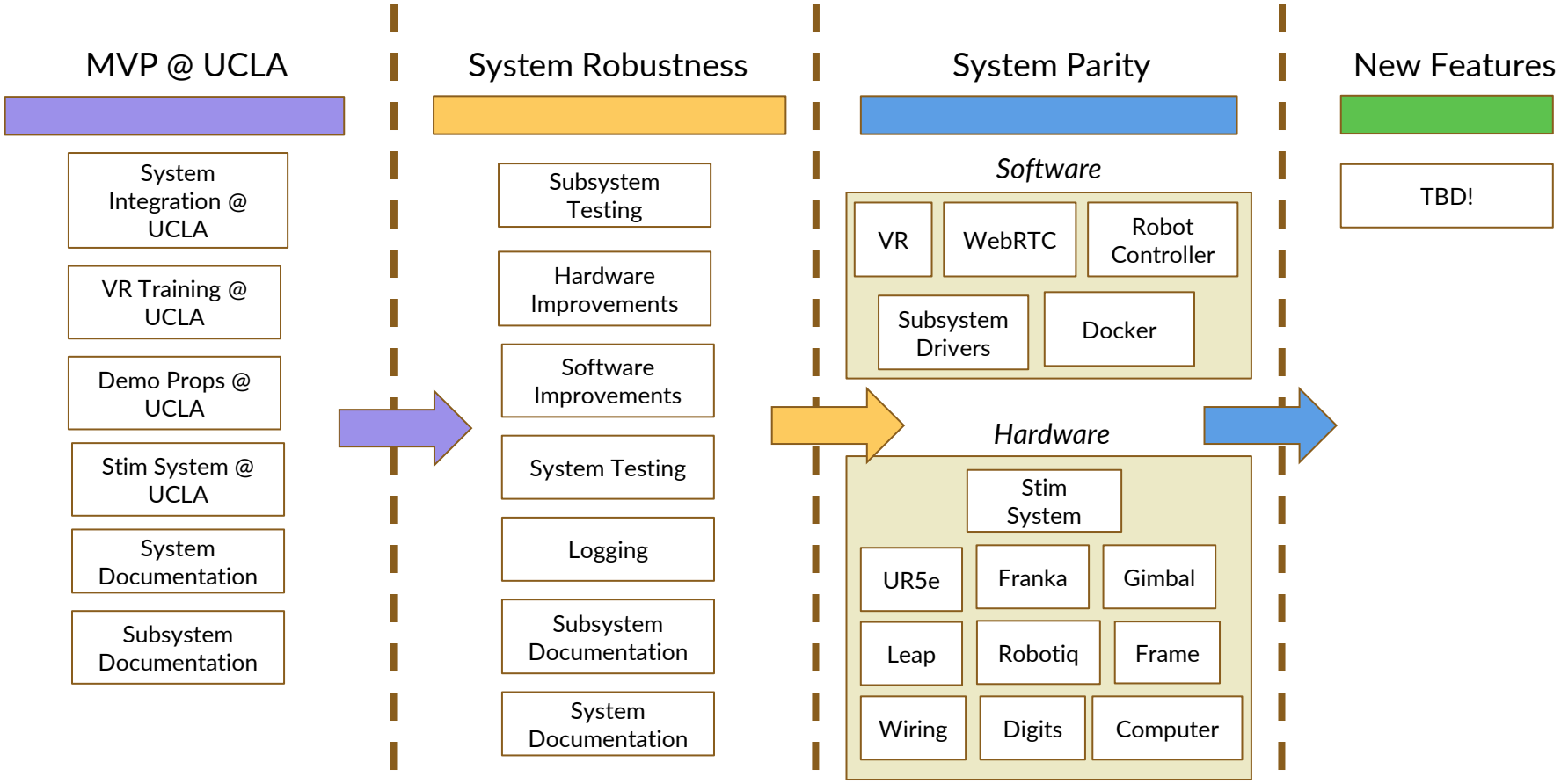
- October system implementation @ UCLA
- System requirements definition + scheduling
- System interface definitions
- Software design definition + software robustness to over 15+ repositories across entire robot system
  - Proper driver design + ROS2 design
  - GUI, launch system, visualization + telemetry
  - Digital Twin
- Robotic system Testing
  - Full end to end test definition
  - Over 200+ automated, hardware in the loop, ROS2 based unit tests
  - Robot controller validation w/ unit tests through simulation and hardware
  - Automatic test report generation
- Hardware design definition
  - Full system CAD
  - System coordinate frame definition
  - Hardware updates to cabling, grippers, gimbal, Digits
  - 4 Hardware releases for LEAP Hand
- Full system release(s) + documentation
  - Software release versioning + tagging
  - Hardware release via. EPDM release system
  - Full system BoM with over 400 parts + links
  - Technical documents / assembly instructions

Systems / Project Management Updates

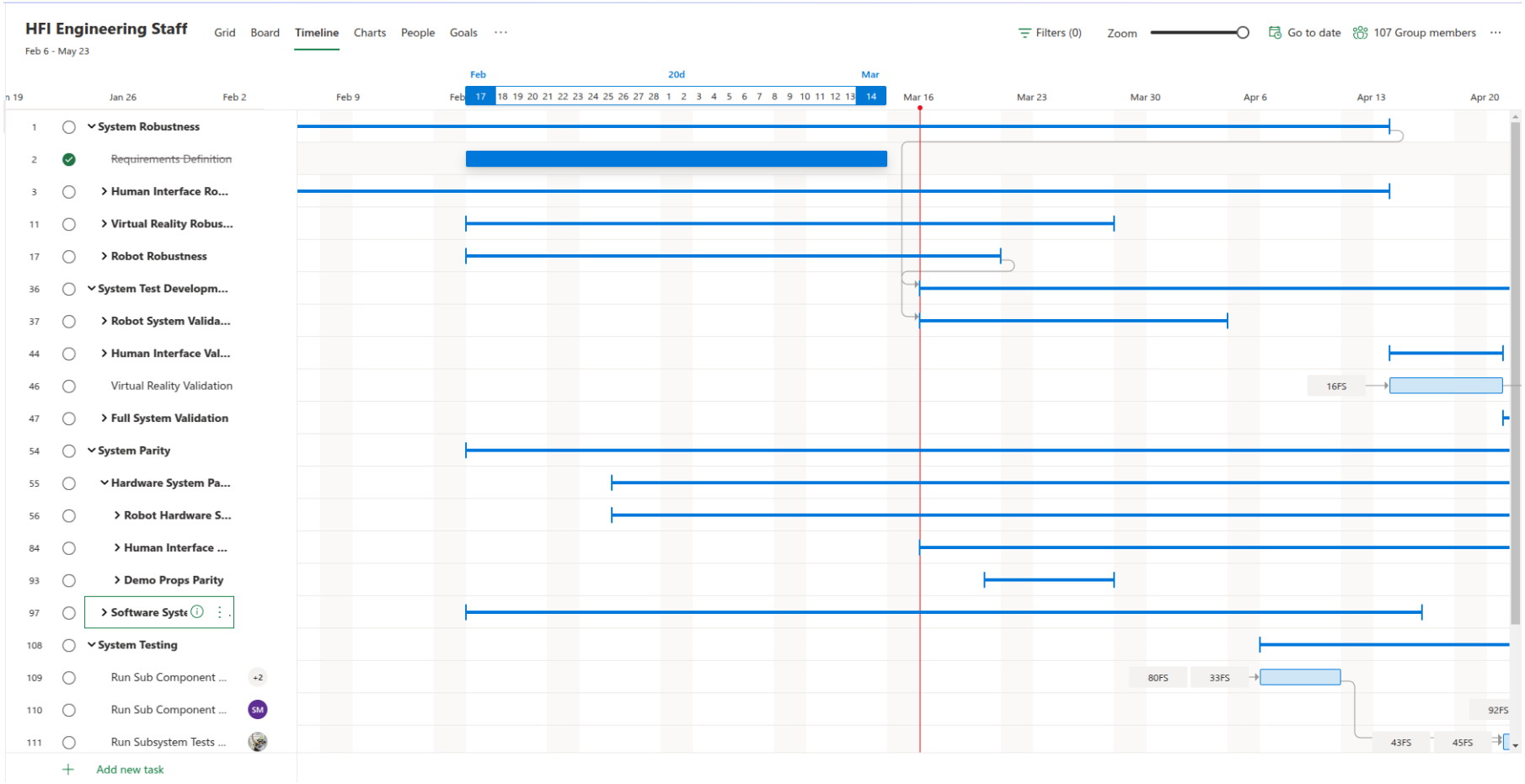
Defining Release Schedule



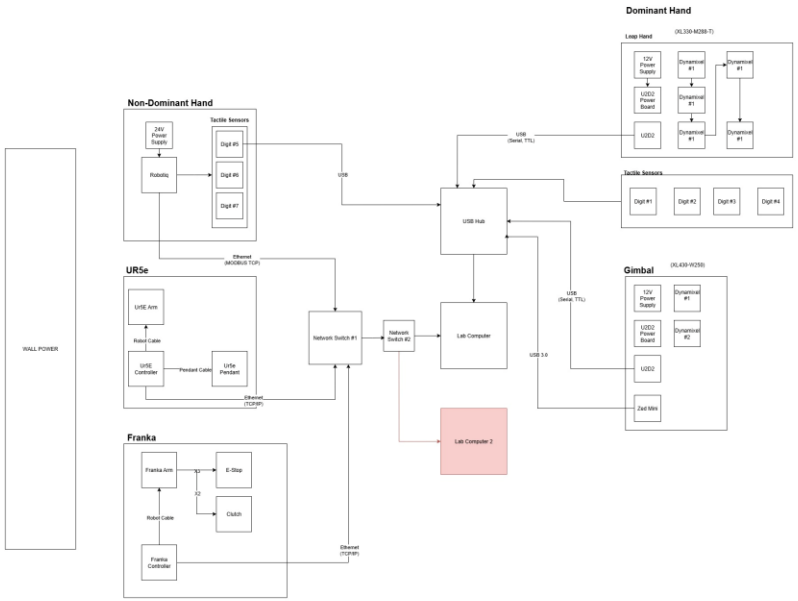
Technical Debt Progression



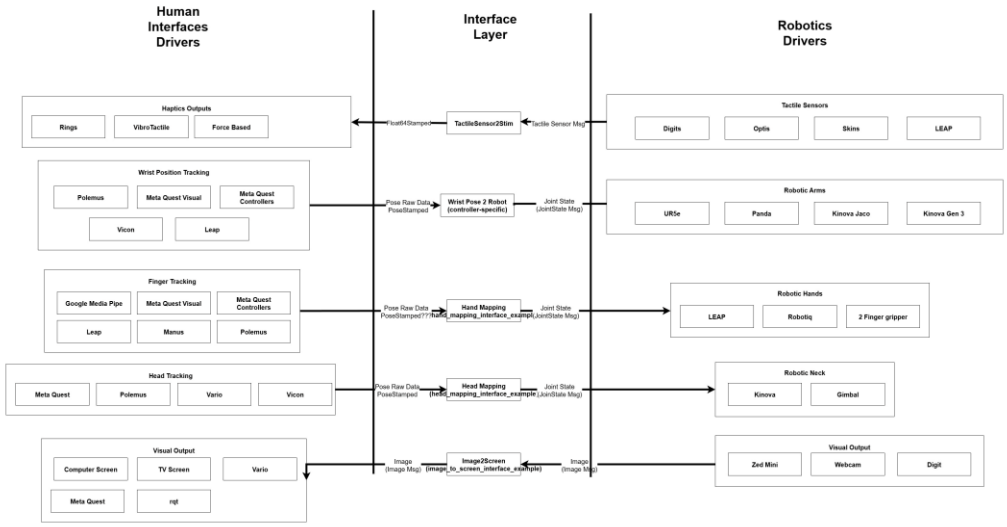
Schedule Coordination



System Diagrams

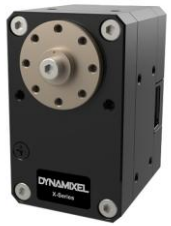


System Block Diagram

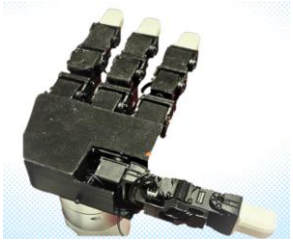


Interface Control Document

System Test Definitions

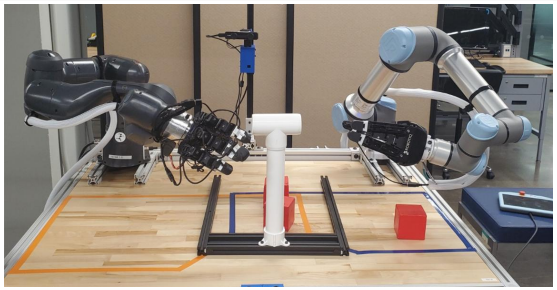


Subcomponent

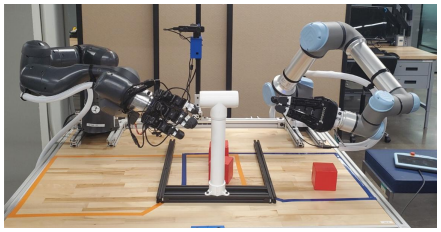


Component

Subsystem



Robotic System



Full System

System Test Definitions

Subcomponent

- **Lowest level of testing**, where the actual low level drivers (motors, sensors, etc) are being tested prior to be integrated into any component **(NO ROS2)**
- Using Pytest (Python) / Catch2 (C++)

Component

- Testing where **integrated subcomponents** are tested as an individual component that will interface to other components **(ROS2)**
- Using Pytest, colcon test, launch_pytest (ROS2)

Subsystem

- Testing where components are **specifically integrated with other components** as a subsystem

Robot System

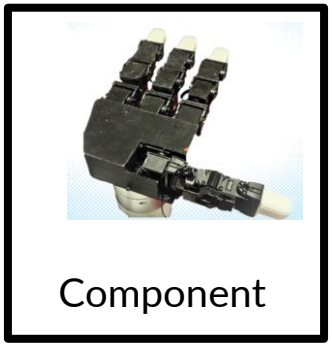
- Testing where specific interactions of the system are tested when **integrated in an entire robotic system**

Full System

- Testing where specific interactions of the full system (**which includes external human interfacing systems**)

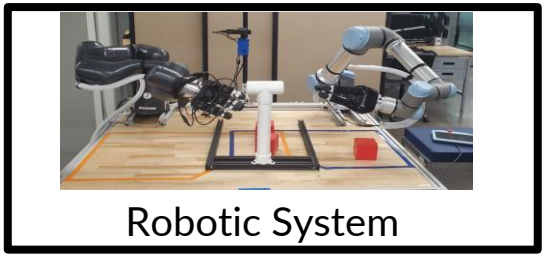
Test Examples

Test dynamixel PID gains are set properly

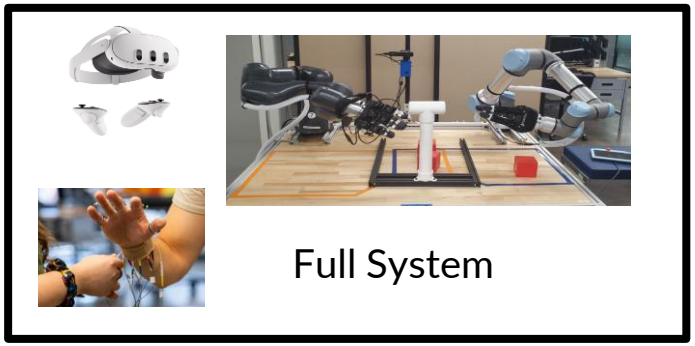


Test ROS2 Leap Node publishes / subscribes correctly

Test tactile sensor output when arm + hand is moving to pick up

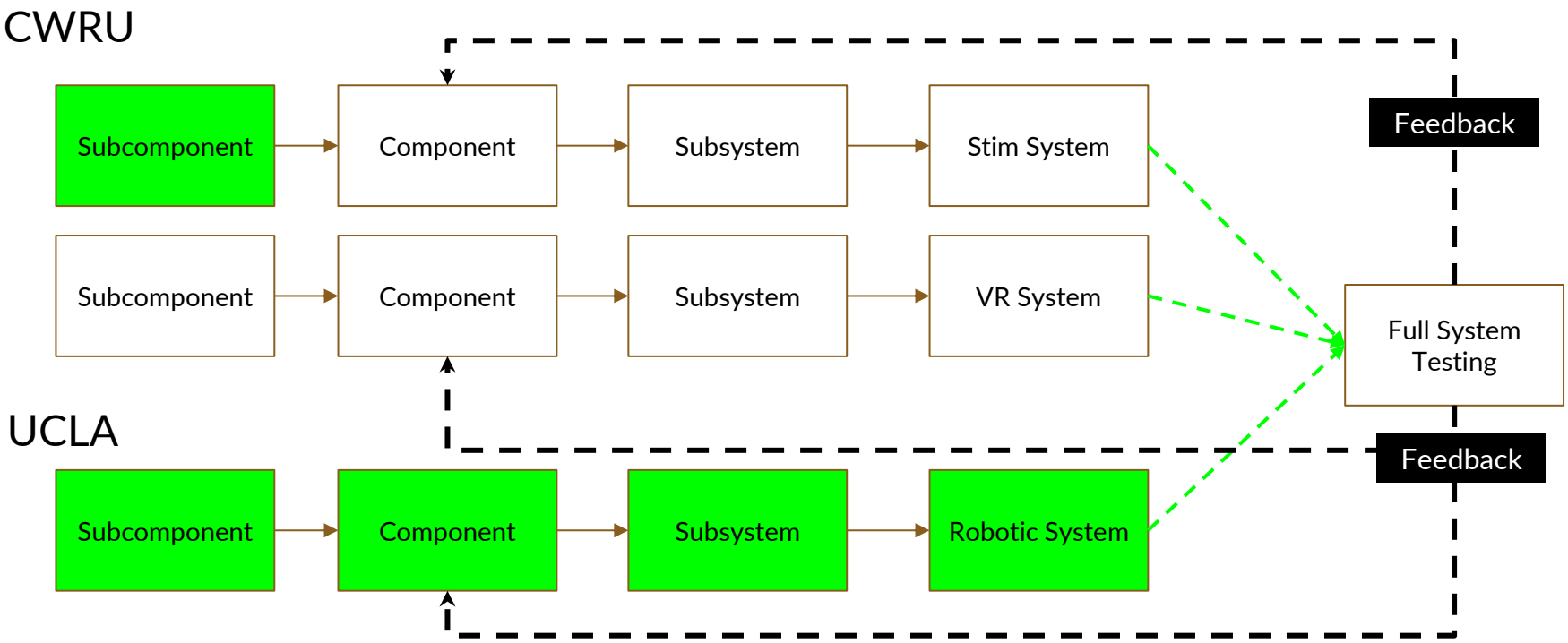


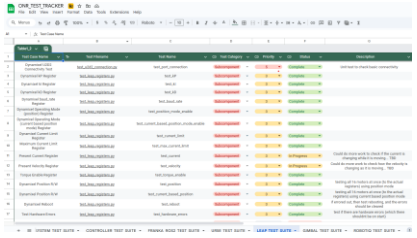
Test handover task between robot arms with tactile feedback



Test VR + haptic feedback while picking up blocks of different stiffnesses

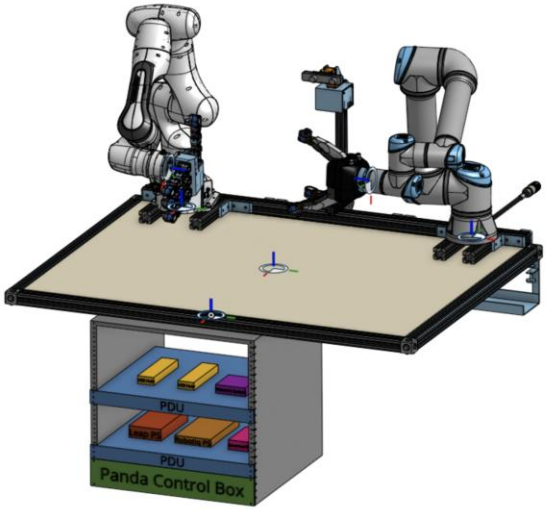
Testing Progression

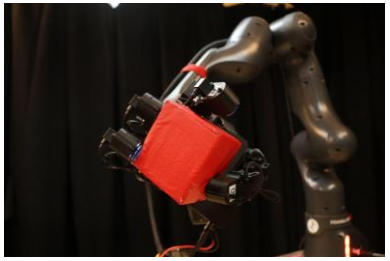


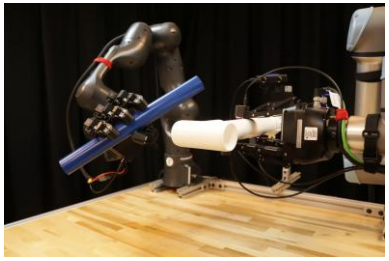


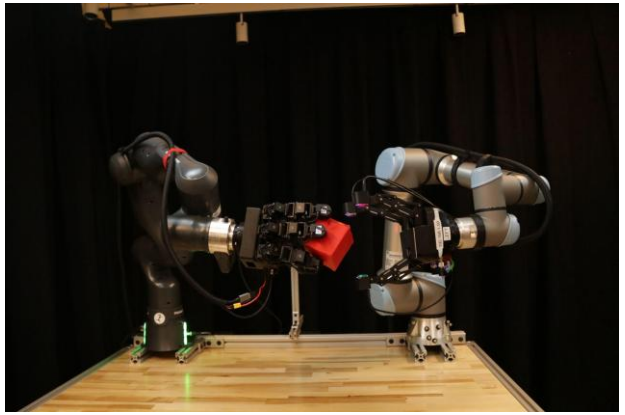
Over 200+ Unit Tests

Photoshoot

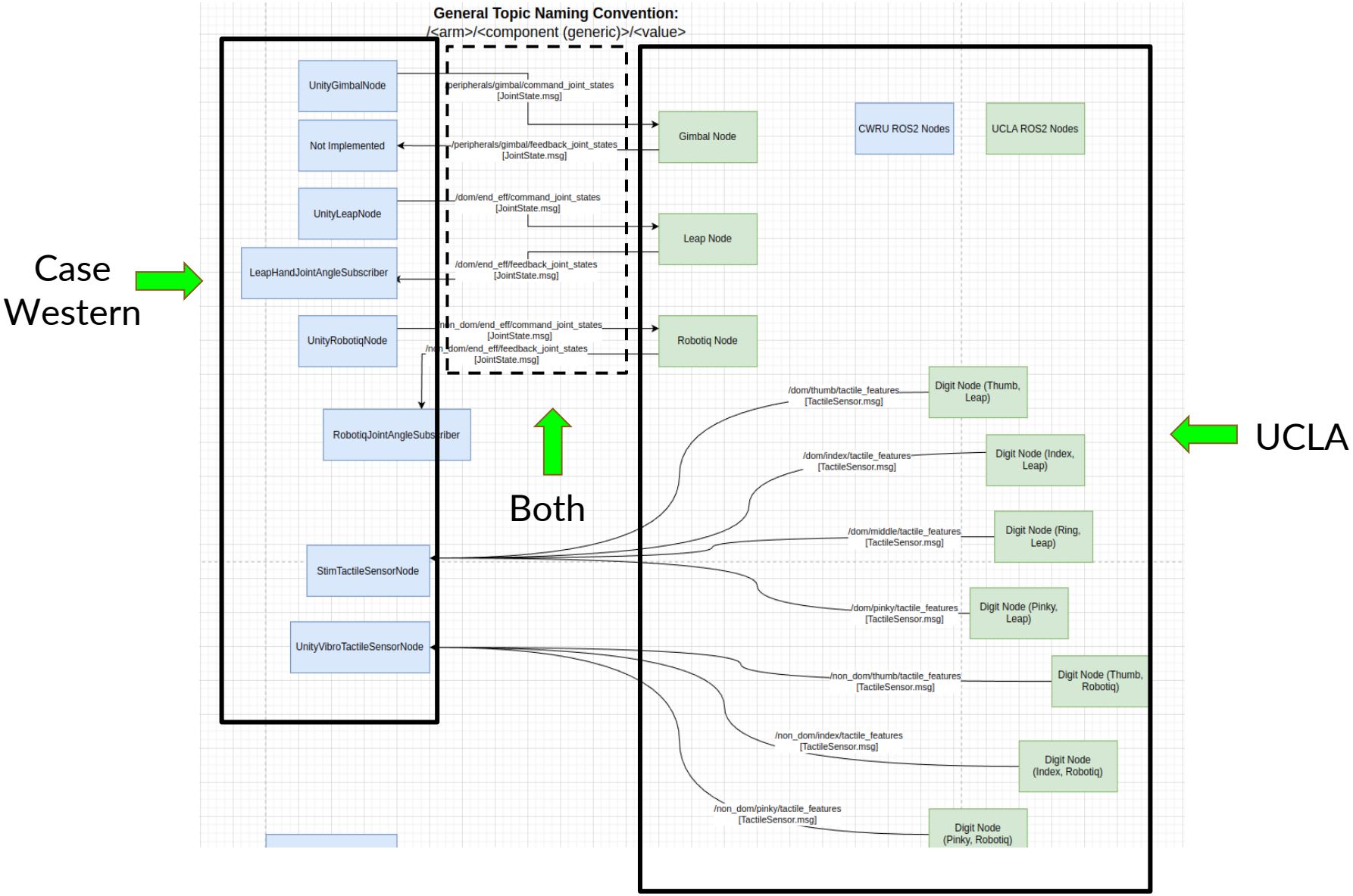








System Interface Control Document



Project Releases

ONR Robotic System Release

File Edit View Insert Format Data Tools Extensions Help

Menus 100% $ % .0 .00 123 Defaul...

E14

	A	B	C
1	**Release Specifications**		
2	**System Release**	**v1.0**	
3	**Description**	Initial system release of ONR Robotic system which includes improved drivers for main repositories apart from controller, GUI and unit tests for key components. Includes full system CAD and BoM with major hardware improvements for cabling, LEAP, Robotiq.	
4	**Overall System CAD**	RELEASE A	LINK
5	**ONR System Code**	onr-demo-v3.0.0	LINK
6	**System Interface Document**	v2.0	LINK
7	**Release Date**	5/16/2025	
8	**Signed By**	Stephen Mock	smock@ucla.edu
9		Ben Forbes	benforbes@ucla.edu
10			
11	**Manufacturing**		
12	**Type**	**Specifications**	
13	**3D Printing**	20% Infill (unless specified in technical document), tree supports, 0.4mm nozzle, 0.16mm layer height	
14			
15	**Unit Tests @ UCLA**		
16	**Repository Name**	**Success**	**Date Passed**
17	digit_ros2	84/84	05/16/25
18	gimbal_ros2	30/30	05/16/25
19	leap_ros2	41/41	05/16/25
20	robotiq_ros2	23/23	05/16/25

onr-demo-v4.0.0 Latest

sjmock99 released this 5 days ago onr-demo-v4.... 988a890

Initial release of the ONR Robotic software that is linked to a [SYSTEM RELEASE: 2.0] SDIB5_fc7XnK-VJOc/edit?gid=895105116#gid=895105116)

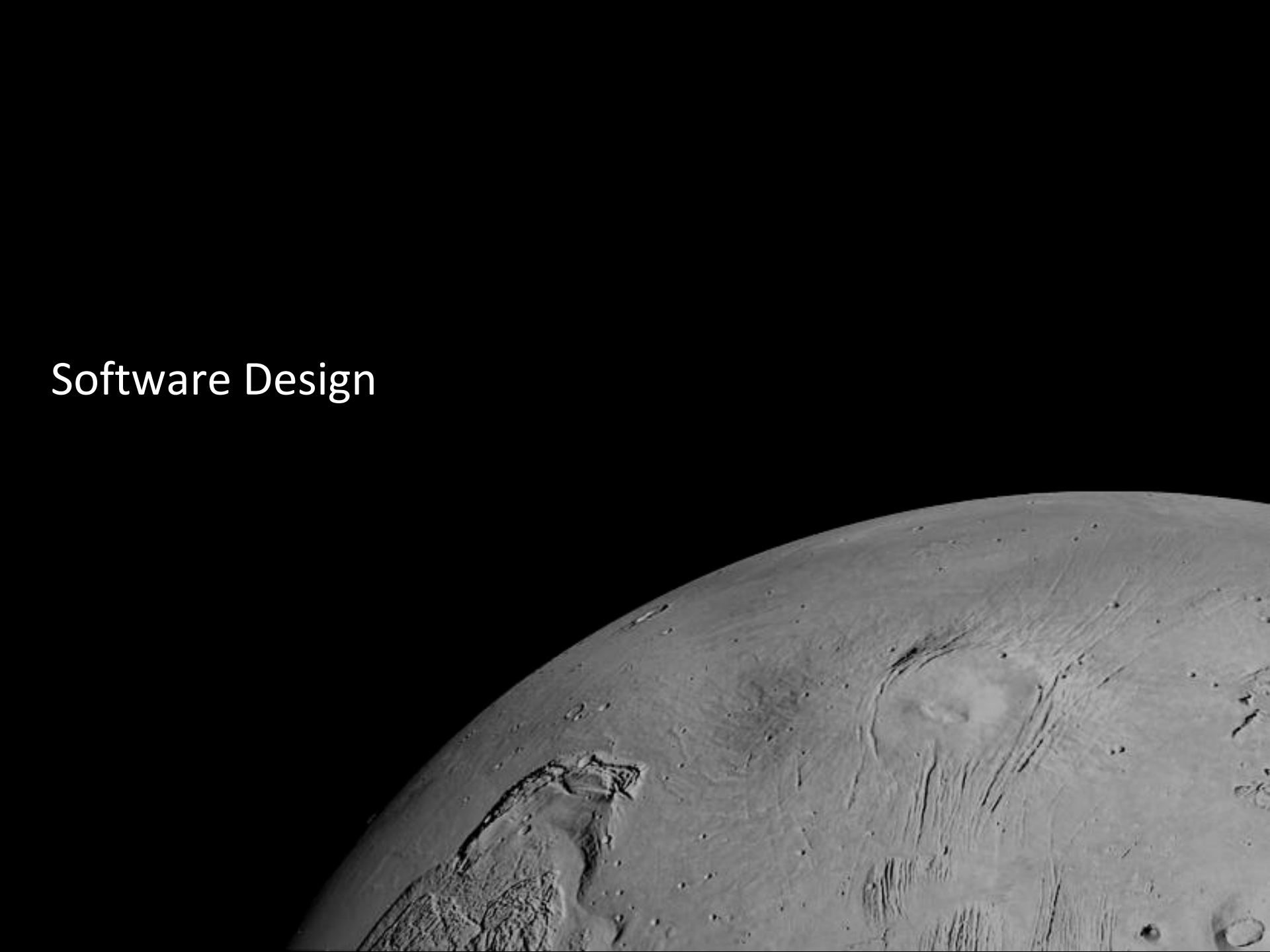
This release is for the ONR August 2025 Summer Meeting.

Review Release: Release A - Full Robotic System

Search	Revision	State	Part number
ONR Full Robotic Syst... Assembly	A	Released	ASM-403
Franka_Research_... Assembly	A	Released	ASM-272
Gimbal Assembly Assembly	A	Released	ASM-119
e-Series_UR5e Assembly	B	Released	ASM-352
table Assembly	A	Released	ASM-391
leap_hand_assm Assembly	C	Released	ASM-027
Assembly 1 Assembly	A	Released	ASM-379
Robotiq 3 Finger Gr... Assembly	A	Released	ASM-181

	SYSTEM BOM						
22							
23	Part						
24	Leap Hand + more						
25	Robotiq + more						
26	Server Rack + more						
27	Gimbal + more						
28	Table Hardware	System	Assembled	A		1	LINK to BOM
29	Franka + more	Franka	Received	A		1	LINK to BOM
30	UR5e + more	UR5e	Received	B		1	LINK to BOM
31	Cable Management Organizer	Cable Management	Received		Home Rolling Cable Management Cord Power Strip/Surge Protector Storage Organizer	1	LINK
32	Cable Management Tray	Cable Management	Received		Metal Cable Management Tray Under Desk with Clamp	3	LINK
33	Cable Management Spacers	Cable Management	Ready to Order	A		6	LINK
34	USB A Extension, M-F (16ft)	Cable Management	Received		USB A extension, 3.0, 16feet	2	LINK
35	Cable Sleeve (1/2in, 25ft)	Cable Management	Received		Cable Sleeve (1/2in, 25ft)	2	LINK
36	Velcro Cable Ties	Cable Management	Received			100	LINK
37	MicroUSB Cable (3 ft)	Gimbal, Cable Management, Server Rack	Received		MicroUSB (3 feet) for Gimbal U2D2	1	LINK
38	MicroUSB Cable (16 ft)	Leap, Cable Management	Received		MicroUSB (16 feet) for Leap Hand U2D2	1	LINK

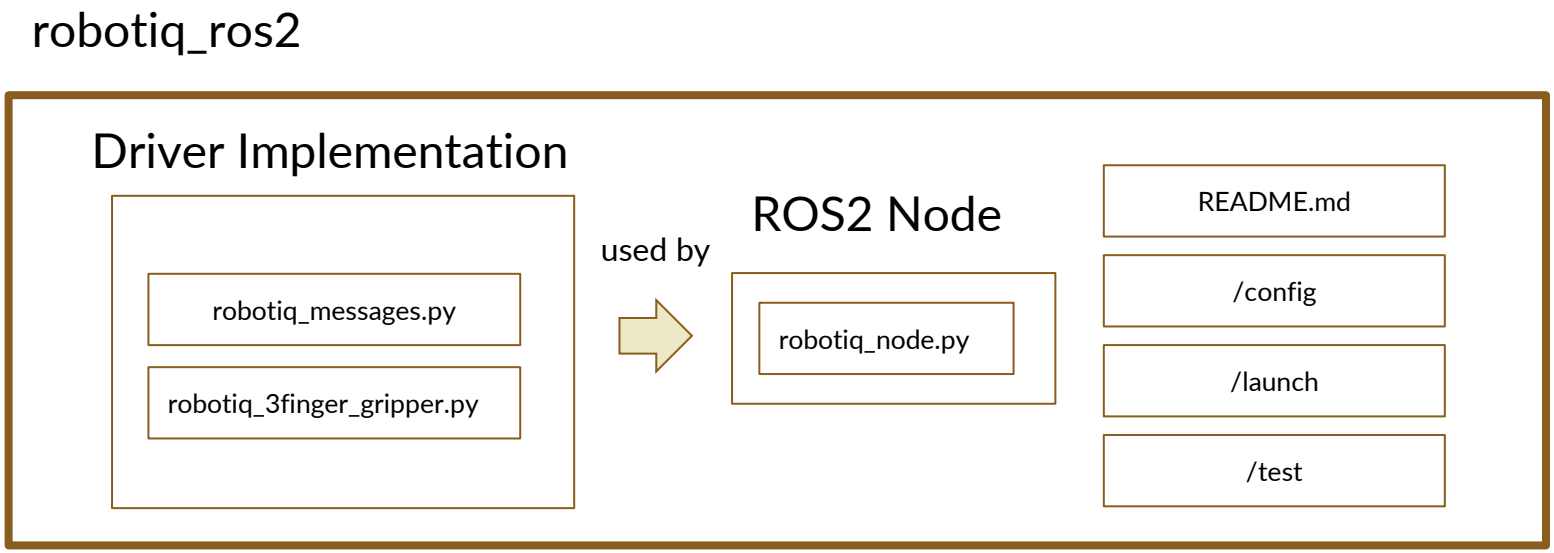
Software Design



Software Robustness

- **Every** repository used uses proper git architecture (main, develop, feature) and pull request rules
  - Every repository must have a working READMe
  - **Each ROS2 package** has a driver and also a ROS node that uses the driver
  - **Drivers** should use correct encapsulation techniques for their respective language
- General code and file structure cleanup (CMake, package.xml, launch/config, naming, etc)
  - Refactoring
  - Bugs, Incorrect implementation
  - Feedback from Unit Testing
- All functions should have proper declarations and typing
- Docstrings / Doxygen Strings / Comments are required

High Level Software Architecture Example



Unit Testing Objective

- **Unit testing** is a software development process that involves testing individual parts of a software application
  - Our application is a little different because we are also using hardware in the loop (HIL)
  - This means that both hardware and software changes can result in a unit test failing
- **Functionally checking out** and diagnosing components / subcomponents prior to full system integration
  - How do we know that our robotic system is going to the right positions, sensor readings, etc?
  - How do we know our implementation is even correct?
  - If changes occur to the hardware or software, do the components still meet their function?
  - Diagnostic tool when integrating with CWRU

Unit Testing Definition

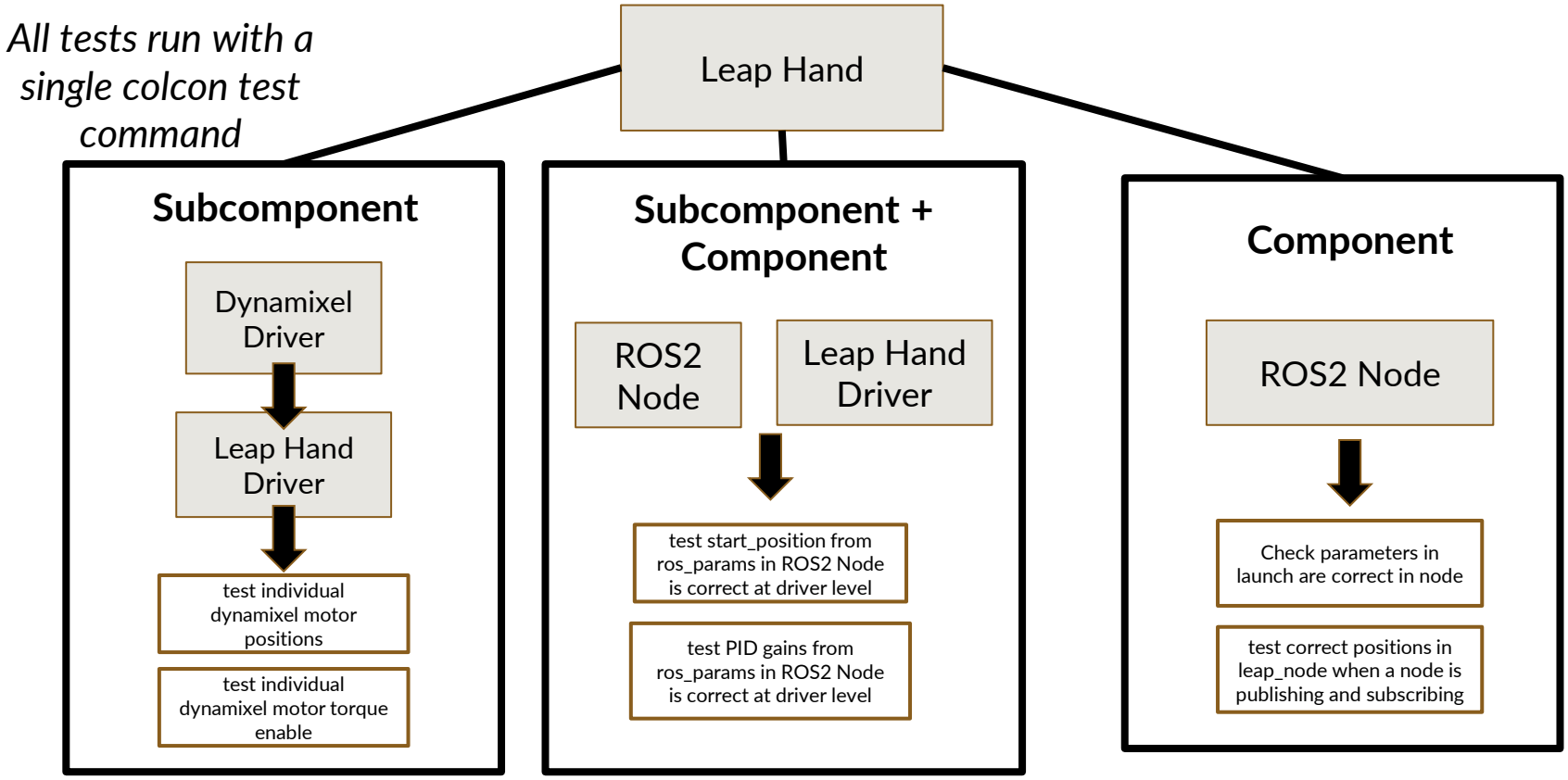
Definitions:

- **Subcomponent Level:** low level drivers (motors, sensors, etc) **[NO ROS2]**
  - **Ex:** a single dynamixel motor
- **Component Level:** integrated **subcomponents** tested as an individual **component [ROS2]**
  - **Ex:** a Leap Hand w/ ROS2 Node
- **Subsystem**: components are specifically integrated with other **components** as a **subsystem**
  - **Ex:** a Leap Hand on a UR5e Arm
- **System**: Integration of **all subsystems**

Framework:

- **Pytest** (Python), **Catch2** (C++)**, ROS Integration** (pytest, launch_pytest)
- **Colcon Test** to run all tests at once

Leap Hand Unit Testing Example

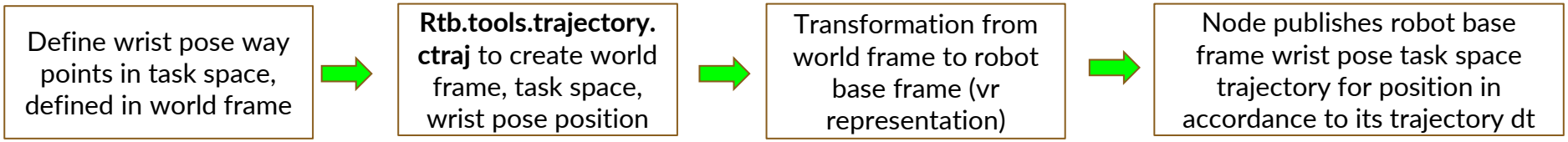


Colcon Test Process

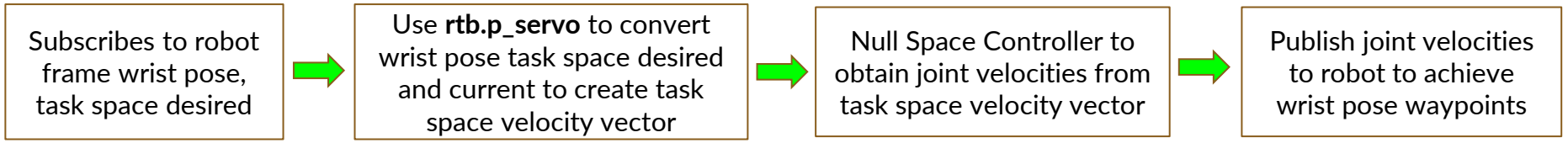
1. Remove build, install folders (since old test data can often go in here and conflict with test results)
   - `rm -rf /build /install`
2. Build workspace
   - `colcon build`
3. Run individual unit tests (if needed)
   - For launch based unit tests!
     i. `launch_test /path/to/test.py`
   - For strictly unittest
     i. `python-3 /path/to/test.py`
4. Run test suite
   - `colcon test --event-handlers console_direct+ --packages-select <package_name>`
5. Process Tests
   - `colcon test-result --all`
6. Review Results
   - `xunit-viewer -r build/<package_name>/test_results -c`

Null Space Controller Development

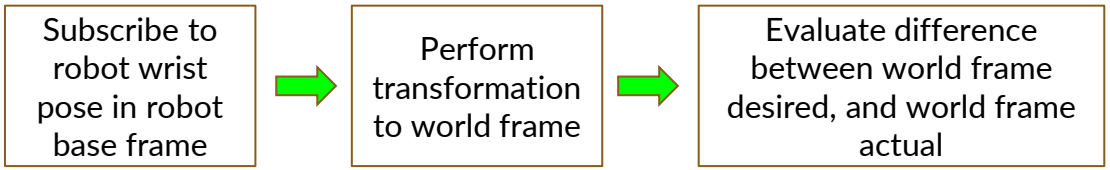
Trajectory Creation Node



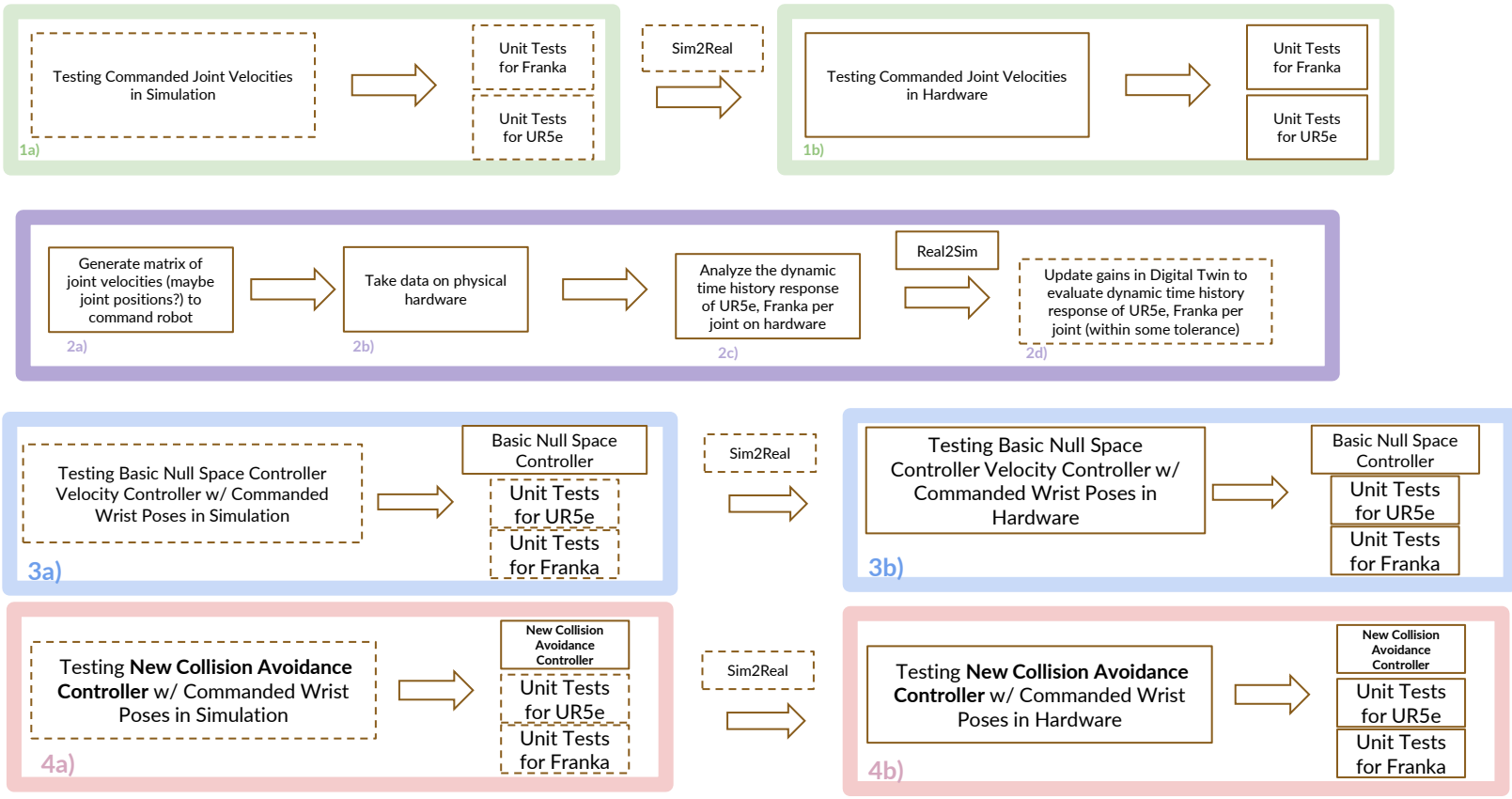
Controller



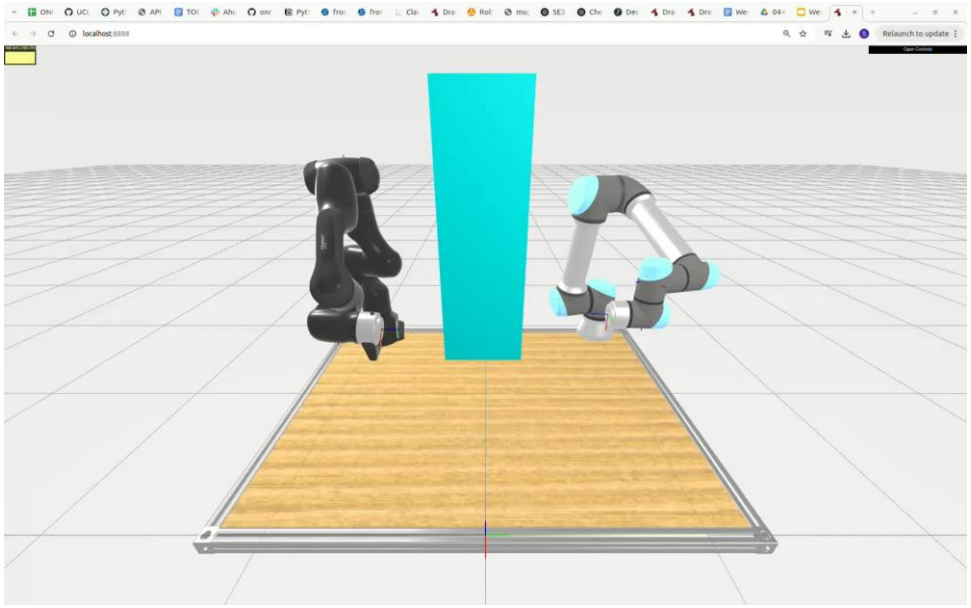
Trajectory Creation Node (Validation)

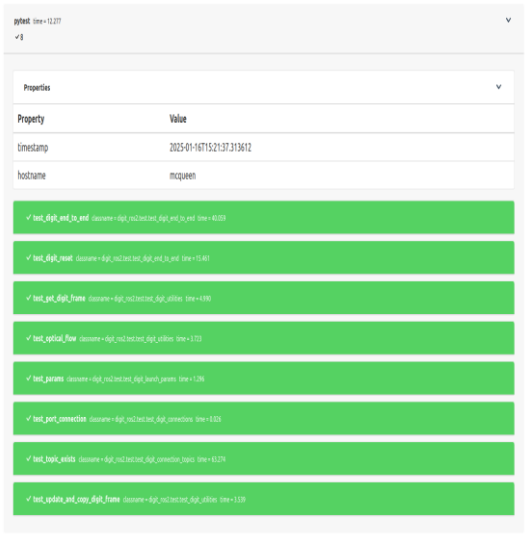


Digital Twin + Controller Verification Plan



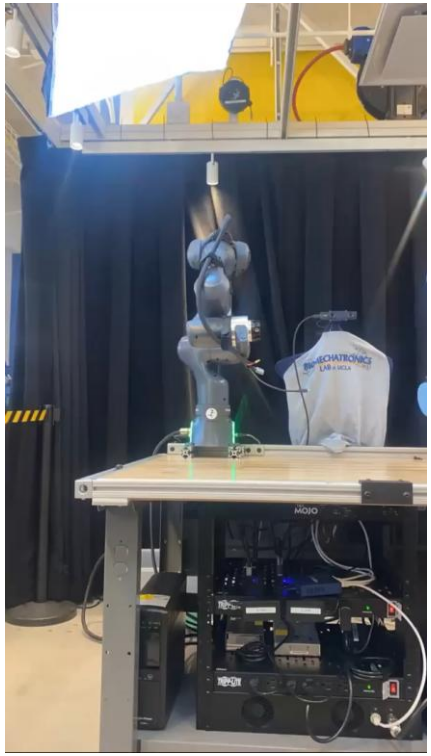
Digital Twin + Controller Unit Tests / Validation

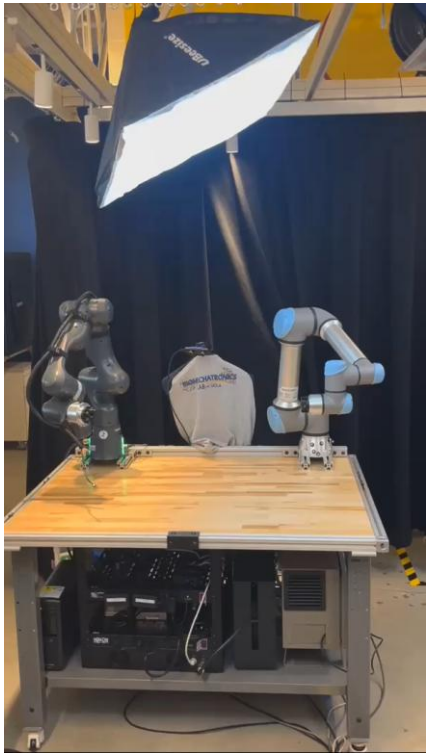




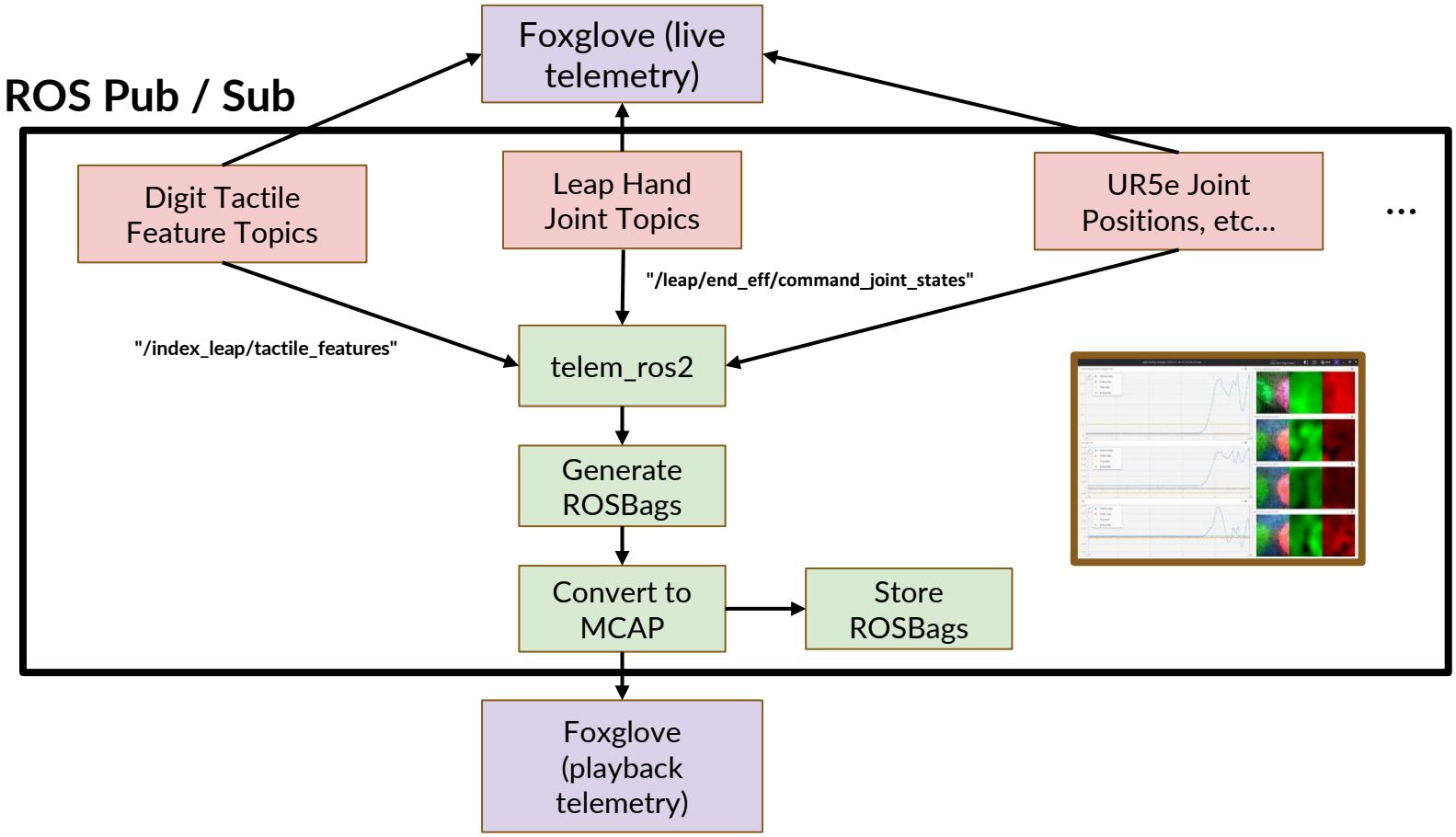
Automated test reporting, ROS2 Hardware in the Loop Unit Tests

Hardware in the Loop Unit Test Demonstration

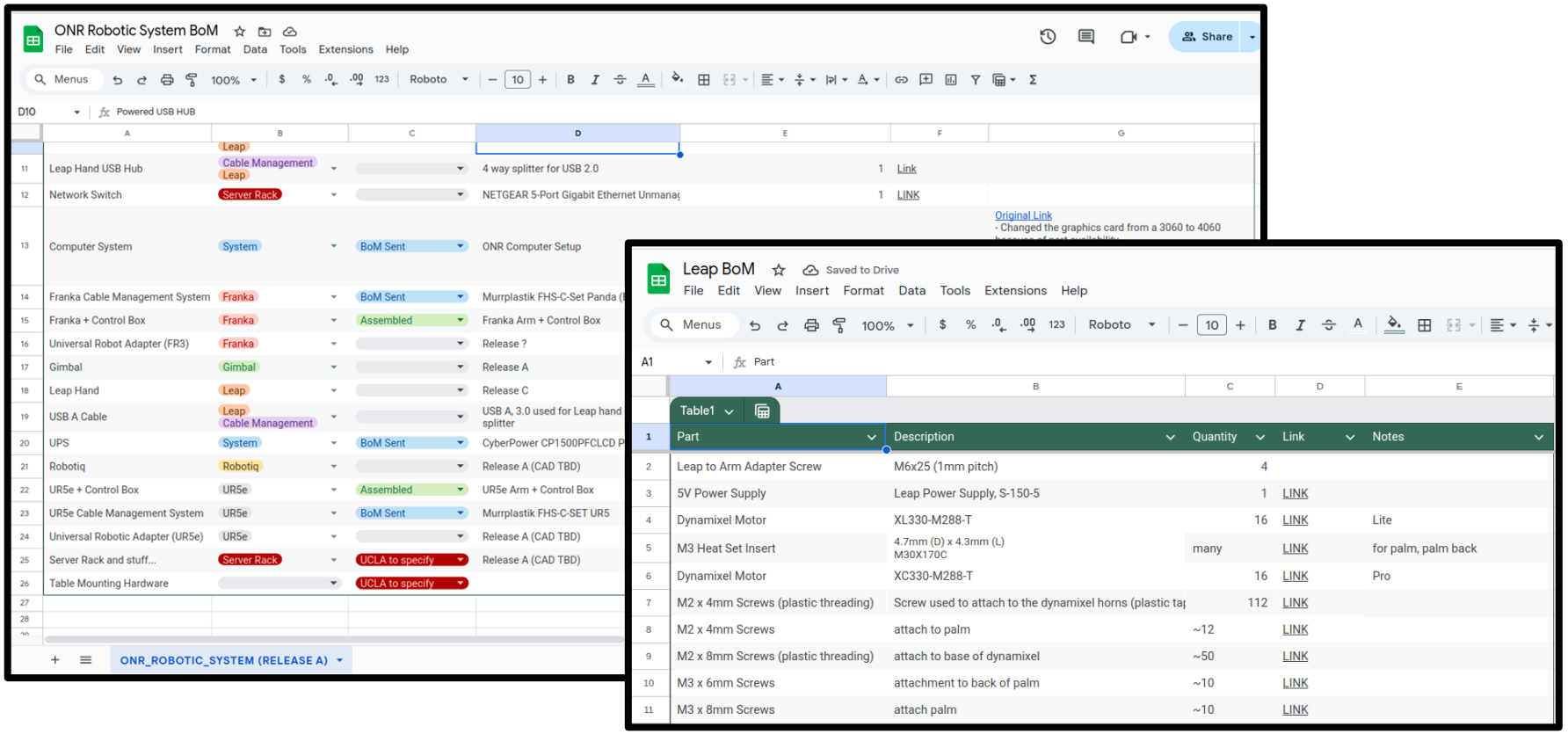




Data Visualization + Telemetry



Bill of Materials



ONR Robotic System BoM

11	Leap Hand USB Hub	Leap, Cable Management, Leap		4 way splitter for USB 2.0	1	Link
12	Network Switch	Server Rack		NETGEAR 5-Port Gigabit Ethernet Unmanag	1	LINK
13	Computer System	System	BoM Sent	ONR Computer Setup		Original Link - Changed the graphics card from a 3060 to 4060
14	Franka Cable Management System	Franka	BoM Sent	Murrplastik FHS-C-Set Panda (		
15	Franka + Control Box	Franka	Assembled	Franka Arm + Control Box		
16	Universal Robot Adapter (FR3)	Franka		Release ?		
17	Gimbal	Gimbal		Release A		
18	Leap Hand	Leap		Release C		
19	USB A Cable	Leap, Cable Management		USB A, 3.0 used for Leap hand splitter		
20	UPS	System	BoM Sent	CyberPower CP1500PFCLCD P		
21	Robotiq	Robotiq		Release A (CAD TBD)		
22	UR5e + Control Box	UR5e	Assembled	UR5e Arm + Control Box		
23	UR5e Cable Management System	UR5e	BoM Sent	Murrplastik FHS-C-SET UR5		
24	Universal Robotic Adapter (UR5e)	UR5e		Release A (CAD TBD)		
25	Server Rack and stuff...	Server Rack	UCLA to specify	Release A (CAD TBD)		
26	Table Mounting Hardware		UCLA to specify			

ONR_ROBOTIC_SYSTEM (RELEASE A)

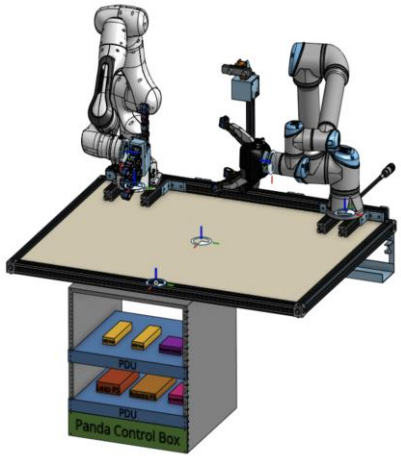
Leap BoM

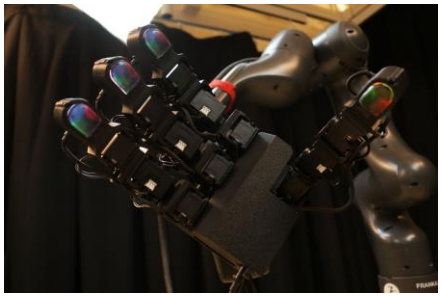
Saved to Drive

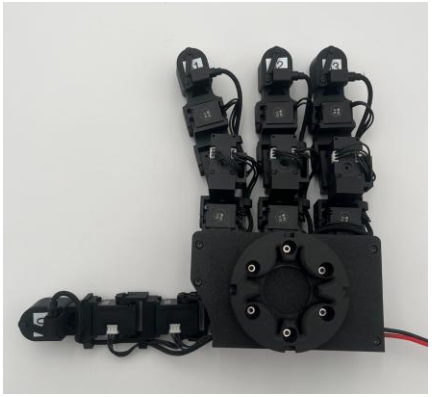
	Part	Description	Quantity	Link	Notes
2	Leap to Arm Adapter Screw	M6x25 (1mm pitch)	4		
3	5V Power Supply	Leap Power Supply, S-150-5	1	LINK	
4	Dynamixel Motor	XL330-M288-T	16	LINK	Lite
5	M3 Heat Set Insert	4.7mm (D) x 4.3mm (L) M30X170C	many	LINK	for palm, palm back
6	Dynamixel Motor	XC330-M288-T	16	LINK	Pro
7	M2 x 4mm Screws (plastic threading)	Screw used to attach to the dynamixel horns (plastic ta	112	LINK	
8	M2 x 4mm Screws	attach to palm	~12	LINK	
9	M2 x 8mm Screws (plastic threading)	attach to base of dynamixel	~50	LINK	
10	M3 x 6mm Screws	attachment to back of palm	~10	LINK	
11	M3 x 8mm Screws	attach palm	~10	LINK	

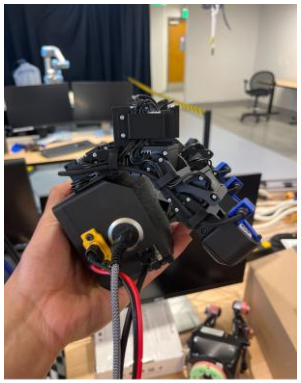
Hardware Design

Robotic System CAD

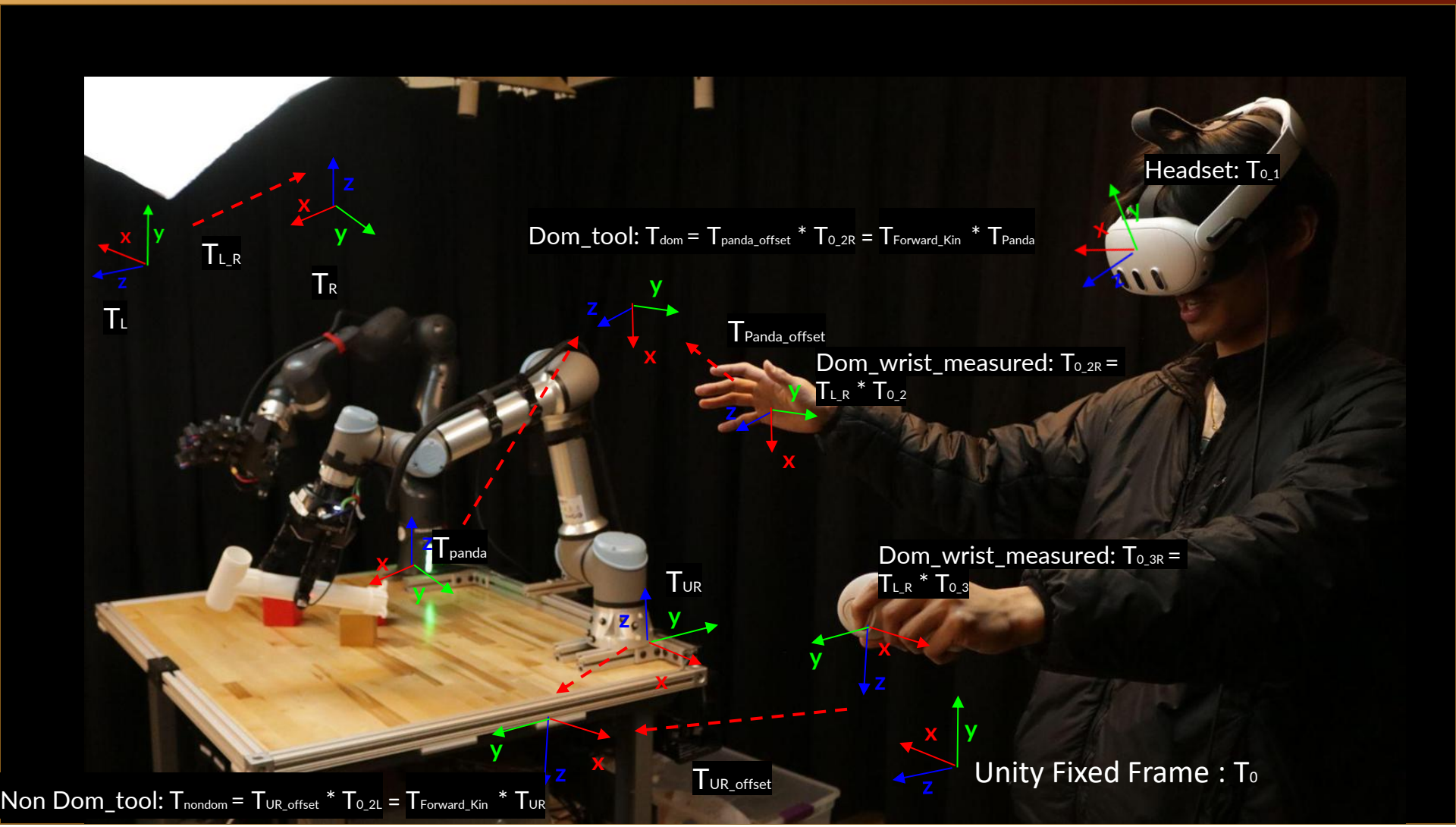




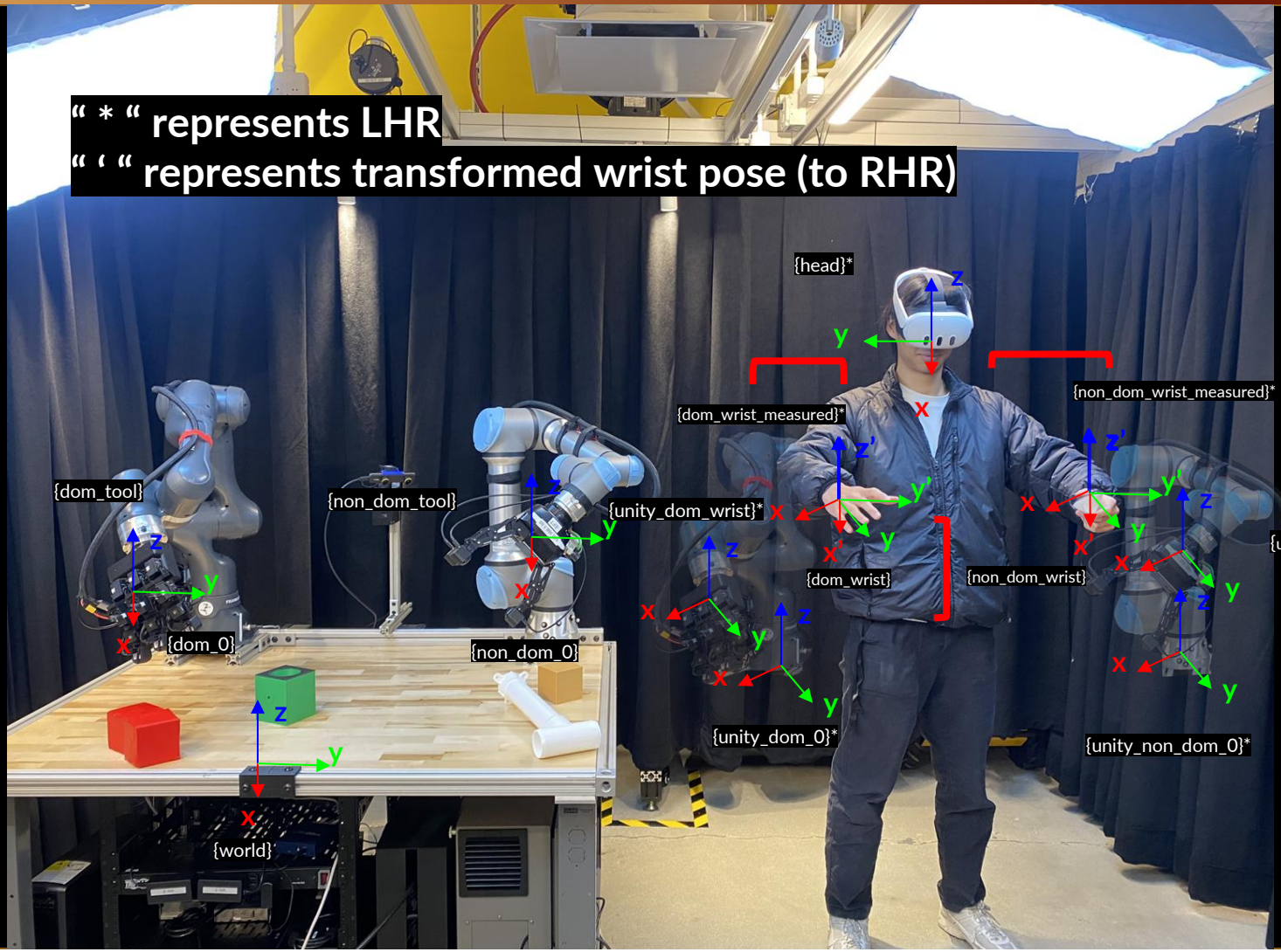




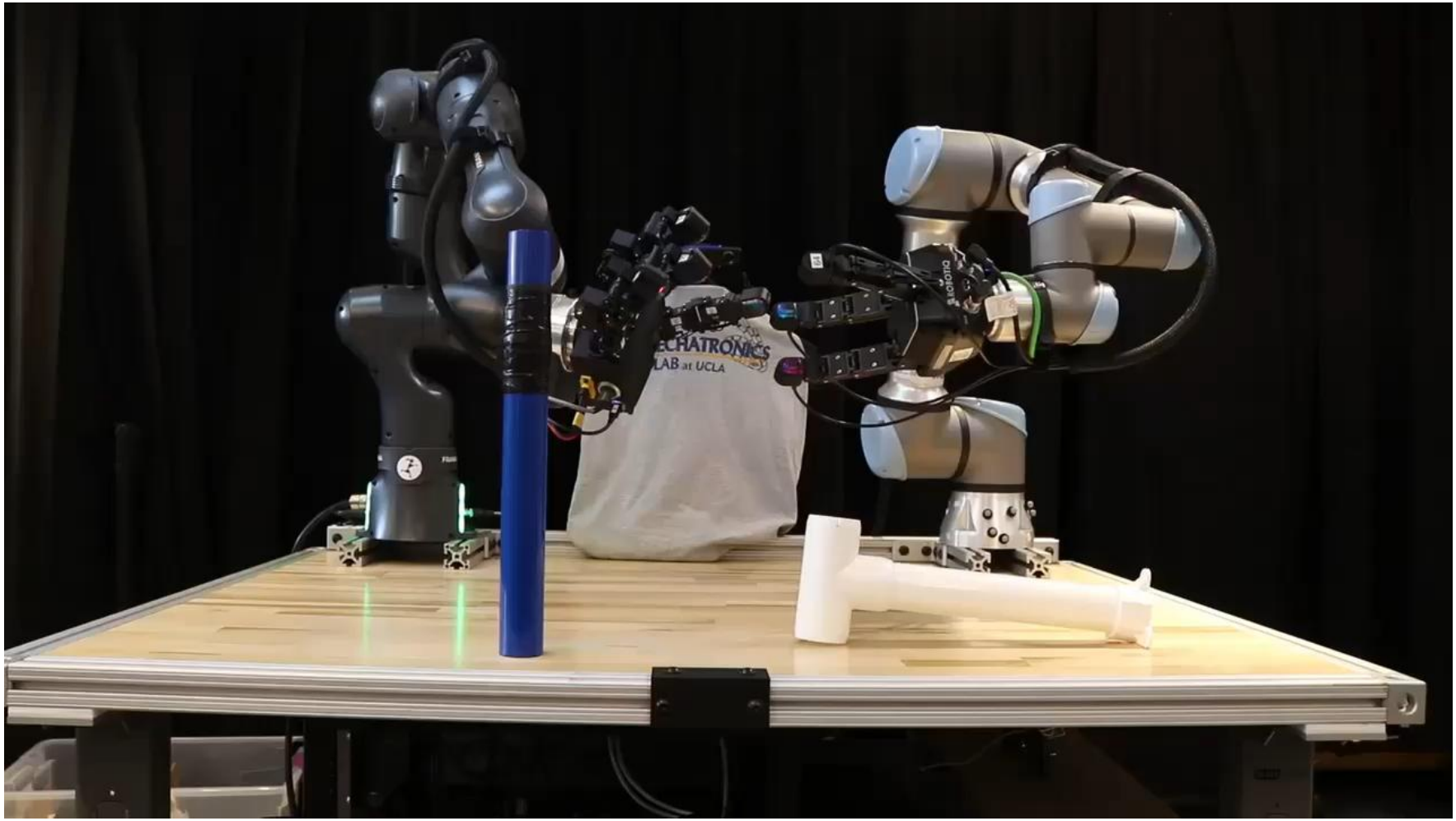
Robot Coordinate Frames



Robot Coordinate Frames



Other Videos / Media



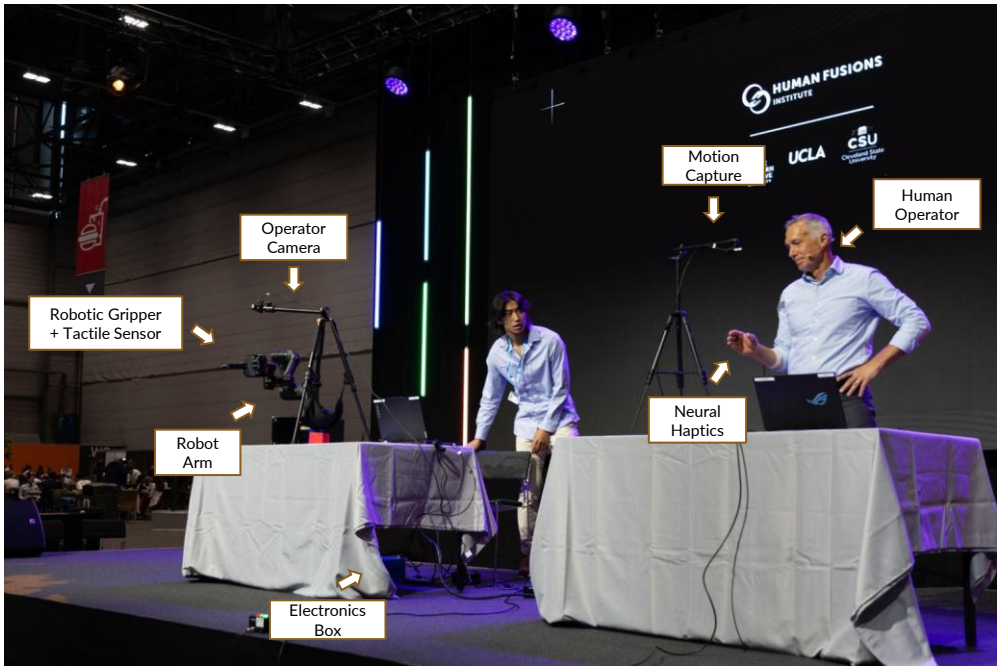
AI for Good Portable Demo

UCLA Biomechatronics Lab

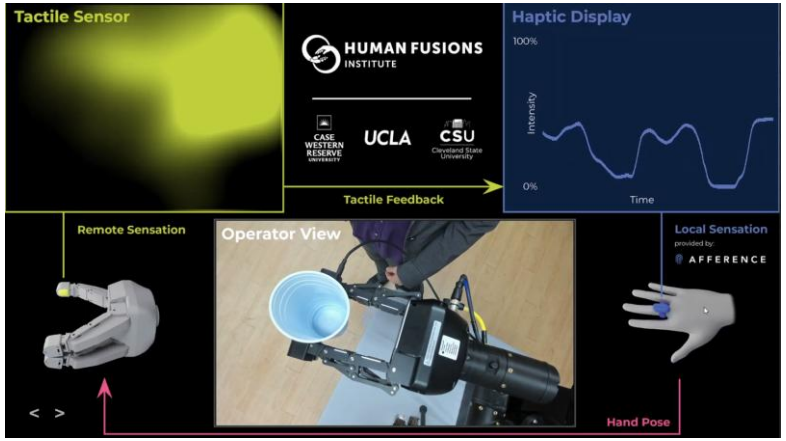
April 2025 - July 2025

AI for Good Portable Demo

System Layout



Visualization



HUMAN FUSIONS INSTITUTE

CASE WESTERN RESERVE UNIVERSITY

CSU Cleveland State University

Link

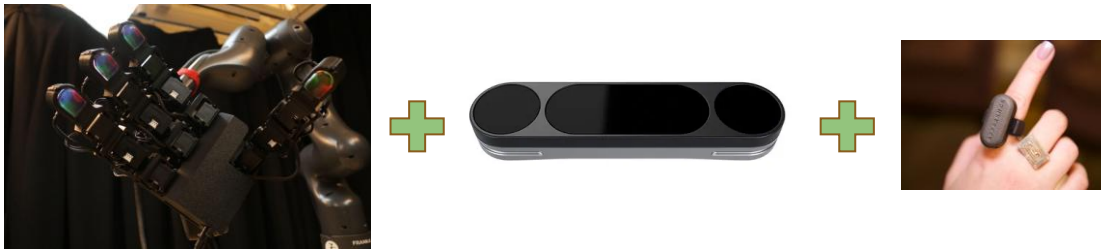
ONR / United Nations AI for Good Demo

High Level Objective:

1. To create a **highly portable, standalone** demo to demonstrate the capabilities of remote touch through a robotic interface with limited computational overhead and hardware.

Specifications:

1. Tactile Sensor(s)
2. Haptic Display(s)
3. Motion Capture(s)
4. Robotic End Effector(s)



More practically:

Tactile Sensor(s) / Robotic End Effectors(s)

- Robotiq / LEAP hands with a **DIGIT 1.5** on the index finger

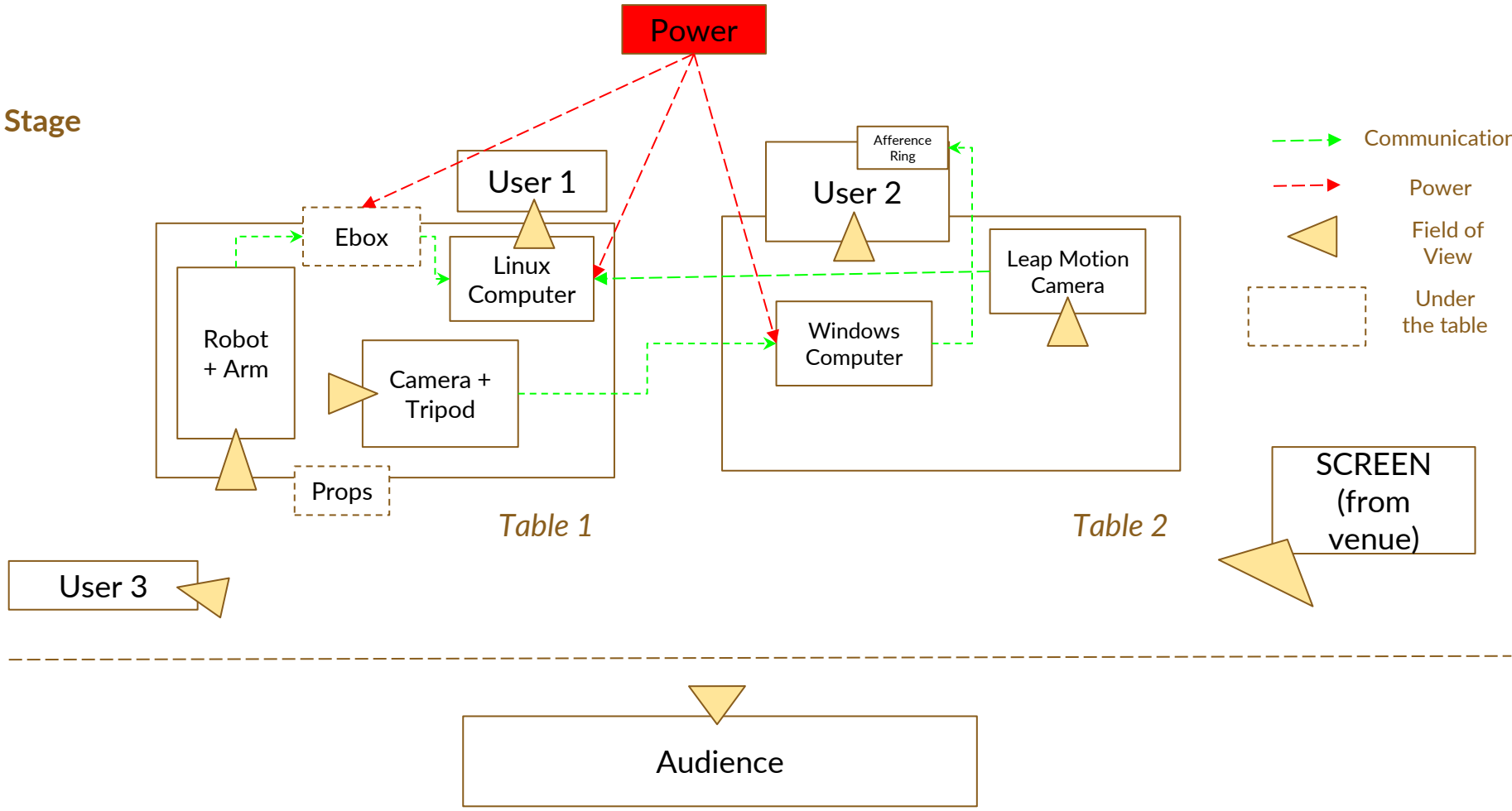
Haptic Display(s)

- A single Afference ring for the index finger per user
- Calibration program for the user prior to demo, based on CES 2025 protocol

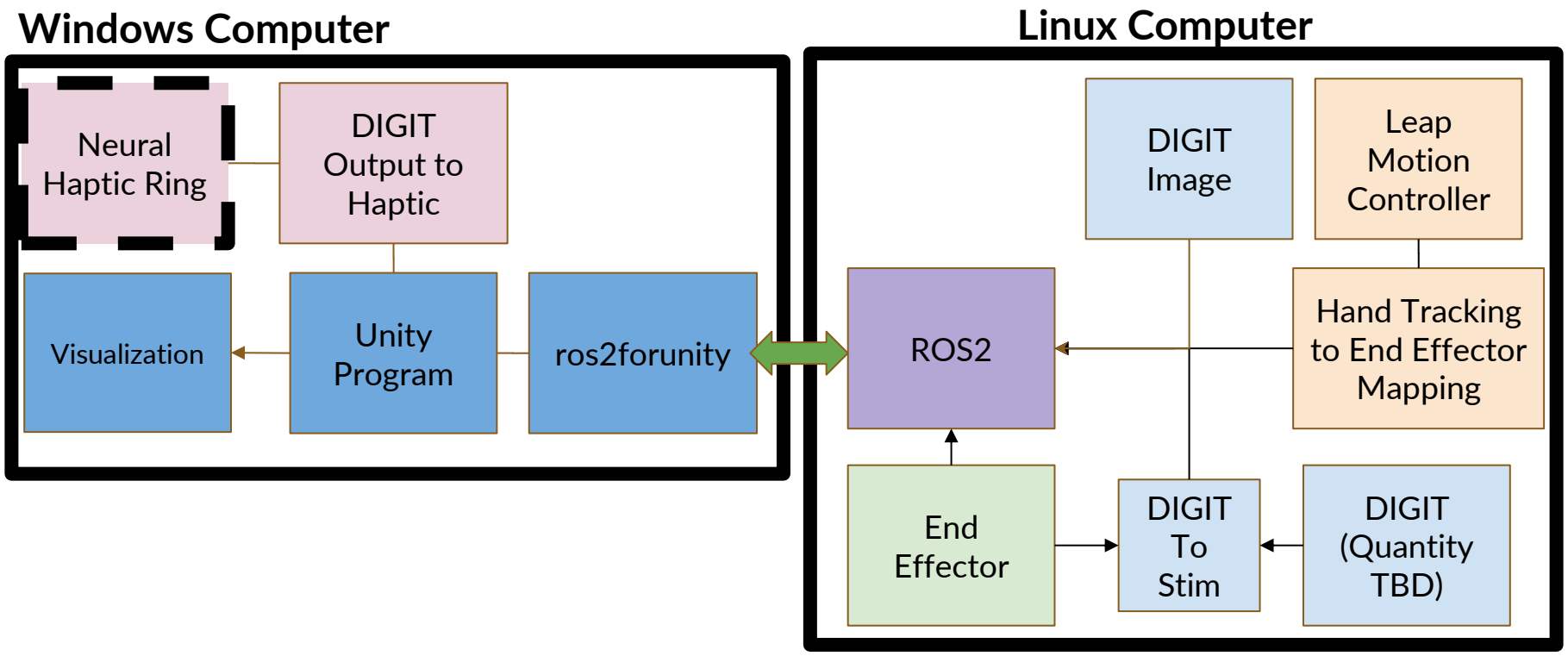
Motion Capture(s)

- Leap Motion Controller to detect hand pose
- Mapping between user hand pose and end effector pose

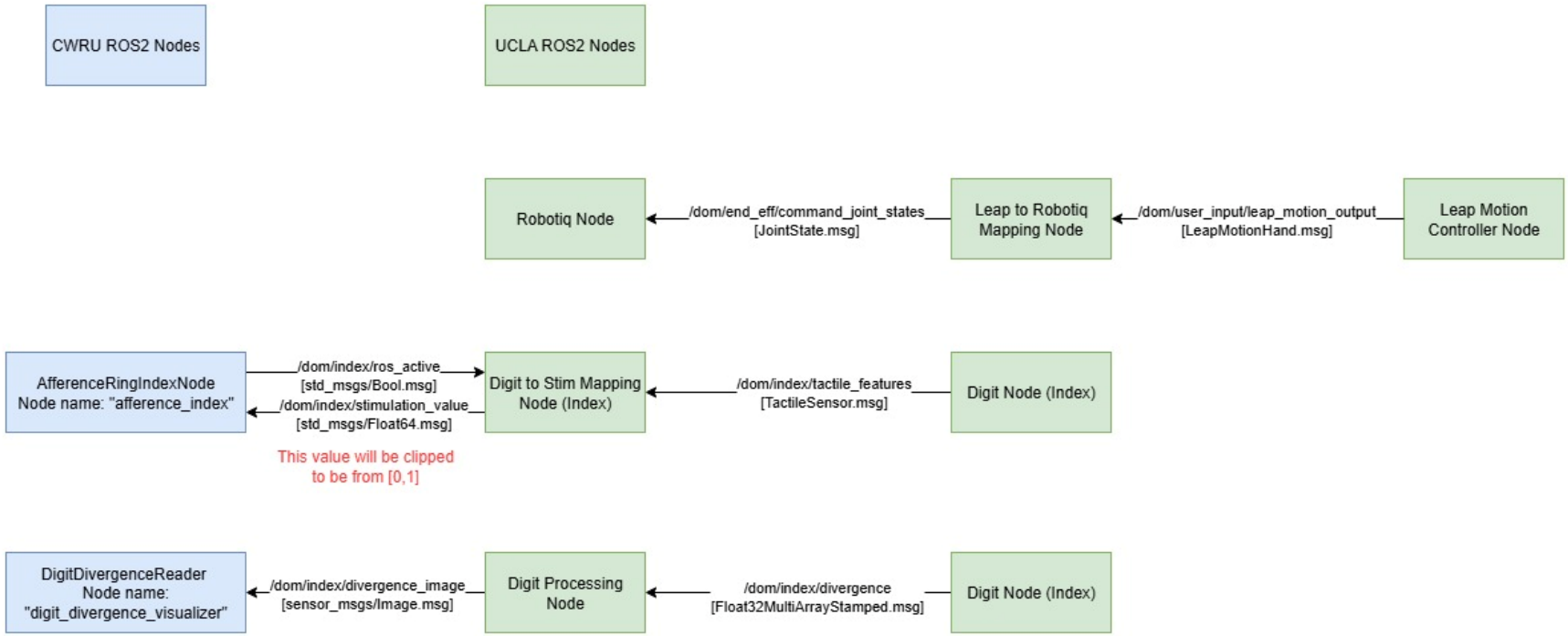
Demo Layout (v3)



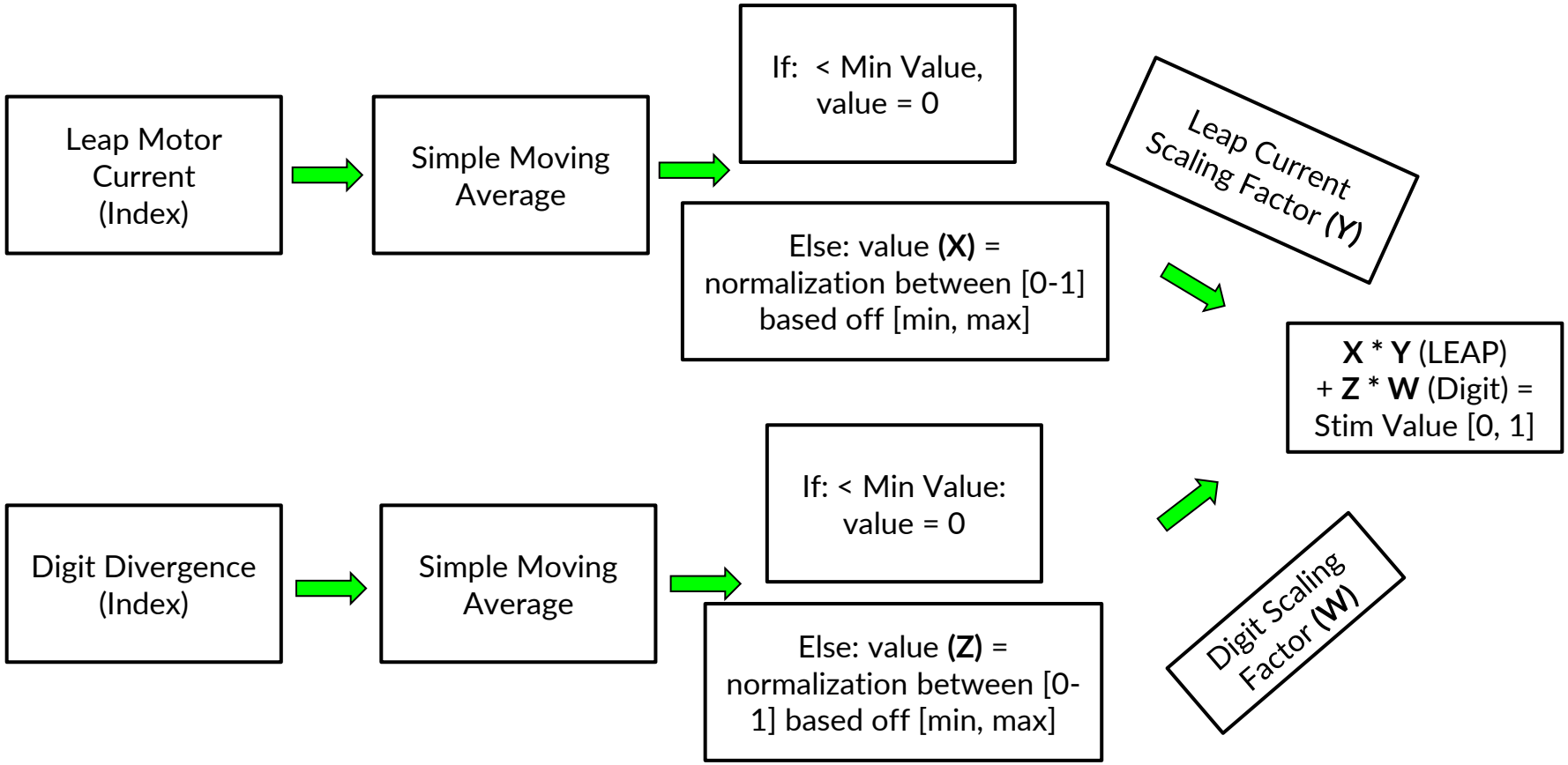
System Diagram



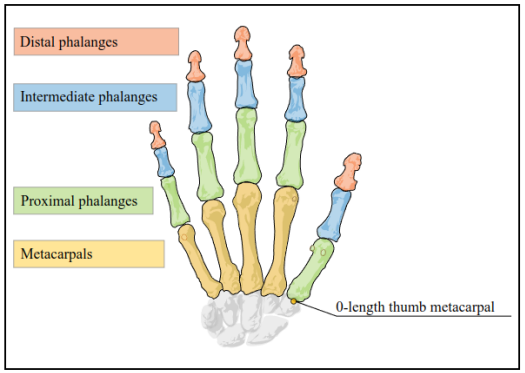
Portable Demo System Interface



Tactile Sensor to Stim Mapping



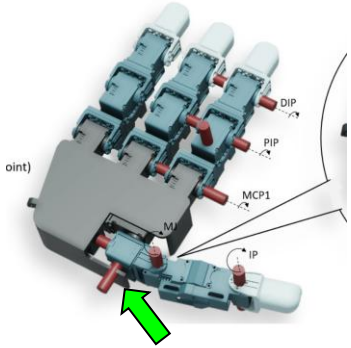
Leap to LEAP Hand Mapping



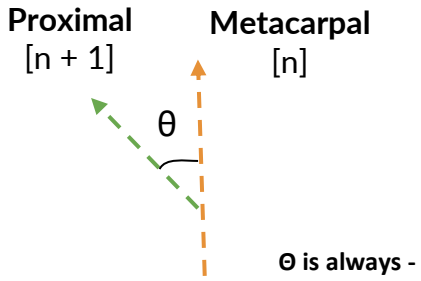
General Mapping Algorithm

Obtain orientation of [Joint n+1] and [Joint n] → Convert to Quaternions → R_[Joint n + 1]^-1 * R_[Joint n] → rel_angle = 2 * arccos(q_w) →

Take abs() * -1 because each successive joint is always negative of the previous joint → Multiple by scaling factor to map from human to LEAP geometry → Multiple by scaling factor to emphasize specific fingers → Remap to range where dynamixel can operate within ([-180° to 180°])



Thumb mapping is weird here...



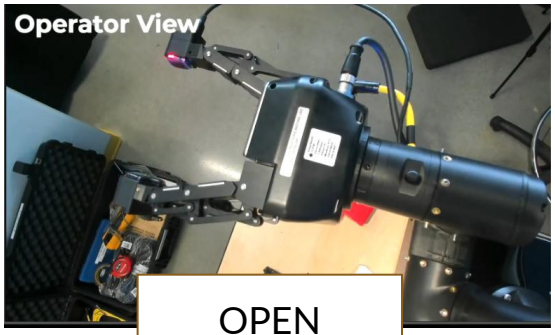
Θ is always -

leap_ros2 changes

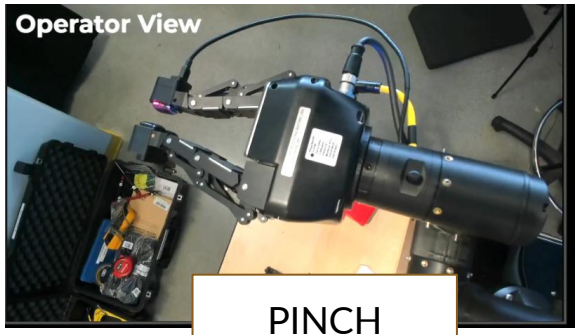
- Updated LEAP to use current based position mode
- Updated LEAP to recover if entering torque out condition

Hand Mapping Improvements

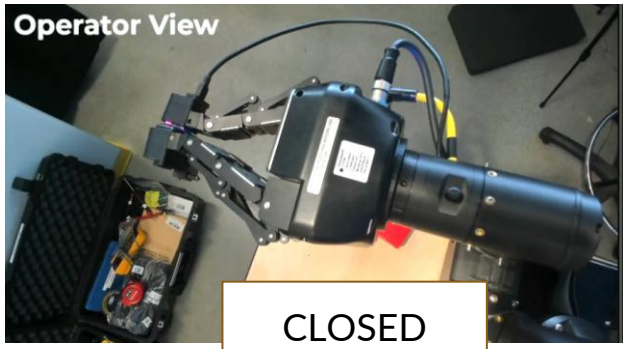
- Switch from LEAP Hand to Robotiq for simpler single trajectory mapping
- Chose three specific poses [Open, Pinch, Closed] ->
  - Associated 3 values with these positions that are used for calibration
  - Created points in between these trajectories to linear interpolate
- Created a region where the gripper moves slowly, and that corresponds to the value of the “pinch”



OPEN

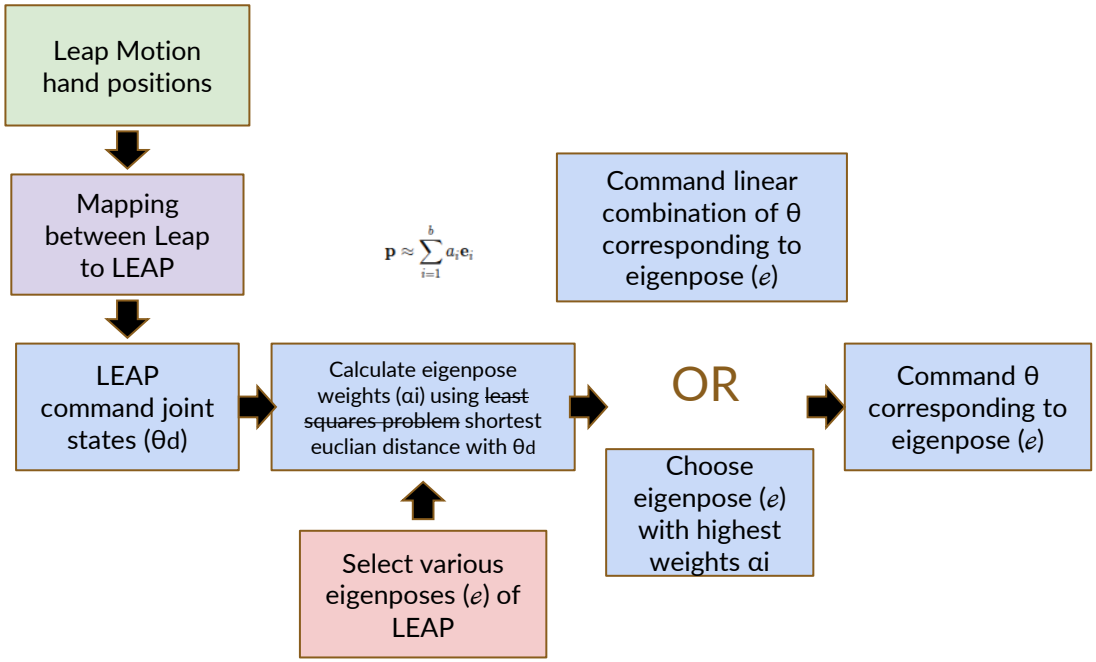


PINCH



CLOSED

Eigengrasp Approach



Example Workflow

If you **do center** the pose, the workflow looks like this:

1. **Center the pose:**

$$\mathbf{p}_{\text{centered}} = \mathbf{p}_{\text{actual}} - \bar{\mathbf{p}}$$

2. **Solve for amplitudes:**

$$\mathbf{a} = (\mathbf{E}^T\mathbf{E})^{-1}\mathbf{E}^T\mathbf{p}_{\text{centered}}$$

3. **Reconstruct pose (optional):**

$$\mathbf{p}_{\text{approx}} = \bar{\mathbf{p}} + \sum_{i=1}^{b} a_i \mathbf{e}_i$$

1. Use the Actual Pose Directly

Since you don't need to subtract a mean pose, you can work directly with the **actual pose** $\mathbf{p}_{\text{actual}}$.

- Let $\mathbf{p}_{\text{actual}}$ be your input pose vector, representing the joint angles of the hand in some configuration.
- Let $\mathbf{e}_1, \mathbf{e}_2, \ldots, \mathbf{e}_b$ be your **eigengrasps** (non-orthogonal vectors).

2. Set Up the Linear System

You want to represent $\mathbf{p}_{\text{actual}}$ as a **linear combination** of the eigengrasps:

$$\mathbf{p}_{\text{actual}} = a_1\mathbf{e}_1 + a_2\mathbf{e}_2 + \cdots + a_b\mathbf{e}_b$$

This can be written as a **matrix equation**:

$$\mathbf{E} \cdot \mathbf{a} = \mathbf{p}_{\text{actual}}$$

Where:

- $\mathbf{E}$ is a $d \times b$ matrix where each column $\mathbf{e}_i$ is an eigengrasp.
- $\mathbf{a} = [a_1, a_2, \ldots, a_b]^T$ is the vector of amplitudes, i.e., the **weights** you want to solve for.
- $\mathbf{p}_{\text{actual}}$ is the **actual hand pose** vector.

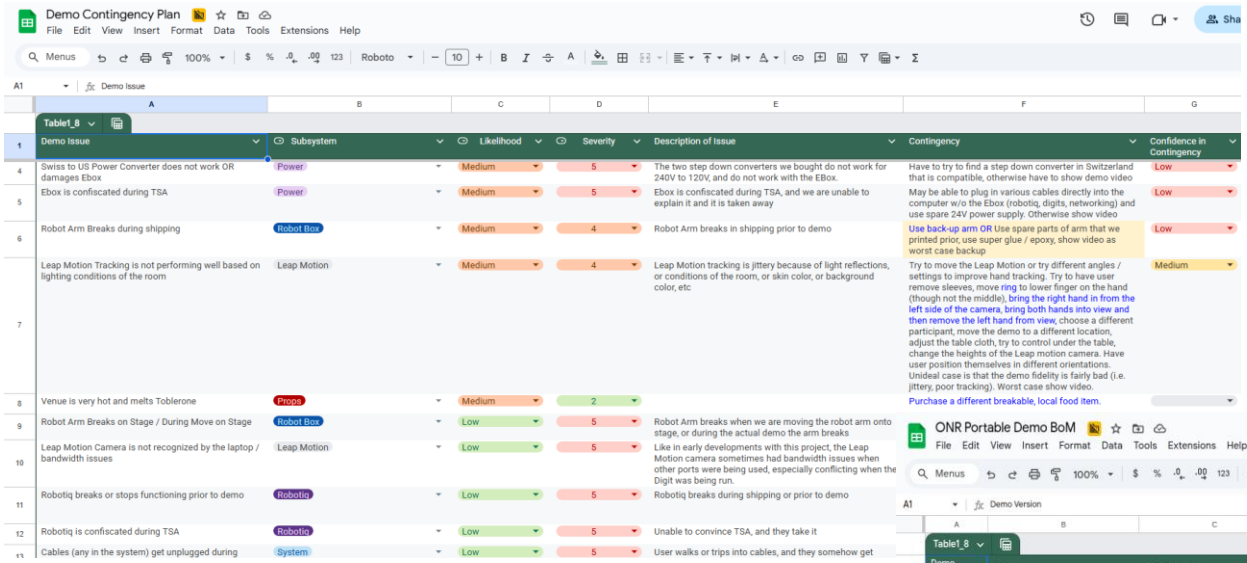
3. Solve for the Amplitudes

To solve for the **amplitudes** $\mathbf{a}$, you can use the **least-squares solution** since the system is usually overdetermined (i.e., you have more dimensions in $\mathbf{p}_{\text{actual}}$ than eigengrasps).

The solution is given by:

$$\mathbf{a} = (\mathbf{E}^T\mathbf{E})^{-1}\mathbf{E}^T\mathbf{p}_{\text{actual}}$$

BoM & Contingency Plan

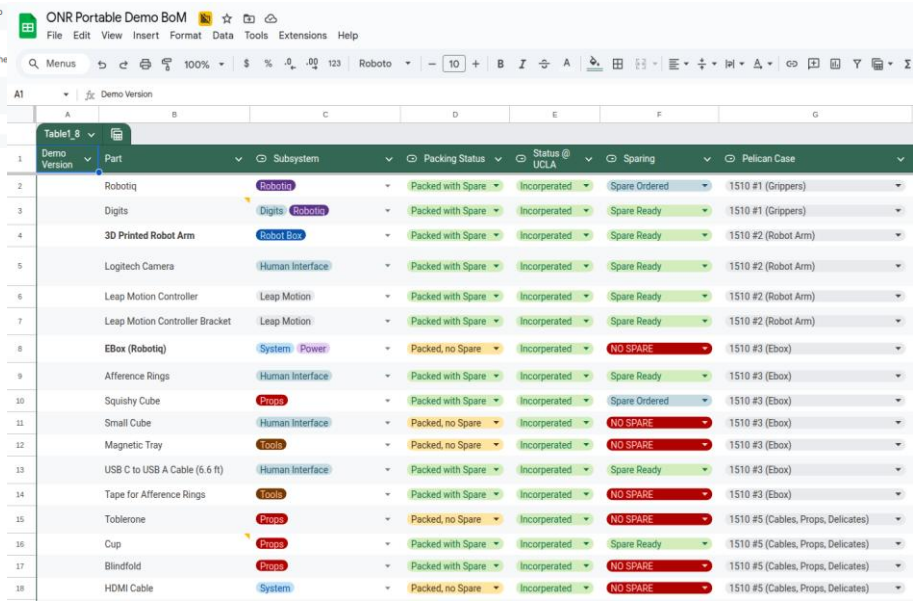


Demo Contingency Plan

File Edit View Insert Format Data Tools Extensions Help

Table1_8

Demo Issue	Subsystem	Likelihood	Severity	Description of Issue	Contingency	Confidence in Contingency
Swiss to US Power Converter does not work OR damages Ebox	Power	Medium	5	The two step down converters we bought do not work for 240V to 120V, and do not work with the EBox.	Have to try to find a step down converter in Switzerland that is compatible, otherwise have to show demo video	Low
Ebox is confiscated during TSA	Power	Medium	5	Ebox is confiscated during TSA, and we are unable to explain it and it is taken away	May be able to plug in various cables directly into the computer w/o the Ebox (robotiq, digits, networking) and use spare 24V power supply. Otherwise show video	Low
Robot Arm Breaks during shipping	Robot Box	Medium	4	Robot Arm breaks in shipping prior to demo	Use back-up arm OR Use spare parts of arm that we printed prior, use super glue / epoxy, show video as worst case backup	Low
Leap Motion Tracking is not performing well based on lighting conditions of the room	Leap Motion	Medium	4	Leap Motion tracking is jittery because of light reflections, or conditions of the room, or skin color, or background color, etc	Try to move the Leap Motion or try different angles / settings to improve hand tracking. Try to have user remove sleeves, move ring to lower finger on the hand (though not the middle), bring the right hand in from the left side of the camera, bring both hands into view and then remove the left hand from view, choose a different participant, move the demo to a different location, adjust the table cloth, try to control under the table, change the heights of the Leap motion camera. Have user position themselves in different orientations. Unideal case is that the demo fidelity is fairly bad (i.e. jittery, poor tracking). Worst case show video.	Medium
Venue is very hot and melts Toblerone	Props	Medium	2		Purchase a different breakable, local food item.	
Robot Arm Breaks on Stage / During Move on Stage	Robot Box	Low	5	Robot Arm breaks when we are moving the robot arm onto stage, or during the actual demo the arm breaks		
Leap Motion Camera is not recognized by the laptop / bandwidth issues	Leap Motion	Low	5	Like in early developments with this project, the Leap Motion camera sometimes had bandwidth issues when other ports were being used, especially conflicting when the Digit was being run.		
Robotiq breaks or stops functioning prior to demo	Robotiq	Low	5	Robotiq breaks during shipping or prior to demo		
Robotiq is confiscated during TSA	Robotiq	Low	5	Unable to convince TSA, and they take it		
Cables (any in the system) get unplugged during	System	Low	5	User walks or trips into cables, and they somehow get		



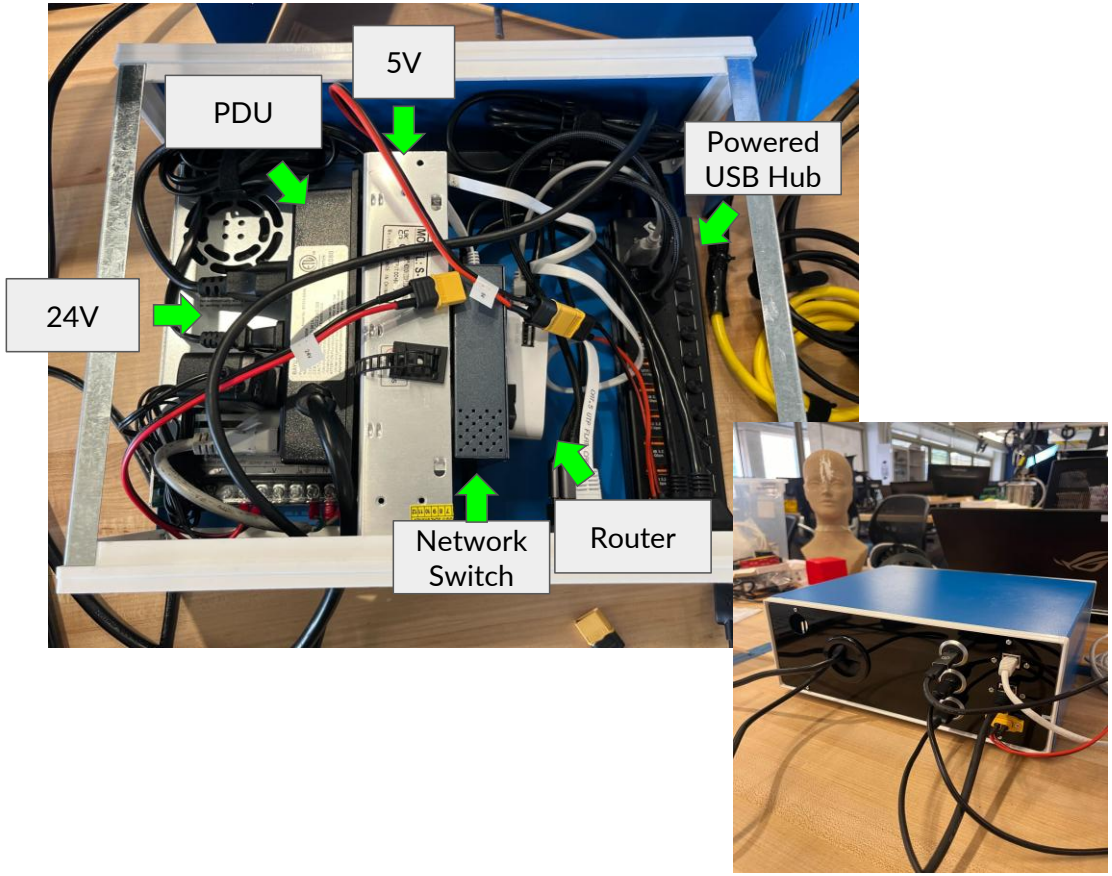
ONR Portable Demo BoM

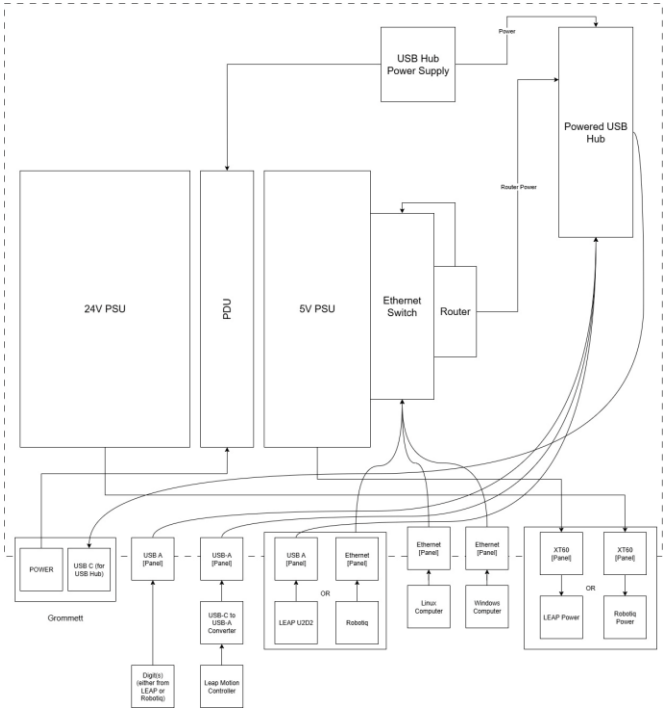
File Edit View Insert Format Data Tools Extensions Help

Table1_8

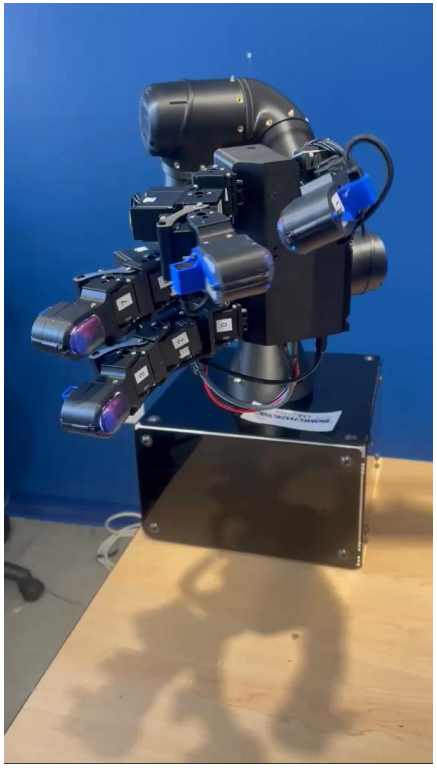
Demo Version	Part	Subsystem	Packing Status	Status @ UCLA	Sparing	Pelican Case
	Robotiq	Robotiq	Packed with Spare	Incorporated	Spare Ordered	1510 #1 (Grippers)
	Digits	Digits Robotiq	Packed with Spare	Incorporated	Spare Ready	1510 #1 (Grippers)
	3D Printed Robot Arm	Robot Box	Packed with Spare	Incorporated	Spare Ready	1510 #2 (Robot Arm)
	Logitech Camera	Human Interface	Packed with Spare	Incorporated	Spare Ready	1510 #2 (Robot Arm)
	Leap Motion Controller	Leap Motion	Packed with Spare	Incorporated	Spare Ready	1510 #2 (Robot Arm)
	Leap Motion Controller Bracket	Leap Motion	Packed with Spare	Incorporated	Spare Ready	1510 #2 (Robot Arm)
	EBox (Robotiq)	System Power	Packed, no Spare	Incorporated	NO SPARE	1510 #3 (Ebox)
	Afference Rings	Human Interface	Packed with Spare	Incorporated	Spare Ready	1510 #3 (Ebox)
	Squishy Cube	Props	Packed with Spare	Incorporated	Spare Ordered	1510 #3 (Ebox)
	Small Cube	Human Interface	Packed, no Spare	Incorporated	NO SPARE	1510 #3 (Ebox)
	Magnetic Tray	Tools	Packed, no Spare	Incorporated	NO SPARE	1510 #3 (Ebox)
	USB C to USB A Cable (6.6 ft)	Human Interface	Packed with Spare	Incorporated	Spare Ready	1510 #3 (Ebox)
	Tape for Afference Rings	Tools	Packed with Spare	Incorporated	NO SPARE	1510 #3 (Ebox)
	Toblerone	Props	Packed, no Spare	Incorporated	NO SPARE	1510 #5 (Cables, Props, Delicates)
	Cup	Props	Packed with Spare	Incorporated	Spare Ready	1510 #5 (Cables, Props, Delicates)
	Blindfold	Props	Packed with Spare	Incorporated	NO SPARE	1510 #5 (Cables, Props, Delicates)
	HDMI Cable	System	Packed, no Spare	Incorporated	NO SPARE	1510 #5 (Cables, Props, Delicates)

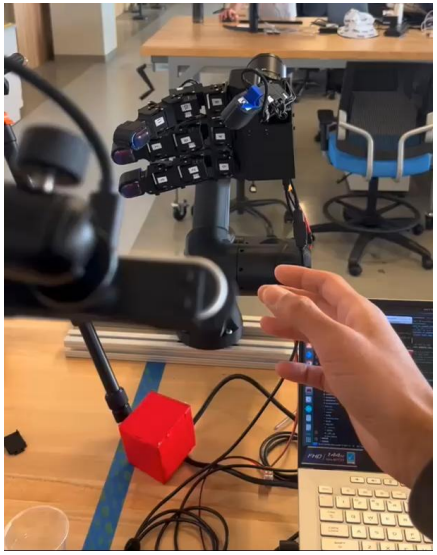
Electronics Box



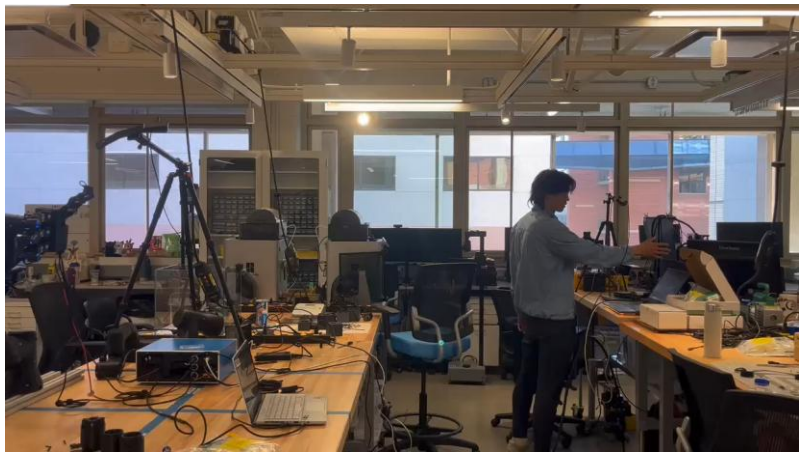


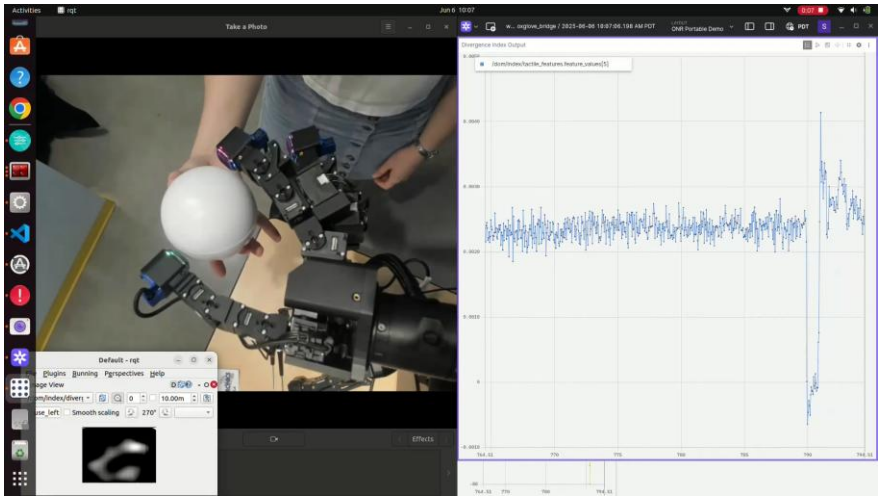
Operation Videos LEAP



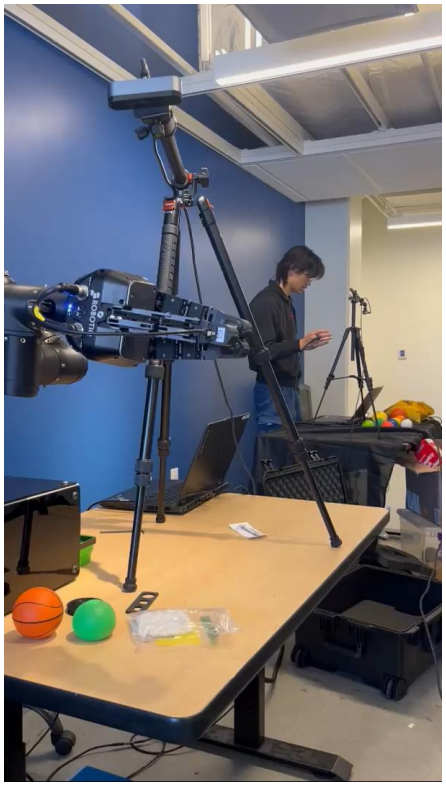


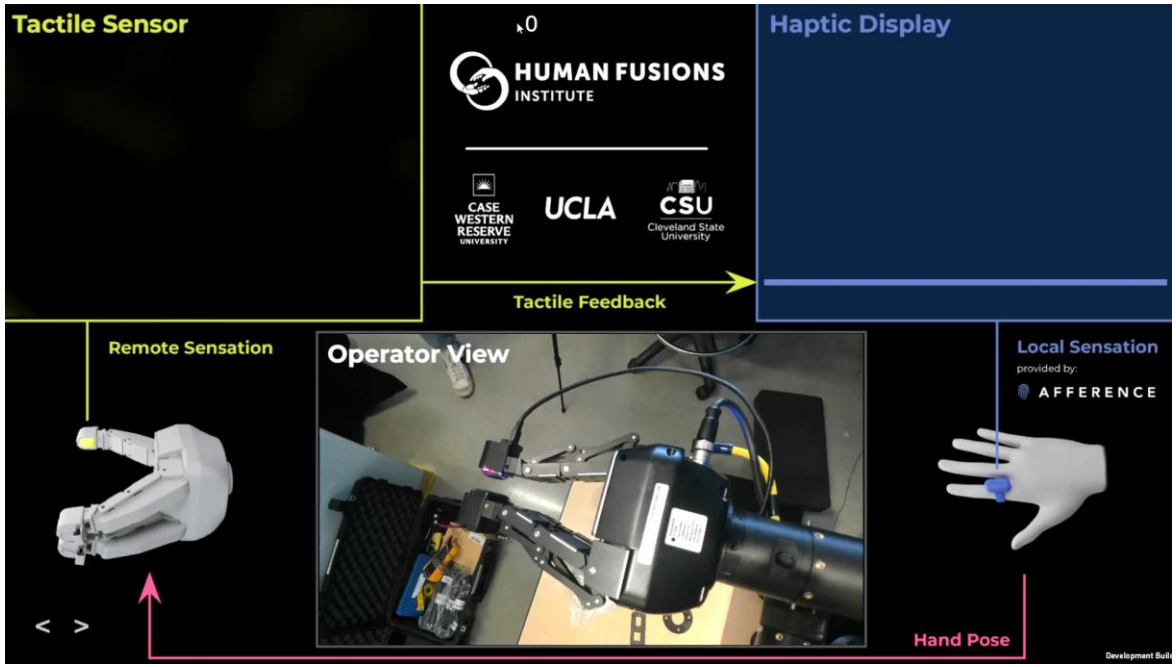
Operation Videos LEAP (cont.)



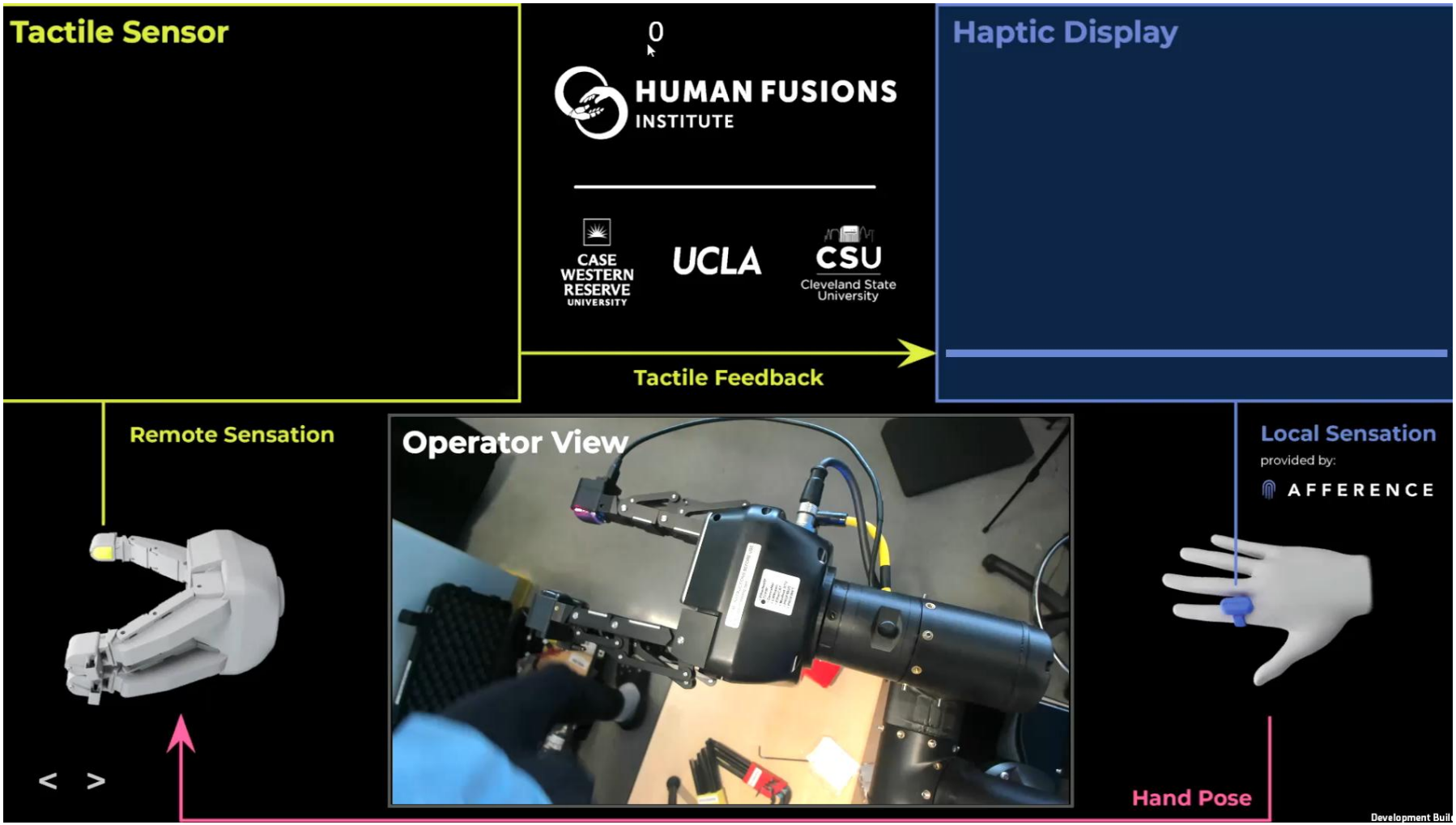


Operation Videos (Robotiq)

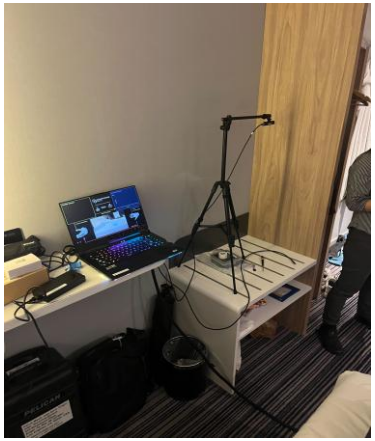


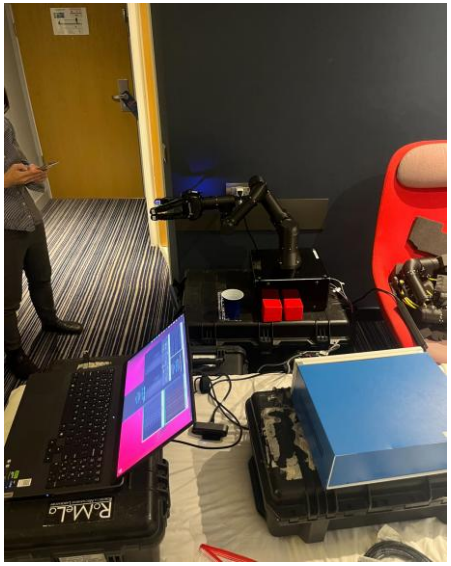


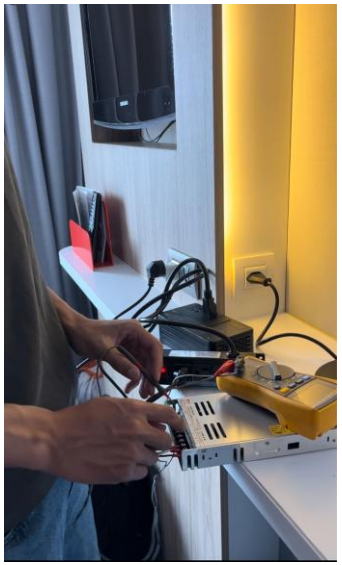
Digit Output Visualization

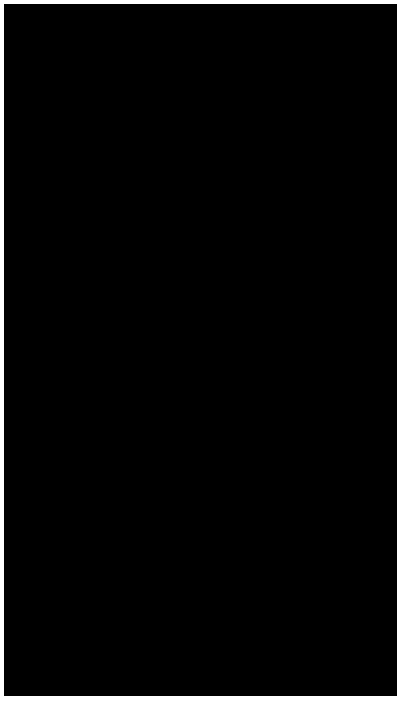


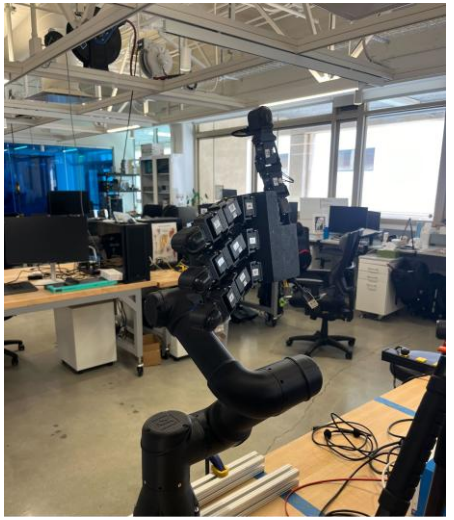
Extra Photos + Videos











Hardware Release Guidelines

1. Prior to doing a release on the overall ONR System CAD, each of the subcomponents should have their own release
    1. This is so that a release of the overall ONR System CAD, is simply matching the revisions of all the components
2. Release candidates should be named: RELEASE <LETTER> - <BRIEF DESCRIPTION>
3. In ONR BoM for subcomponents that have CAD representation, specify the release
4. Within the BoM for the subcomponent, specify a line for CAD that links to the correct CAD revision, as well as the overall technical document
5. Within the technical document, create a tab called “Releases (Mechanical)”, where information regarding the high level changes that were done during the release are enumerated, as well as details such as print settings
6. Within the technical document, add a tab called “Assembly Instructions (Mechanical)”, which will have assembly instructions for the specific release

Software Design Guidelines

1. When ready to release, create a tag of onr_robotic_system, which should ideally only be off of develop or main
   a. **Ideally,** all subcommits of submodules that are referenced in onr_robotic_system would be referencing develop or main
2. Tags should follow semantic versioning where `onr-demo-v<MAJOR.MINOR.PATCH>`
3. For things in development going to CWRU, often it will be a major version and minor versions are smaller updates
   a. EX: MAJOR -> a new controller
   b. EX: MINOR -> a new ReadMe, small QoL updates to the system
   c. Probably not using PATCH for now…
4. After tags are established, create a release in Github based off of the tag
   a. Include high level description of the release notes
5. The releases should be designed to just download the release for simplest integration
   a. `git tag -a onr-demo-v<....> -m 'message'`
   b. git tag -a onr-demo-v3.0.0 -m "First official software release of the ONR robotic system"
   c. `git push origin onr-demo-v<....>`

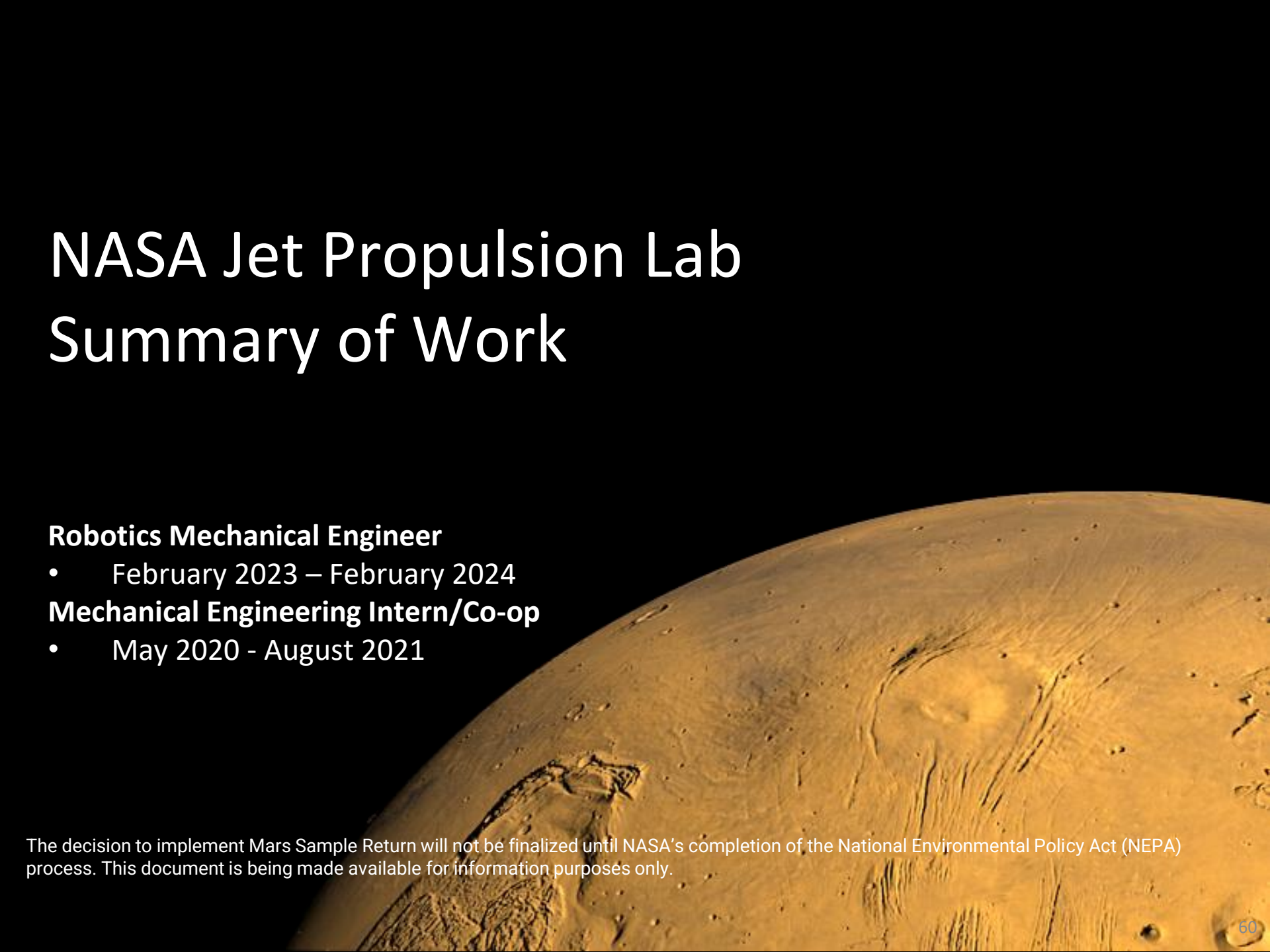
NASA Jet Propulsion Lab
Summary of Work

Robotics Mechanical Engineer

- February 2023 – February 2024

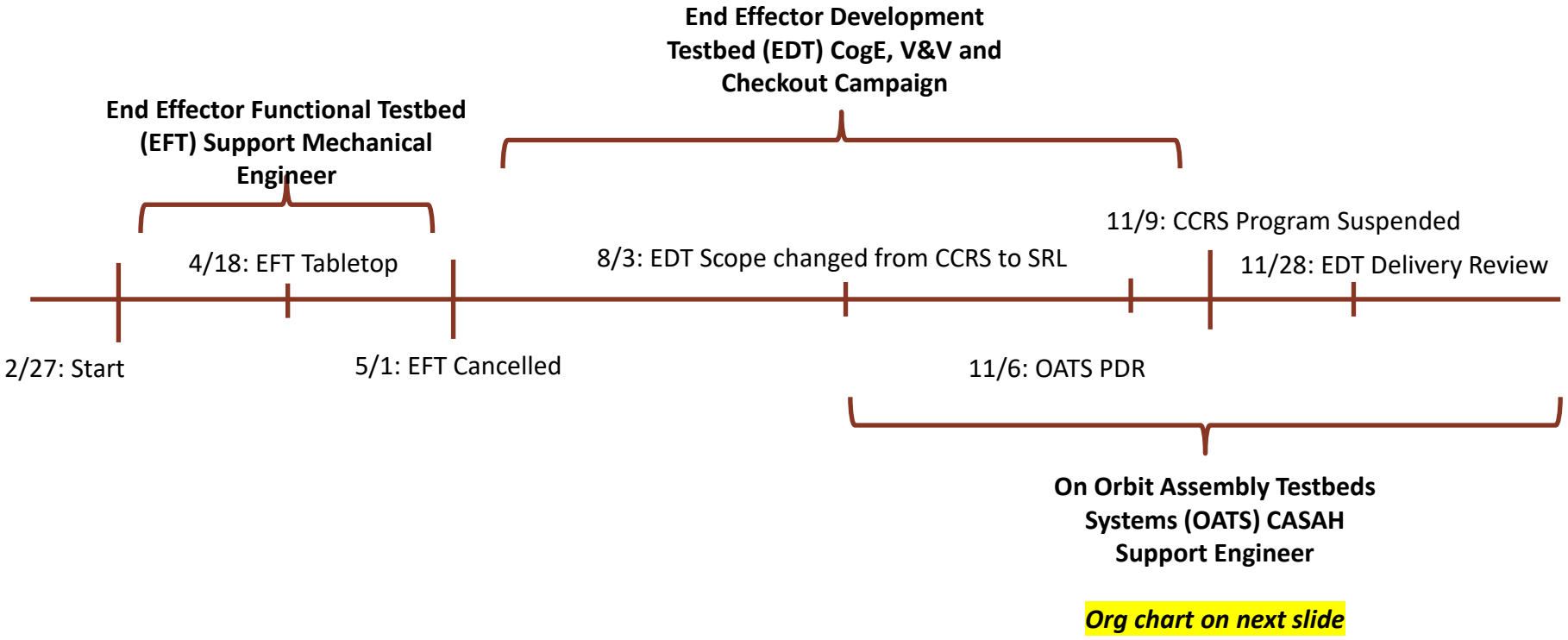
Mechanical Engineering Intern/Co-op

- May 2020 - August 2021



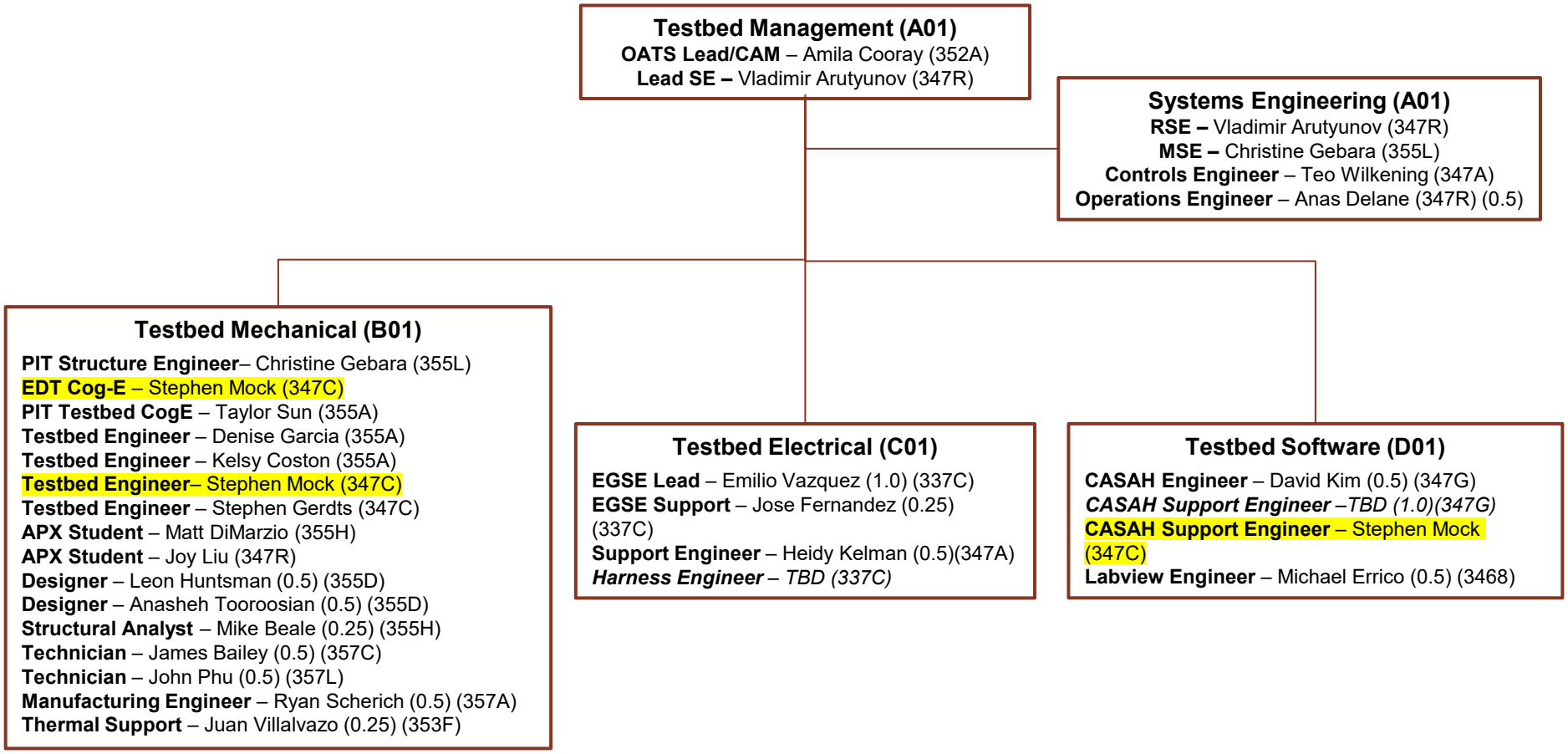
The decision to implement Mars Sample Return will not be finalized until NASA's completion of the National Environmental Policy Act (NEPA) process. This document is being made available for information purposes only.

Overview – CCRS Testbeds roles held by Stephen Mock



Org chart on next slide

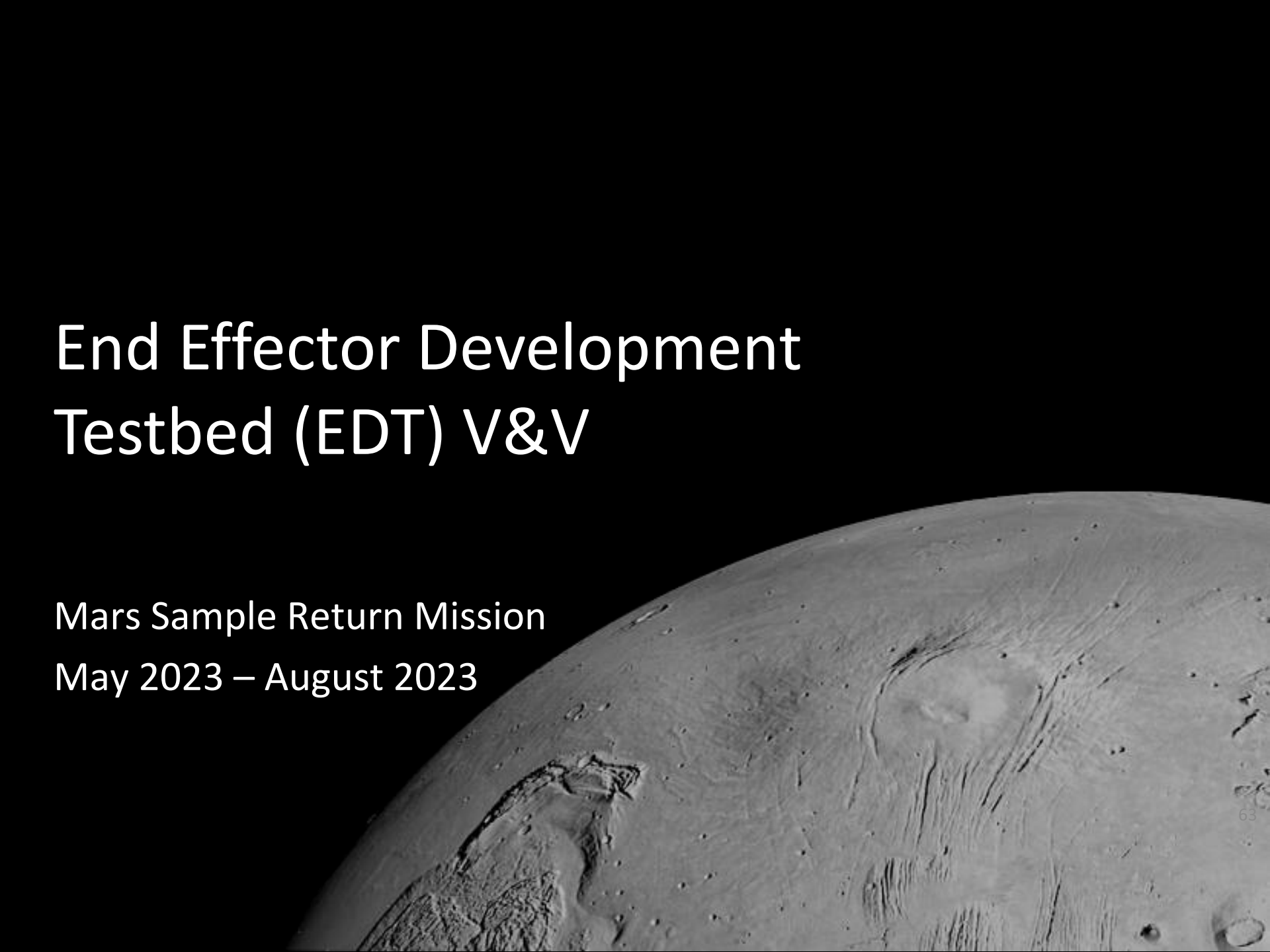
On Orbit Assembly Testbeds Systems (OATS) Org Chart



End Effector Development Testbed (EDT) V&V

Mars Sample Return Mission

May 2023 – August 2023



Preface

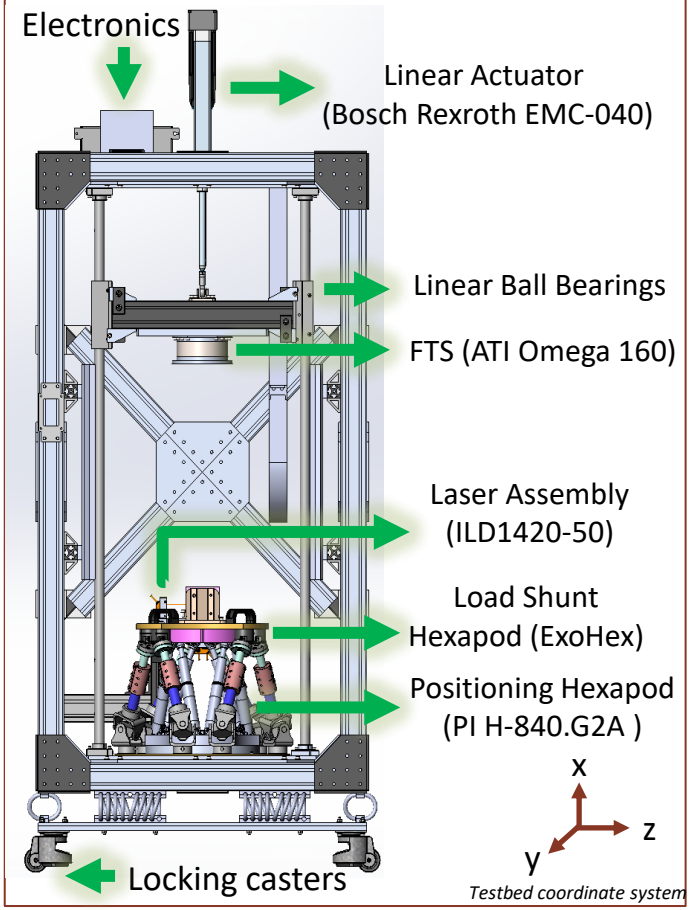
History

The End Effector Testbed (EDT) was created as part of the Capture Contain Return System (CCRS) Testbeds team to provide a venue to test prototype CCRS end effectors starting in Summer 2022. The objective of the testbed was to measure force and torque data during insertion for misaligned interfaces. It was a successor to a previous testbed for which the inner hexapod was originally purchased. With increases in load requirements, a larger 8020 structure and linear actuator were implemented for high axial loading and clocking moments. An ExoHex was designed to enable the inner hexapod to still be used for precise positioning, without having to survive high loads. EDT V&V started in May 2023, but was later rescoped in August 2023 to be delivered to the Sample Retrieval Lander (SRL) team in November 2023. As a result, the purpose of the testbed and checkout tests were focused on general functionality rather than CCRS specific implementation. This package highlights the capabilities of EDT, as well as the performed checkouts, and reference information. The checkouts relate to validating the basic functionality of the testbed, particularly for safety purposes.

JPL Team

- **EDT V&V**: Stephen Mock (347C)
- **EDT Software**: Michael Errico (3468)
- **EDT Design / Build / History:**
  - Vladimir Arutyunov (347R)
  - Stephen Gerdts (347C)
  - Jake Chesin (347B)
  - Heidy Kelman (347A)
- **EDT CAD:** Heidy Kelman (347A)

EDT Overview



Testbed coordinate system

Design Intent: Simulate misalignments in 6DoF such that forces/torques can be measured during end effector functions

CONOPS

Loosen ExoHex Legs → Move Inner Hexapod to Position → Lock down ExoHex Legs → Demate Inner and ExoHex → Linear Actuator moves to contact* → Linear Actuator moves to preload* → End Effector Functions → Linear Actuator backs off and resets to free space

**Forces/Torques measuring during whole operation*

Main Component Capabilities

Bosch Rexroth EMC-040
- 305 mm Stroke
- 3.4 kN rated peak axial load
- Absolute encoder
- Optional limit switches (not installed)

PI H-840.G2A
- 392N Fx load capacity (normal orientation)
- ±50mm lateral
- ±25mm vertical
- ±15° rotation (tip/tilt)
- ±30° rotation (clocking)
- Absolute encoder

ATI Omega 160
- Dual Calibration
- High Load: SI-2500-400
- Low Load: SI-1000-120

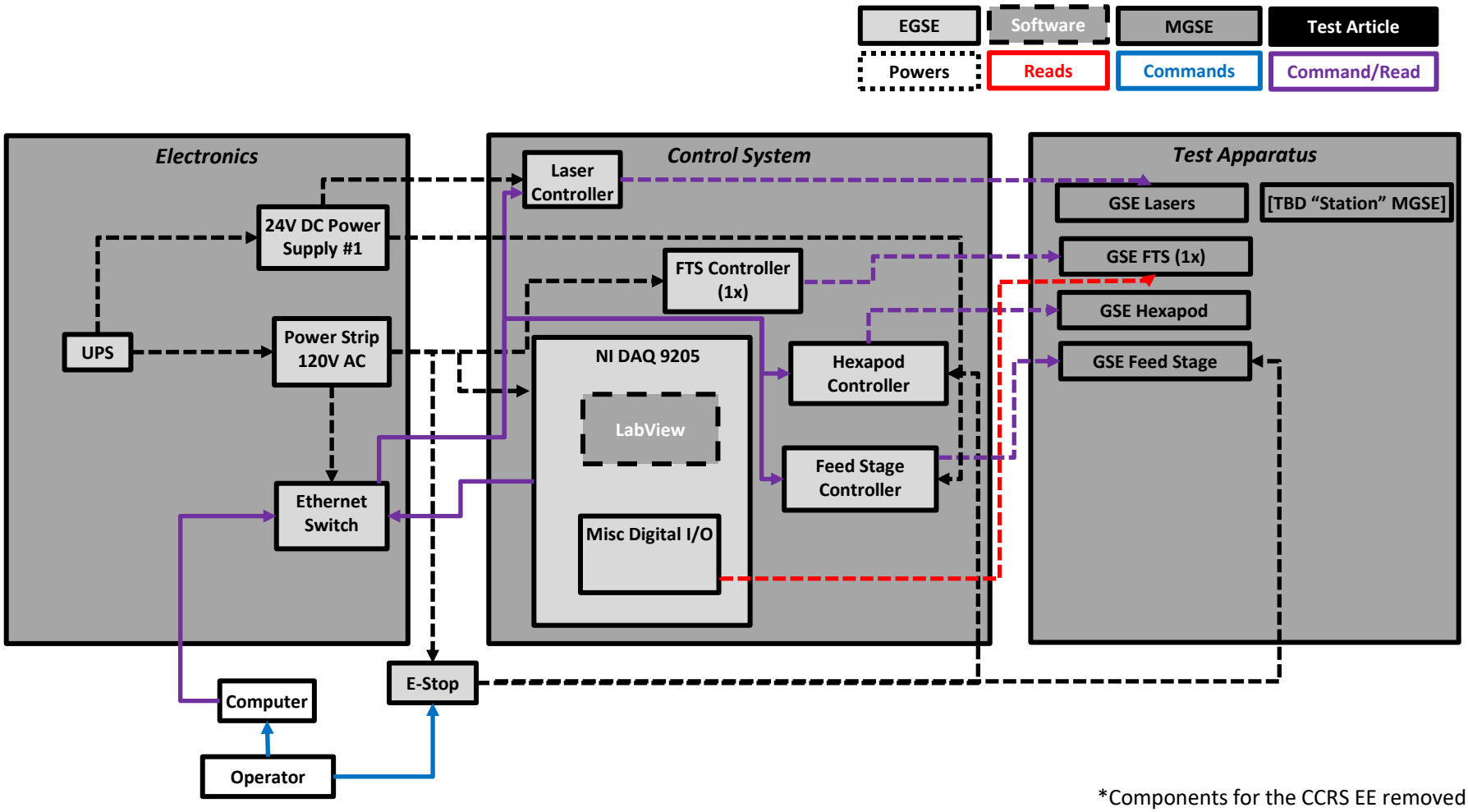
ExoHex
- 31kN axial load capacity at zero position
  - based off single leg proof test to 5500N
  - full assembly not proofed
- ±10mm lateral
- ±1.15° rotation (tip/tilt)
- ±0.57° rotation (clocking)
  - *Tested values*

Functional Capabilities

- ExoHex for high loads
  - Inner hexapod for precise positioning
  - ExoHex can always be detached
- LabView software for Operator GUI and Control
- FTS recording during test and for force limiting
  - NI DAQ 9205
- Interlock-based E-stop
- Feed Motion Functionality
  - Freespace Move
  - Move to Contact
  - Move to Preload / Move to No Load
- Hexapod coordinate frame changes
- Laser distance measuring capability

Not Fully Checked Out:
- Stiffness Characterization via. laser assembly
- ExoHex assembly proof loading
- Linear actuator proof loading
- Hexapod coord. frame changes in LabView

EDT Functional Block Diagram



*Components for the CCRS EE removed

Main Program Software Design

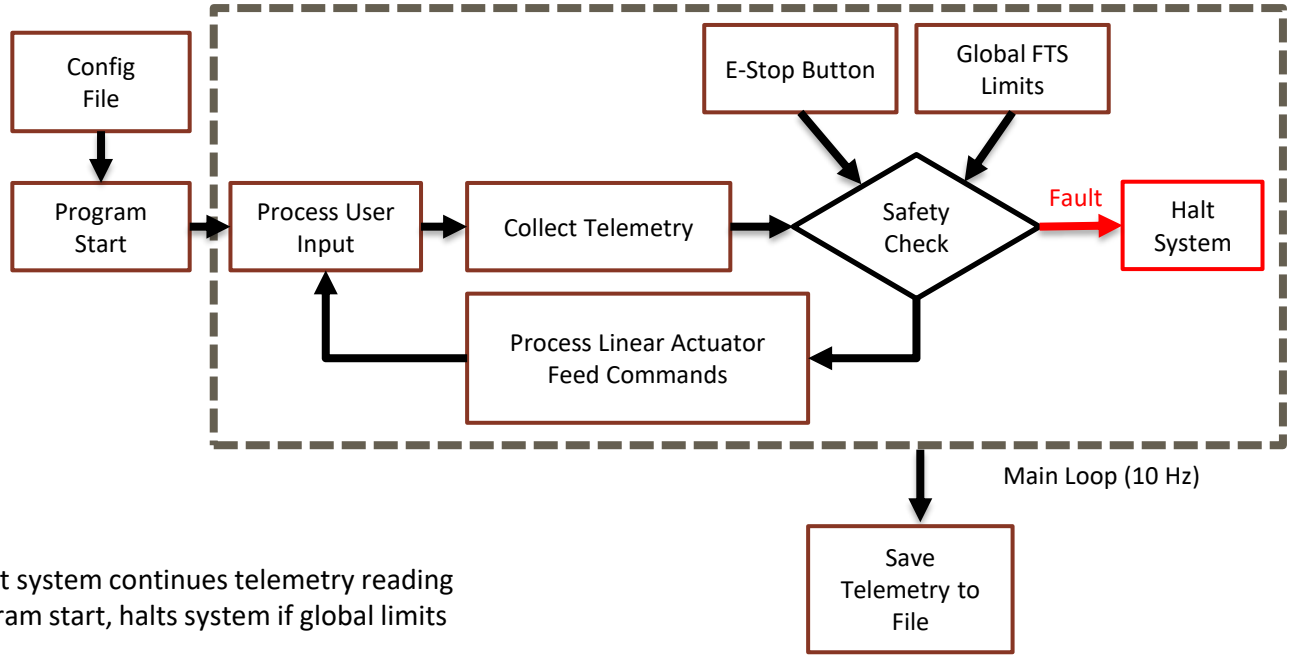
Software Written by **Michael Errico**

Configuration File:
- Linear Actuator Feed Command Parameters (position/force limits, speeds)

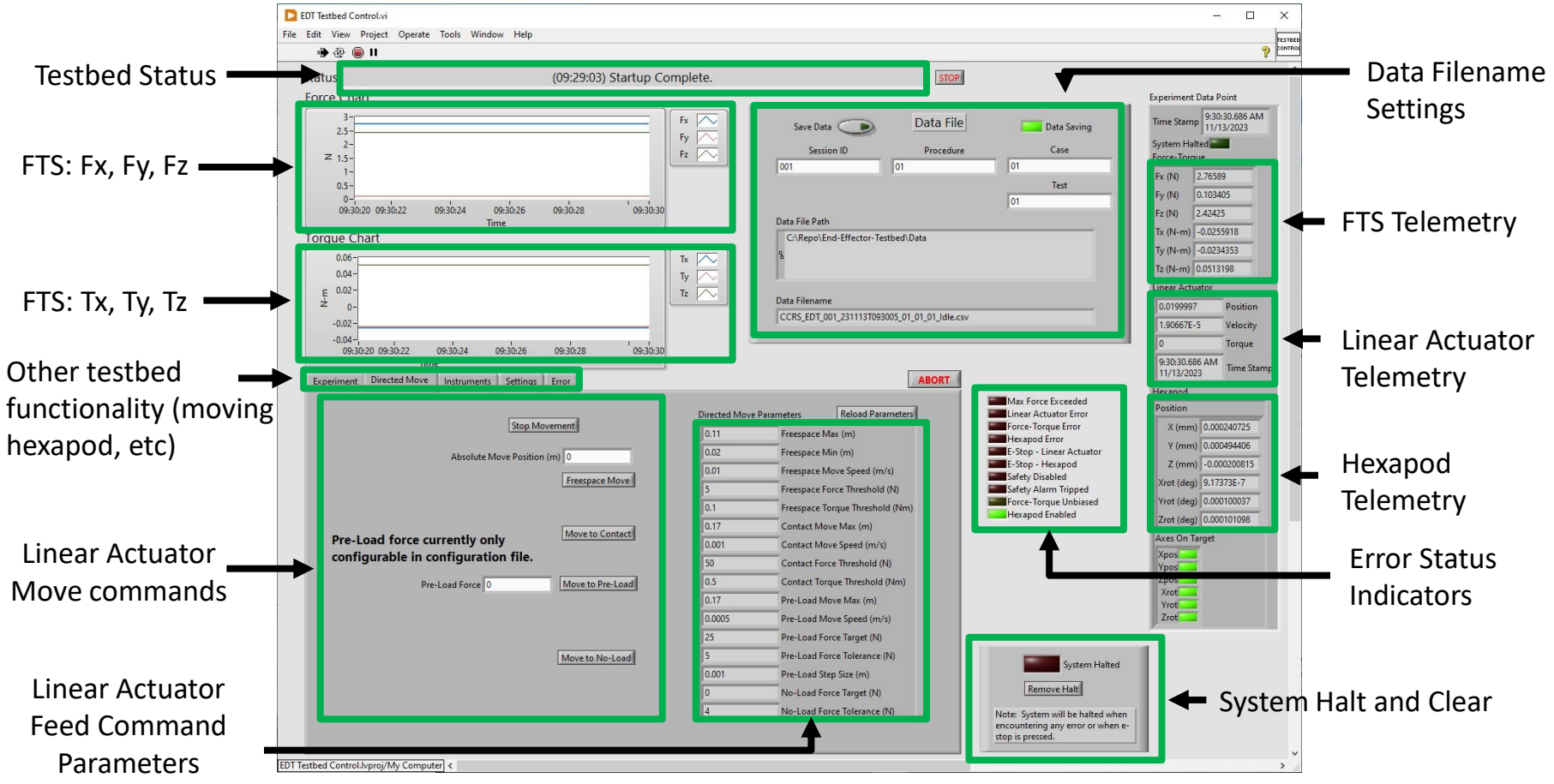
User Inputs:
- Linear Actuator Feed Commands
  - Free-space move
  - Move to contact
  - Move to pre-load
  - Move to no-load
- Hexapod Free-space Position move
  - Hexapod speed

Telemetry:
- Hexapod
  - Absolute Position (X, Y, Z)
  - Absolute Rotation (XRot, YRot, ZRot)
- FTS
  - Force (Fx, Fy, Fz)
  - Torque (Tx, Ty, Tz)
- Linear Actuator
  - Absolute Position
  - Speed

E-Stop: Full system halt with physical E-stop but system continues telemetry reading
Global FTS Limits: Hard-coded and set on program start, halts system if global limits are exceeded during any operation

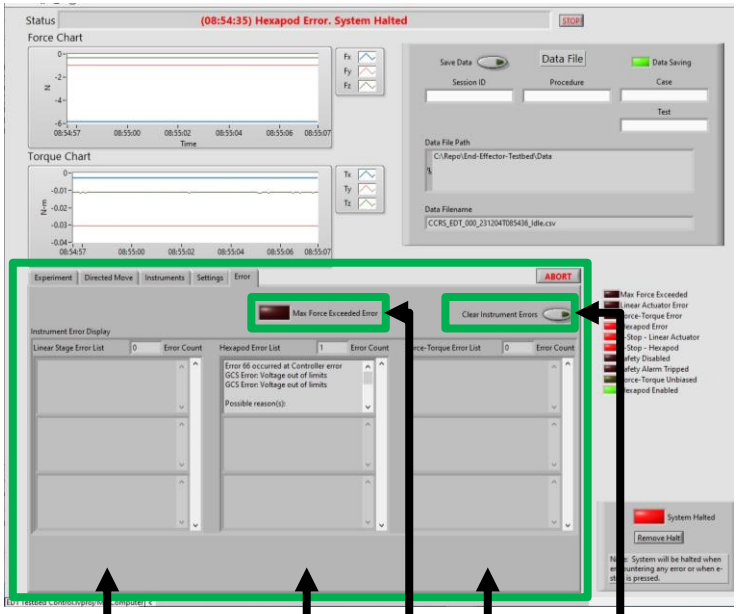


Operator GUI Screen

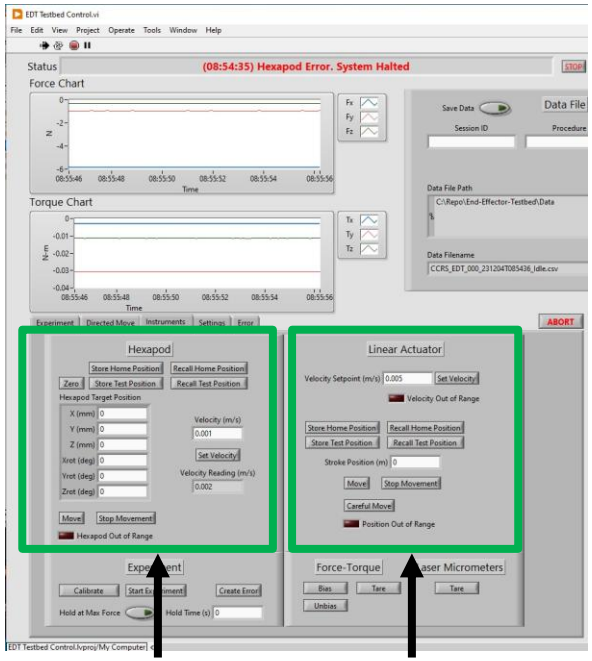


Operator GUI Screen cont.

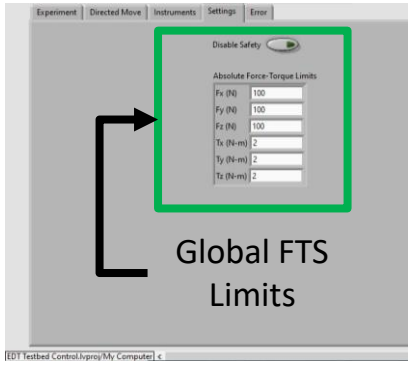
Error Tab



Instrument Tab

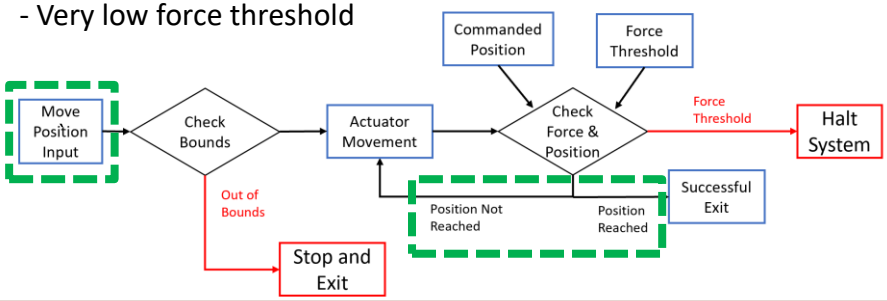


Settings Tab

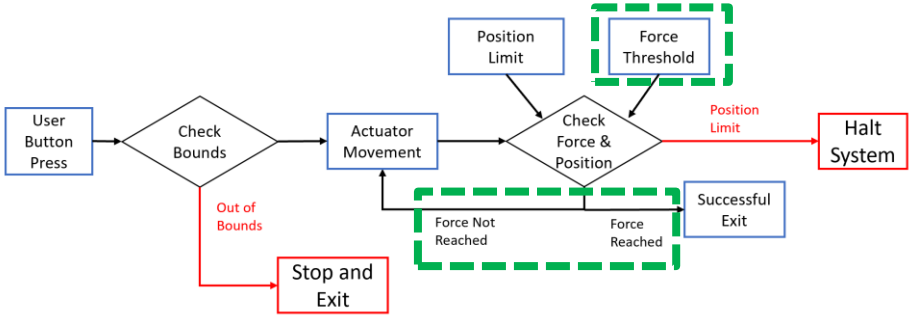


Linear Actuator Movement Block Diagrams

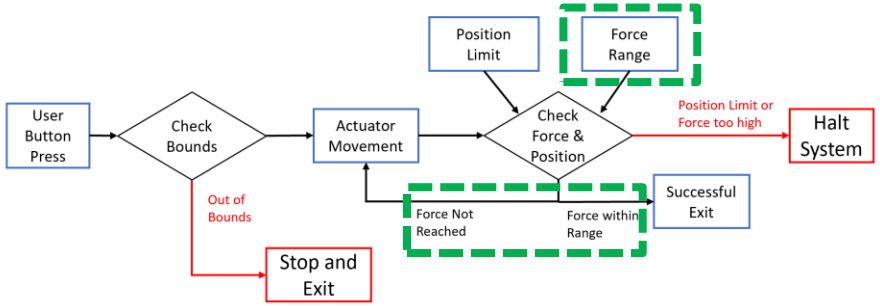
Linear Actuator Free-space Move:
- Higher speed position move with pre-defined limits
- Very low force threshold



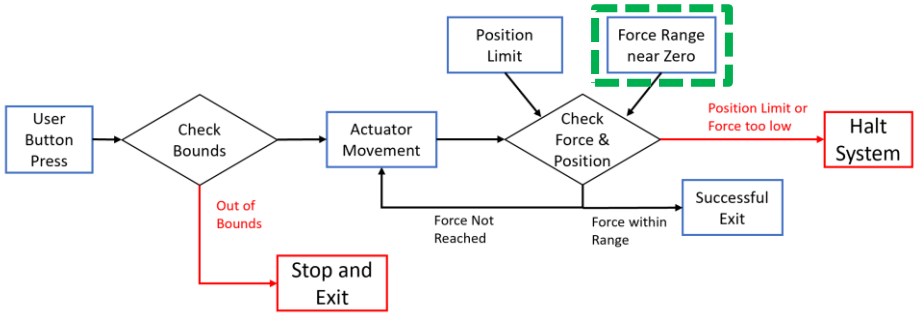
Linear Actuator Move to Contact:
- Slow movement to contact until force threshold is passed



Move to Preload:
- Slow forward movement until target force is within force range



Linear Actuator No-Load Movement:
- Reverse movement to reduce force to near-zero

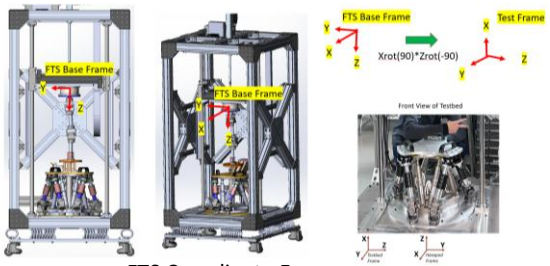


Checkout Summary

Basic E-Stop Checkout [10] ✅
Objective: Verify E-stop capability to stop motion of both the hexapod and linear actuator during operation, yet still maintain connection and FTS recording. Test how system halts are handled and cleared. Additional testing to see how MicroMove responds to an E-stop being pressed.
Result: E-stop halts motion, maintains connection and continues to record FTS data. System halt can be cleared when E-stop is removed. MicroMove will error when E-stop is pressed, and hexapod can be restarted after E-stop is unpressed.

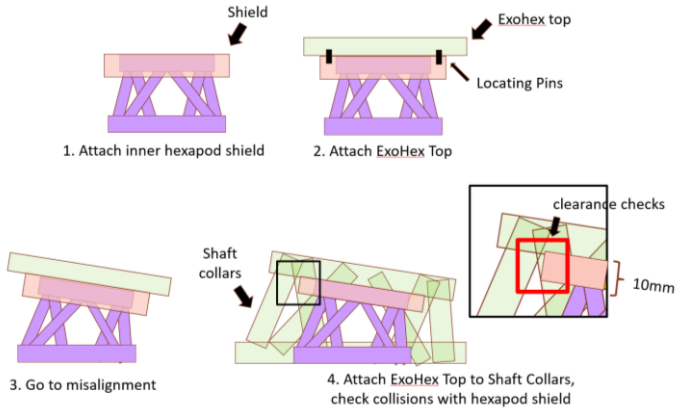
Basic Hexapod Movement [06] ✅
Objective: Move to the maximum 1DoF travel ranges of the hexapod using the testbed coordinate frame. Additionally, test the behavior of the hexapod to stop under global force overload error.
Result: Hexapod moves in accordance with testbed coordinate frame (using vendor provided software) and responds to force overload error in LabView.

Basic FTS Checkout [02] ✅
Objective: Confirm the coordinate system of the FTS for future coordinate transformations.
Result: FTS frame tracks as expected, and coordinate frame change to testbed frame maps correctly



FTS Coordinate Frame

ExoHex Misalignment [09] ✅
Objective: Check for potential collisions between ExoHex strut legs and Inner Hexapod top plate during misalignment, and during potential demate motions (simulated by a hexapod shield).
Result: With the tested subset of misalignments (27 tests), no ExoHex struts were close to collisions. This however is only done for a smaller subset of misalignment and should be performed with actual test misalignments.



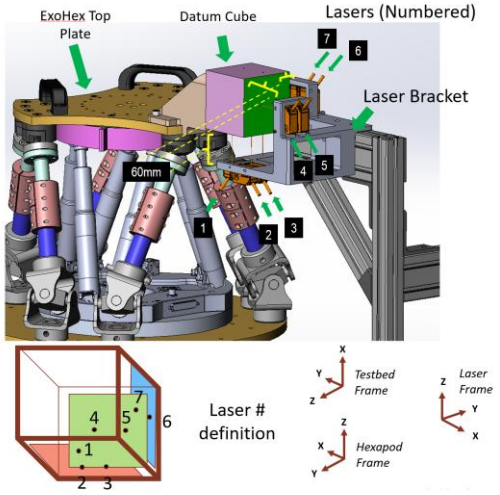
ExoHex Misalignment Test CONOPs

Checkout Summary (continued)

Basic Laser Checkout [03] ✅

Objective: Understand the capabilities of the laser nest assembly when measuring a static cube moving to different positions. Could potentially be used for future stiffness characterization

Result: Lasers measure relative movement accurately, but small errors exist which are likely due to overall misalignment of laser assembly to hexapod.



EDT Laser Nest and Coordinate Frames

Basic Feed Motion Checkout [05] ✅

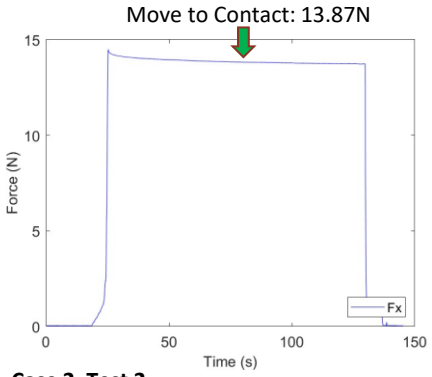
Objective: Test main linear actuator movements (freespace move, move to contact, and move to preload, move to no load).

- *Freespace Move*: higher speed movement to bring the linear actuator to a specific position, with very low force threshold.
- *Move to Contact*: Lower speed movement which moves to a force threshold and stops when it is exceeded (no tolerance).
- *Move to Preload*: Lower speed movement which moves to a specific force value with a given +- tolerance. Meant to reach the desired preload given by the test requirements.
- *Move to No Load*: Reverse "Move to Contact", in which the actuator moves away from contact so that Free-space Moves can be commanded.

Result:

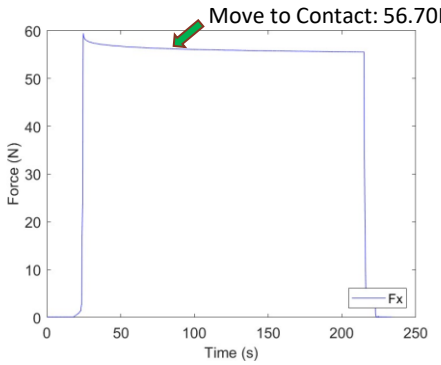
- Linear actuator program demonstrated its intended use for all four different types of movements through applying a specific preload to an aluminum can.
- Linear actuator triggers halt when overall testbed force thresholds are exceeded, preventing users from inputting new commands.

Basic Feed Motion Checkout Results



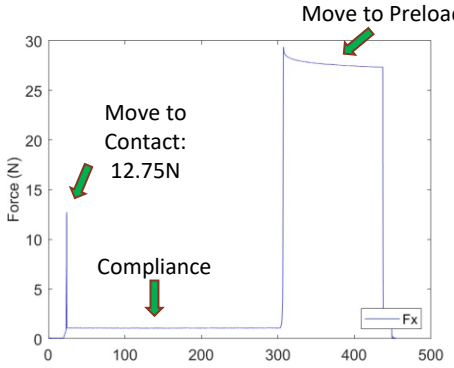
Case 2, Test 2

- Move to Contact to 10N, system stopped at 13.87N



Case 3, Test 3

- Move to Contact to 50N, system stopped at 56.70N

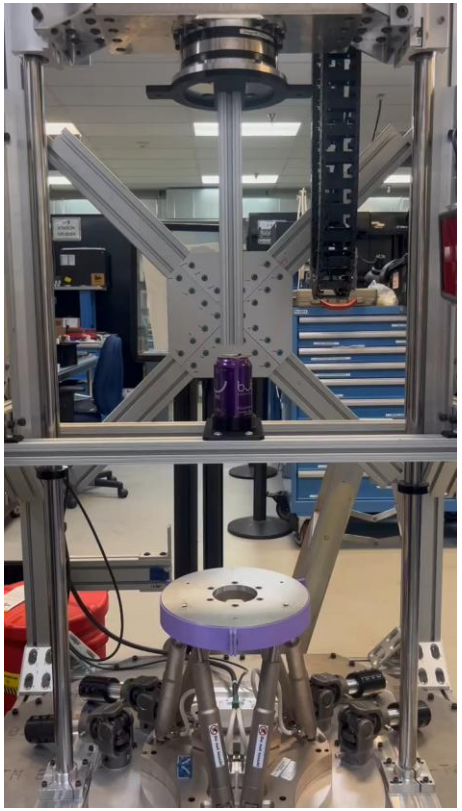


Case 3, Test 2

- Move to Contact to 10N, system stopped at 12.75N
- Move to Preload to 25N, system stopped at 27.64N

Notes:

- The system does not halt perfectly as the force build-up occurs quickly; thus, the system does not stop exactly when the force threshold is crossed.
  - Move to contact speed: 1mm/s
  - Move to preload speeds 0.5mm/s
- There is some compliance in the aluminum can such that when the linear actuator stops, the force decreases



Move to Contact Demonstration (video)

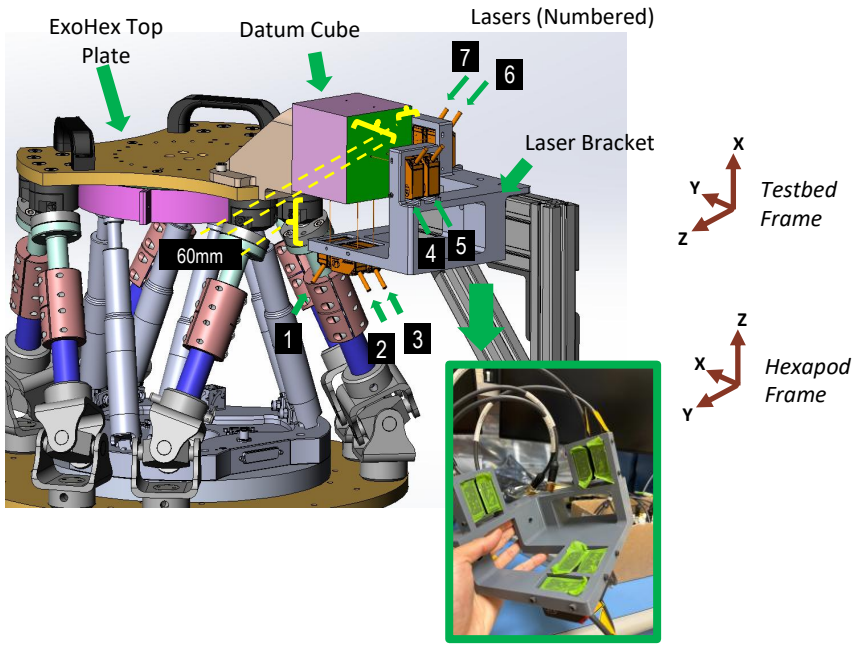
Laser Transform Module for End Effector Development (EDT) Testbed

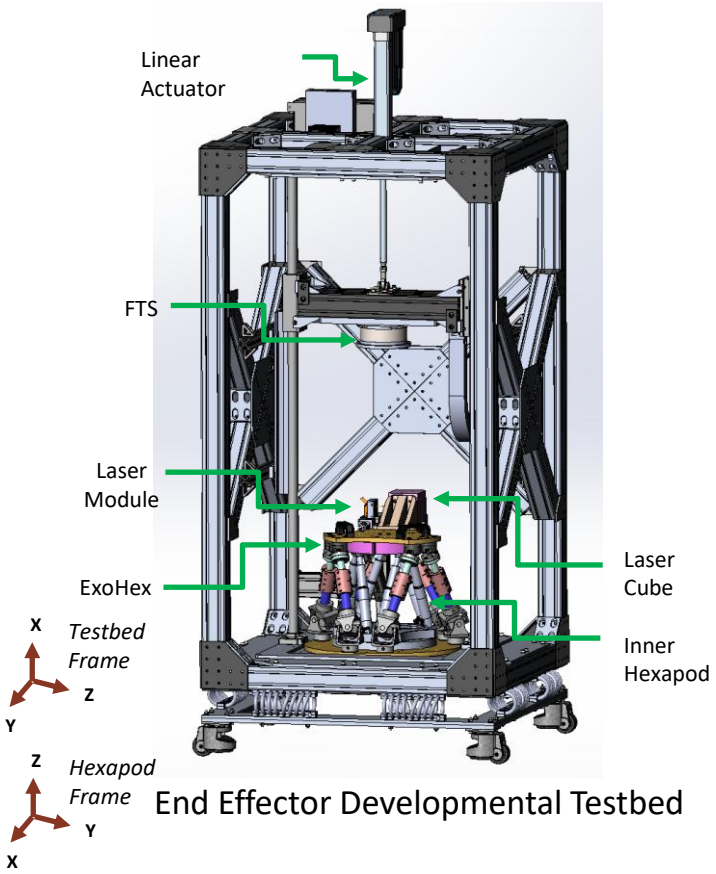
Mars Sample Return Mission
May 2023 – August 2023

Testbed Background + Objective

Objective: Characterize stiffness of ExoHex top plate under proof load

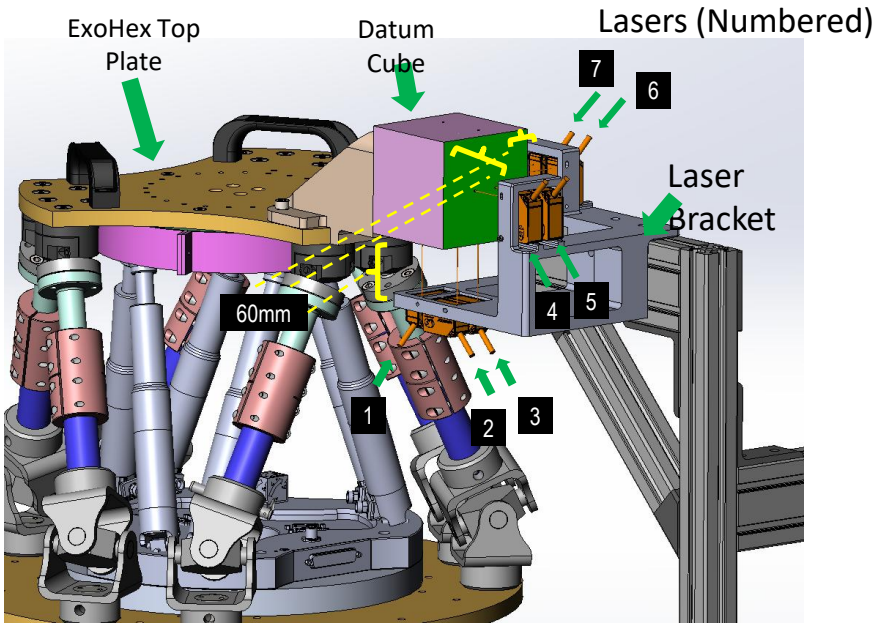
1. Datum (cube) mounted to top plate is considered rigid with hexapod top plate which will deform under external load
2. Lasers points to cube and measure changes in position in free space due to distortions
3. Using 7 lasers, perform transform calculation to define full homogenous transform of cube

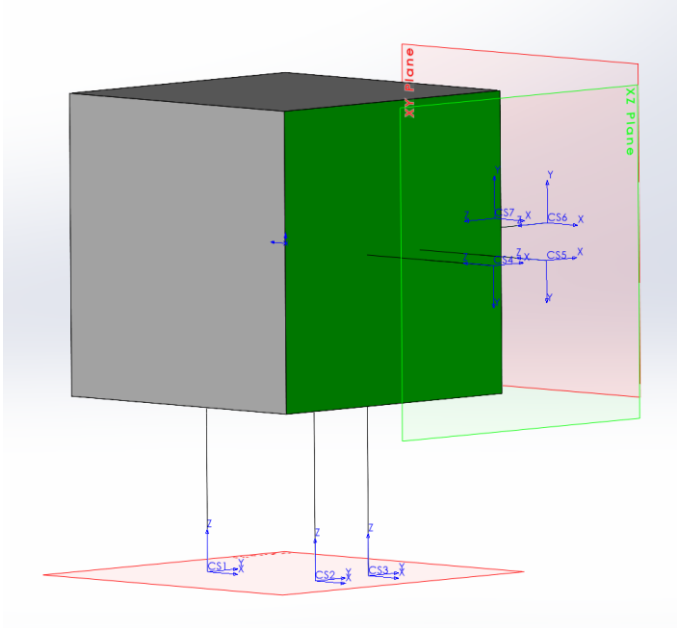


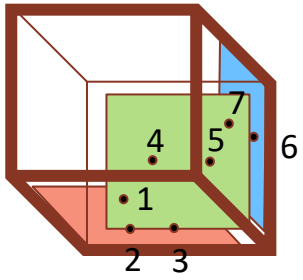


End Effector Developmental Testbed

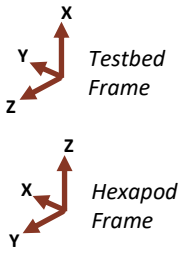
Coordinate Frame Definition

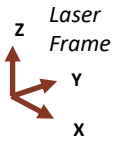






Laser # definition

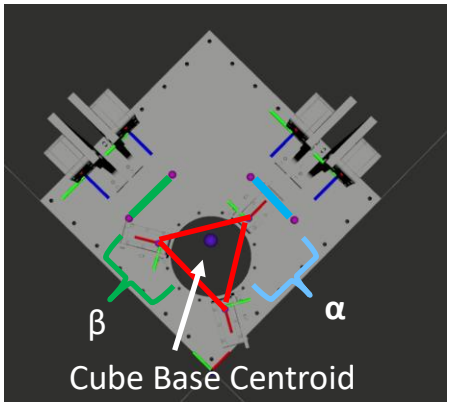




- Separate CAD model to simulate rotations / translations
- Lasers represented as (very small diameter) extrusions up to surface for ground truth generation from nominal laser positions → SolidWorks sensors
- Laser Frame → Testbed Frame is Ry(-90)*Rx(90)

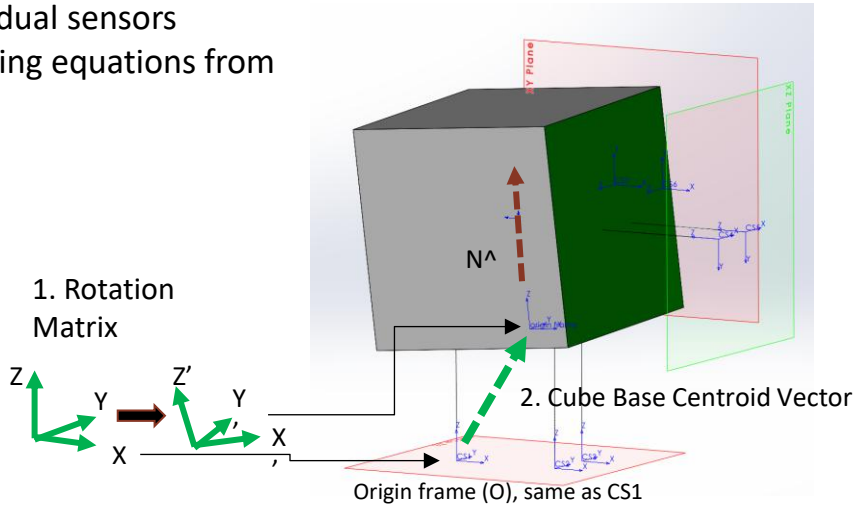
Implementation Approach

- Define frames of our 7 lasers such that we can perform the IK (inverse kinematics) to define the full transform of the cube
  - Coordinate system is defined on bottom of 3 planes, where n^ is defined
  - α and β are constants defined to surface of where lasers hit cube face
    - In our case alpha and beta are cube side lengths (4in / 2)
    - Reference

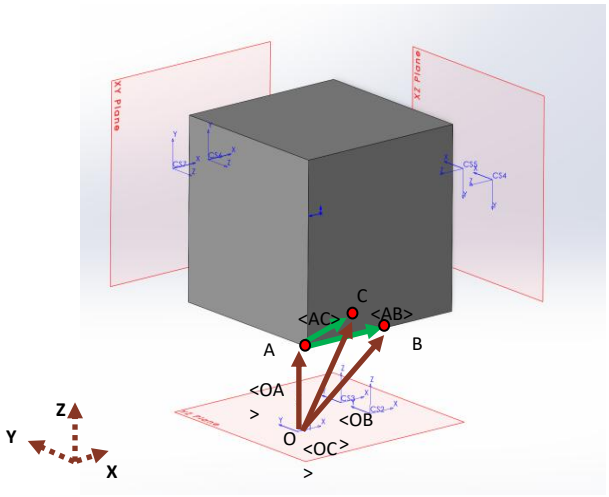


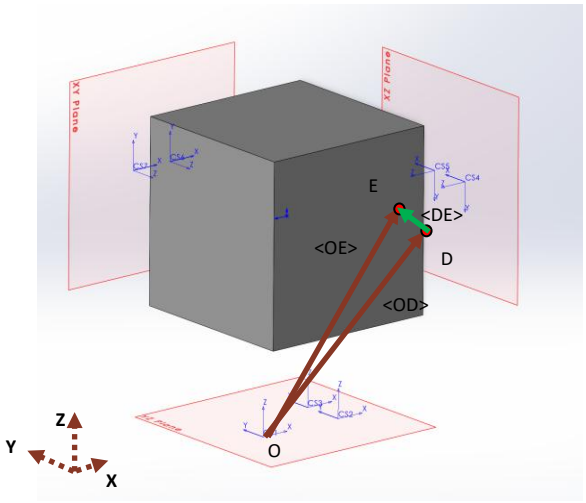
Setup

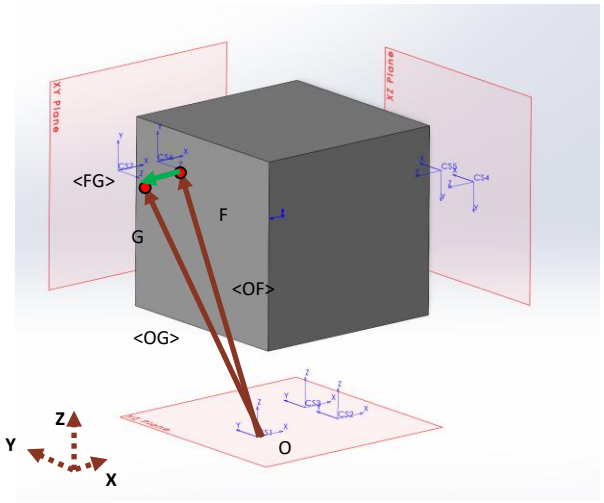
1) Choose a world coordinate frame (origin frame) for lasers
   1) Origin frame set at first laser frame (CS1)
2) Create transformations to each of the frames on each of the lasers
   1) Z-axis always the pointing towards the cube
   2) H_O1, H_O2 (ETC)
   3) Pure Z-translation in that coordinate frame from each individual sensors
3) Solve for transform from laser readings in laser frame on cube using equations from paper for
   1) Rotation Matrix
   2) Cube Base Centroid Vector
4) Perform transformation from world frame to testbed frame



Main Vector Definition







Origin Frame (coincident with CS1)

Defined in Origin Frame
Vector of Interest: <AC> = <OC> - <OA>, <AB> = <OB> - <OA>

Where <OC> = H03*P3, <OB> = H02 * P2, <OA> = H01*P1 where P1, P2, P3 are the magnitude of the laser (in Z axis)

$$\hat{n} = \frac{\vec{AB} \times \vec{AC}}{\|\vec{AB} \times \vec{AC}\|} \tag{3}$$

```
353    Eigen::Vector3d ab = b - a;
354    Eigen::Vector3d ac = c - a;
355
356    Eigen::Vector3d n = ab.cross(ac).normalized();
```

Defined in Origin Frame
Vector of Interest: <DE> = <OE> - <OD>

Where <OE> = H05*P5, <OD> = H04*P4 and P4, P5 are the magnitude of the laser (in Z axis)

$$\frac{\hat{n} \times \vec{DE}}{\|\hat{n} \times \vec{DE}\|} \cdot \mathscr{O} = \frac{\hat{n} \times \vec{DE}}{\|\hat{n} \times \vec{DE}\|} \cdot D - \alpha \tag{9}$$

```
358    Eigen::Vector3d de          = e - d;
359    Eigen::Vector3d n_cross_de = n.cross(de).normalized();
```

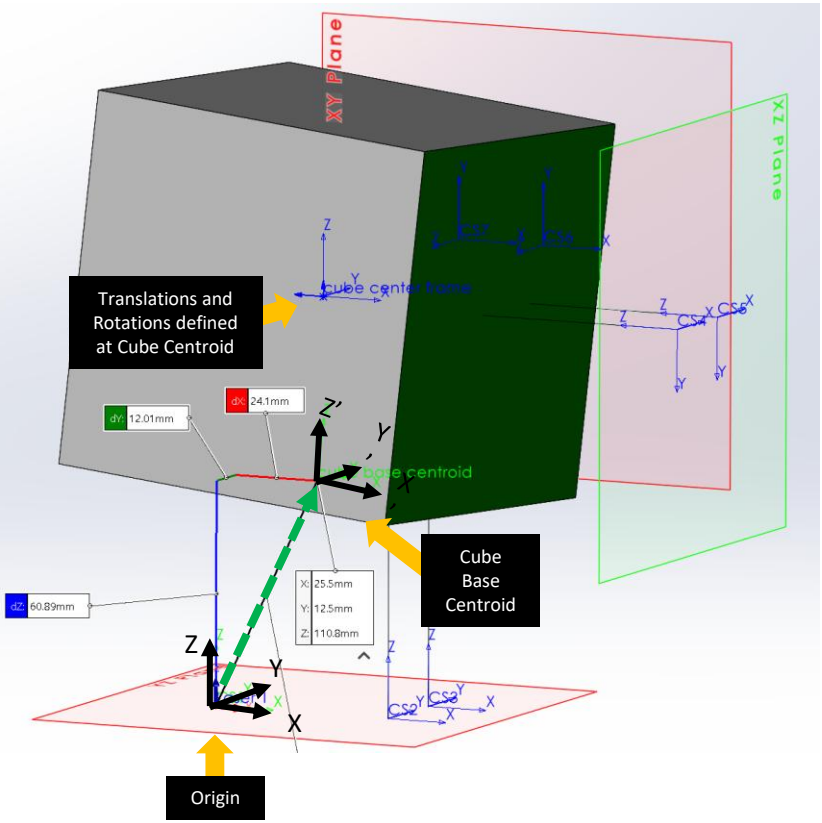
Defined in Origin Frame
Vector of Interest: <FG> = <OG> - <OF>

Where <OG> = H07*P7, <OF> = H06*P6 and P6, P7 are the magnitude of the laser (in Z axis)

$$\frac{\hat{n} \times \vec{FG}}{\|\hat{n} \times \vec{FG}\|} \cdot \mathscr{O} = \frac{\hat{n} \times \vec{FG}}{\|\hat{n} \times \vec{FG}\|} \cdot F - \beta \tag{12}$$

```
361    Eigen::Vector3d fg          = g - f;
362    Eigen::Vector3d n_cross_fg = n.cross(fg).normalized();
```

Ground Truth Test Approach



Perform 3 types of ground truth tests

1. Rotations
2. Translations
3. Combined

- Using CAD, rotate the cube in a specific order
  - Laser output from CAD informs inverse kinematics and creates ground truths
  - Check rotations by outputting Euler Angles in (ZYX) format
- For tests cases, Δ translations and rotations defined at **CUBE centroid**
  - Cube Base Centroid (centroid of 3 lasers on cube face) may have some "parasitic" translations due to rotation about a different pivot

Inputs

1. Laser values from CAD
2. Rotations (Euler Angles) for comparison
3. Position vector between Cube Base Centroid and Origin Frame for comparison

Outputs

1. Rotations (Euler Angles)
2. Position vector between Cube Base Centroid and Origin Frame

Compare

1. Ground Truth Rotation vs. Inverse Kinematic Rotation solution
2. Ground Truth Position vector vs. Inverse Kinematic Position solution

Rotation (Orientation) Ground Truth Tests

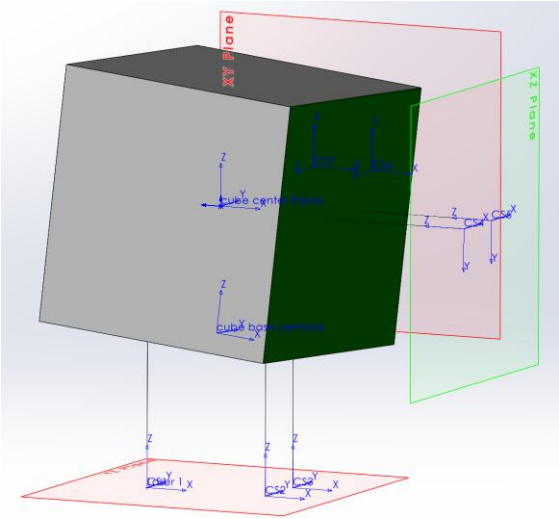
or in rotation matrix form:

$$R = \begin{bmatrix} (\frac{\hat{n}\times\vec{DE}}{\|\hat{n}\times\vec{DE}\|})^T \\ (\frac{\hat{n}\times\vec{FG}}{\|\hat{n}\times\vec{FG}\|})^T \\ -\hat{n}^T \end{bmatrix} \quad (17)$$

```
393    Eigen::Matrix3d R_ltm;
394    R_ltm << -n_cross_de(0), -n_cross_fg(0), n(0), -n_cross_de(1), -n_cross_fg(1),
395        n(1), -n_cross_de(2), -n_cross_fg(2), n(2);
```

Ground Truth Test Cases

Test #	Δ X (mm)	Δ Y (mm)	Δ Z (mm)	Zrot (deg)	Yrot (deg)	Xrot (deg)	IK Rot Match?	IK Pos. Match?
1	0	0	0	0	0	0		
2	0	0	0	1	0	0		
3	0	0	0	0	1	0		
4	0	0	0	0	0	1		
5	0	0	0	1	2	0		
6	0	0	0	1	2	3		
7	0	0	0	-2	-5	3		



Rotation Test Cases

- Test individual rotations, as well as rotations in sequence, as well as with either +/- signage
- Correctly tracked Euler Angles (for both positive and negative), as well as sequences of Euler Angles
  - Rounded (to the first decimal place) → might be due to sig-figs on laser output from CAD
- When rotating about centroid of cube, the cube base centroid also translates, which was captured in the positional inverse kinematics

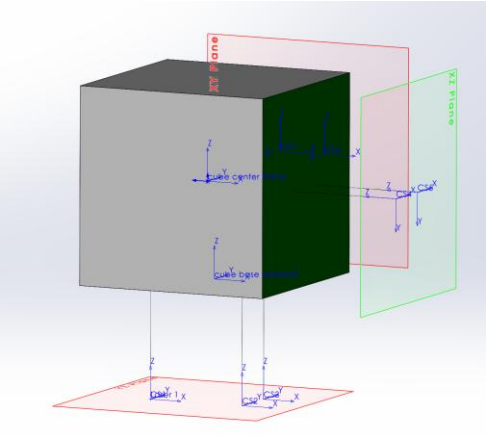
Translation Ground Truth Tests

$$\begin{bmatrix} \hat{n}^T \\ (\frac{\hat{n}\times\vec{DE}}{\|\hat{n}\times\vec{DE}\|})^T \\ (\frac{\hat{n}\times\vec{FG}}{\|\hat{n}\times\vec{FG}\|})^T \end{bmatrix}_{(3\times3)} \{\underline{\mathcal{O}}\}_{(3\times1)} = \begin{Bmatrix} \hat{n}\cdot A \\ \frac{\hat{n}\times\vec{DE}}{\|\hat{n}\times\vec{DE}\|}\cdot D - \alpha \\ \frac{\hat{n}\times\vec{FG}}{\|\hat{n}\times\vec{FG}\|}\cdot F - \beta \end{Bmatrix}_{(3\times1)} \quad (13)$$

```
364    Eigen::Matrix3d A;
365    A << n(0), n(1), n(2), n_cross_de(0), n_cross_de(1), n_cross_de(2),
366        n_cross_fg(0), n_cross_fg(1), n_cross_fg(2);
367
368    Eigen::Vector3d rhs;
369    rhs << n.transpose() * a,
370        n_cross_de.transpose() * d + laser_target_tool_axis_offset_,
371        n_cross_fg.transpose() * f + laser_target_tool_axis_offset_;
372
373    Eigen::FullPivLU<Eigen::Matrix3d> lu(A);
374    Eigen::Vector3d                   origin = lu.solve(rhs);
```

Ground Truth Test Cases

Test #	Δ X (mm)	Δ Y (mm)	Δ Z (mm)	Zrot (deg)	Yrot (deg)	Xrot (deg)	IK Rot. Match ?	IK Pos. Match?
1	0	0	0	0	0	0		
2	+2.5	0	0	0	0	0		
3	-2.5	0	0	0	0	0		
4	0	+2.5	0	0	0	0		
5	0	0	+2.5	0	0	0		
6	+2.5	+1	0	0	0	0		
7	+2.5	0	+1	0	0	0		
8	0	+2.5	+1	0	0	0		
9	+2.5	+1	+5	0	0	0		



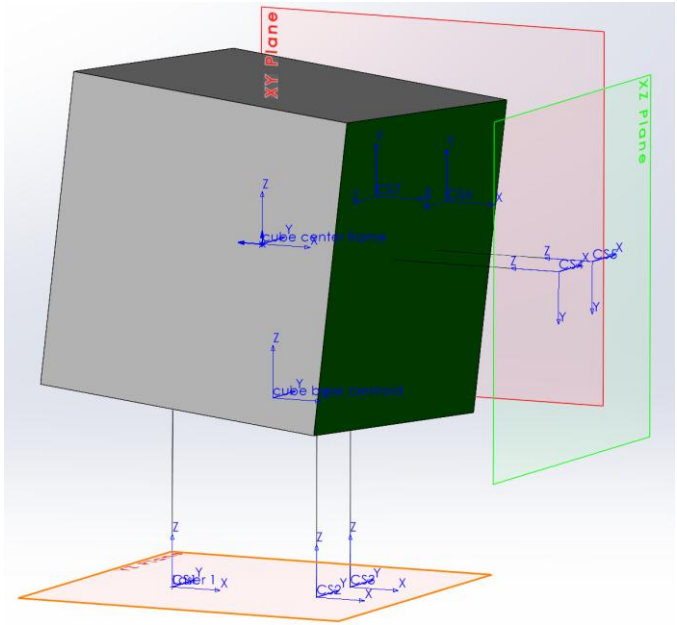
Translation Test Cases

- Test individual translations, as well as multiple translations, as well as with either +/- signage
- Tests worked in accordance with translations (and had no rotations)
- Rounded (to the first decimal place) → might be due to sig-figs on laser output from CAD

Combined Ground Truth Tests

Ground Truth Test Cases

Test #	Δ X (mm)	Δ Y (mm)	Δ Z (mm)	Zrot (deg)	Yrot (deg)	Xrot (deg)	IK Rot. Match ?	IK Pos. Match?
1	5	7.5	10	-2	-5	3		
2	-5	-7.5	-10	-2	-5	3		
3	-5	-7.5	-10	2	5	-3		
4	1	3	5	2	0	0		
5	5	3	1	0	3	0		
6	3	2	1	-5	0	0		



Combined Test Cases

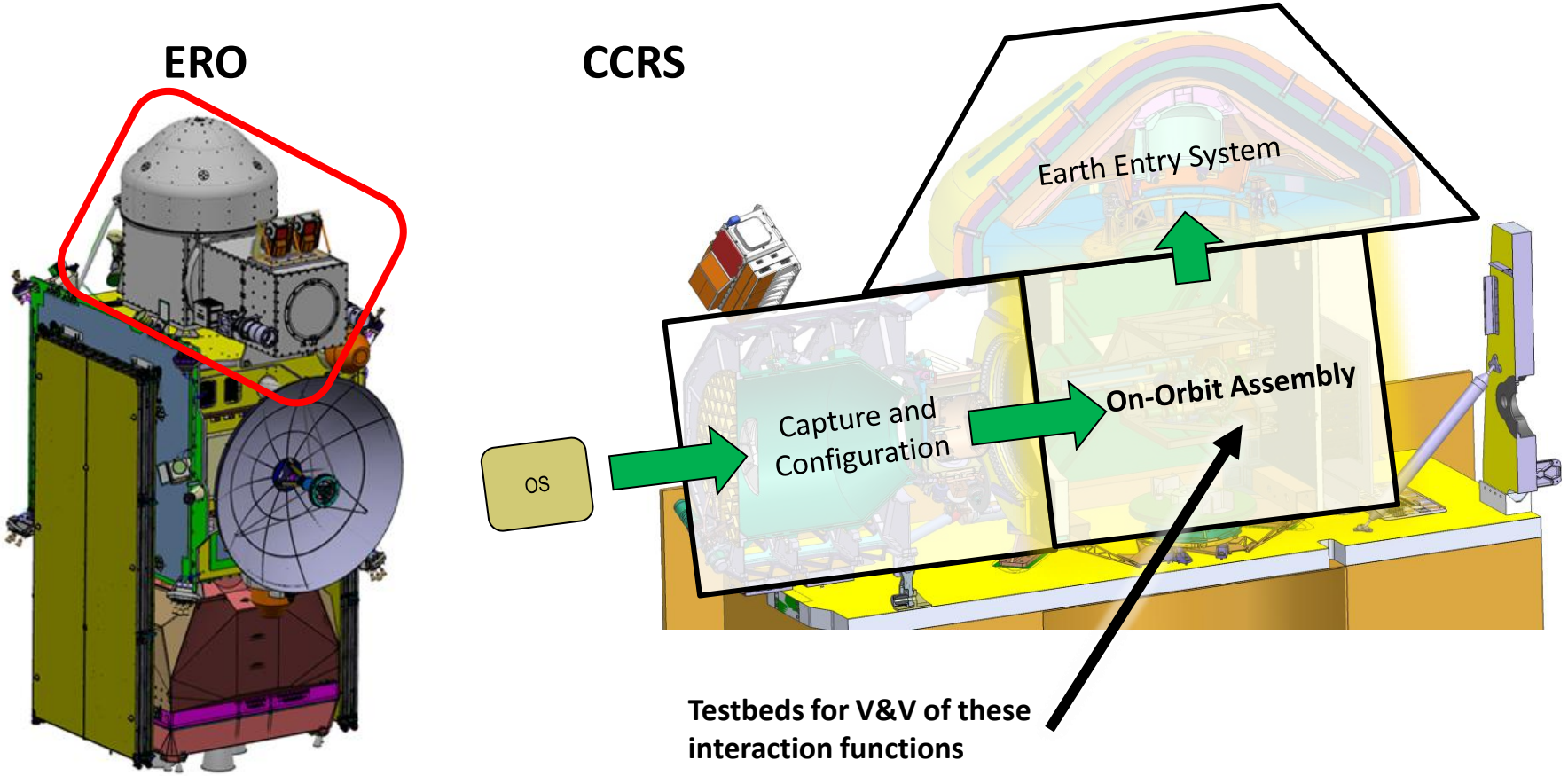
- Test both translations and rotations at centroid of cube
  - Every case works for the inverse kinematics!
- Rounded (to the first decimal place) → might be due to sig-figs on laser output from CAD

Software Development Summary of Work

Mars Sample Return Mission

August 2023 - January 2024

Capture, Containment, and Return System (CCRS) in ERO Context



CCRS Overview for Testbeds context

PIE Station Interaction Functions

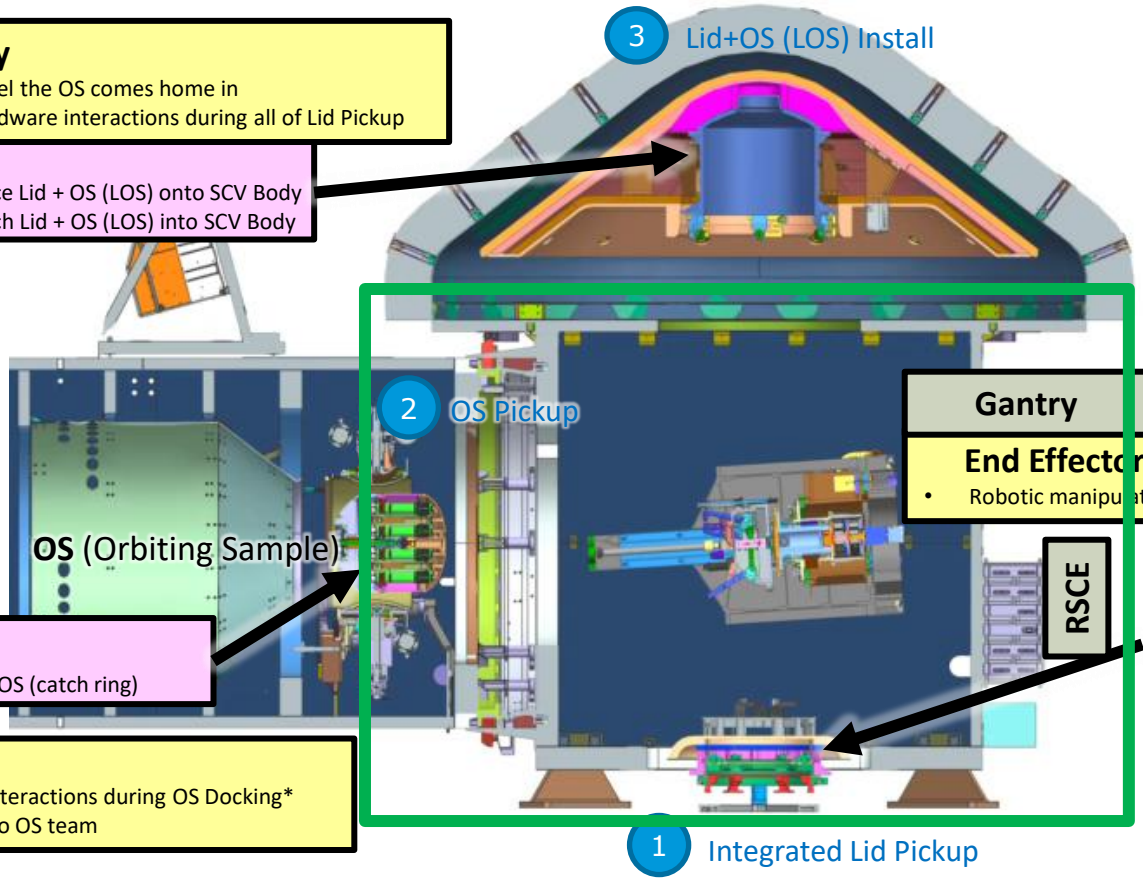
PIE Hardware

RTAS Hardware

Acronyms:
- Robot Interface to Payload (RIP)
- Secondary Containment Vessel (SCV)
- Aerothermal Closeout (ATC)
- SCV-OS Latch, Align, and Restrain (SOLAR)
- Integrated Lid Assembly (ILA)
- Lid Release Mechanism (LRM)
- Orbiting Sample (OS)
- End Effector (EE)

SCV Body
- The vessel the OS comes home in
- Only hardware interactions during all of Lid Pickup

Station 3
- Using EE, Place Lid + OS (LOS) onto SCV Body
- Using EE, Latch Lid + OS (LOS) into SCV Body



Gantry

End Effector (EE)
- Robotic manipulator

Station 1
- Dock EE to ILA
- Release ILA from LRM
- Using EE, Extract ILA from LRM

Integrated Lid Assembly (ILA) on Lid Release Mechanism (LRM)
- Only hardware interactions during all of Lid Pickup
- **ILA**: RIP, SCV Lid, ATC, SOLAR

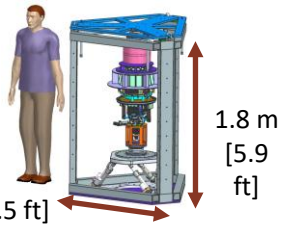
Station 2
- Using EE, Dock ILA to OS (catch ring)

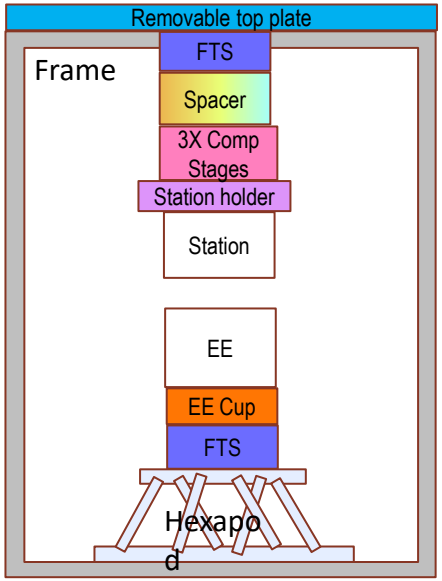
Catch Ring
- Only hardware interactions during OS Docking*
- Provided by PIE to OS team

PIT Overview

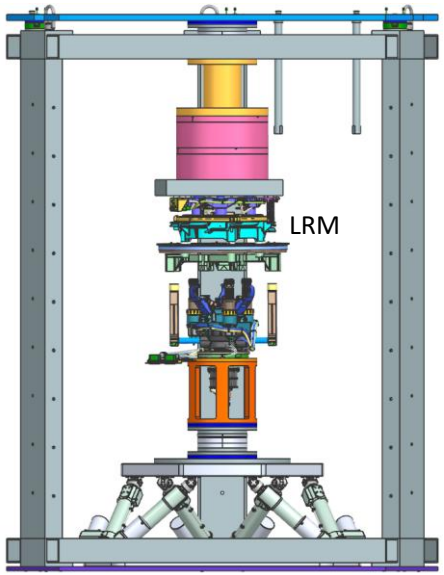
PIT: Pickup and Installation Testbed

Goal: Test and measure station and tool interactions between CCRS end effector and various interfaces given a specific misalignment in **TVAC** environment

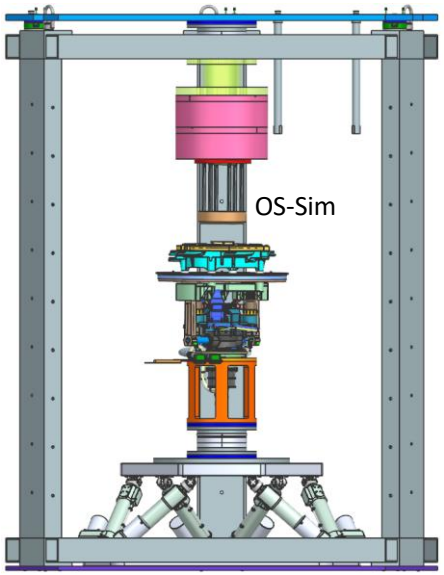




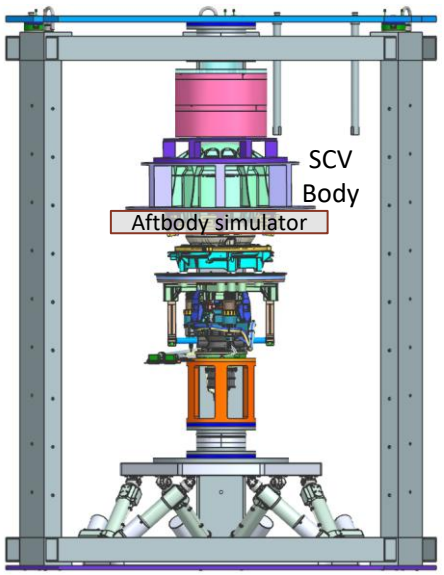
Generalized Configuration



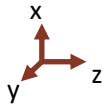
Lid Pickup



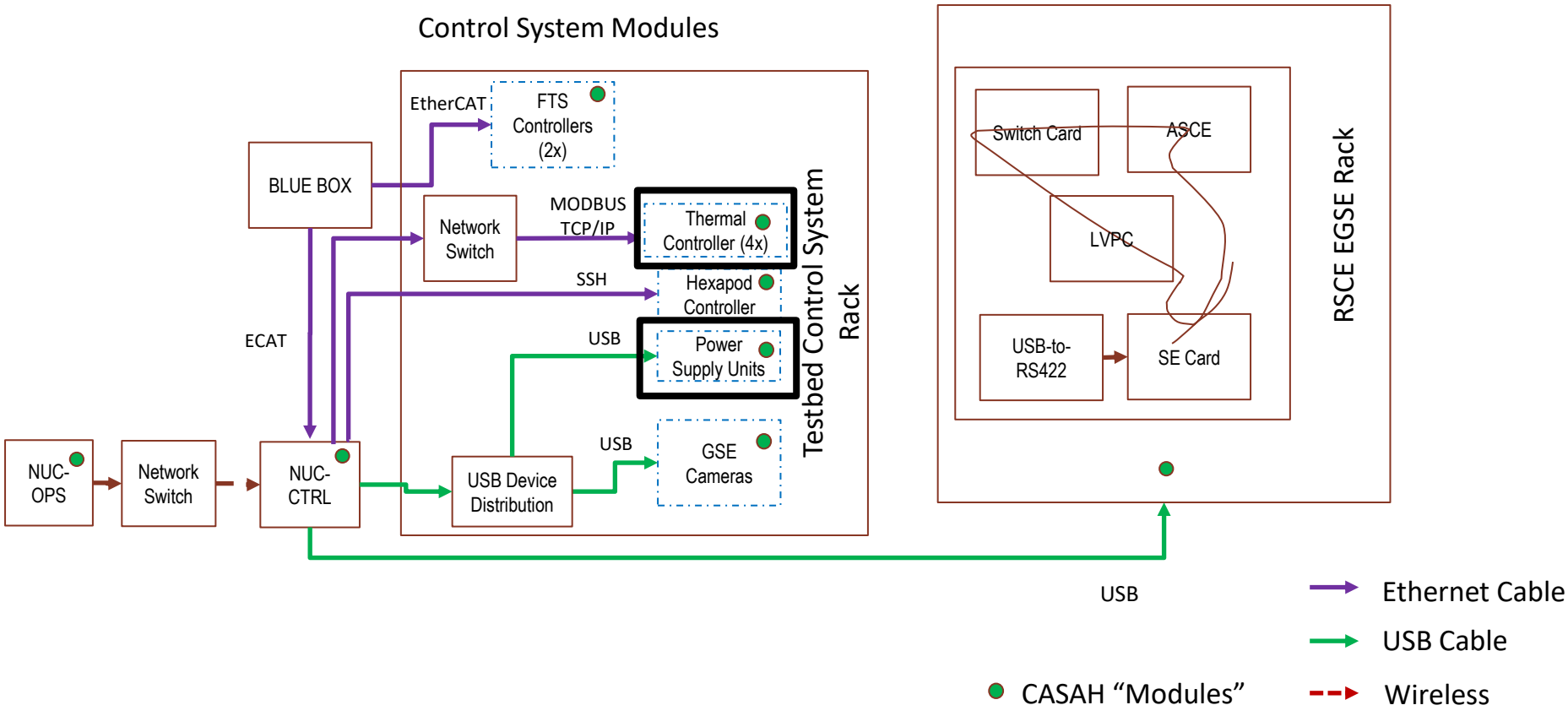
OS Pickup



LOS Install



PIT Software Functional Block Diagram



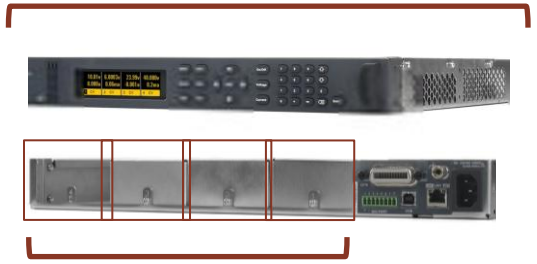
PSU_MGR:
ROS2 Power Supply Package

Keysight Power Supply

- Objective: Create a CASAH Module that can manage multiple Keysight Power Supplies (PSU) with the core functionality of
  1. Initialize PSUs
  2. Query and Publish Telemetry
  3. Allow operators / other modules to
     1. Clear errors
     2. Change channel outputs (ON/OFF)
- Intended use was to turn ON/OFF motors for RSCE Rack Sequencing
- Previous scope included error management -> later moved to FP_MGR

Keysight Series N6700

Mainframe (1x)



Channels (4x)

Nomenclature

- **Module** (CASAH Module): represented by **psu_mgr**, manages multiple instances of a Power Supply Class
- **Mainframe:** Refers to an instance of the Power Supply Class and represents a single mainframe which houses multiple channels
- **Channel:** One of the four smaller power supplies present in a mainframe which have their own voltage, current, power requirements
  - *Assumes each frame has four channels – some frames have two channels combined - have not tested this behavior*

PSU Output Channels

Main Functionality of UPS

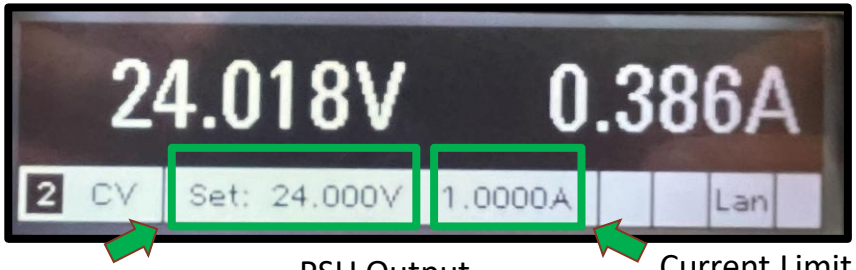
Hardware Pre-existing Functionality:

- Over Protection settings to Protect Hardware
  - Channel will stop outputting during Overvoltage (OV) or Overcurrent (OC) event

Needed Software Functionality:

- Turn on/off Power Supply Channels
- Read/Set Voltage Set Points, Current Limit
- Read/Set Overvoltage Set Points
- Read/Set Overcurrent Set Points
- Read Overvoltage Errors
- Read Overcurrent Errors
- Read Voltage Output
- Read Current Output

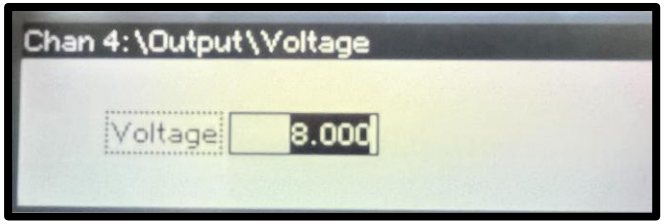
PSU can be separately programmed via. screen



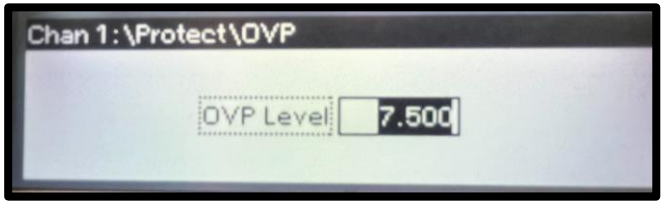
Voltage Set Point

PSU Output Screen

Current Limit



PSU Voltage Settings



PSU Overvoltage Protection Settings

Keysight Power Supply Wrapper

KeysightPowerSupply.py

Instance Data:

- ID
- usb_addr
- output_states []
- currents []
- current_limits []
- voltages []
- voltage_setpoints []
- error []
- error_enum[]
- overvoltage_setpoints []
- overcurrent_enables []
- fault

Functions:

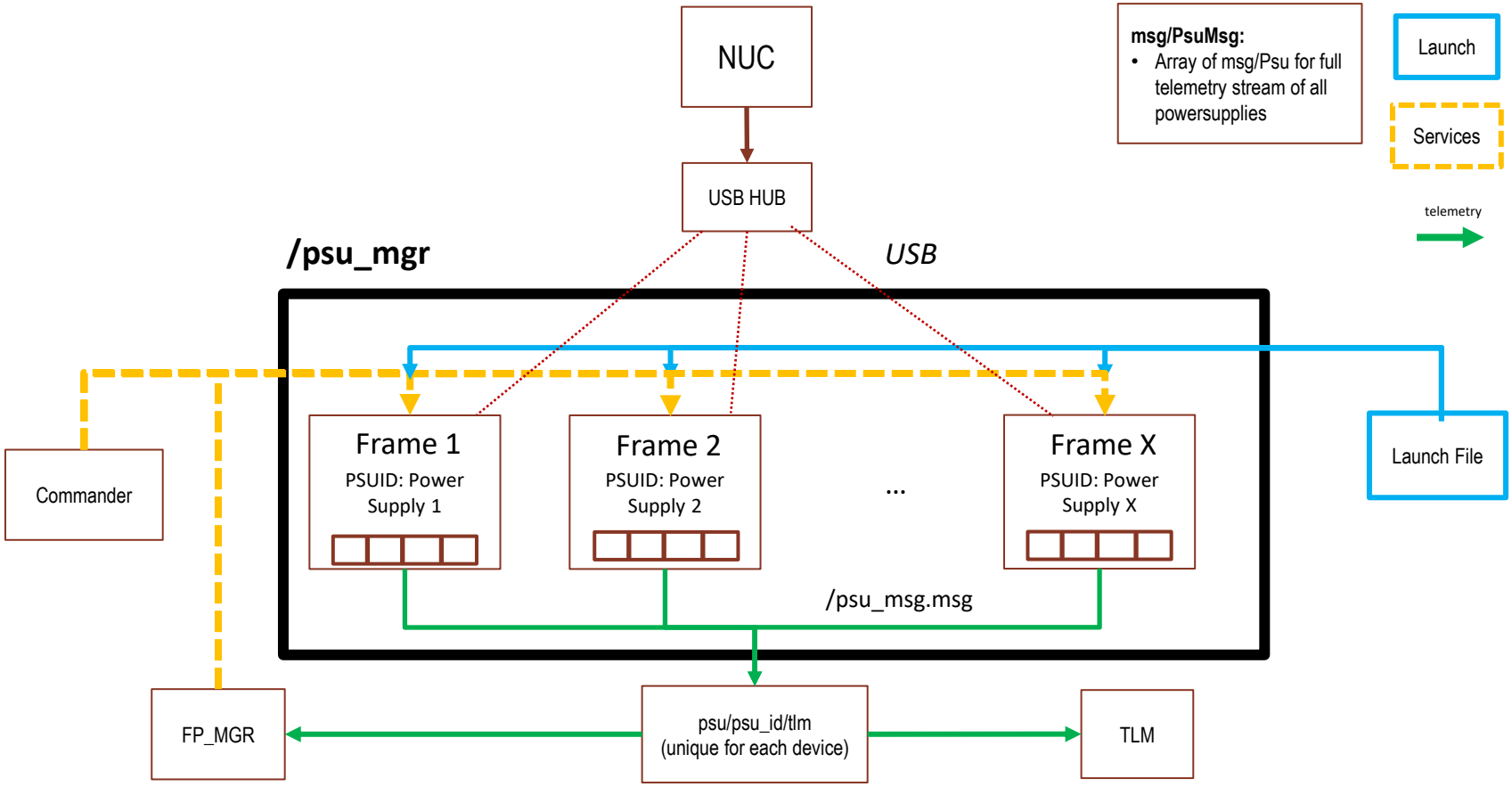
- Set/Read Output States
- Set/Read Current Limit Set Points
- Set/Read Voltage Set Points
- Set/Read Overvoltage Set Points
- Set/Read Overcurrent Enable State
- Read Error
- Read Error Enum
- Read Current Measurement
- Read Voltage Measurement
- Read Fault

- Fault defined as any error on any channel for a single frame

Communication & API:

- Uses Virtual instrument Software Architecture (VISA) API
  - Can communicate using TCP / USB
    - VISA Layer gives specific “VISA” address to hardware
    - NI-VISA Library
- Commands are sent through Standard Commands for Programmable Instruments (SCPI)
  - EX: OUTP ON, (@2)
    - Set channel 2 OUTPUT to ON
- Used **PyVISA** library -> module written in Python
  - Library supports multithreading
- Index of the array corresponds to channel of mainframe

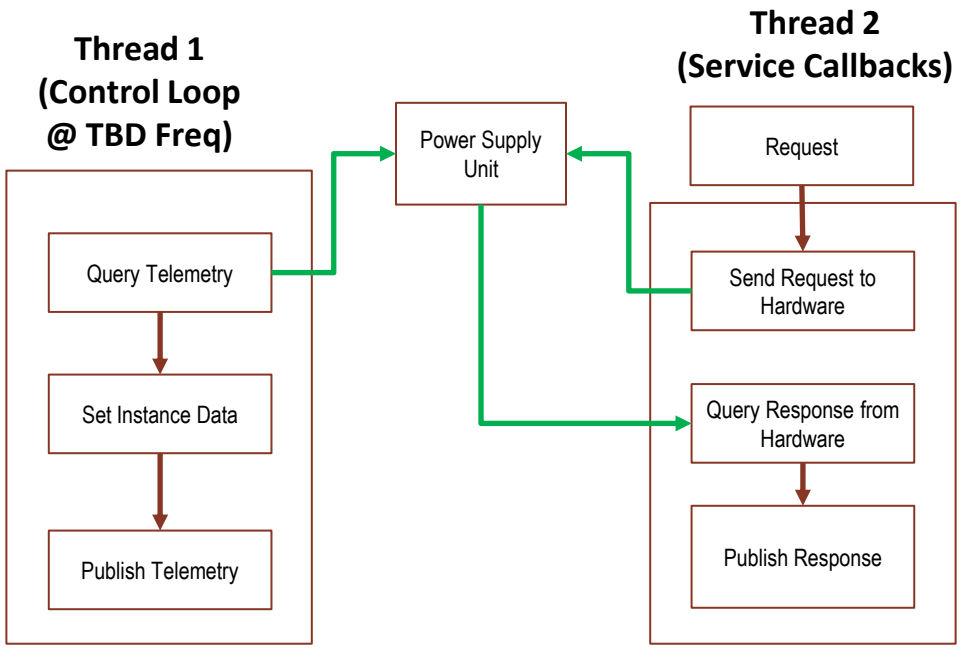
PSU_MGR Setup



Architecture #1: Asynchronous Control, Multithread

Assumed Requirements:

1. Query / Publish telemetry at specific frequency every time
2. Timing of service call timing is not strict; can be performed whenever possible



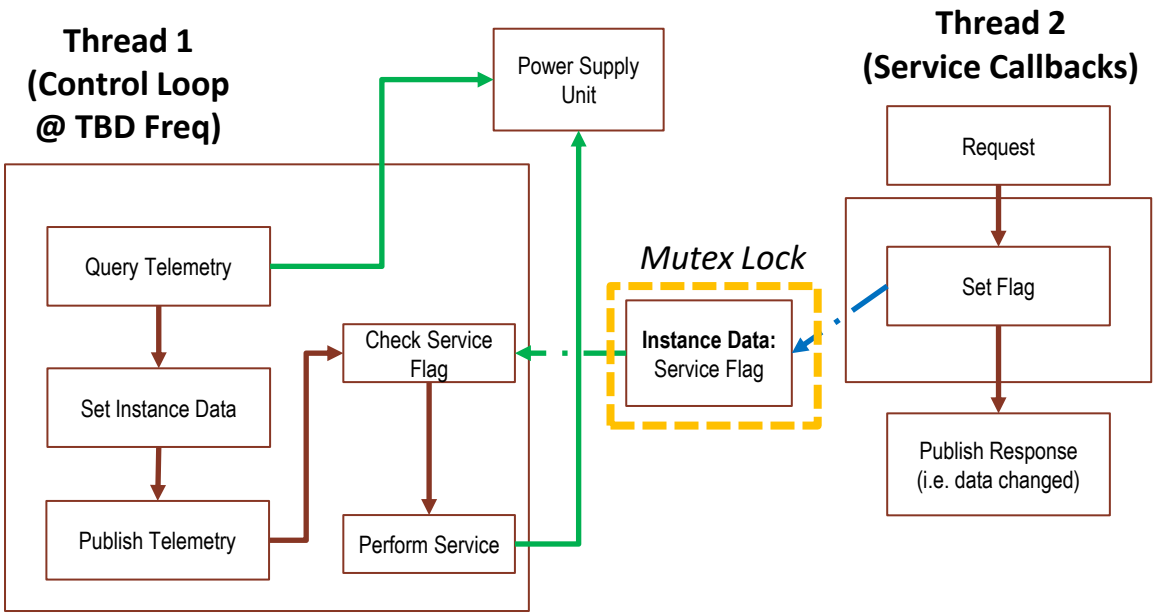
Notes:

- Asynchronous control scheme
- Requires multiple communication interfaces with different threads
  - i.e. multiple TCP Clients to the same server (hardware)
- **Keysight Power Supply** does not support multiple interfaces
  - Tested multiple clients and multithreading
    - Did not work (I/O errors)
  - Vendor claims that the PSU cannot query multiple requests at the same time

Architecture #2: Synchronous Control, Single Loop

Assumed Requirements:

1. Query / Publish telemetry at specific frequency every time
2. Timing of service call timing is not strict; can be performed whenever possible



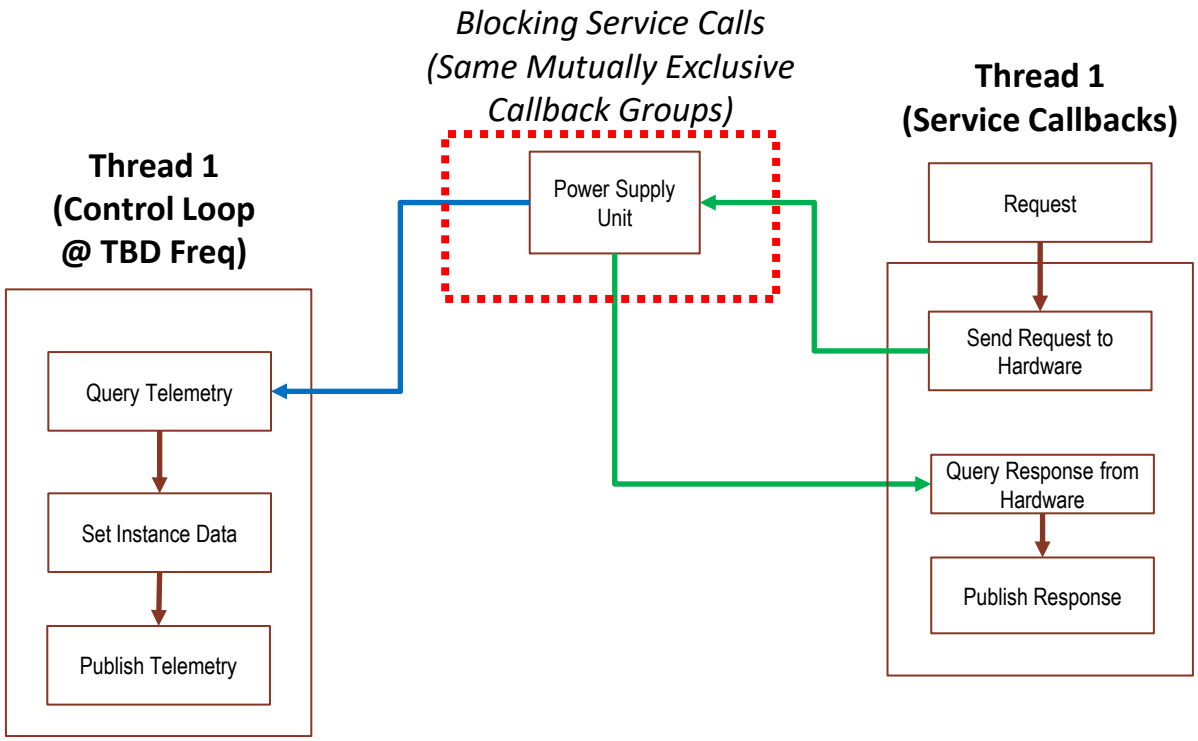
Notes:

- Synchronous Control Loop
- Only one thread can access hardware at a time
  - risks "overrun" in a single cycle if performing service takes a long time compared to required control loop frequency

Architecture #3: Asynchronous Control, Single Thread

Assumed Requirements:

1. Query / Publish telemetry at specific frequency is not critical if **delayed**
2. Service calls should be performed when available (even if blocking)

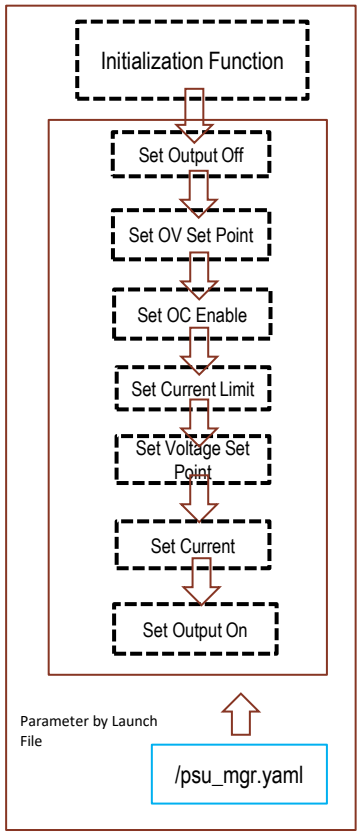


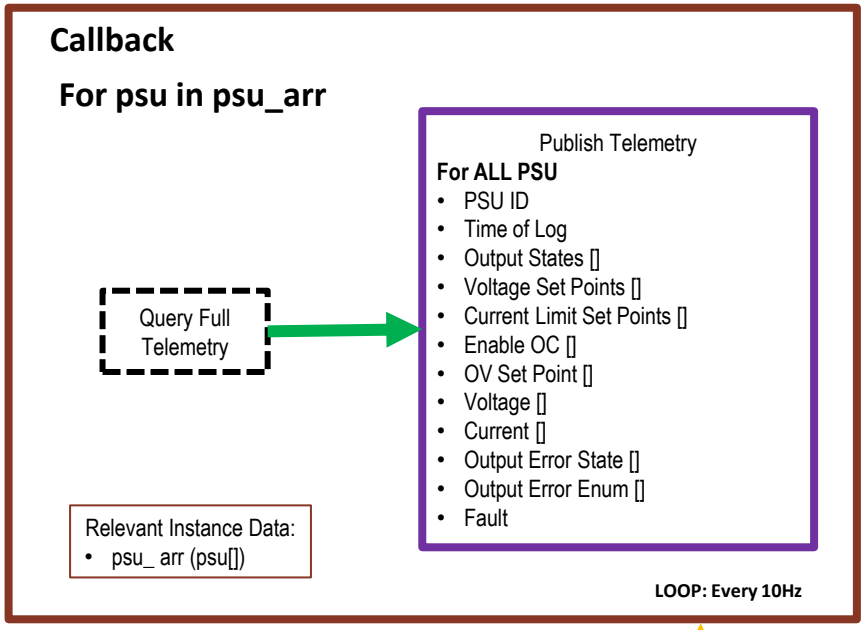
Notes:

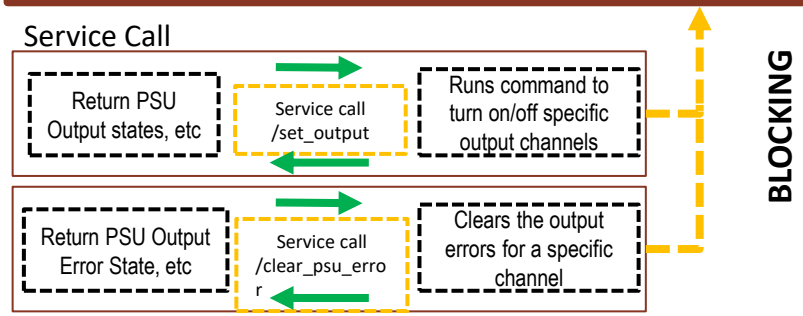
- Asynchronous Control Scheme
  - Only one callback runs and interfaces with hardware at a time
- Assumes that telemetry output can be delayed if service call takes too long
  - Need requirements on control loop frequency
- **Chosen architecture since hardware does not support multithreading**

State Machine for Single Power Supply Node

On Node Launch



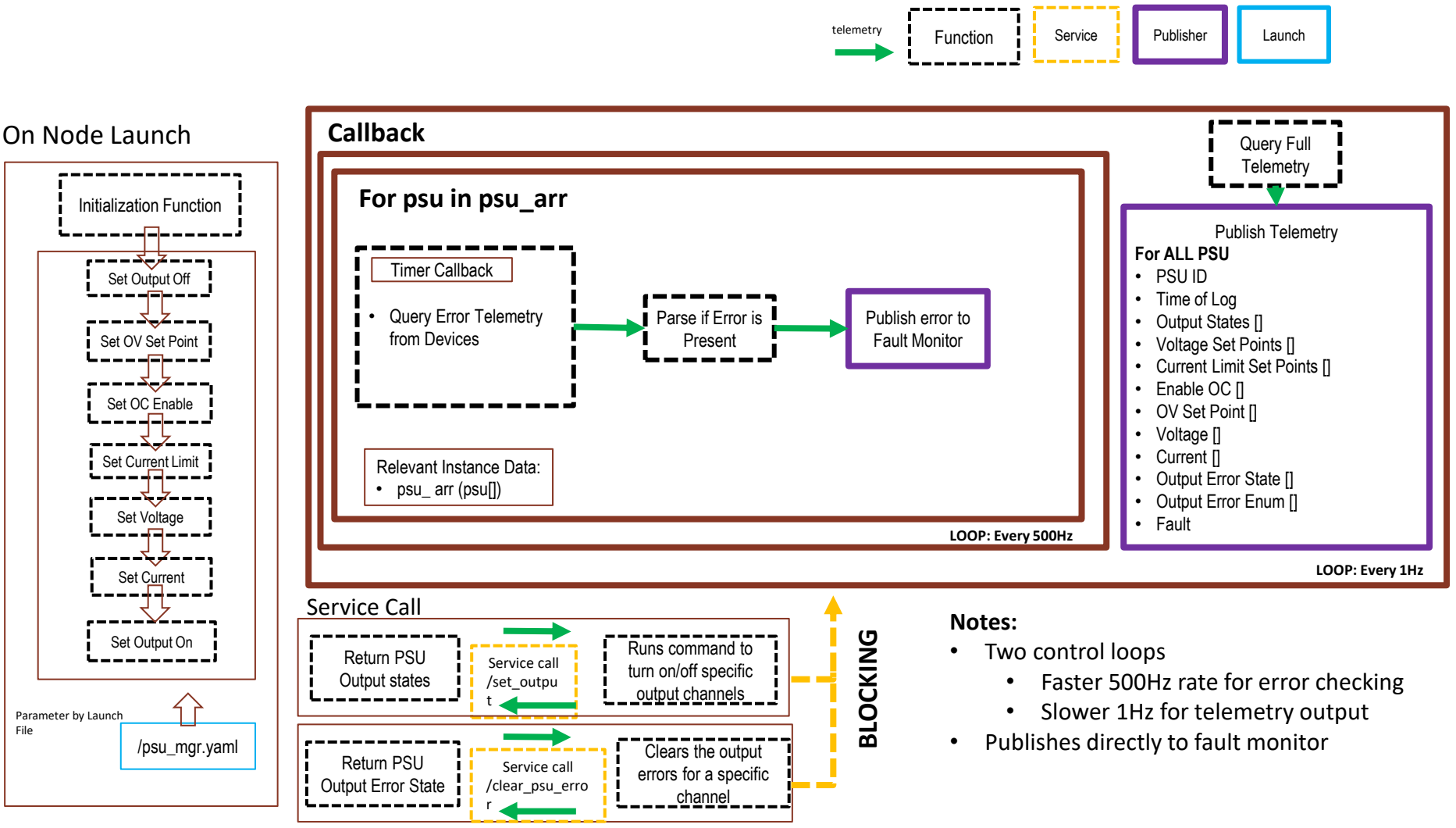




Notes:

- Single control loops through every PSU and generates telemetry messages
- Service calls are **blocking** (mutually exclusive callbacks)
- Output can be changed during a fault via. service call
  - Handled by FP_MGR
- Stress testing with 1 NUC, 2 PSU, USB:
  - **10Hz control loop frequency**

Alternative Architecture



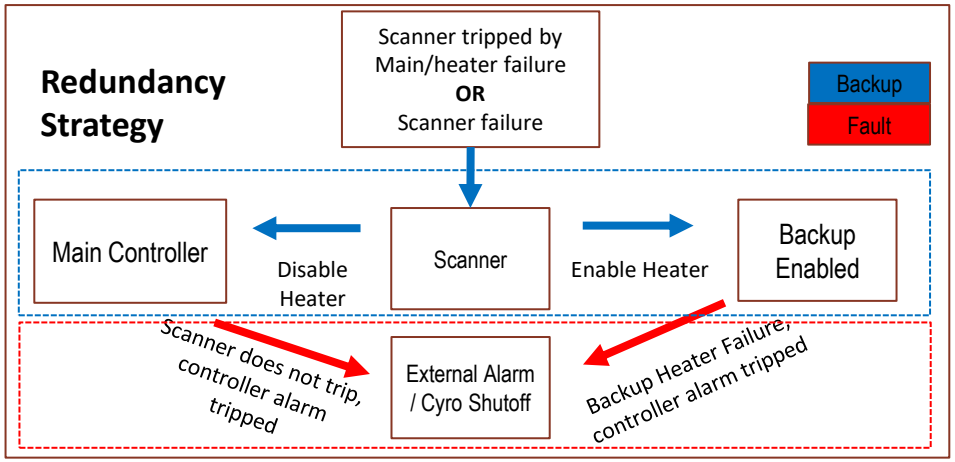
Notes:

- Two control loops
  - Faster 500Hz rate for error checking
  - Slower 1Hz for telemetry output
- Publishes directly to fault monitor

THM_MGR:
ROS2 Thermal Controller Package

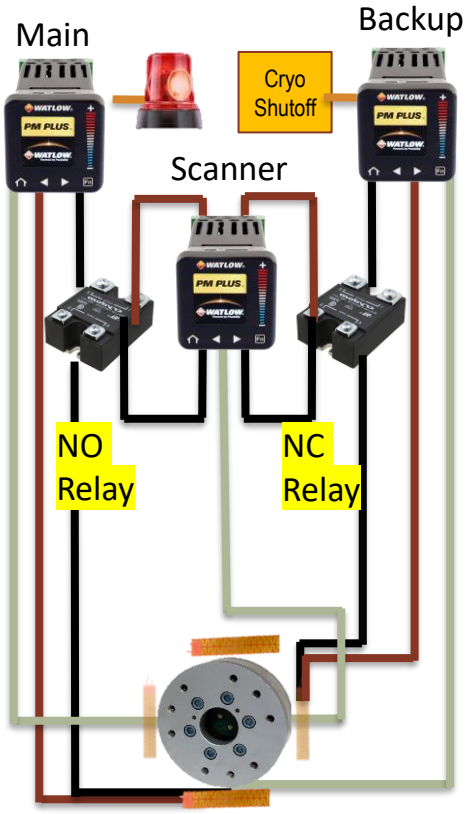
Thermal Manager Objective

- Objective: Create a CASAH Module that can manage multiple Thermal Controllers the core functionality of:
  1. Initializing Controllers
  2. Query and Publish Telemetry
  3. Allow operators / other modules to
     1. Clear errors
     2. Change Alarm Set Points
     3. Change Heating Control Set Point
- Controllers originally to be used for TVAC Testing [-50C to 70C]
  - Heaters and Thermocouples for closed loop control to set point
  - "Thermal Zones" for Single Redundancy
  - Watlow PM PLUS PID & Integrated Limit Controller, Omega Heaters, Crydom DC Relays



Single Failure - Redundancy

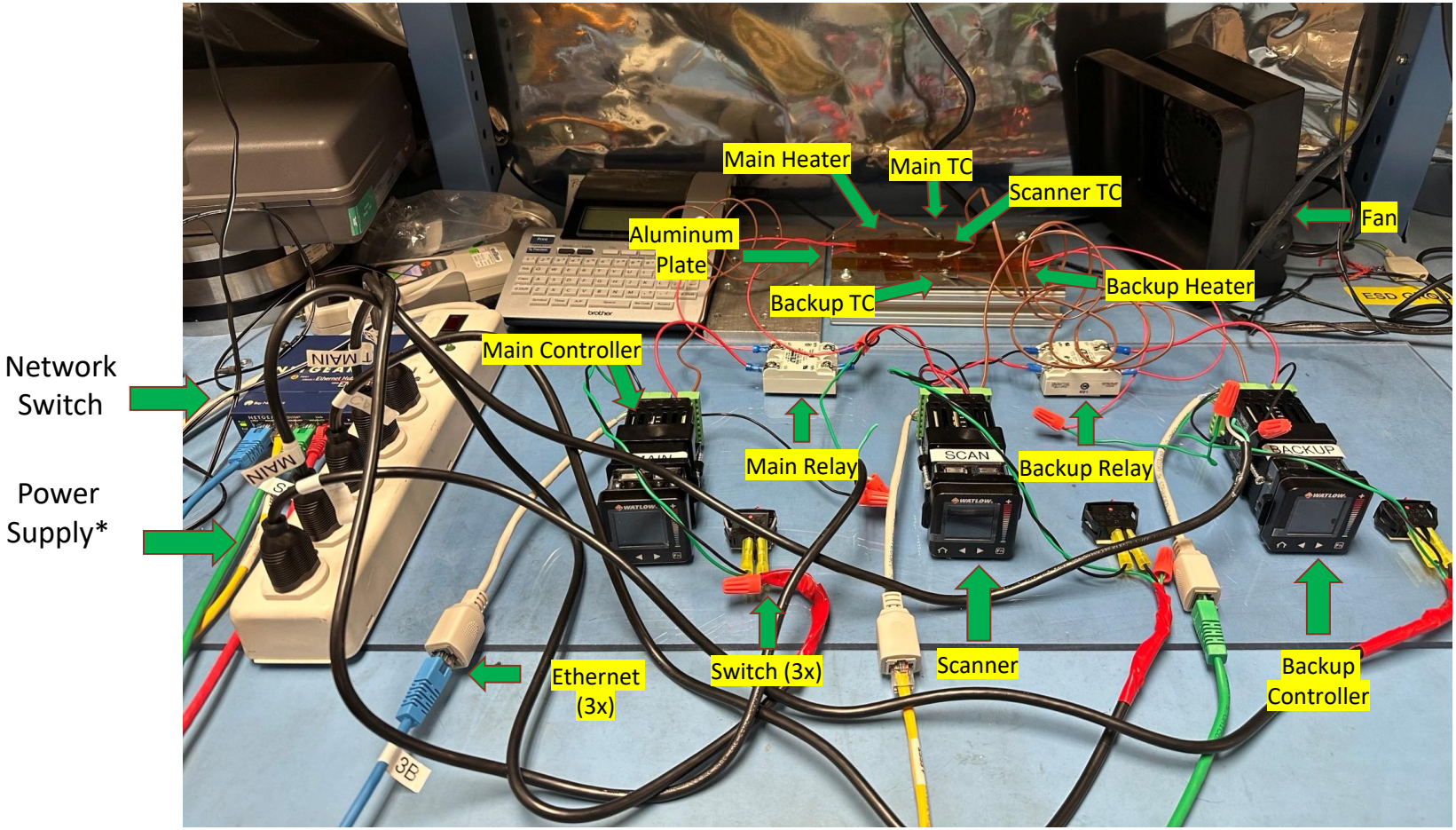
Double Failure - Hardware at Risk



Thermal Zone Concept

FlatSat Physical Setup

* 120VAC or 24V DC



Thermal Controller Required Functions (Manual & Commanded)

Main Controller

1. Set Control Set Point to **T_Set_Point**
2. Set Alarm 1 Set Points to **T_Alarm_Low, T_Alarm_High**
3. Heat until **T_Set_Point** [Output 1]

Scanner

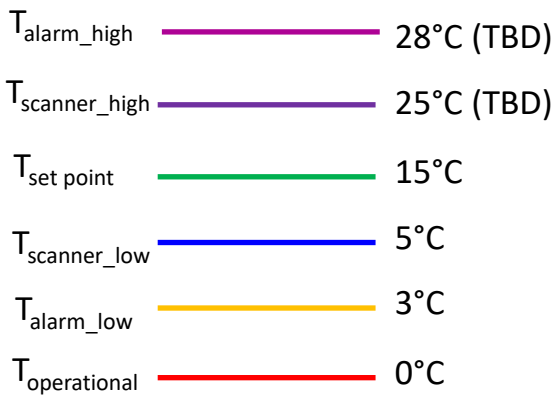
1. Set Alarm 1,2 Set Points to **T_Scanner_Low, T_Scanner_High**
2. Disable main heaters if **T_Scanner_Trip** is reached [Output 2]
3. Engage Relay for Backup Heaters **if T_Scanner_Trip** is reached [Output 1]

Backup Controller

1. Set Control Set Point to **T_Set_Point**
2. Set Alarm 1 Set Points to **T_Alarm_Low, T_Alarm_High**
3. Heat until **T_Set_Point** [Output 1]

Hardware Details

- All heating / alarm control is done by controllers onboard processing
- Controller sends PWM signals to heaters to reach temperature based on TC readings
  - Closed Loop (PID)
- Each controller has **2x** outputs, alarms
- If alarm is triggered, relays enable / disable outputs with latching
- Alarm set points must be set during operation
  - Otherwise, alarm will trip when trying to reach control loop set point



Main Functionality of THM_MGR

Hardware Pre-existing Functionality:

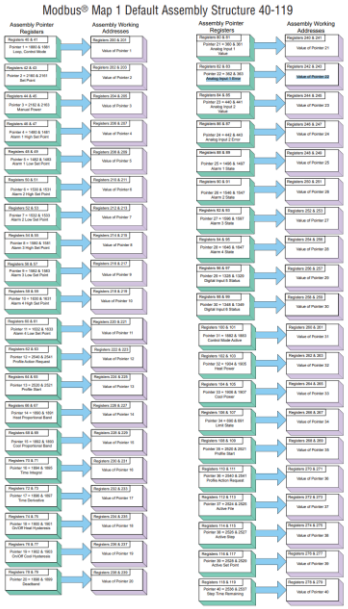
- PID Heating
- Alarm Output Behavior

Needed Software Functionality:

- Set Control Set Point
- Set Alarm (High / Low)
- Read Temperature
- Read TC Error
- Read Alarm 1,2 State
- Read Heat Power
- Read Alarm 1,2 Set Points (high / low)
- Read Control Set Points

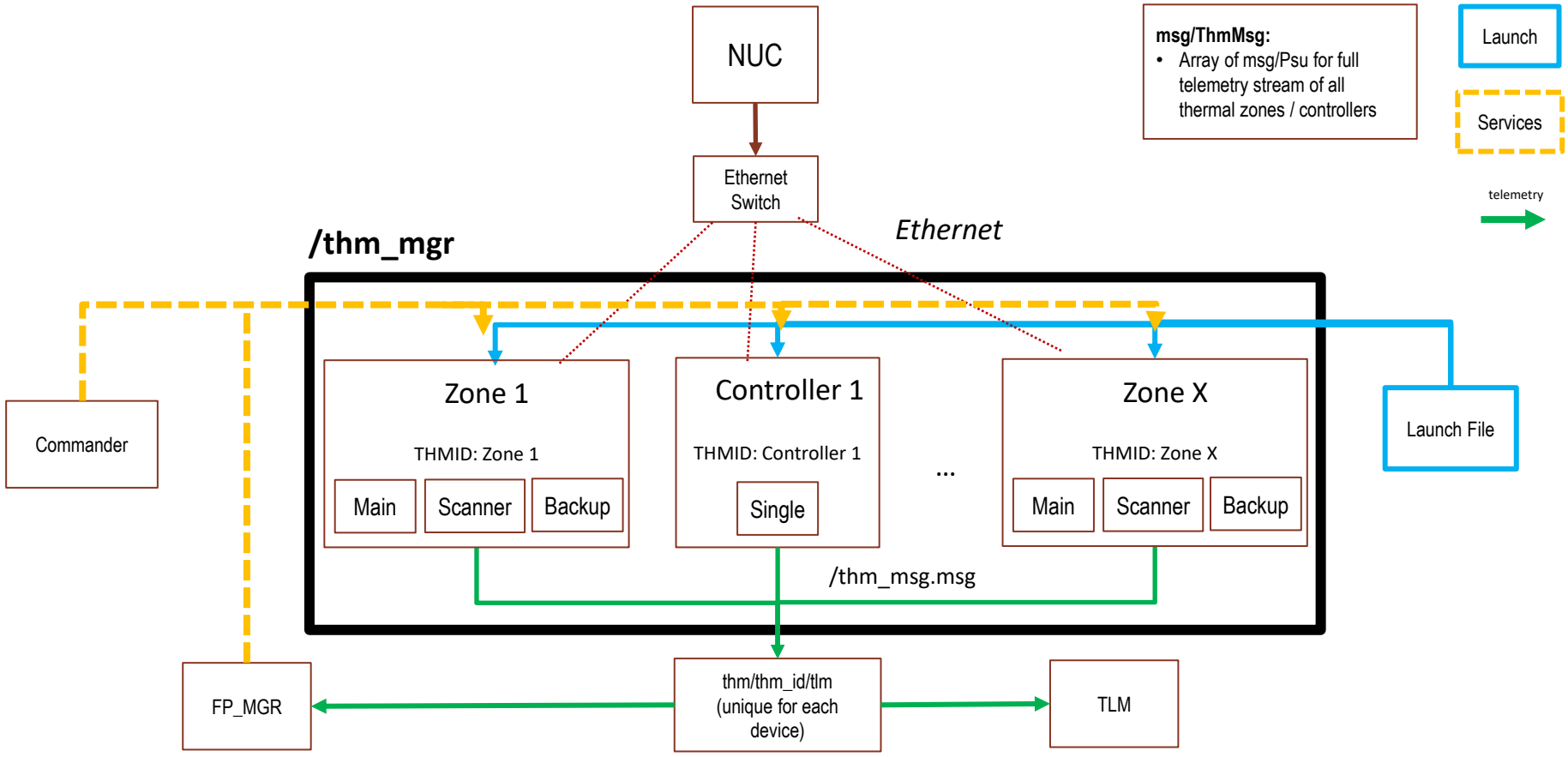
Thermal Controller can be separately programmed via. screen

- Watlow Controllers use MODBUS TCP
  - Write/Read from specific registers (32bit)
  - Using PyModbus library – **does not support multithreading**



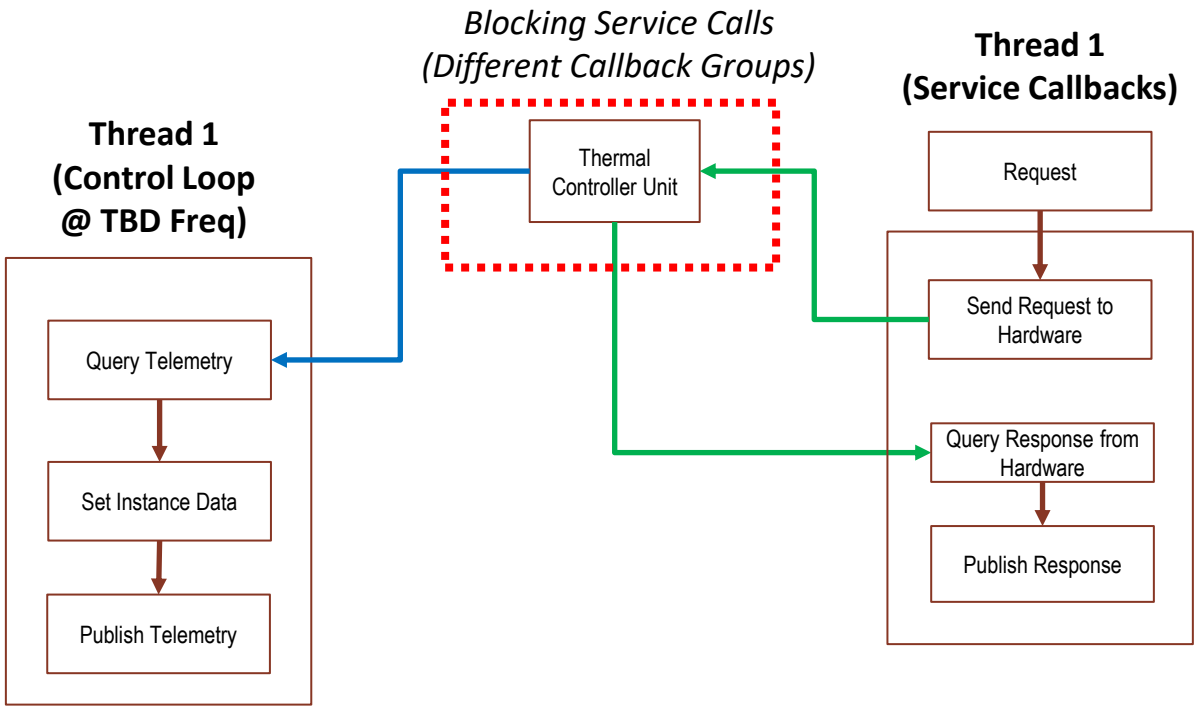
MODBUS Register List

THM_MGR Setup



Architecture #3: Asynchronous Control, Single Thread

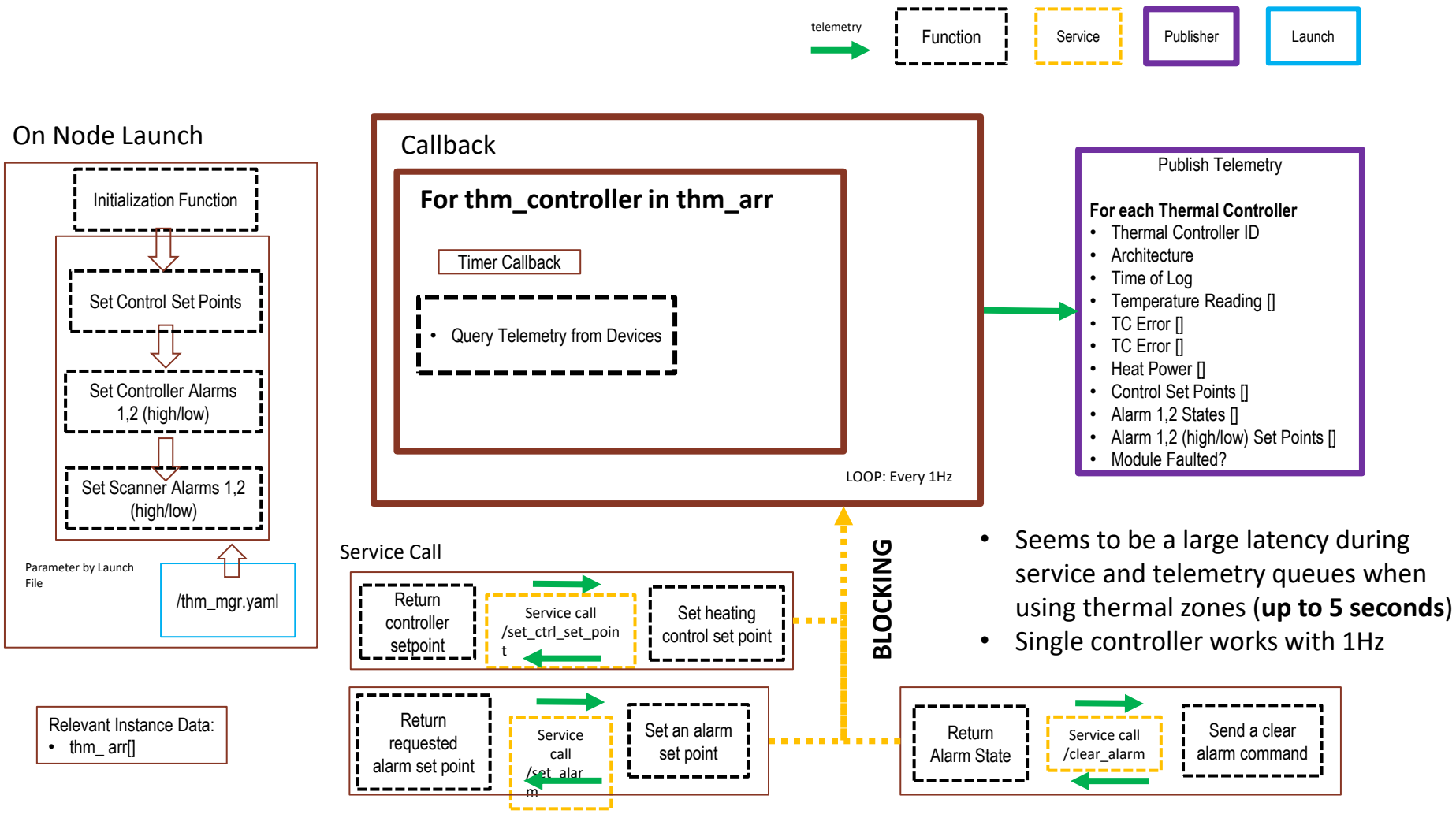
Assumed Requirements:
1. Query / Publish telemetry at specific frequency is not critical
2. Service calls should be performed when available (even if blocking)



Notes:

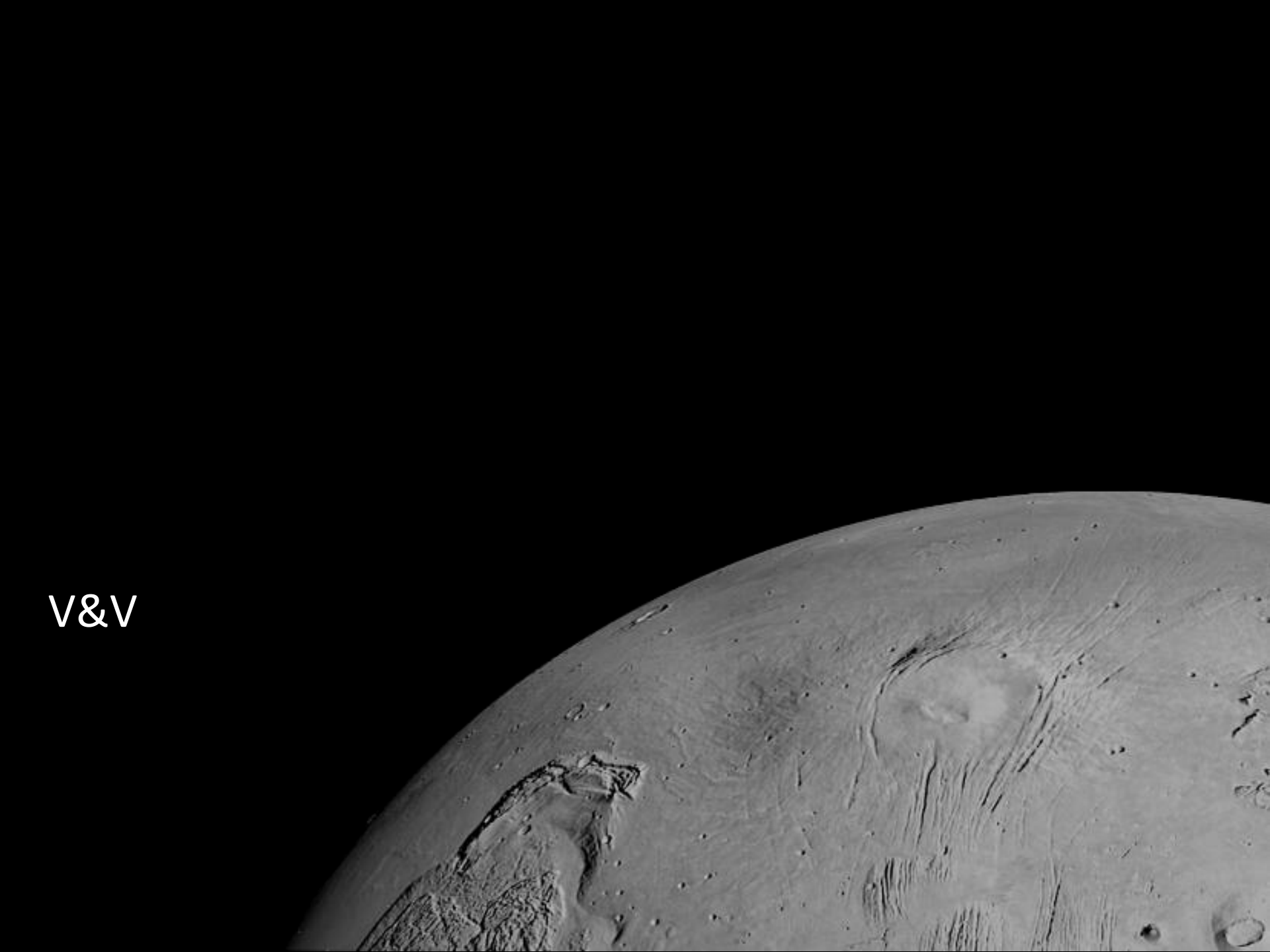
- Asynchronous Control Scheme
  - Only one callback interfaces with hardware at a time
- Assumes that telemetry output can be delayed if service call takes too long
  - Need requirements on control loop frequency
- **Chosen architecture since PyModbus does not support multithreading**

State Machine for Thermal Manager



- Seems to be a large latency during service and telemetry queues when using thermal zones (**up to 5 seconds**)
- Single controller works with 1Hz

V&V



hxpd_mgr

- Performed basic checkouts for Symétrie Hexapod
  - Range of Motion
  - Stopping with FTS
- Helped with test procedure / actually interfacing with CASAH operator tools, testbed deployment

End Effector Initial Developmental Testbed

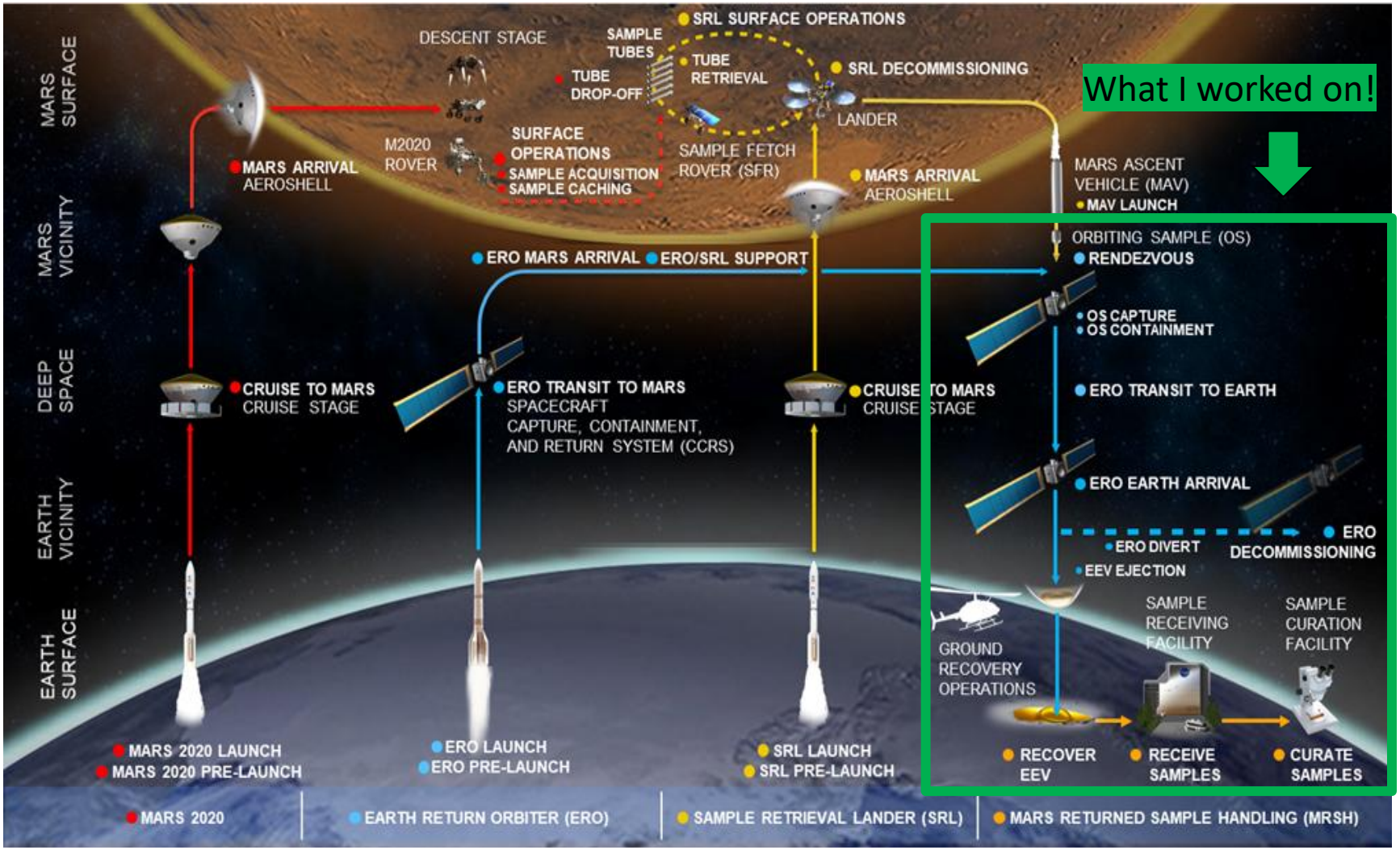
Mars Sample Return Mission

Spring 2020 - Summer 2021

Disclaimer

- The decision to implement Mars Sample Return will not be finalized until NASA's completion of the National Environmental Policy Act (NEPA) process
- This document is being made available for information purposes only
- The information presented has been approved through export control and has been released to be shown to the public

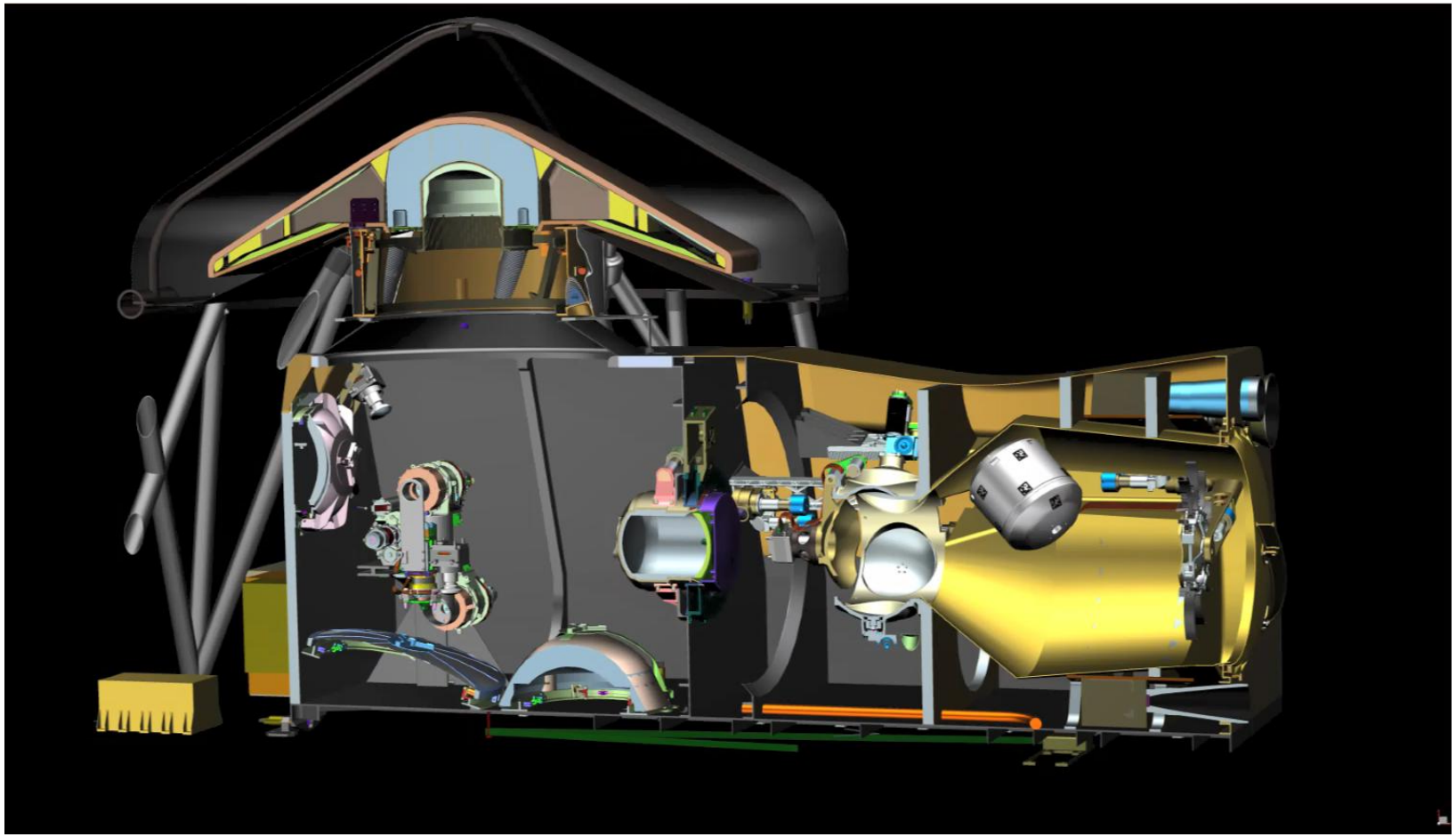
Mars Sample Return Planning Overview



Pre-Decisional Information – For Planning and Discussion Purposes Only

CCRS Containment Operations

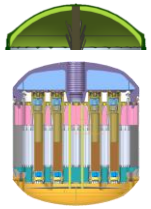
The CCRS Capture and Containment Module uses an end effector on the Robotic Transfer Arm to perform a series of assembly tasks to contain the OS, assemble the OS into the EEV, and maintain the Earth clean zone



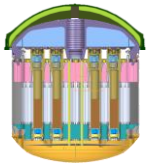
Artist's Concept

Assembly Operations

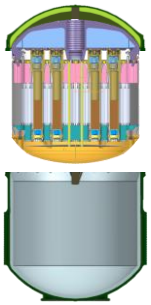
OS = Orbiting Sample
PCV = Primary Containment Vessel
SCV = Secondary Containment Vessel
CAM = Containment Assurance Module
EEV = Earth Entry Vehicle



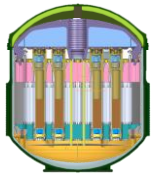
1. PCV Lid aligned with OS



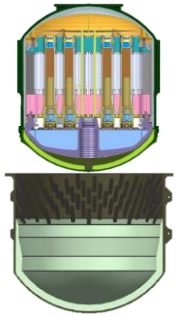
2. PCV Lid mated with OS



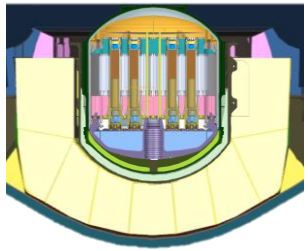
3. OS aligned with PCV Base



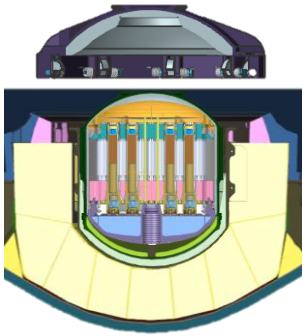
4. PCV Base and Lid mated and brazed



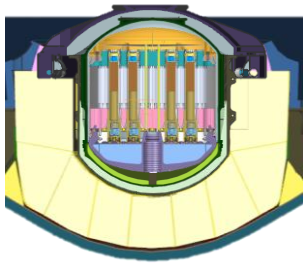
5. PCV aligned with the SCV Base



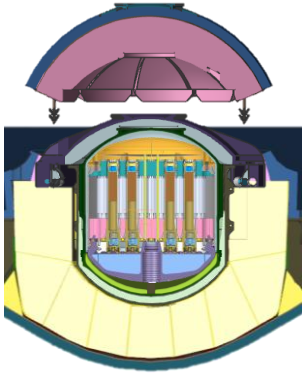
6. PCV inserted into SCV Base/EEV



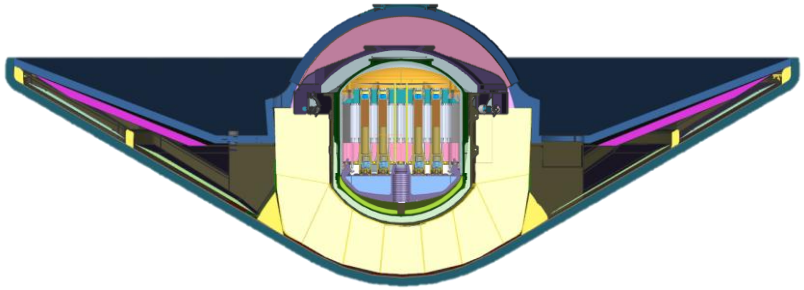
7. SCV Lid aligned with the SCV Base/EEV



8. SCV Lid mated with the SCV Base/EEV



9. CAM Lid aligned with the EEV

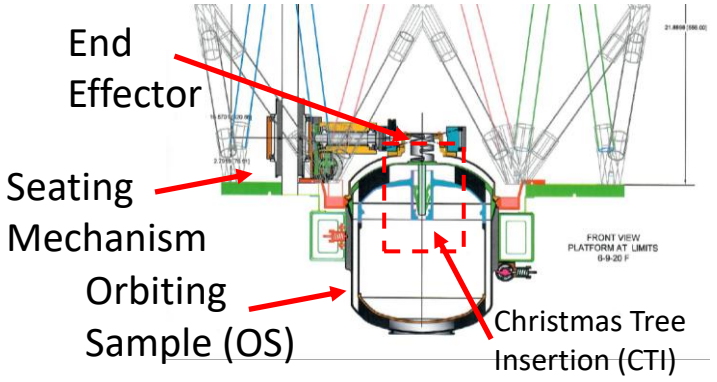


10. CAM Lid mated with the EEV

The Problem: End Effector Misalignment

End Effector Misalignment can be due to a variety of issues such as:

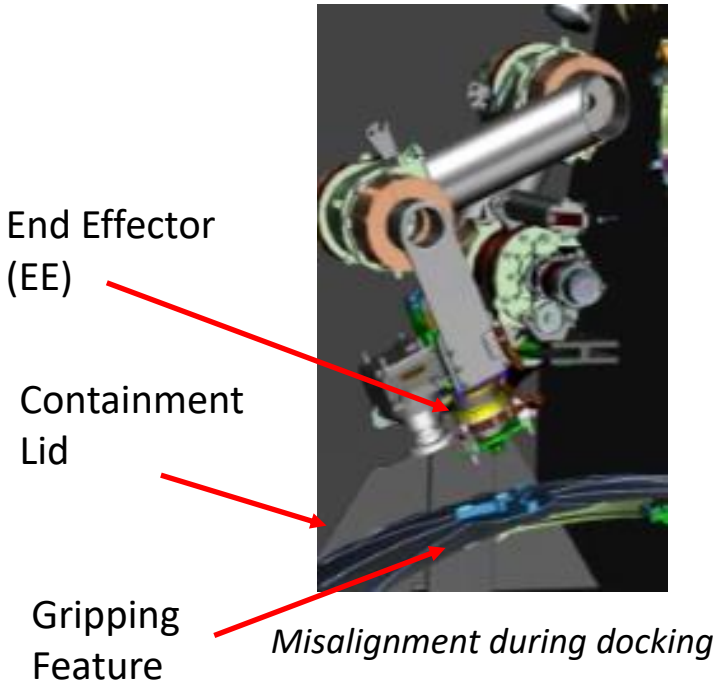
- **Joint errors** (encoder inaccuracy/sensitivity)
- **Kinematic Errors** (model is not 100% true to real geometry)
- **Non-kinematic errors** (backlash, stiffness, temperature effect)



PCV Lid Insertion into OS

Objectives:

1. Create a platform to test behavior of Robotic Transfer Arm (RTA) end effectors when misalignment is present
   - **Not testing static preloading of seals due to higher load requirements**
2. Test the capability of mechanical alignment strategies (e.g., Christmas Tree Insertion, end effector posts)
3. Measure the forces and torques present during docking/insertion procedures



Misalignment during docking

Testbed Requirements

Load Requirements

Function	Force Required (N)	75% Margined Load Required (N)*	Rationale
CTI Insertion	80	140	Load Estimates calculated and provided from previous work. Magnitudes of values quite similar to those presented in MSL Bit Box and M2020 SHA insertion tests.
Braze Insertion	80	117	
PCV Grip	200	350	
PCV Place	200	350	
SCV Lid Grip	200	350	
SCV Lid Place	200	350	
CAM Lid Grip	200	350	
CAM Lid Place	100	175	
ERM Lid Grip	200	350	
ERM Lid Place	200	350	

MSL Bit Box

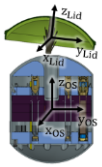
M2020 SHA

* 75% added margin accounts for uncertainty in force required

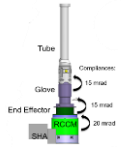
Displacement Requirements

Offset	Required	Rationale
Position (along x-axis)	+/- 10 mm	All of the requirements are based off the SHA EE Misalignments. Each of the following requirements have ≈200% Margin. Additionally, the magnitude of the errors are similar to those present in the MSL Drill and Bit Box Tests
Position (along y-axis)	+/- 10 mm	
Angular (about x-axis)	+/- 2.86 deg	
Angular (about y-axis)	+/- 2.86 deg	
Clocking (about z-axis)	+/- 2.86 deg	

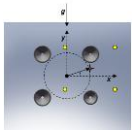
EE Misalignment



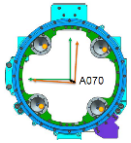
OS Pin Insertion



SHA EE RCCM (M2020)



Drill and Bit Box (MSL)



SCS Bit Exchange (M2020)

Testbed Requirements

Stroke Requirements

Function	Stroke Required (mm)	50% Margined Stroke Required (mm)*	Rationale
CTI Insertion	65.8	98.7	Stroke Estimates calculated and provided from current CAD models of CCRS Architecture.
Braze Insertion	184.6	276.9	
PCV Grip	11.3	17.0	
PCV Place	201.6	302.4	
SCV Lid Grip	10.4	15.6	
SCV Lid Place	88.6	132.9	
CAM Lid Grip	19.5	29.3	
CAM Lid Place	133.5	200.3	
ERM Lid Grip	10.4	15.6	
ERM Lid Place	17.6	26.4	

* 50% added margin accounts for uncertainty in stroke required

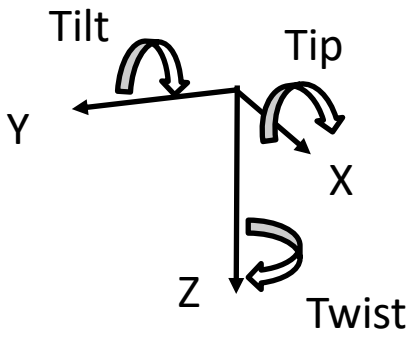
Testbed Requirements

Functional requirements the testbed:

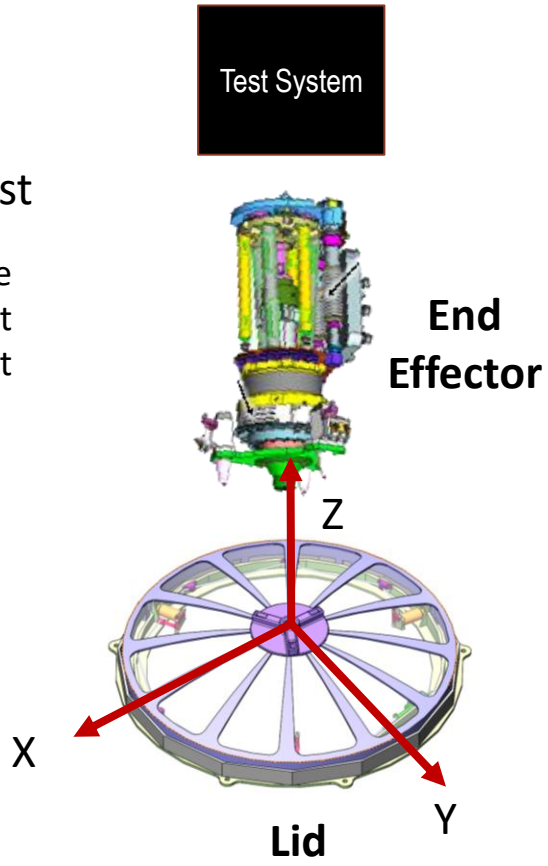
- Apply Force
- Provide Motion (6-DoF)
  - Provide Initial Alignment Error
    - Lateral
    - Angular
    - Clocking
- Measure Forces (6-DoF)

These requirements can be met using:

- Stewart Platform
- Linear Actuator
- 6-Axis Force Torque Sensor

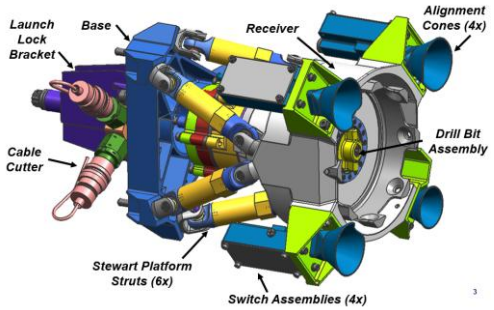


Lateral – XY plane
Angular – XY Tip/Tilt
Clocking – Z Twist

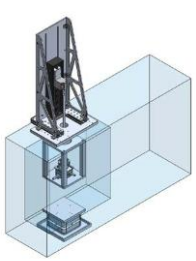


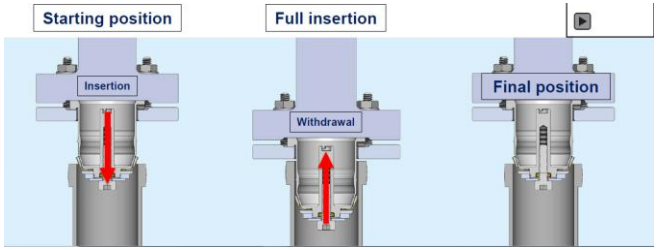
Previous Flight Project Testbeds

MSL Bit Exchange Development Test (2008)

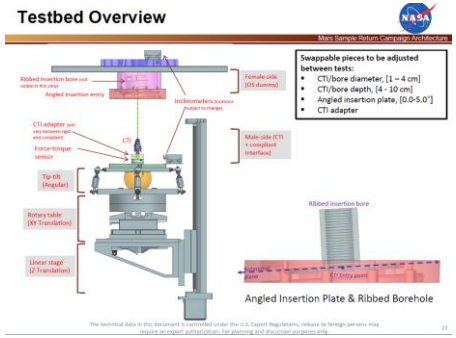


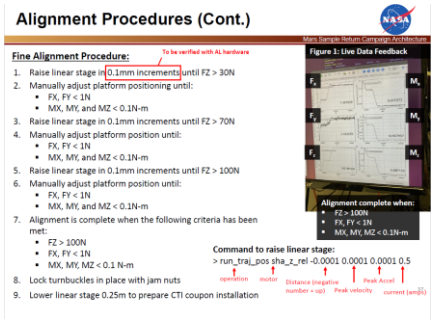
M2020 Tube Retainer Performance Characterization Testing (2020)

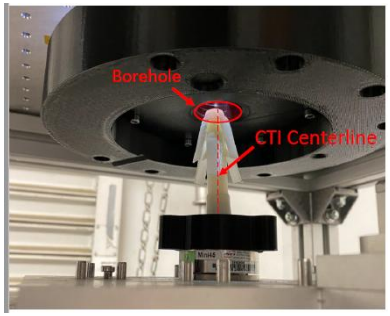


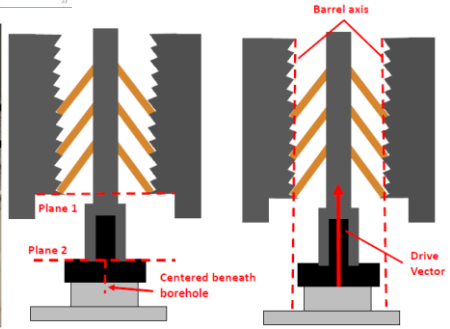


IFACT: Insertion Force & Alignment Characterization Testbed

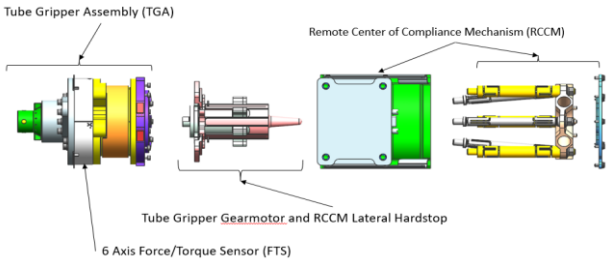


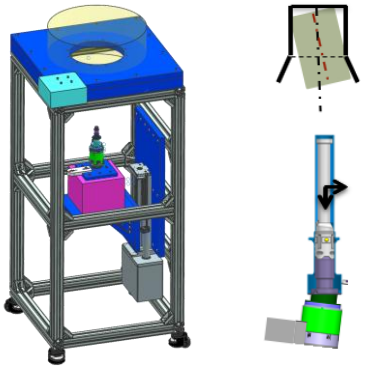




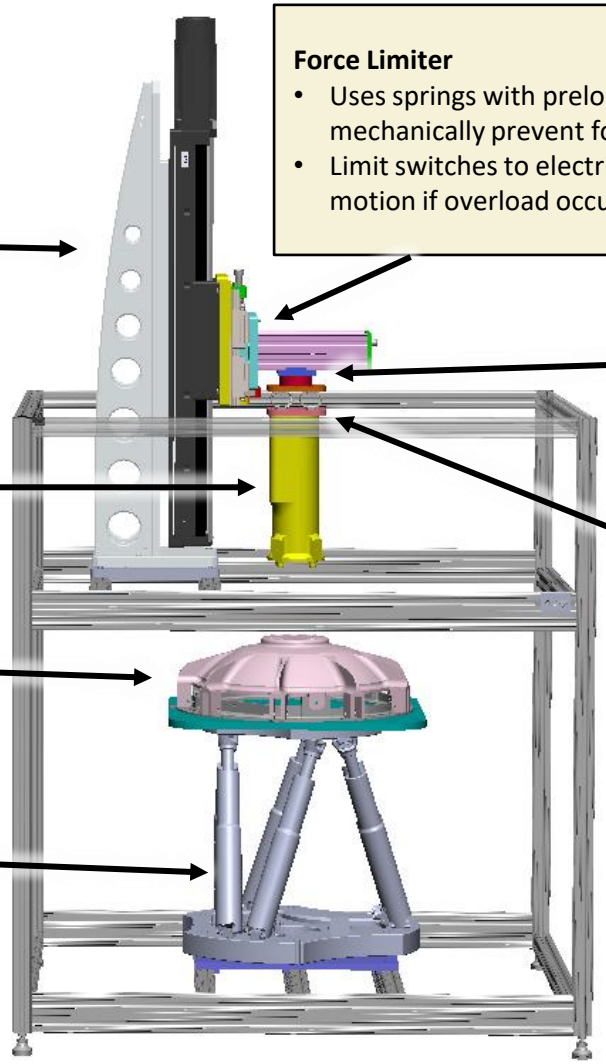


M2020 SHA Insertion/Misalignment Testing (2017)





Testbed Configuration



Force Limiter
- Uses springs with preload in order to mechanically prevent force overload
- Limit switches to electrically stop all motion if overload occurs

Linear Actuator
- Attaches to post assembly
- Provides force and travel in Z-direction

Force Torque Sensor (FTS)
- Records docking Force/Torques between EE and Lid/OS
- Feedback control if overloading

End Effector (EE)*
- Grip/Place various containment lids/OS
- Test alignment strategies (cones/CTI)

Remote Center of Compliance (RCCM)
- Helps to correct for misalignment during insertion
- Final design being produced (currently wire rope isolator stand in)

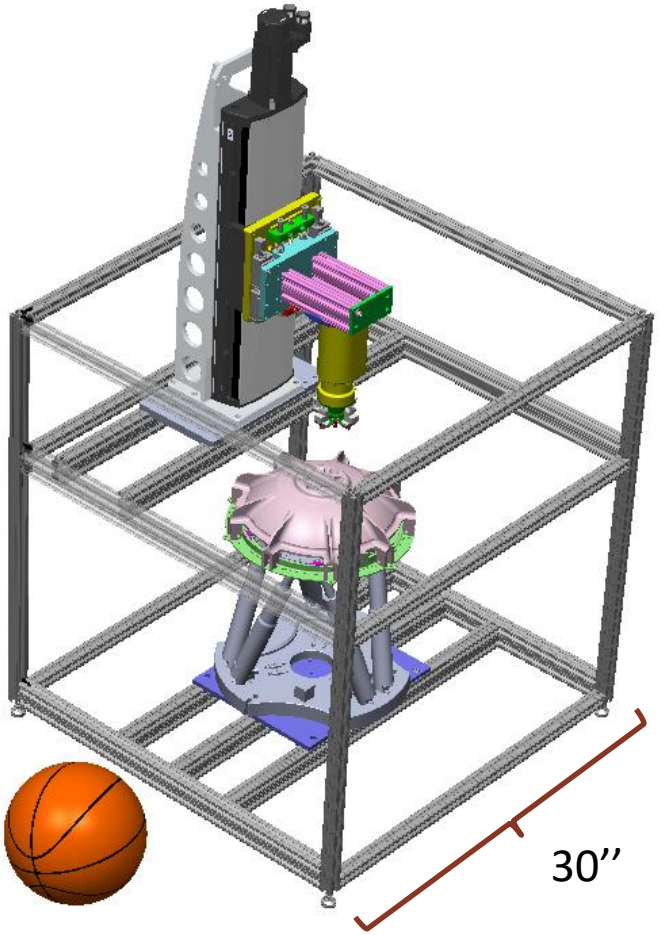
Containment Lid*

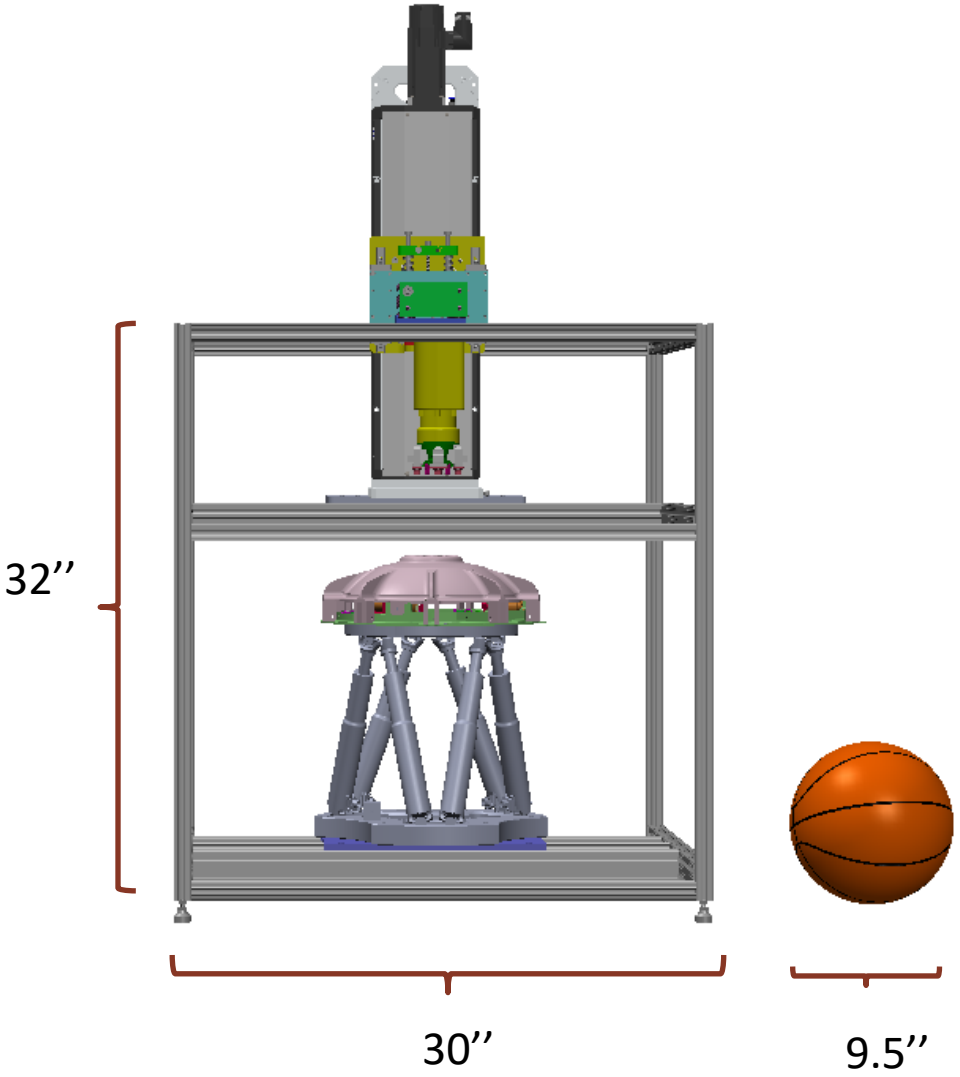
Hexapod
- Movement in 6 DoF
- Mounts SCV (or various) Lids
- Provides misalignment for EE/Lid interface
- Supports EE/Lid/OS interface load

- Conceptualized and designed testbed
- Hardware trade studies
- Obtained quotes from vendors
- Procured parts (BoM), assembly instructions

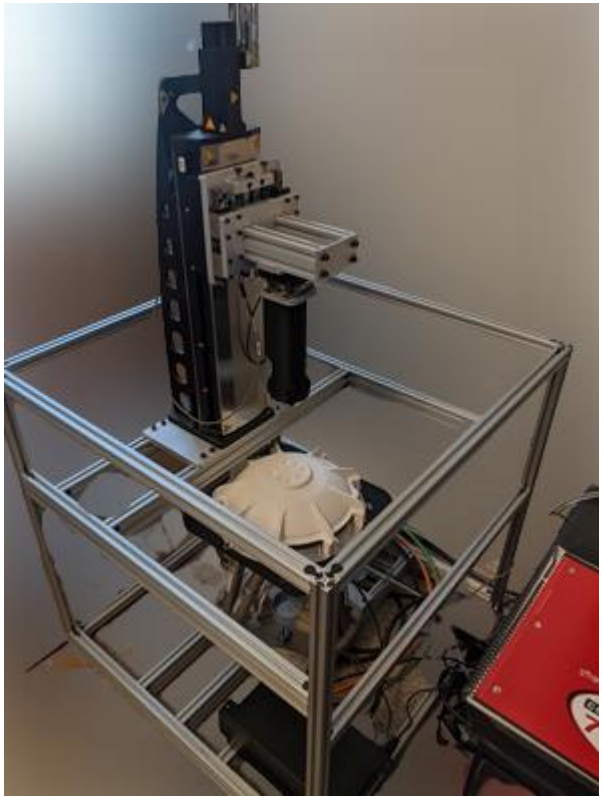
*These pieces are still in development, not final models

Test Bed Dimensions

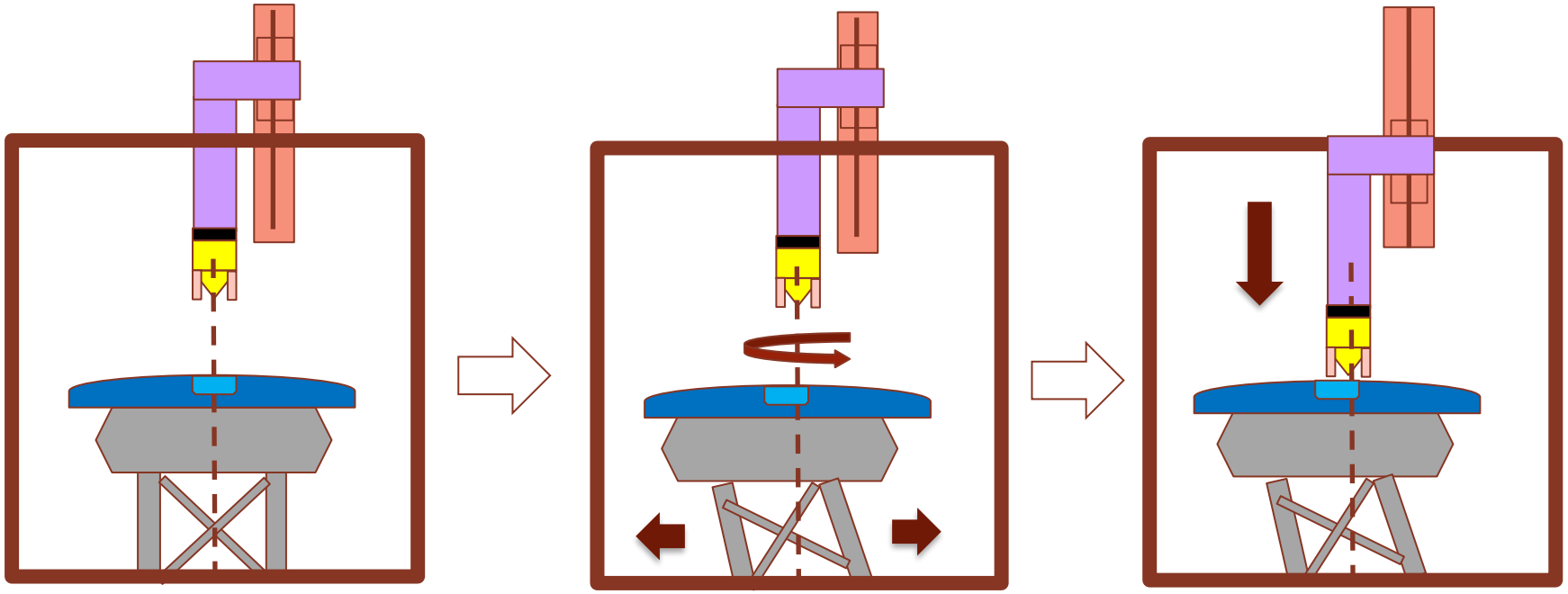




Additional Photos



Testbed Operational Concept



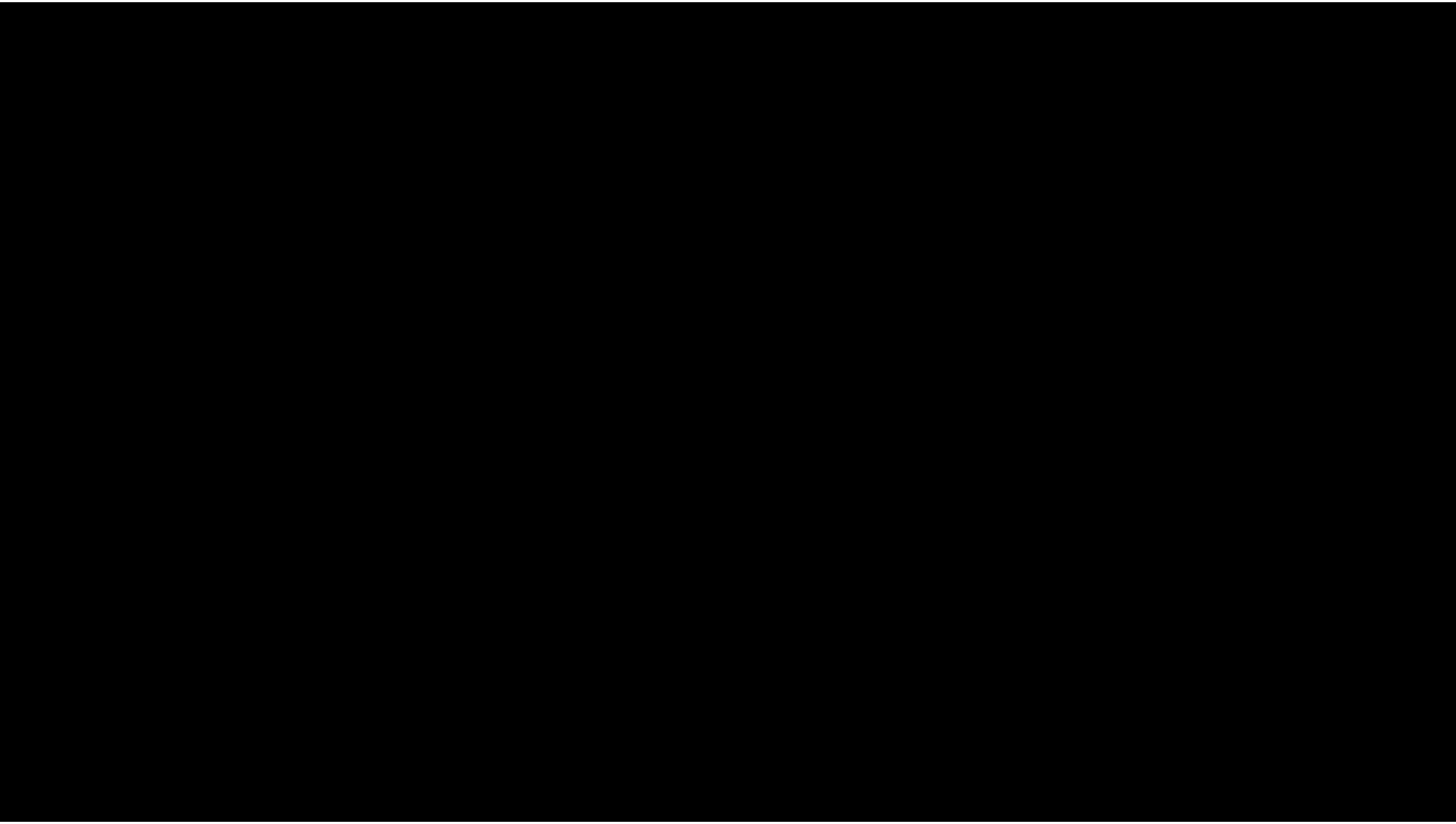
1. Calibrate and align hexapod to end effector

2. Move the hexapod in 5 DoF (lateral, angular, clocking) to create misalignment

3. Move Linear Actuator down vertically, begin docking until
- Load reached (350N)
- Switches Triggered
- Timeout

Record FTS Data and reset alignment

Real Life Photos/Demonstration



Mars Sample Return Handling Concept of Operations

Mars Sample Return Mission

Winter 2021

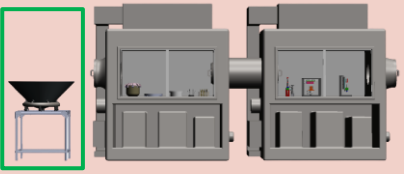
Disassembly Components

EEV Disassembly

COS Disassembly

Tube Disassembly

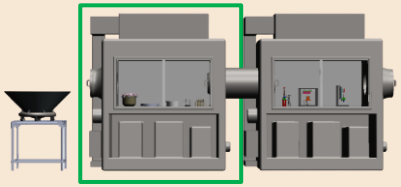
EEV Disassembly



Breach Room

ISO Level: 6
Temperature: 20C
Pressure: 1 atm
Atmosphere: Air
Humidity: 30%

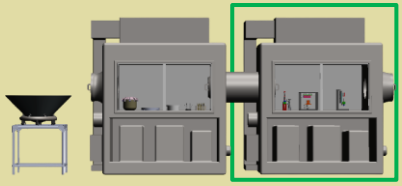
COS Disassembly



First Isolator

ISO Level: 4
Temperature: 20C
Pressure: 1 atm
Atmosphere: Nitrogen
Humidity: 0%

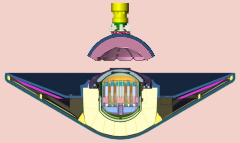
Tube Disassembly



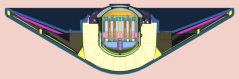
Second Isolator

ISO Level: 3
Temperature: 20C
Pressure: 1 atm
Atmosphere: Nitrogen
Humidity: 0%

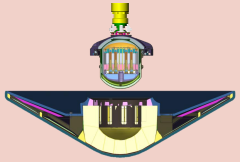
1. CAM Lid retention bolts released



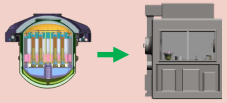
2. CAM Lid lifted away from EEV



3. SCV Base – EEV bolts removed

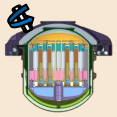


4. SCV removed from EEV

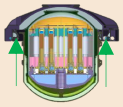


5. SCV moved to first isolator

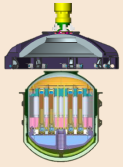
SCV Disassembly



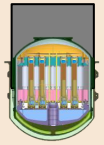
6. SCV has hole drilled to equalize pressure



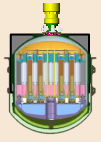
7. SCV Lid latches are pressed to release Lid



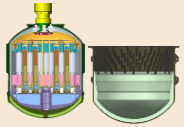
8. SCV Lid is lifted away from the SCV Base



9. Sleeve deflects pawls



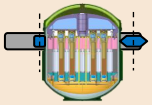
10. COS is gripped by gripper



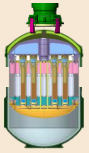
11. COS is removed from SCV Base

PCV Disassembly

12. PCV has hole drilled to equalize pressure

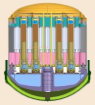


13. PCV cut along some TBD location below the braze line



14. OS is removed from the PCV Base

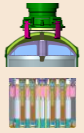
OS Disassembly



15. OS has bottom cap bolts removed

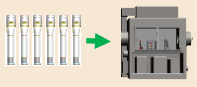
16. OS bottom cap is lifted away from the OS and atmosphere sample is curated

17. OS Lid latches are pressed to release lid

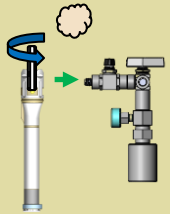


18. OS body removed from OS and PCV Lids

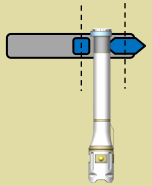
19. RSTA are removed one by one from OS body



20. RSTA are moved to second isolator



21. RSTA are vented and gasses are collected

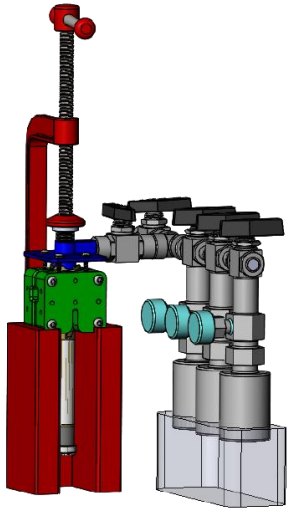


22. RSTA are cut below hermetic seal and above alumina

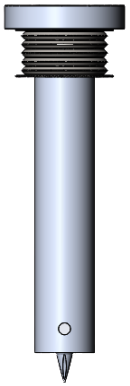
23. RSTA halves are separated and samples are curated

Tube Disassembly Tools

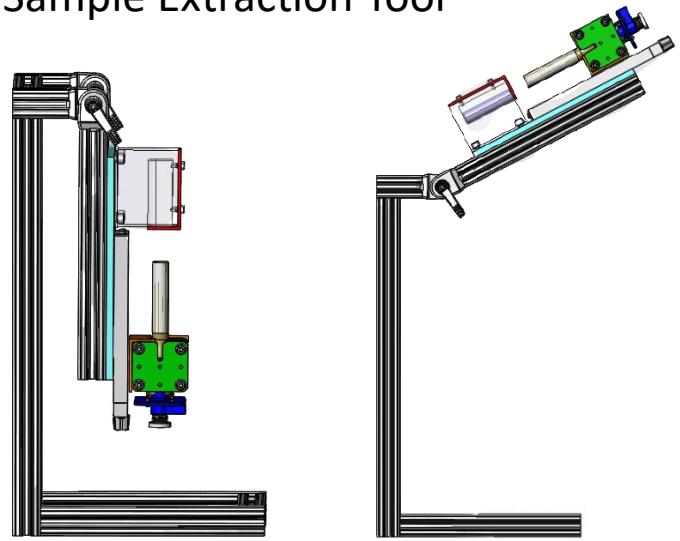
Gas Extraction Tool

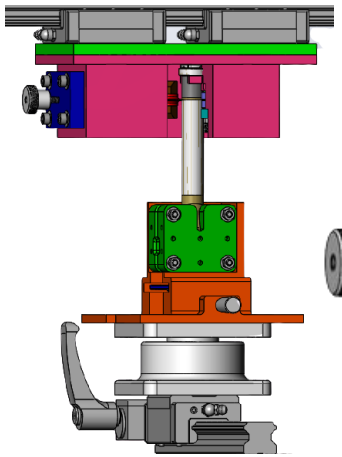


Puncture Tool

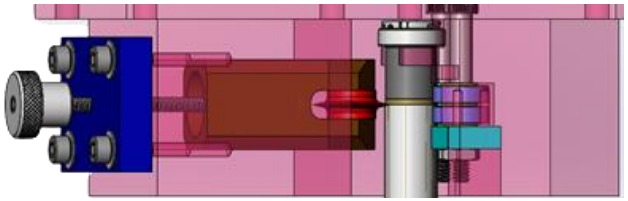


Sample Extraction Tool





Tube Cutting Tool



- Three conceptual tools
  - Demonstrate the various tube opening processes
- Published IEEE Paper!
- High level; no detailed design

Gas Puncture Trade Space

Process	Vibration	Chip/Dust Production	Potential Loss of Gas	Tube Deformation	Sample Composition Affected	Introducing Contaminant	Complexity	Overall Risk
Center Punch with Arbor Press	Medium	Low	Low	Low	No	Low	Low	Medium
Center Punch with Jackscrew	Low	Low	Low	Low	No	Low	Low	Low
Standard Drill Bit	High	High	Medium	Medium	Low	Low	Low	High
Step Bit	Medium	Medium	Medium	Medium	Low	Low	Low	Medium
Rotary Cutting Disc	High	High	Medium	Low	Medium	Low	Low	High
Slide Hammer	High	Medium	Low	Medium	Medium	Low	Low	Medium
Laser Cutter	Low	Low	High	Low	High	High	High	High
Melting and Inserting Tool	No	No	Low	High	Very High	No	High	High

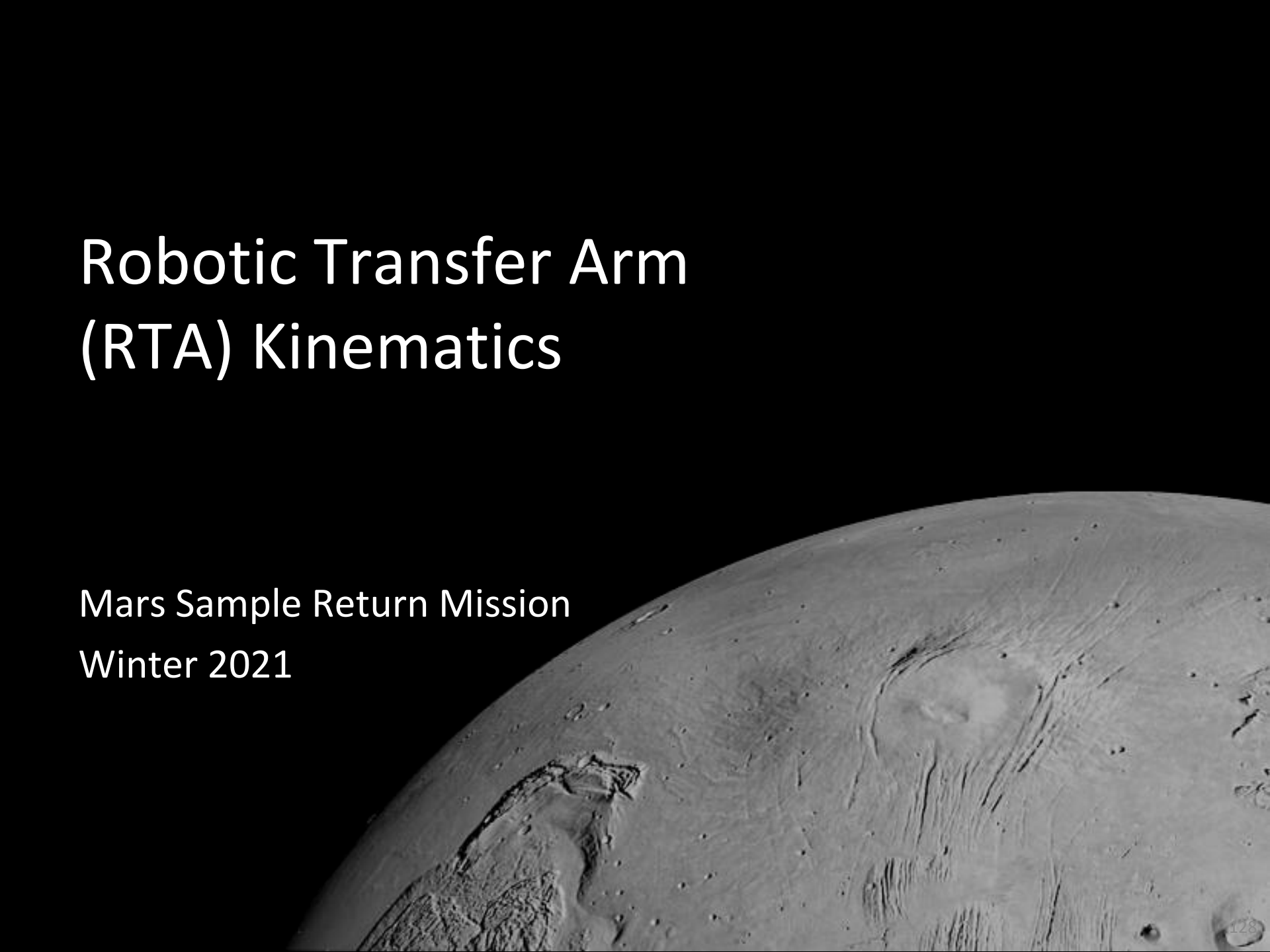
Suggestion: Using a center punch with a jackscrew to create a slow and continuous pressing motion. Small hole the size of tool tip will be created with little chip production for gas extraction

Testing has proven the capability of this tool with arbor press but jackscrew design has not been tested

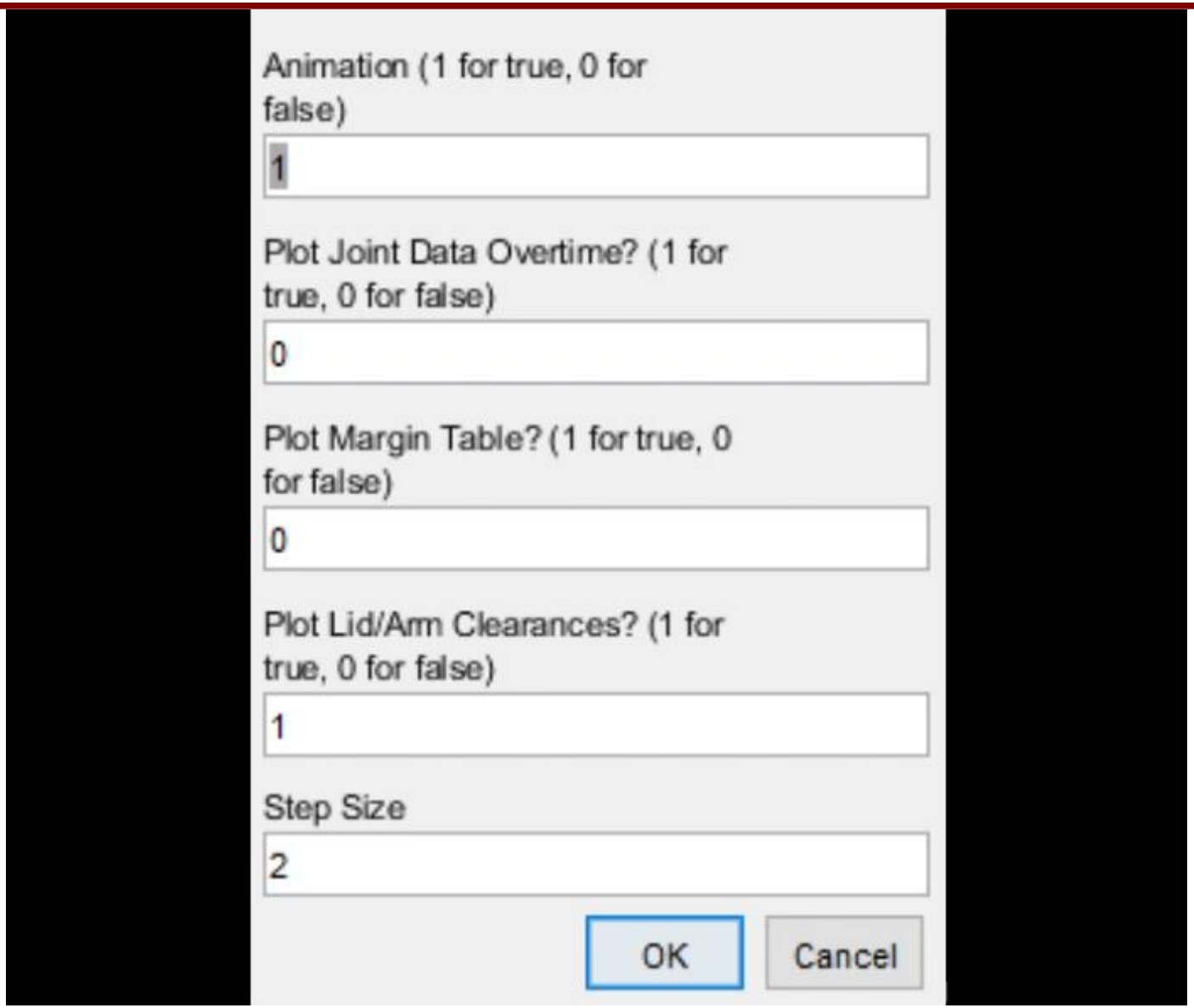
Robotic Transfer Arm (RTA) Kinematics

Mars Sample Return Mission

Winter 2021

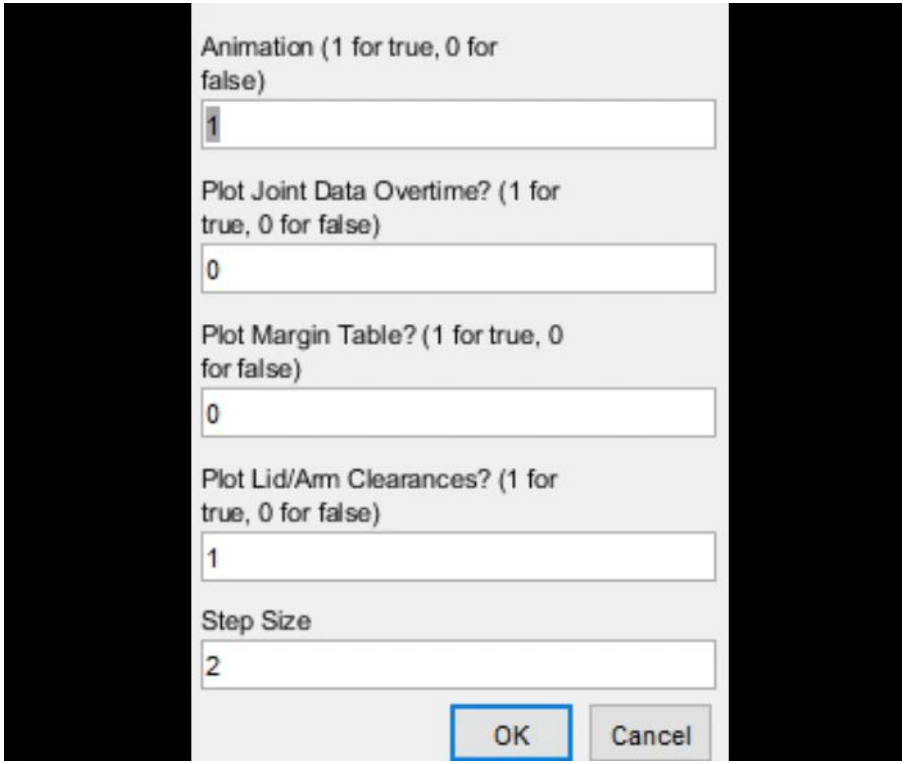


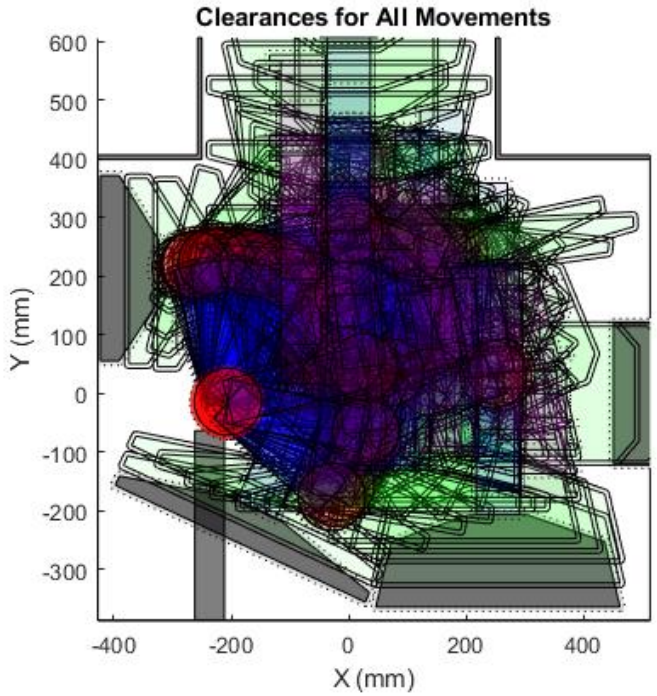
RTA Kinematics Simulation



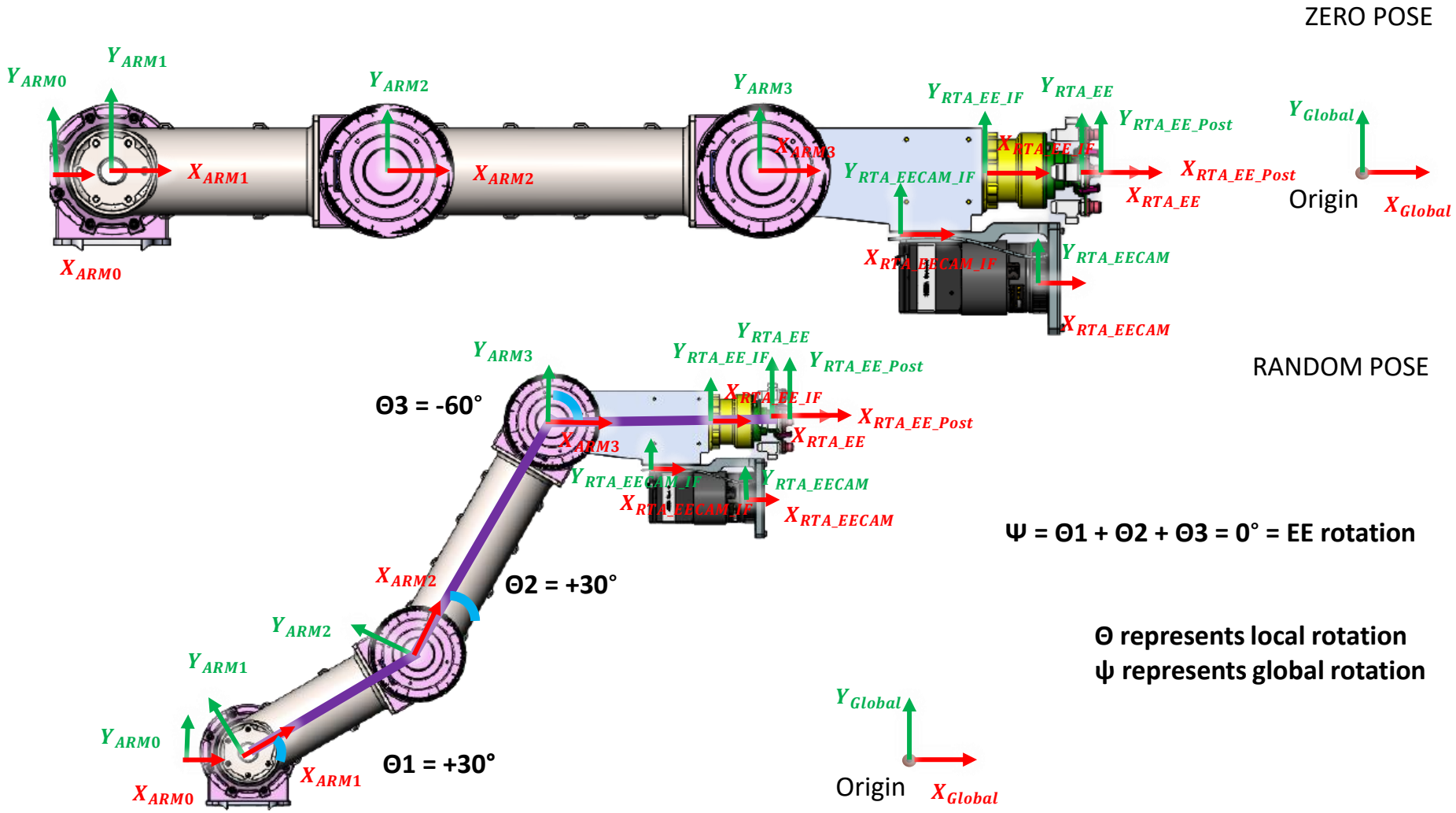
Program Features

- Forward and Inverse Kinematic simulation of 3DoF Planar RTA
- Specific poses, linear path planning, step size, animation, elbow transitions
- Calculates joint torques
  - Assuming arm moves slowly; static analysis
- Clearance checks with NTE Volumes
- Optimizing script to decrease link length and decrease joint torque
  - Parameter Search





Arm Frames


ZERO POSE
Y_{ARM0}
Y_{ARM1}
Y_{ARM2}
Y_{ARM3}
$Y_{RTA\_EE\_IF}$
Y_{RTA_EE}
$Y_{RTA\_EE\_Post}$
Y_{Global}
X_{ARM1}
X_{ARM2}
X_{ARM3}
$Y_{RTA\_EECAM\_IF}$
$X_{RTA\_EE\_IF}$
$X_{RTA\_EE\_Post}$
X_{RTA_EE}
Origin
X_{Global}
X_{ARM0}
$X_{RTA\_EECAM\_IF}$
Y_{RTA_EECAM}
X_{RTA_EECAM}
RANDOM POSE
Y_{ARM3}
Y_{RTA_EE}
$Y_{RTA\_EE\_IF}$
$Y_{RTA\_EE\_Post}$
Θ3 = -60°
$X_{RTA\_EE\_IF}$
$X_{RTA\_EE\_Post}$
X_{ARM3}
X_{RTA_EE}
$Y_{RTA\_EECAM\_IF}$
Y_{RTA_EECAM}
$X_{RTA\_EECAM\_IF}$
X_{RTA_EECAM}
Ψ = Θ1 + Θ2 + Θ3 = 0° = EE rotation
X_{ARM2}
Θ2 = +30°
Y_{ARM2}
Y_{ARM1}
Θ represents local rotation
ψ represents global rotation
Y_{ARM0}
X_{ARM1}
Θ1 = +30°
X_{ARM0}
Y_{Global}
Origin
X_{Global}

MATLAB Joint Optimization

- Created a cost function that seeks to minimize total link length, and ultimately find the lowest torque generated
- Brute force, optimization

INPUTS: Length of Link 1, Link 2, Link 3 as well as Joint 1 X,Y position
OUTPUTS: Configuration with lowest total torque

PSUEDO CODE

For Link1 Length Bounds:
For Link2 Length Bounds:
For Link3 Length Bounds:
For J1 X position Bounds:
For J1 Y Position Bounds:
if: iterations L1, L2, L3, X1, Y1 can meet the main kinematic frames, store this combination and the sum of link lengths
else: record the combination and move onto next iteration

→ For solutions that close, choose lowest torque out of the available results

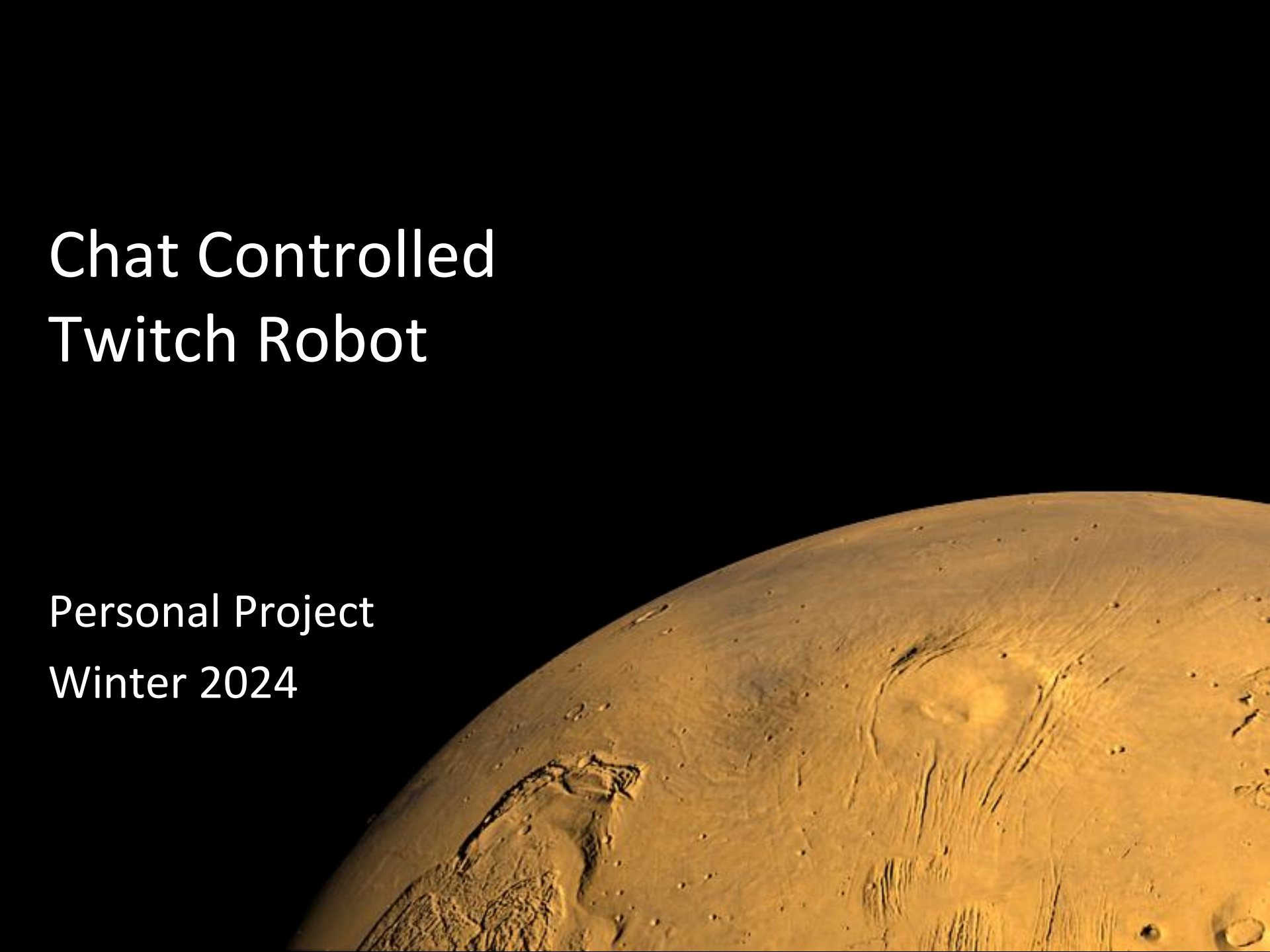
Results:

- Took a day to run (using multithreading) but ultimately worked!
- Optimized link lengths informed 3D printed RTA design

Chat Controlled Twitch Robot

Personal Project

Winter 2024



Objective

Background:

- In 2014, Twitch Plays Pokemon was a popular streaming channel where users completed the game through Twitch Chat commands
- My personal objective was to create a robot that streams itself and is remotely controllable through Twitch Chat

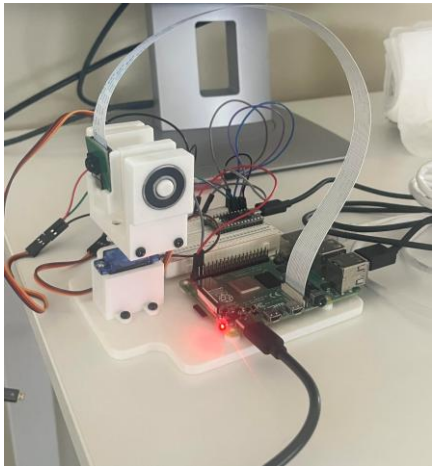
Twitch Plays Pokemon

Requirements:

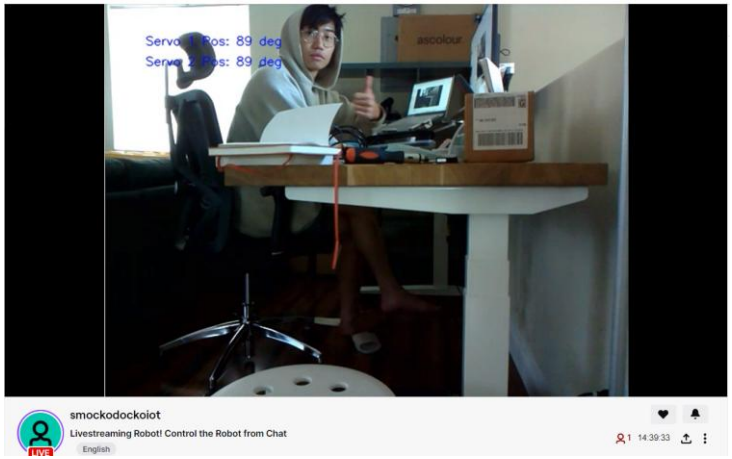
- Fully autonomous (stand-alone system)
- 2DoF camera control (pan/tilt)
- Chat integration
- Permanently streaming

Technologies Used:

- ROS2
- MicroROS
- Teensy / Raspberry Pi / Servos
- Arduino C/C++
- Networking (UART, SSH)
- OpenCV, Video4Linux (V4L)
- Linux (Ubuntu)



Twitch Robot Setup



Live Twitch Stream

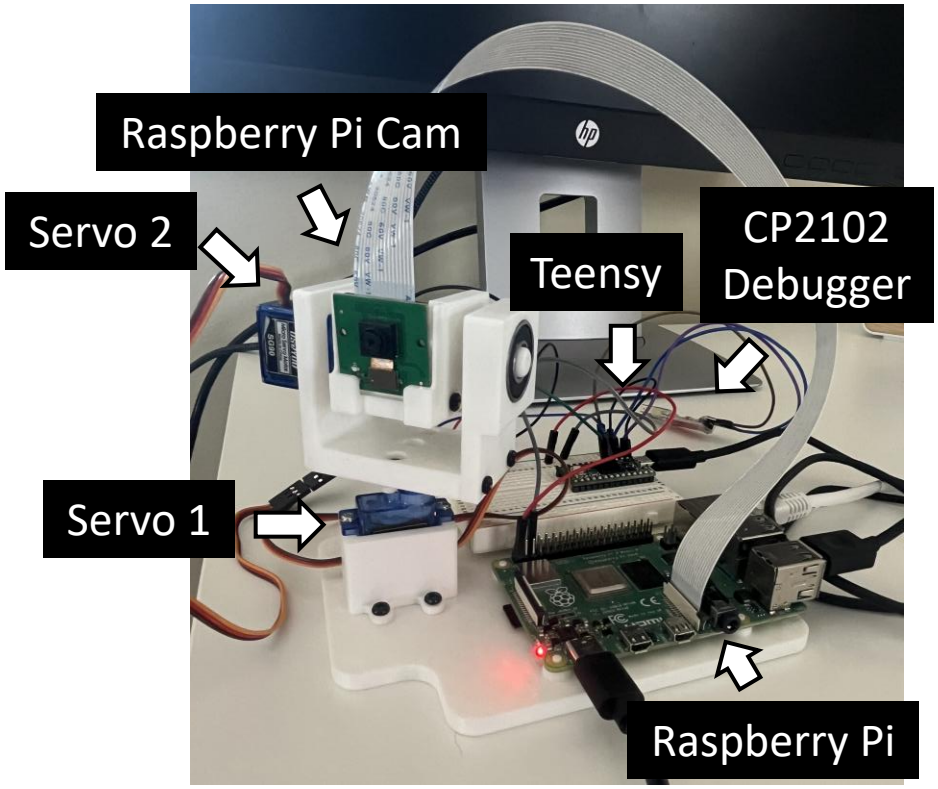
Twitch Stream Link

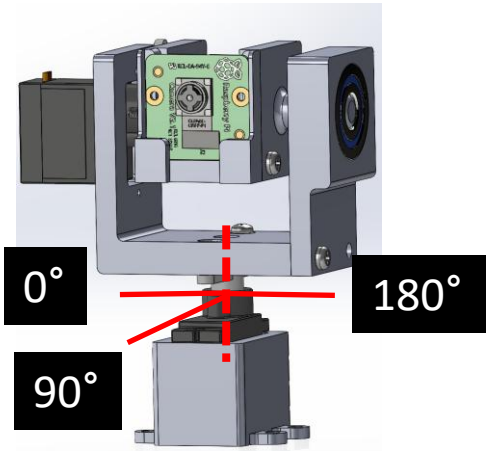
Demonstration Video

Twitch Robot Demonstration

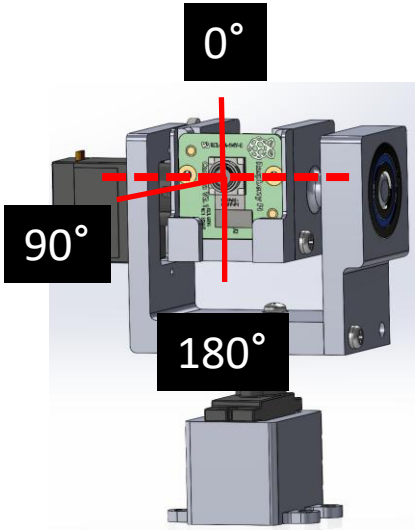
Mechanical Design

- Simple Pan-Tilt Camera Design
  - SG90 Servos use 5V from Raspberry Pi (convenience)
- 2 Degrees of Freedom (Pan, Tilt) that go from 0-180deg
- Camera works with Raspberry Pi by using Video4Linux (V4L) drivers / ROS2 package sourced from online





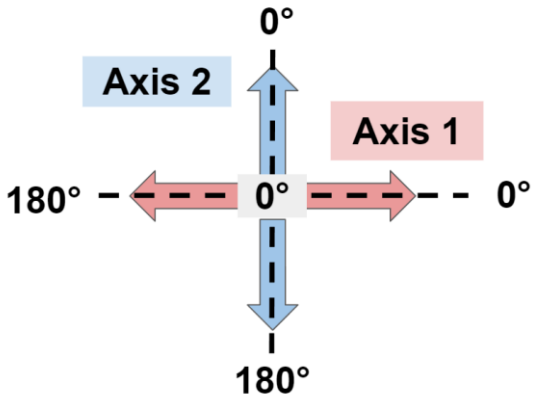
Servo 1 Axis



Servo 2 Axis

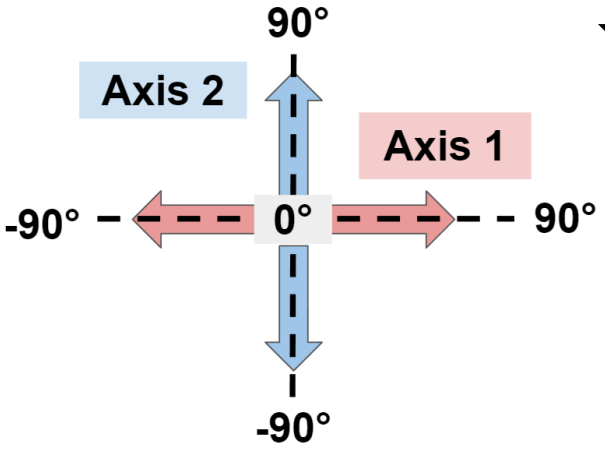
Axis Orientation

- Objective is to map the coordinates that the viewer sees, to the actual coordinates of the servo motors
- Makes for intuitive user experience as a centered position is [0,0]



Servo-based Definition

90-X



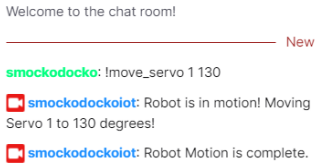
Camera Frame Definition

High Level CONOPS

1. Live Stream is Started

2. User Inputs Chat Command

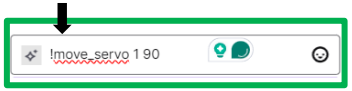
Command: !move_servo <servo_num> <servo_pos>

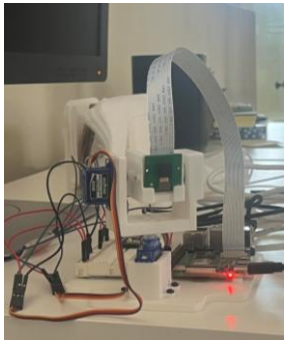


3. Message is parsed by Chat Bot for:

1. Servo Number (servo_num, [1 or 2])
2. Servo Position (servo_pos, [0 to 180°])

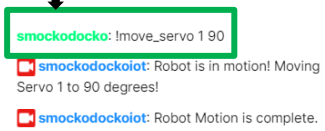
Chat message

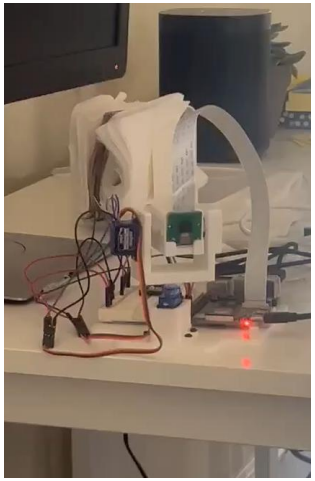




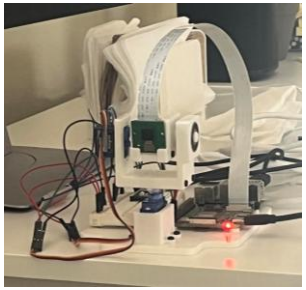
Start: Servo 1 Pos = 130°

Chat message sent



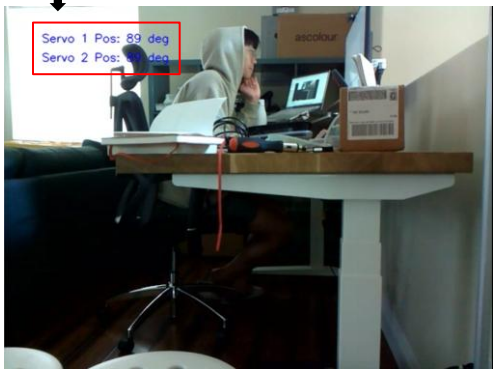


Movement from 130° to 90°



End: Servo 1 Pos = 90°

Position Updated



4. Command is sent to servo via. Teensy

5. Servo moves to new position

6. Teensy reports motion is complete

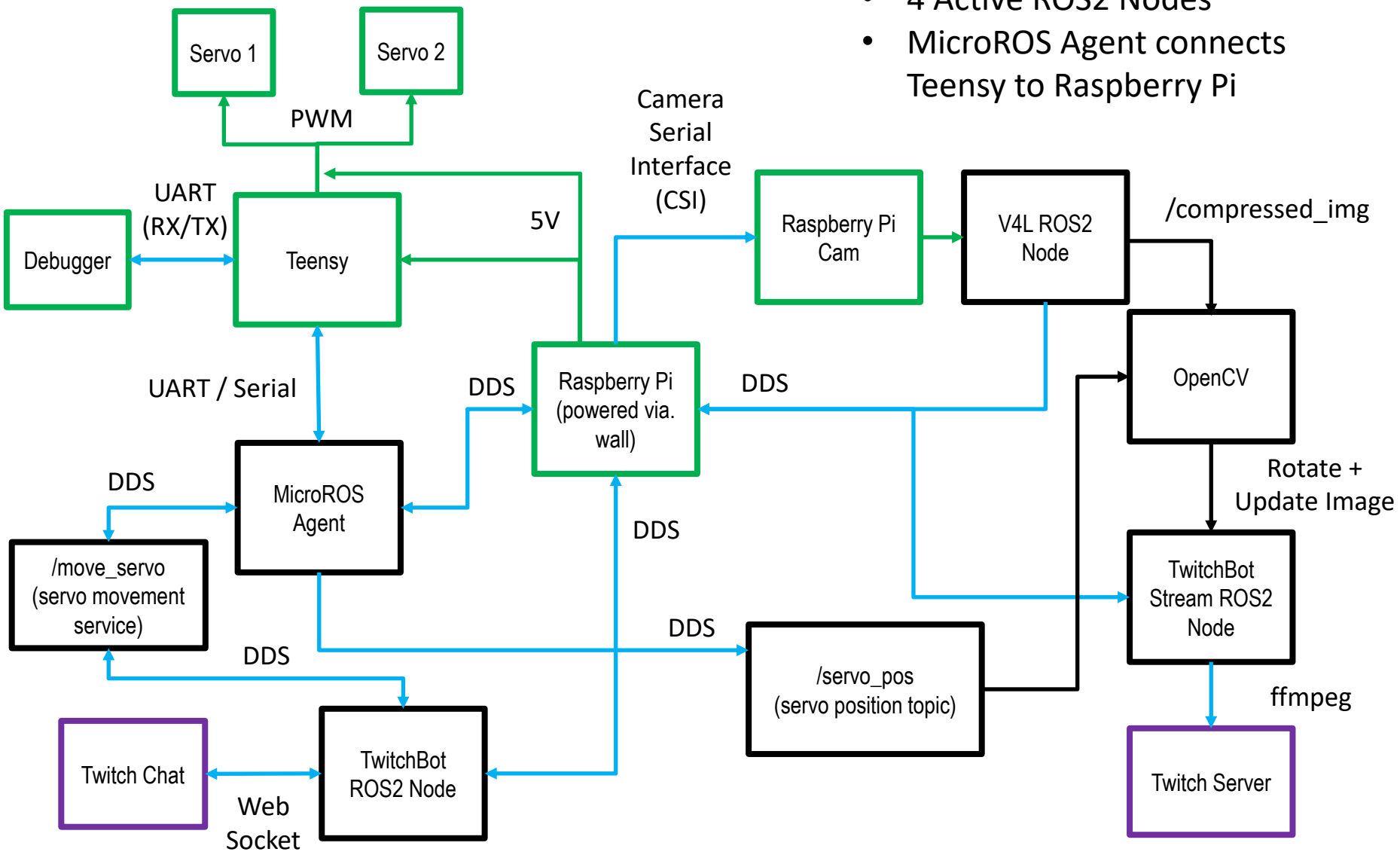
7. Chat bot is open to new commands

8. Camera image to stream is updated (delay exists)

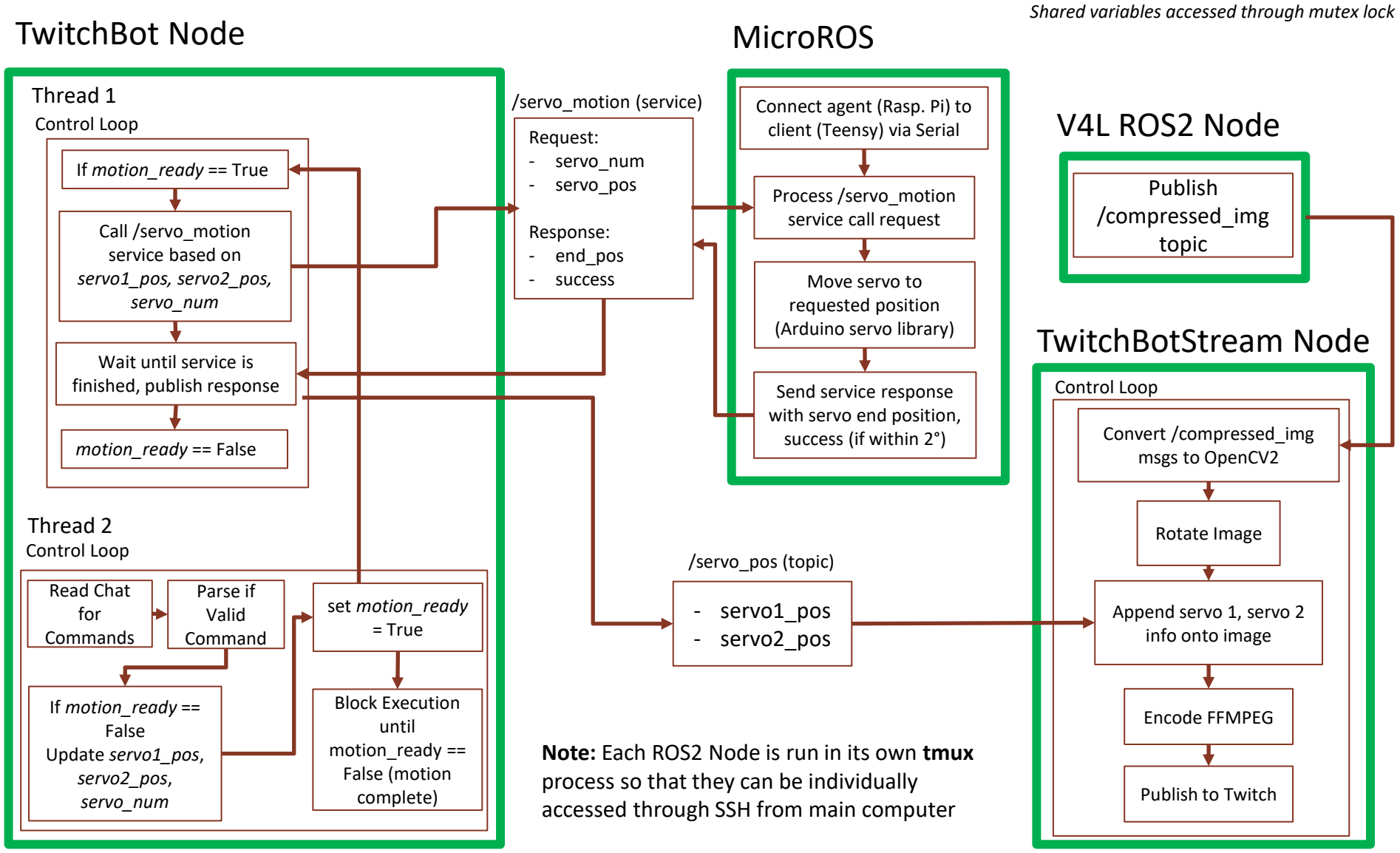
System Diagram

Hardware | Networking / Interface | ROS2 | Twitch

- 4 Active ROS2 Nodes
- MicroROS Agent connects Teensy to Raspberry Pi



ROS Architecture



Project Summary

Summary

- Twitch Bot is currently working as expected and is accessible on Twitch!
- Has been tested to run for a week straight without any disruptions or network drops
- Latency mainly depends on user's internet speed (Twitch App works best)

Potential Improvements:

- Improve internet connection for higher quality upload speeds on Raspberry Pi
- Add additional commands to move to specific waypoints (i.e. !kitchen, !couch)
- Improve clarity of command arguments (add diagrams to the stream)
- Improve mechanical setup (higher quality servos)

MedMate

Personal Project

Flowers IoT Hackathon Winning Submission

Fall 2020

MedMate Description

Flower Invention Studio IoT Hackathon Objective: Design a custom IoT Prototype within two weeks to solve a home automation task

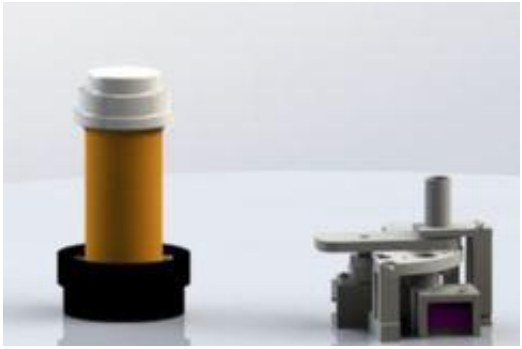
Solution:

- MedMate, a device that helps patients and caretakers monitor medicine intake

  Two Forms: pill bottle monitor and a pill dispenser
- MedMate tracks:
  1. when a user should take their prescription
  2. senses when a user has taken their medicine, and then
  3. logs this information in a database that is presented on a webpage
- Winning submission for Flowers Invention Studio IoT Hackathon
  - Worked with one other partner who focused primarily on web development
  - I designed the product, state machines, device code
  - Completed remotely!

References

- Hackathon Submission Link (with video)
- GitHub Link



Pill Bottle Monitor **Pill Dispenser**

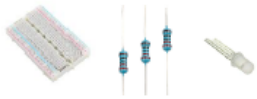
MedMate
demonstration Video

Hardware/Software Used

Tools Used:
- Ender3 Printer (personal printer)
- Solder

Software Used:
- Arduino C (C/C++)
- Eclipse Paho MQTT Python client (Python)
- Cloud Firestore (Google Firebase)
- React (JS)

Protocols:
- MQTT (Message Queuing Telemetry Transport)
- I2C (Inter-Integrated Circuit)

Hardware	Images
ESP32 IoT Microcontroller	
VCNL4010 Proximity Sensor	
DRV5053 Analog-Bipolar Hall Effect Sensor	
Tower Pro SG90	
5V Power Supply	
Breadboard, 220 Ohm Resistors, Multicolor LED	

Project Motivation

Motivation: Help my Dad and caregiver track medications by:

- reminding patients to take medicine
- allow caregiver to monitor intake history

Design Philosophy:

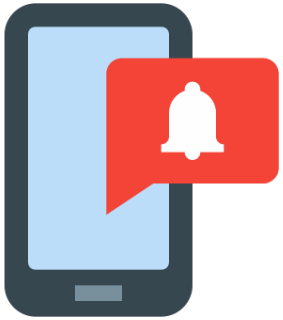
1. Low profile
2. Cheap
3. Simple hardware
4. Simple user interface
5. Interfaces with any standard pill bottle

ESP32 was our chosen controller because it is configurable with the Arduino client and has full IoT functionality. The chosen network protocol was MQTT, as it is designed for simple communication between a controller and host server.

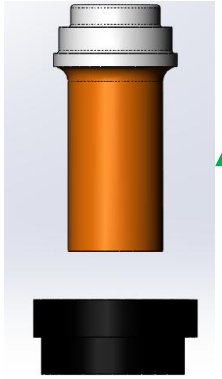
MedMate Pill Bottle Monitor Storyboard

1. User enters their prescription information

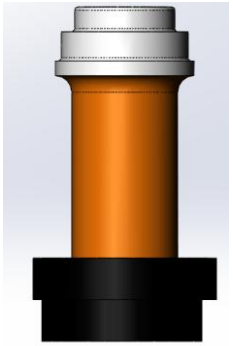
2. Server keeps track of when user should take medicine



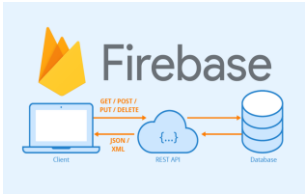
3. User is notified to take medicine



4. Pill bottle is taken off MedMate



5. Pills are consumed and bottle is put back on MedMate

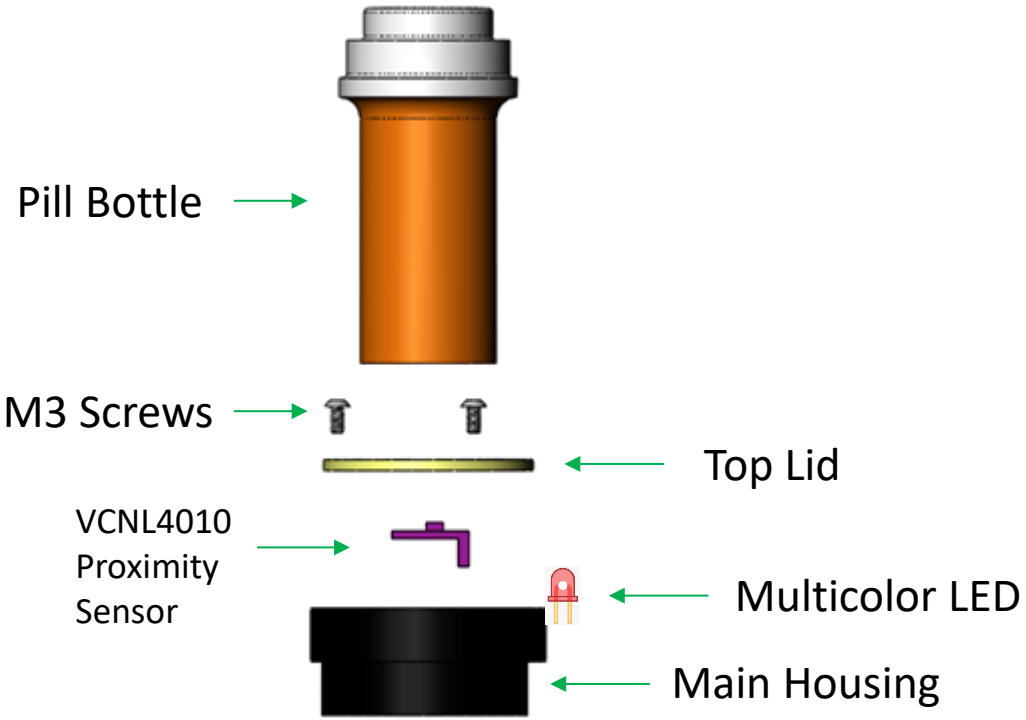


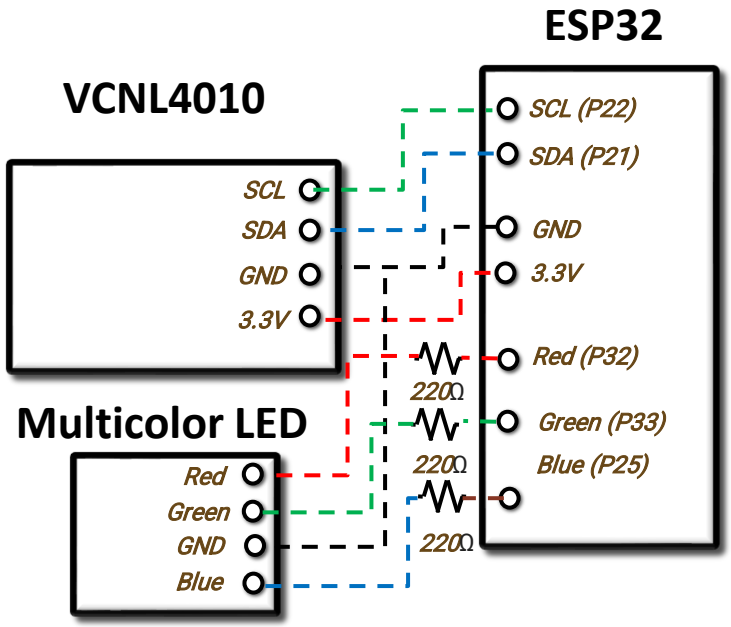
6. Time of Intake is uploaded to Database

7. Patient History shown on Webpage

Final Prototype (Pill Bottle Monitor)

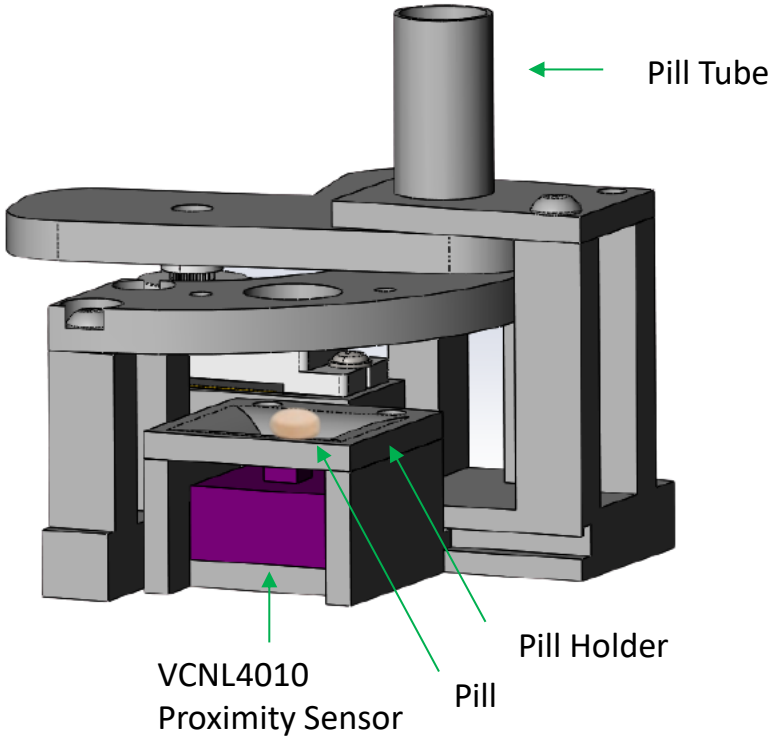
- Uses Proximity Sensor to sense pill bottle presence
- Added LED for debugging/clarity
- Fits common pill bottle diameters

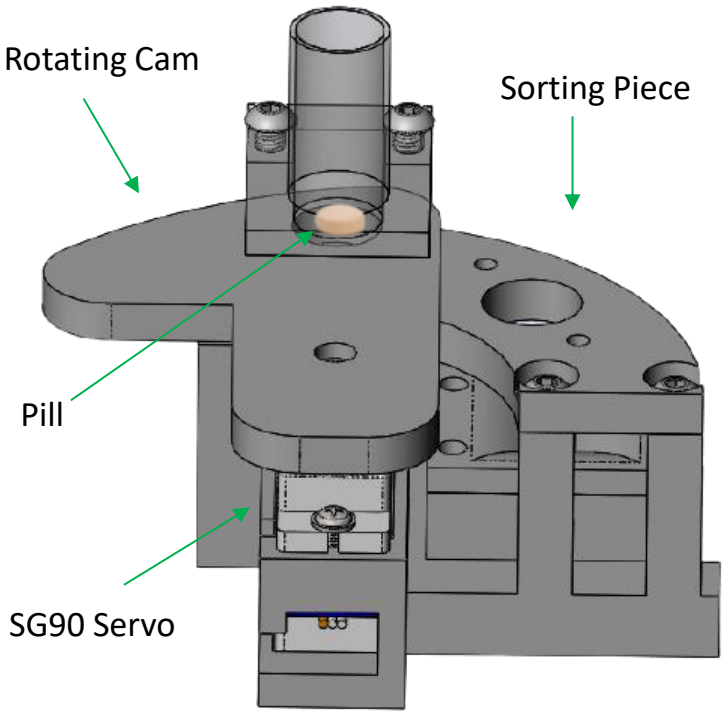




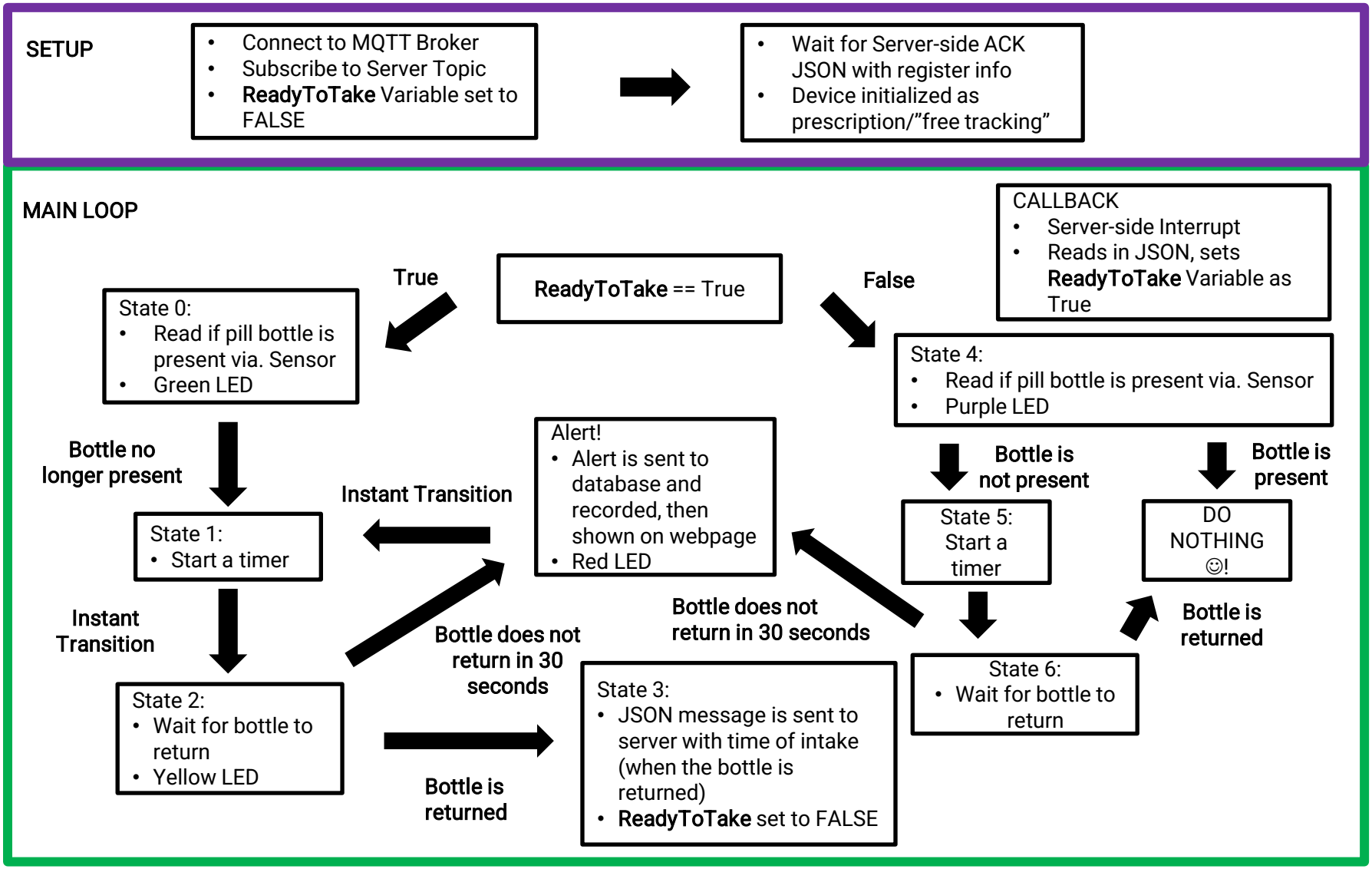
Final Prototype (Pill Dispenser)

- Actuates using servo and rotating cam to release pills at medication time
  - Controls how much medicine the user can take.
- Still uses proximity sensor to detect if pill has been taken
- Can be part of a larger assembly – designed for demonstration purposes





MedMate Pill Bottle Monitor State Machine



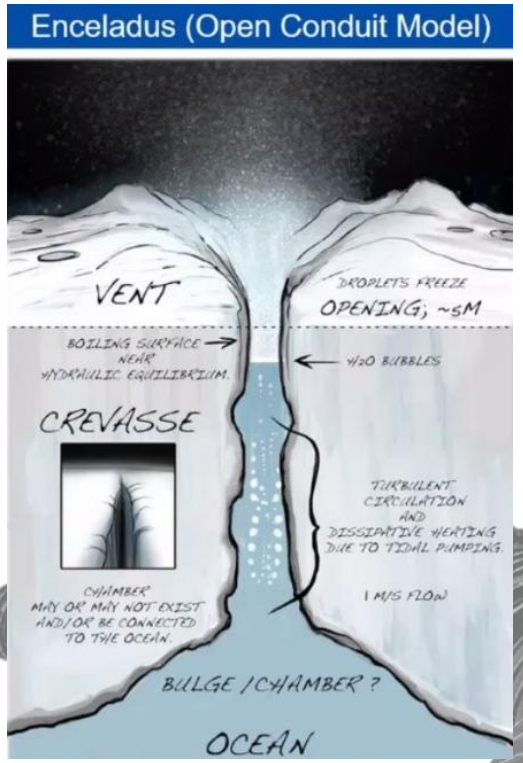
Exobiology Extant Life Surveyor Robot Sampling System

Senior Capstone – JPL

Fall 2021

Background & Problem Statement

- Exobiology Extant Life Surveyor (EELS)
  - Bio-inspired snake-like robot
  - Traverse Enceladus' icy ocean-world terrain to **search for life**
- Front nose segment must be designed to:
  - Collect **sub-glacial liquid samples**
  - Gather **environmental data**
  - Travel over icy terrain and underwater
- Impact
  - Understand factors contributing to **melting glacial icecaps**
  - Explore glacial moulins and crevices traditionally **inaccessible to humans**
  - Determine conditions required to **sustain life**



Personal Contributions: Project Management, Mechatronics / Code Design, Literature Research

System Requirements

EELS System Integration

- Fits within Ø12cm X 50cm cylinder
- Withstands icy environmental conditions
  - -20 to 20°C
  - 0 to 150psi
  - 0 to 2m/s flow
- Withstands traversal through environment

Liquid Sampling System

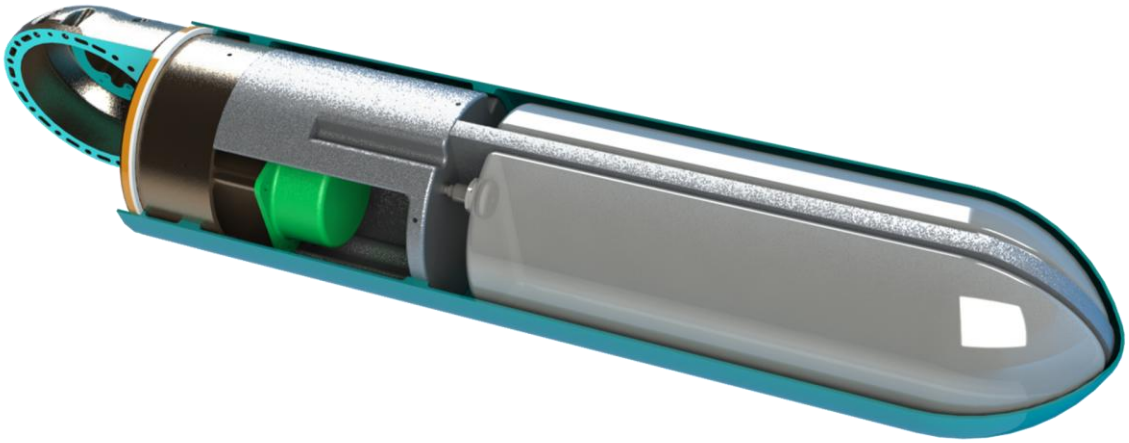
- Acquires 2 separate 1L samples of liquid
- Sterilizable collection system
- Easily removable liquid samples
- Fill at rate of 0.5L/min

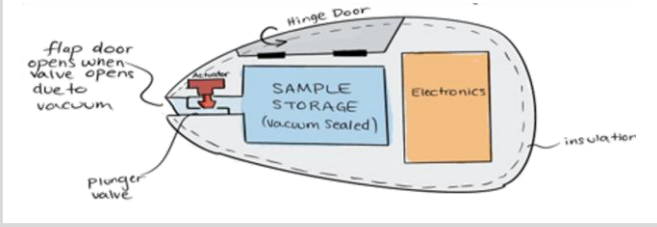
Sensor Data Collection

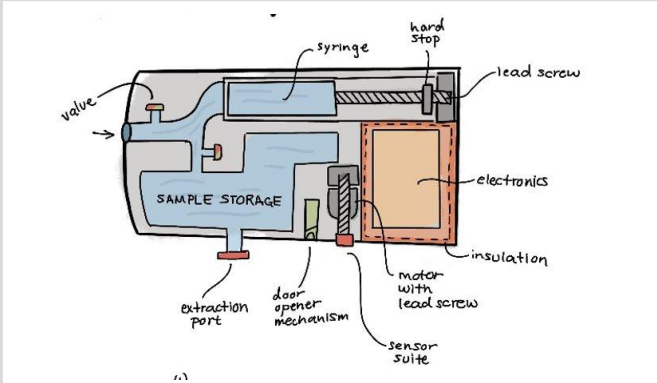
- Gather pressure, temperature, aquatic chemistry, and imagery data
- 0°C minimum sensor operating temperature
- -20°C minimum sensor storage temperature

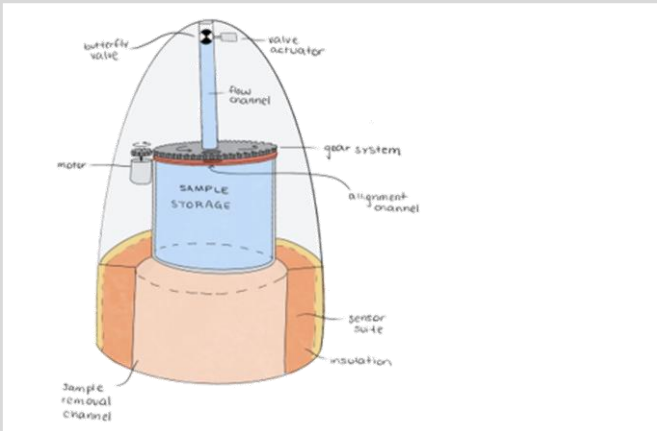
Preliminary Sketches & CAD

- Initial ideation created an outline of the system shape and major mechanical components
- Both **vacuum** and **pump-based** designs were considered

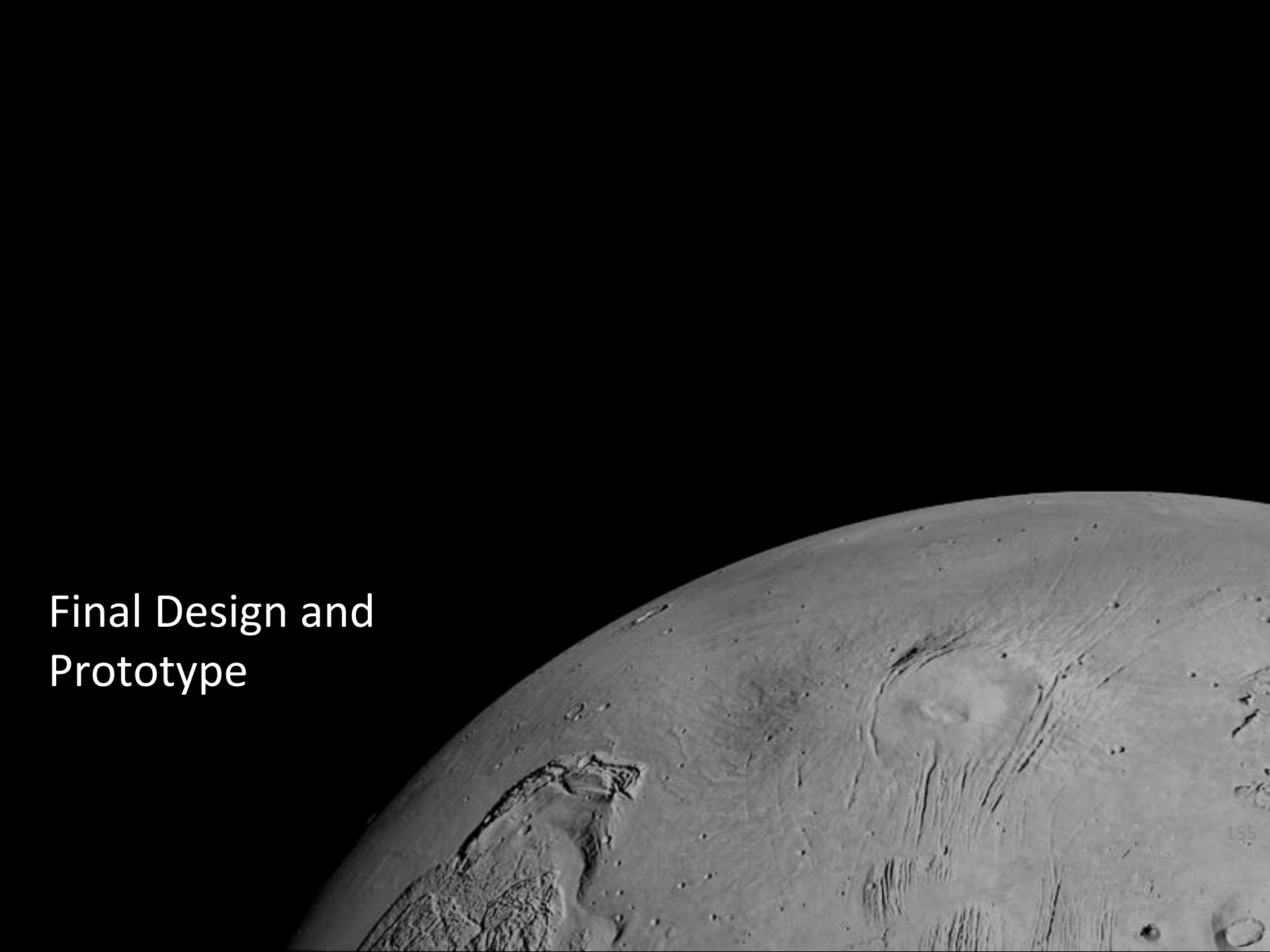




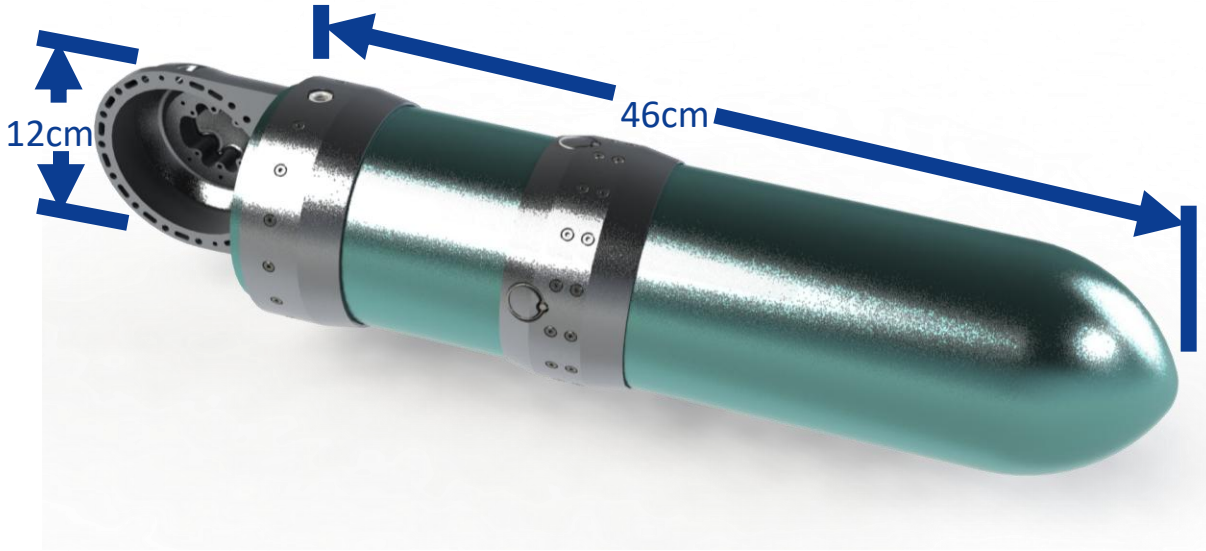


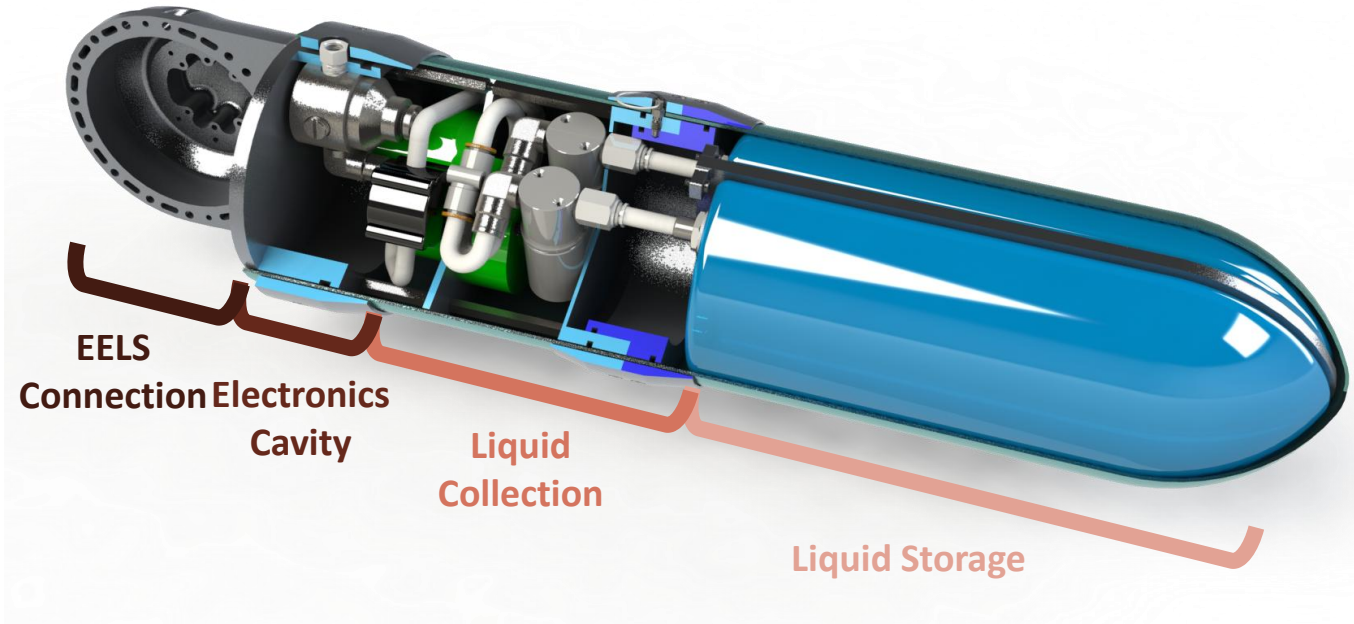


Final Design and Prototype

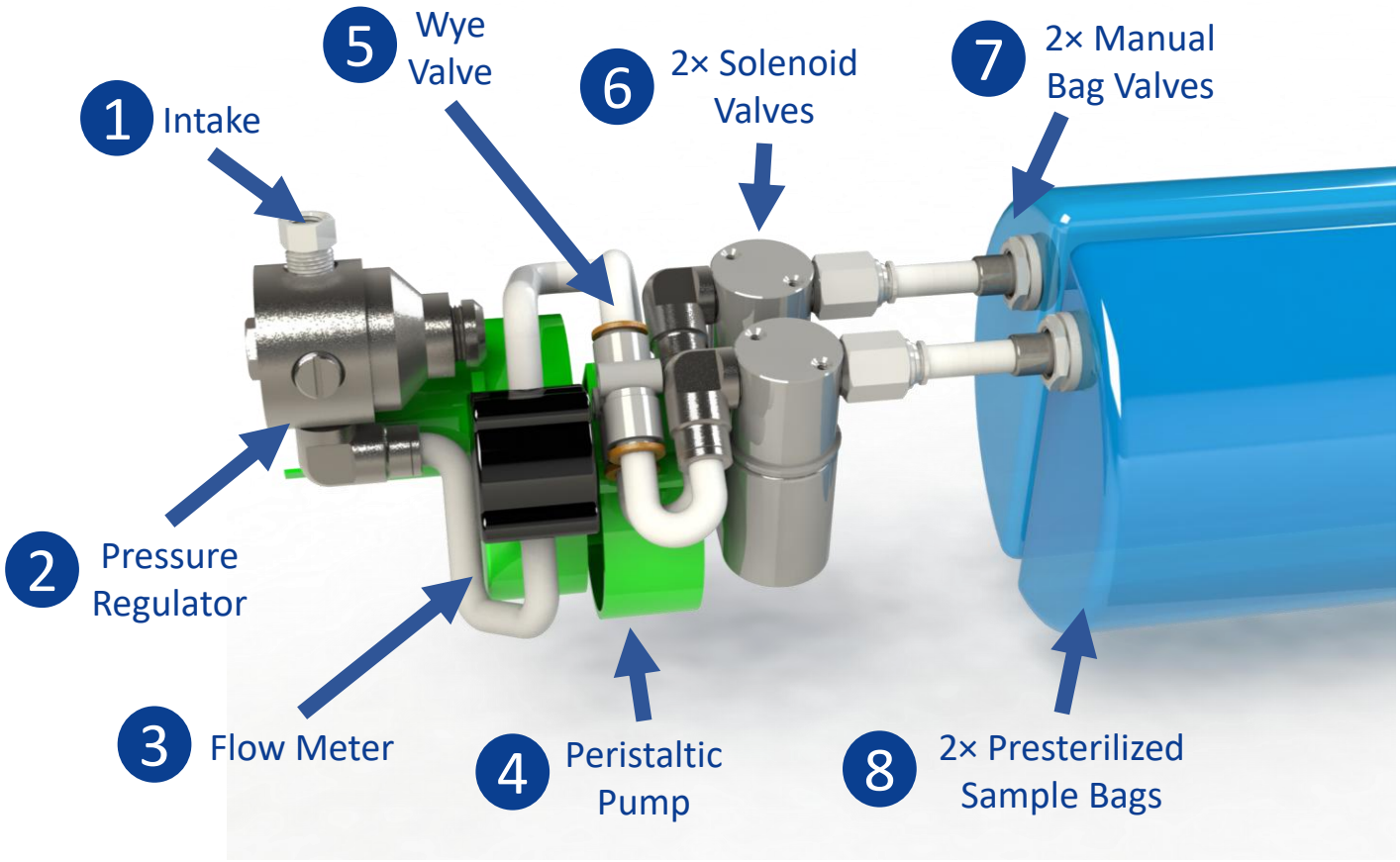


Mechanical: Overview

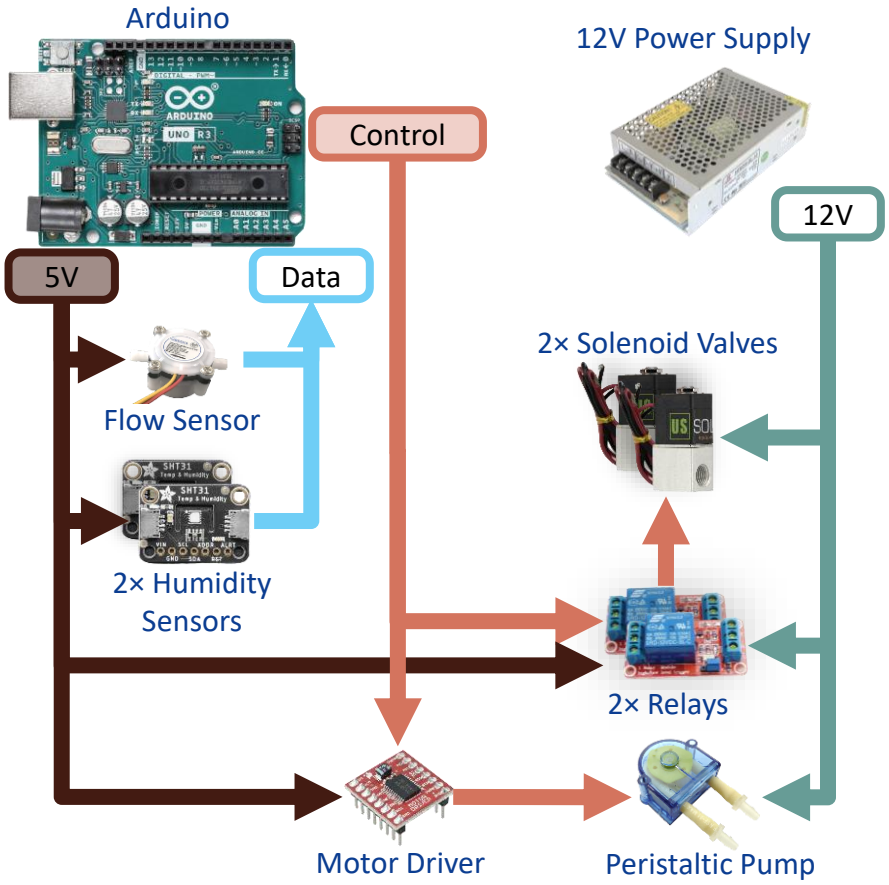




Mechanical: Liquid Collection

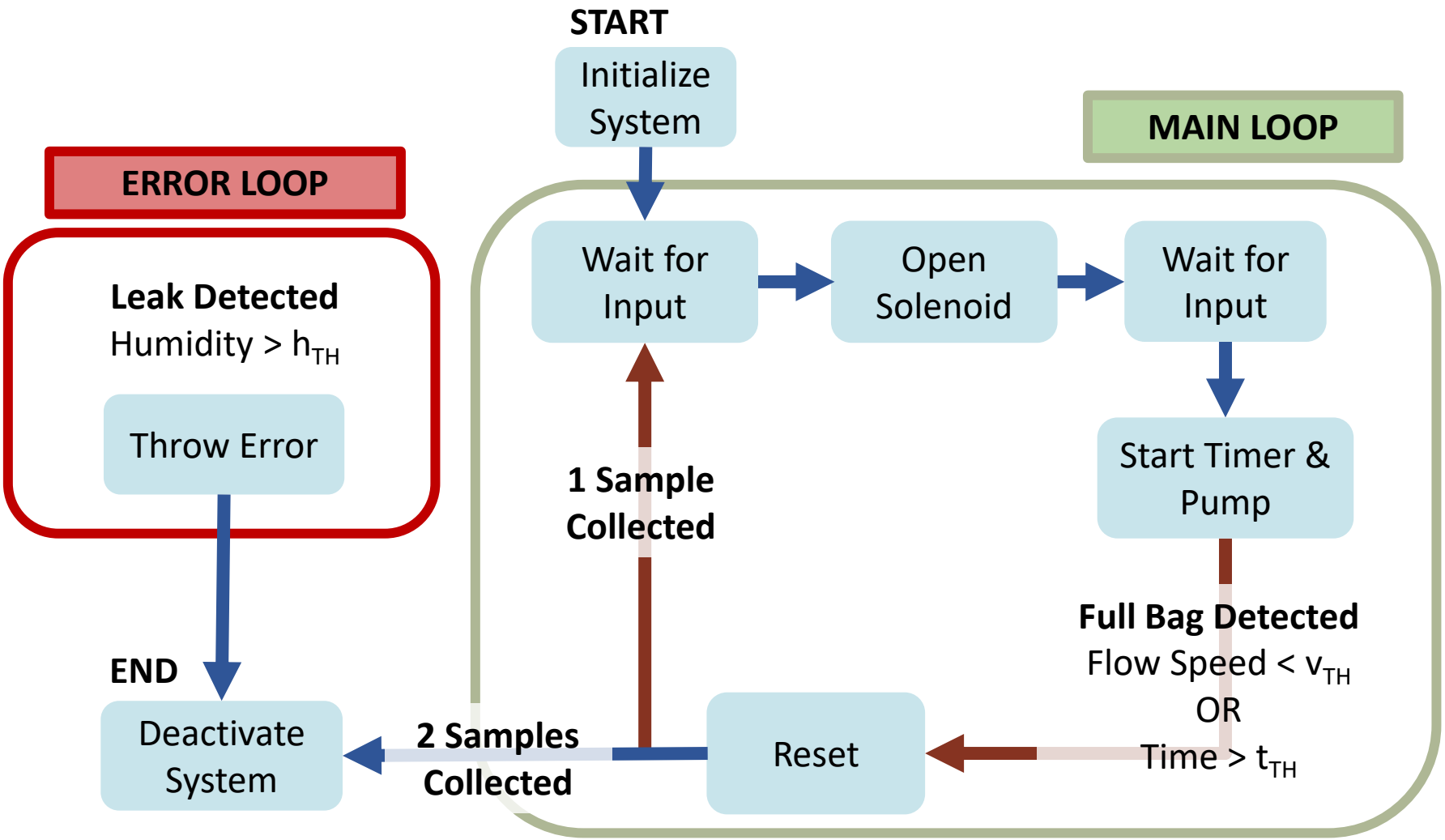


Electrical Overview

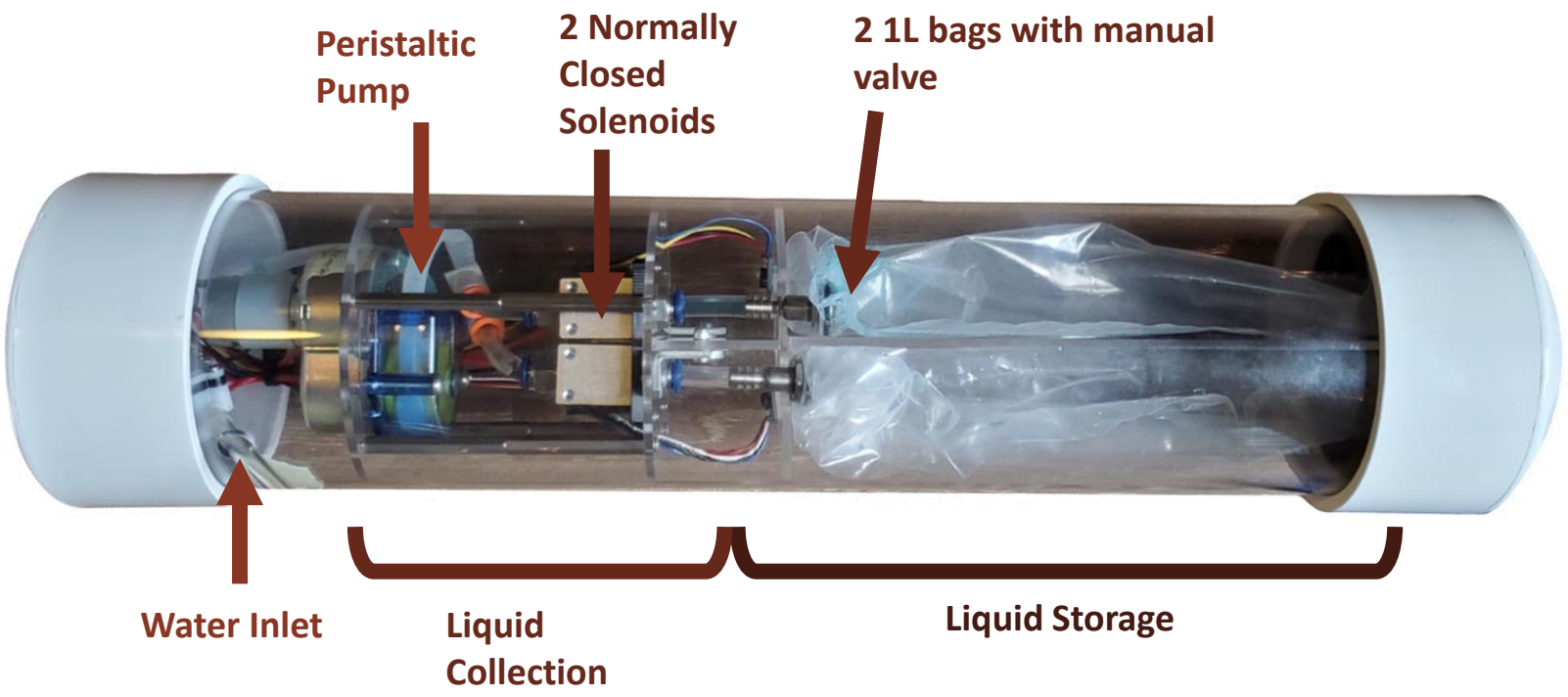


- Flow sensor to detect when bag is full
- Humidity sensor to detect leaks as small as 5mL
- Two solenoid valves to control the filling of bags separately

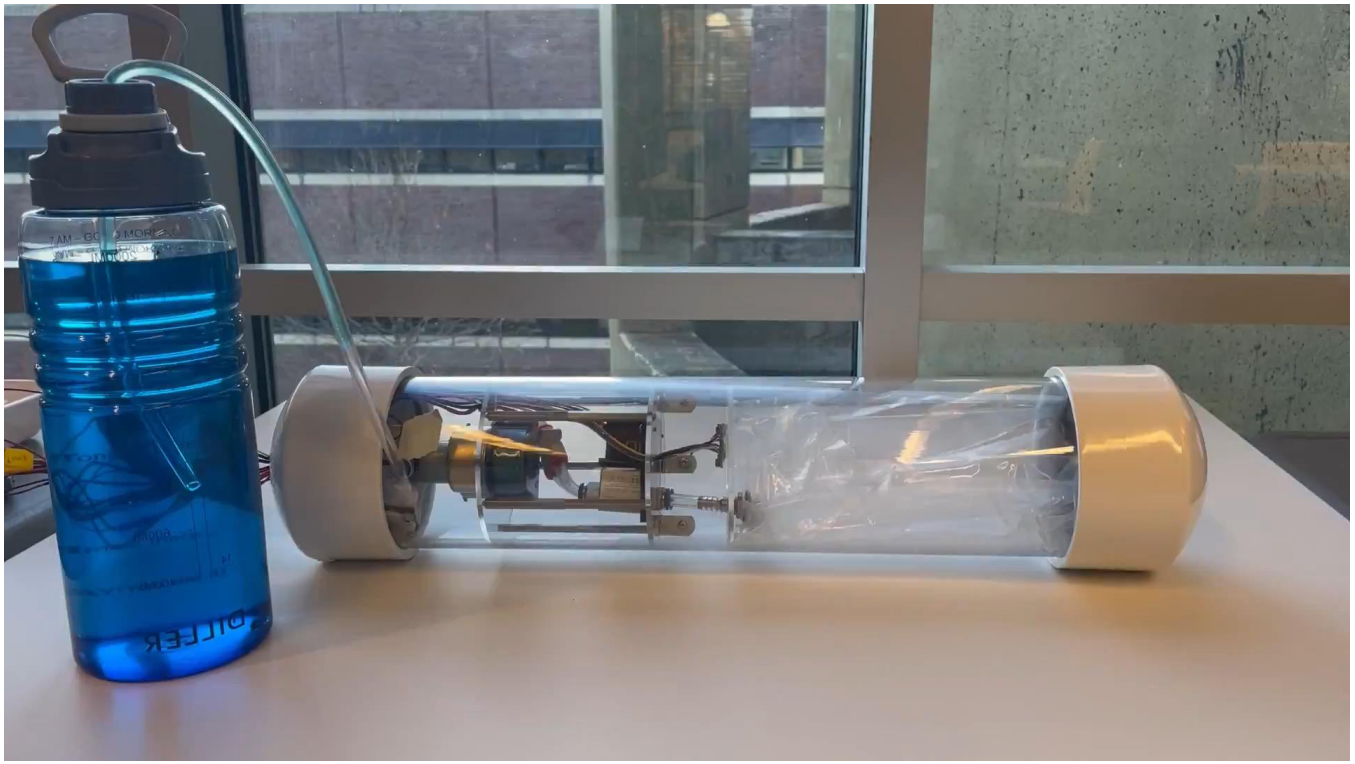
Software Overview



Final Prototype



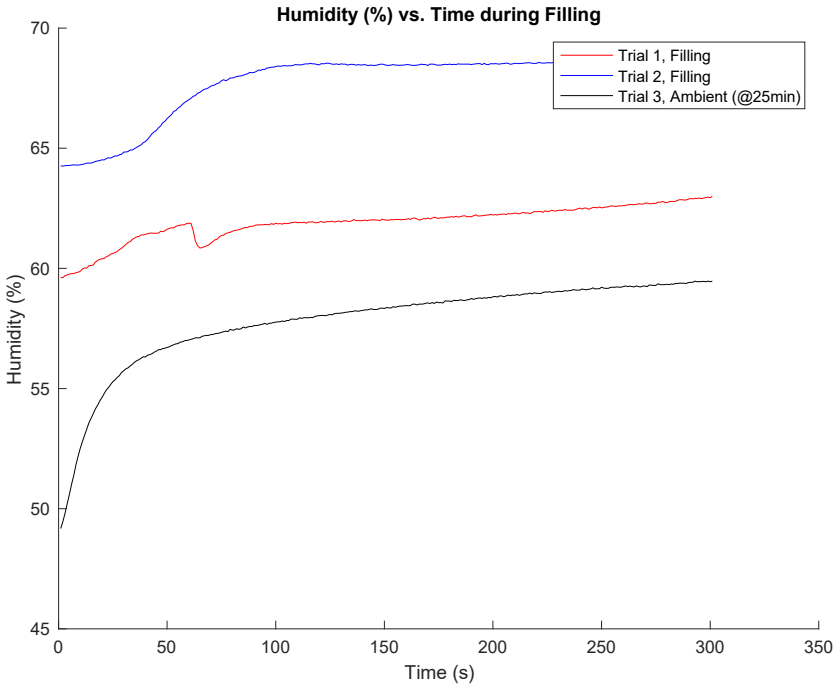
Final Prototype Video

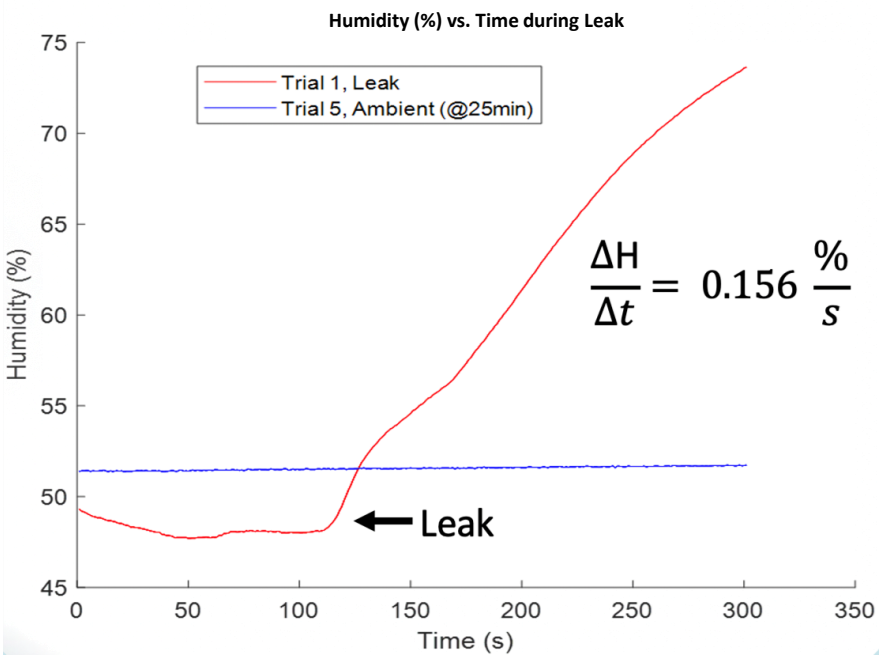


Fill Time: 2min 15s (135s)

Humidity Sensor Testing

- Conducted test trials with previous prototype to determine humidity sensor effectiveness in determining leaks
- Performed ambient, filling, and leak tests
- Rate of change of humidity can be used to determine if leaks occur





Conclusion

Future Work

- More refined prototype rated for Antarctic conditions
- Test while submerged underwater
  - Fit electronics into sampling system
  - Create custom PCB and choose a lower profile microcontroller
- Implement full system control using flow meter and humidity sensors
- Internal air pressurization

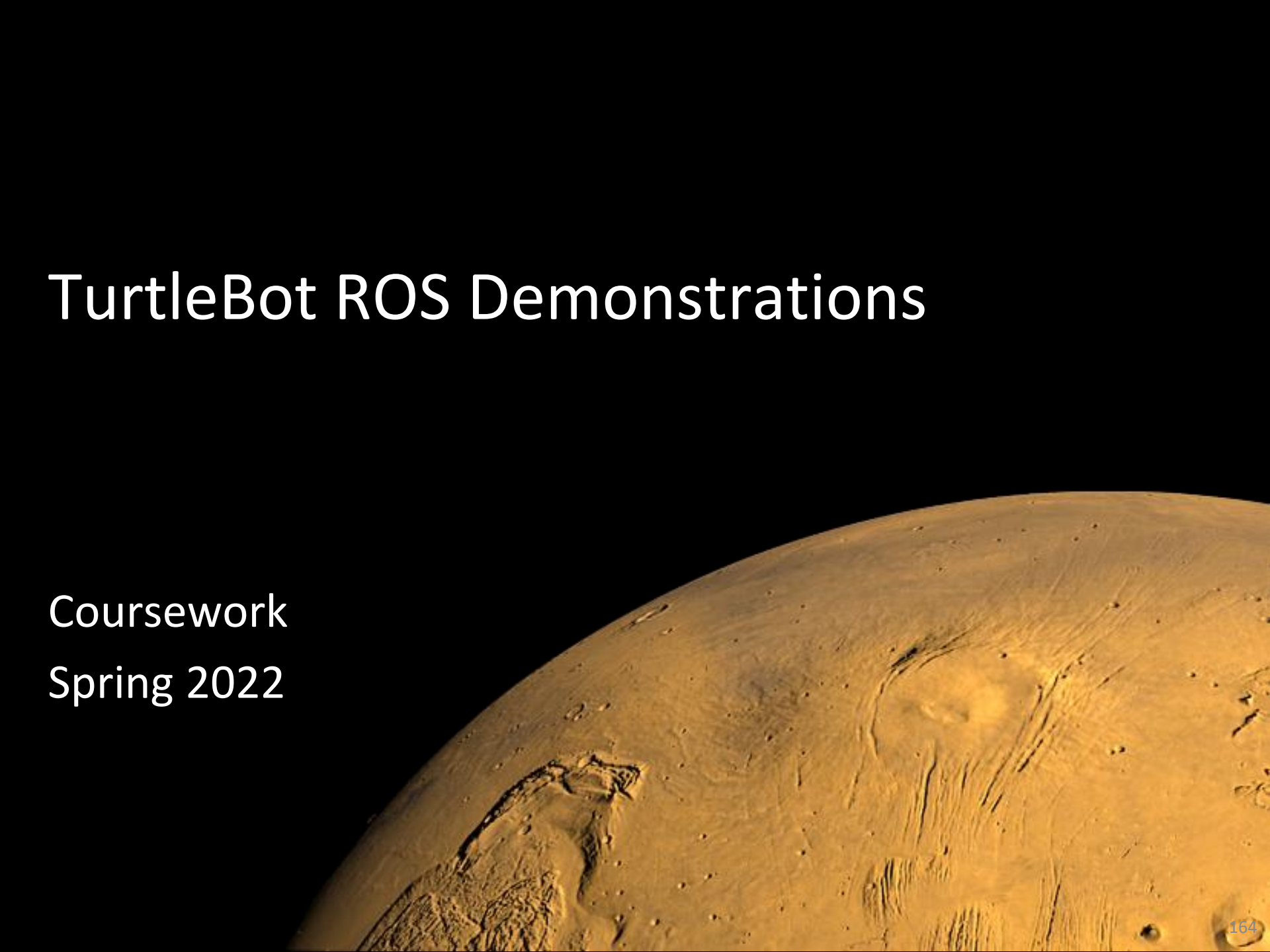
Impact

- Aid in ongoing efforts to understand **melting glacial icecaps** and **inaccessible** glacier moulins
- Provide data of the subglacial environment to **determine conditions suitable to sustain life**

TurtleBot ROS Demonstrations

Coursework

Spring 2022

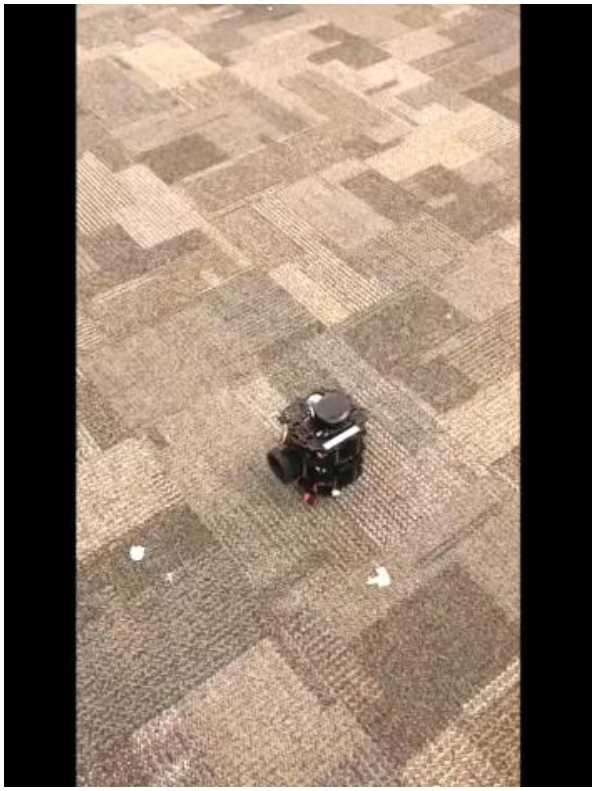


OpenCV Object Following with PID

- Robot to follow a specific object in space while maintaining object in center and at correct distance
  - Used LIDAR to determine distance to object, PID to maintain relative distance/angle
  - Used OpenCV to track object and maintain a specific orientation

Robot Navigation with Odometry/Lidar

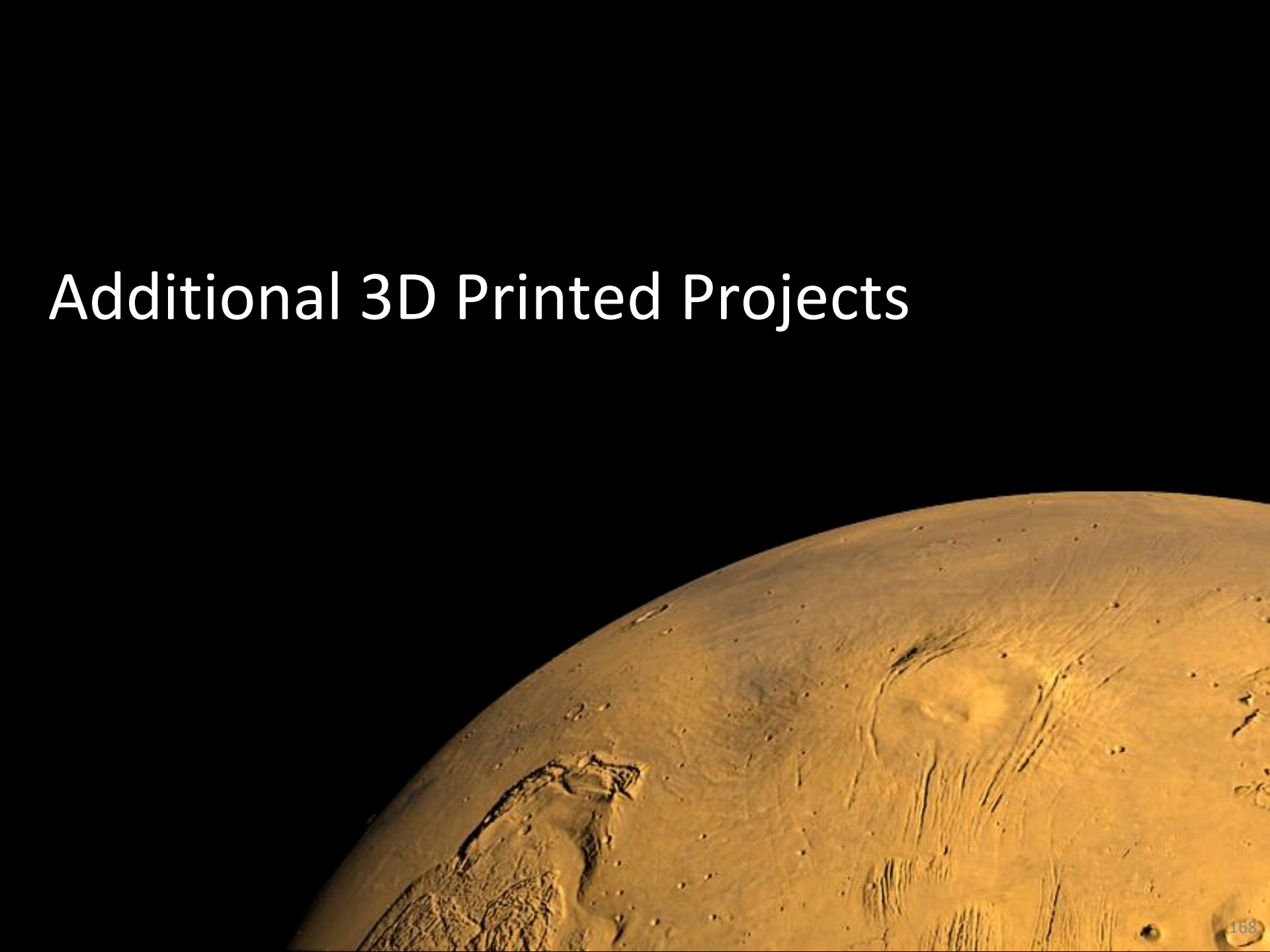
- Using robots onboard odometry and dead reckoning to navigate to various waypoints while avoiding random obstacles
  - Writing and subscribing to odometry nodes, robot kinematics
  - Filtering noisy lidar data
  - Obstacle detection with avoidance routine, while maintaining knowledge of position



Final Project: Sign Classification and Navigation to Goal

- Robot classifies 6 different signs to navigate towards an end goal
  - Used state machine to control robot behavior
    - Implemented behaviors if sign is not correctly classified or if robot is stuck
  - Trained and used KNN ML model to classify signs with image processing
  - LiDAR, filtering, odometry, PID, ROS, image processing, classification

Additional 3D Printed Projects



3D Printed Gearbox

- 9:1 Gear Ratio with 3D Printed Gears and COTS Bearings
- Demonstration of gear feasibility and design with 3D Printed parts
  - Skeleton Modeling